Programowanie w C++

Materiał dydaktyczny

Wersja dydaktyczna — C++11–C++20

Spis treści

1 Kontenery STL i algorytmy **8**

1.1 Cel i zakres rozdziału . . . 8
1.2 Klasyfikacja kontenerów i podstawowe pojęcia . . . 8
1.3 `std::vector`: domyślny kontener sekwencyjny . . . 9
1.4 `std::array`, `std::deque` i listy . . . 11
1.5 Kontenery uporządkowane: `set` i `map` . . . 12
1.6 Kontenery haszujące: `unordered_map` i `unordered_set` . . . 15
1.7 Adaptery: `stack`, `queue`, `priority_queue` . . . 16
1.8 Iteratory, stabilność adresów i unieważnianie . . . 17
1.9 Algorytmy standardowe i wymagania wobec iteratorów . . . 17
1.10 Przypadki projektowe: dobór kontenera do problemu . . . 20
1.11 Złożoność, lokalność pamięci i pomiar wydajności . . . 21
1.12 Najczęstsze błędy i dobre praktyki . . . 22
1.13 Pytania kontrolne . . . 22
1.14 Podsumowanie rozdziału . . . 23
Literatura do rozdziału 13 . . . 23

2 Model obiektowy C++ i polimorfizm **24**

2.1 Po co poznawać model obiektowy? . . . 24
2.2 Obiekty, podobiekty i tożsamość typu . . . 24
2.3 Przeciążanie, przesłanianie i funkcje wirtualne . . . 25
2.4 Destruktor wirtualny i reguły czasu życia . . . 28
2.5 Object slicing: utrata części pochodnej . . . 29
2.6 RTTI: `dynamic_cast` oraz `typeid` . . . 31
2.7 Wielodziedziczenie i wirtualne klasy bazowe . . . 32
2.8 Kowariantne typy zwracane . . . 33
2.9 Klasy abstrakcyjne jako kontrakty i wzorzec NVI . . . 34
2.10 Model implementacyjny: *vptr*, *vtable* i ABI . . . 35
2.11 Alternatywa: polimorfizm statyczny przez szablony i CRTP . . . 36
2.12 Polimorfizm bez dziedziczenia: `std::variant` . . . 38

2.13 Polimorfizm dynamiczny i statyczny — porównanie . . . 38
2.14 Reguła zera/pięciu w klasach polimorficznych . . . 39
2.15 Typowe błędy i pytania kontrolne . . . 39
2.16 Wskazówki dotyczące diagnostyki i pomiarów . . . 40
2.17 Podsumowanie rozdziału . . . 40
Literatura do rozdziału 12 . . . 40

3 Specyfikatory i kwalifikatory deklaracji w C++ 41
3.1 Wprowadzenie: czym właściwie są `static`, `const` i `inline`? . . . 41
3.2 `const`: zakaz modyfikacji przez dany dostęp . . . 42
3.3 `constexpr`: stałe i funkcje dostępne dla obliczeń stałych . . . 43
3.4 `consteval` i `constinit` (od C++20) . . . 44
3.5 `static`: trzy różne zastosowania . . . 44
3.6 `extern`: deklaracja bytu zdefiniowanego gdzie indziej . . . 46
3.7 `inline`: dozwolone definicje w wielu TU . . . 47
3.8 Łączenie specyfikatorów: najważniejsze przypadki . . . 48
3.9 `thread_local`: osobny obiekt dla każdego wątku . . . 49
3.10 `mutable` i `volatile`: dwa inne rodzaje kwalifikacji . . . 49
3.11 `auto` i `decltype(auto)`: dedukcja typu . . . 50
3.12 Powiązane specyfikatory metod i funkcji . . . 51
3.13 Praktyczna organizacja plików: przykład całościowy . . . 51
3.14 Najczęstsze błędy i reguły wyboru . . . 52

4 Wskaźniki i inteligentne wskaźniki w C++ 54
4.1 Po co istnieją wskaźniki? . . . 54
4.2 Podstawy: adres, dereferencja i `nullptr` . . . 54
4.3 Wskaźniki, referencje i kwalifikator `const` . . . 56
4.4 Czas życia obiektu i wskaźniki wiszące . . . 57
4.5 Tablice, arytmetyka wskaźników i `std::span` . . . 58
4.6 `void*`, konwersje i rzutowania wskaźników . . . 59
4.7 Pamięć dynamiczna: `new`, `delete` i ich ograniczenia . . . 59
4.8 RAII i semantyka własności . . . 60
4.9 `std::unique_ptr`: jeden właściciel . . . 61
4.10 `std::shared_ptr`: współdzielona własność . . . 64
4.11 `std::weak_ptr`: obserwacja bez utrzymywania przy życiu . . . 66
4.12 `enable_shared_from_this` i wskaźnik `this` . . . 67
4.13 Zaawansowany przykład: wskaźnik aliasujący `shared_ptr` . . . 67
4.14 Wskaźniki w interfejsach funkcji i klas . . . 68
4.15 PImpl: ukrywanie implementacji przez `unique_ptr` . . . 68

4.16 Wskaźniki do funkcji i składowych klas . . . 69
4.17 Wskaźniki i wielowątkowość . . . 70
4.18 Najczęstsze błędy zebrane w jednym miejscu . . . 71
4.19 Jak wybrać właściwy typ? . . . 71
4.20 Podsumowanie i praktyczne reguły . . . 72

5 Semantyka przenoszenia w C++ 73
5.1 Dlaczego wprowadzono semantykę przenoszenia? . . . 73
5.2 Typ wyrażenia a kategoria wartości . . . 73
5.3 Referencje lvalue i rvalue . . . 75
5.4 Co naprawdę robi `std::move`? . . . 76
5.5 Konstruktor przenoszący i przenoszący operator przypisania . . . 77
5.6 Reguła pięciu, reguła trzech i reguła zera . . . 79
5.7 Przenoszenie a wyjątki: `noexcept` . . . 80
5.8 Zwijanie referencji (*reference collapsing*) . . . 81
5.9 Referencje przekazujące i dedukcja `T&&` . . . 81
5.10 Perfect forwarding i `std::forward` . . . 82
5.11 Perfect forwarding a przeciążenia i szablony klas . . . 83
5.12 Copy elision: kiedy nie ma ani kopii, ani przeniesienia? . . . 84
5.13 Argumenty funkcji: kiedy kopiować, kiedy przenosić? . . . 85
5.14 Rvalue-qualified methods i świadome udostępnianie zasobów . . . 86
5.15 Kontenery, iteratory przenoszące i wzorce użytkowe . . . 87
5.16 Typowe błędy i pułapki . . . 87
5.17 Przykład przekrojowy: kolejka zadań . . . 88
5.18 Praktyczna ściąga i zalecenia projektowe . . . 89

6 Wyjątki i bezpieczeństwo wyjątkowe w C++ 91
6.1 Po co C++ ma wyjątki? . . . 91
6.2 Składnia `try`, `throw` i `catch` . . . 91
6.3 Odwijanie stosu i RAII . . . 92
6.4 Gwarancje bezpieczeństwa wyjątkowego . . . 94
6.5 `noexcept`: deklarowanie i sprawdzanie . . . 95
6.6 Własne klasy wyjątków i hierarchia standardowa . . . 96
6.7 Wyjątki w funkcjach szablonowych i neutralność wyjątkowa . . . 97
6.8 Przenoszenie wyjątków między funkcjami i wątkami . . . 97
6.9 Wyjątki zagnieżdżone i dodatkowy kontekst . . . 98
6.10 `std::uncaught_exceptions` i strażnicy zakresu . . . 99
6.11 Wyjątki a alternatywne modele obsługi błędów . . . 99
6.12 Typowe błędy i dobre praktyki . . . 100

6.13 Przykład końcowy: transakcyjna aktualizacja danych . . . 101
6.14 Podsumowanie rozdziału . . . 101

7 Organizacja pamięci i wydajność 102

7.1 Po co analizować organizację pamięci? . . . 102
7.2 Obiekt, bajt, adres i czas życia . . . 102
7.3 Layout struktury: `sizeof`, `alignof`, padding . . . 104
7.4 Wyrównane alokacje i czas życia obiektów . . . 106
7.5 Alokatory: pamięć, obiekty i bezpieczeństwo wyjątkowe . . . 107
7.6 `std::pmr`: polimorficzne zasoby pamięci . . . 108
7.7 Lokalność danych i hierarchia cache . . . 110
7.8 False sharing i współbieżne liczniki . . . 111
7.9 Jak rzetelnie mierzyć wydajność? . . . 113
7.10 Przykładowy projekt: szybkie przetwarzanie wielu punktów . . . 113
7.11 Jak dobrać technikę? . . . 114
7.12 Typowe błędy i pułapki . . . 114

8 Lambdy i obiekty funkcyjne 116

8.1 Po co istnieją obiekty funkcyjne? . . . 116
8.2 Trzy podstawowe postacie kodu wywoływalnego . . . 116
8.3 Budowa wyrażenia lambda . . . 117
8.4 Przechwytywanie zmiennych (*captures*) . . . 118
8.5 Przechwytywanie `this` i `*this` . . . 119
8.6 Domyślnie stały operator wywołania i `mutable` . . . 120
8.7 Typ lambdy, kopiowanie i konwersja na wskaźnik . . . 120
8.8 Lambdy generyczne i własne parametry szablonu . . . 121
8.9 Wymuszanie sygnatury: `noexcept`, `requires`, zwracanie referencji . . . 122
8.10 `std::invoke`: jeden mechanizm wywołania . . . 122
8.11 `std::function`: jeden typ dla wielu zachowań . . . 123
8.12 Co wybrać: `auto`, szablon, wskaźnik czy `std::function`? . . . 124
8.13 Lambdy ze standardowymi algorytmami . . . 124
8.14 `std::ref`, `std::bind_front` i częściowe wiązanie argumentów . . . 125
8.15 Praktyczny projekt: rejestr callbacków . . . 126
8.16 Rekurencyjne lambdy bez `std::function` . . . 127
8.17 Lambdy w metaprogramowaniu: `std::visit` i `std::apply` . . . 127
8.18 Pakiety przechwyceń i obliczenia odroczone (C++20) . . . 128
8.19 Lambdy w obliczeniach stałych . . . 128
8.20 Koncepcje dla obiektów wywoływalnych . . . 129
8.21 Typowe błędy i trudniejsze przypadki . . . 129

8.22 Zestawienie: kiedy stosować którą konstrukcję? . . . 131

9 Wymazywanie typów i typy sumaryczne 132

9.1 Problem: jak reprezentować różne rodzaje wartości? . . . 132
9.2 `std::optional<T>`: wartość, której może nie być . . . 133
9.3 `std::variant<Ts...>`: zamknięta alternatywa typów . . . 134
9.4 Przykład projektowy: interpreter prostych poleceń . . . 137
9.5 `std::any`: wartość o nieznanym typie . . . 138
9.6 `std::function`: wymazywanie typu obiektu wywoływalnego . . . 139
9.7 Własny mechanizm type erasure: `Printable` . . . 141
9.8 Type erasure a dziedziczenie i polimorfizm statyczny . . . 143
9.9 Wydajność: układ pamięci i koszty dynamicznej dyspozycji . . . 144
9.10 Granice bezpieczeństwa: wyjątki, czas życia i współbieżność . . . 145
9.11 Nowości po C++20 i świadomy wybór standardu . . . 145
9.12 Zasady projektowania i typowe błędy . . . 146
Podsumowanie rozdziału 14 . . . 146
Dokumentacja i literatura rozdziału 14 . . . 146

10 Variadic templates w C++ 147

10.1 Idea i zastosowania . . . 147
10.2 Trzy kropki: deklaracja pakietu a rozwinięcie . . . 147
10.3 Pierwsze praktyczne funkcje . . . 148
10.4 Rekurencyjne przetwarzanie pakietu: metoda C++11 . . . 149
10.5 Fold expressions (C++17) . . . 150
10.6 Operator przecinka i kolejność wykonywania . . . 151
10.7 Pakiety typów i metaprogramowanie . . . 152
10.8 Pakiety wartości i parametry nietypowe . . . 153
10.9 Przekazywanie doskonałe (*perfect forwarding*) . . . 153
10.10 Tworzenie obiektów z dowolnych argumentów . . . 155
10.11 `std::tuple`, `std::apply` i `std::index_sequence` . . . 156
10.12 Dziedziczenie wariadyczne i `std::visit` . . . 158
10.13 Ograniczanie pakietów za pomocą `concepts` (C++20) . . . 158
10.14 Ograniczenia, pułapki i diagnostyka . . . 159
10.15 Variadic templates a funkcje wariadyczne w stylu C . . . 160
10.16 Pełny przykład: prosty rejestrator zdarzeń . . . 161
10.17 Zestawienie: kiedy używać poszczególnych mechanizmów? . . . 162
10.18 Podsumowanie i zadania do samodzielnego sprawdzenia . . . 162

11 Metaprogramowanie w C++ 163

11.1 Czym jest metaprogramowanie? . . . 163

11.2 Instancjowanie szablonów, parametry i zależne nazwy . . . 164
11.3 Metafunkcje i `std::integral_constant` . . . 165
11.4 Specjalizacje: pełne i częściowe . . . 166
11.5 `type_traits`: pytania o typy . . . 167
11.6 Metafunkcje zwracające typy i leniwy wybór . . . 168
11.7 `decltype`, `declval` i konteksty bez obliczania . . . 169
11.8 SFINAE: błędne podstawienie to nie zawsze błąd kompilacji . . . 170
11.9 `if constexpr`: wybór implementacji na podstawie typu . . . 171
11.10 Pakiety typów, sekwencje indeksów i fold expressions . . . 171
11.11 `constexpr`, `consteval` i obliczenia na danych . . . 172
11.12 CRTP: polimorfizm statyczny . . . 173
11.13 Policy-based design: wybór zachowania przez typ . . . 174
11.14 Kompletny przykład: dobór typu akumulatora . . . 175
11.15 Czas kompilacji, diagnostyka i praktyczne ograniczenia . . . 175
11.16 Jaką technikę wybrać? . . . 176
11.17 Typowe błędy i zasady projektowe . . . 176

12 Concepts i requires w C++ 178

12.1 Po co wprowadzono koncepcje? . . . 178
12.2 Pierwsza koncepcja: składnia i zastosowanie . . . 179
12.3 Dwie role słowa `requires` . . . 179
12.4 Cztery rodzaje wymagań w `requires-expression` . . . 180
12.5 Standardowe koncepcje z biblioteki C++20 . . . 182
12.6 Łączenie koncepcji i definiowanie własnych kontraktów . . . 183
12.7 Koncepcje z wieloma parametrami . . . 184
12.8 Przeciążanie funkcji i wybór bardziej szczegółowego ograniczenia . . . 184
12.9 Koncepcje dla szablonów klas i ich specjalizacji . . . 185
12.10 Koncepcje i pakiety parametrów: kontynuacja rozdziału 4 . . . 186
12.11 Concepts a `if constexpr`, `static_assert` i SFINAE . . . 187
12.12 Praktyczny przykład: średnia z zakresu liczb . . . 188
12.13 Koncepcje dla lambd i algorytmów . . . 189
12.14 Granice gwarancji: najczęstsze pułapki . . . 190
12.15 Zasady projektowania: kiedy używać, a kiedy nie? . . . 191
12.16 Podsumowanie i pytania kontrolne . . . 191

13 Iteratory, Ranges i Views 193

13.1 Dlaczego wprowadzono Ranges? . . . 193
13.2 Iteratory: uogólnienie wskaźników . . . 194
13.3 Kategorie iteratorów i ich wymagania . . . 195

13.4 Własny iterator i własny zakres . . . 195
13.5 Zwykłe *range* i *sentinel* . . . 196
13.6 Koncepcje zakresów w praktyce . . . 197
13.7 Algorytmy `std::ranges` i projekcje . . . 197
13.8 Widoki: *view* nie jest synonimem kontenera . . . 198
13.9 Leniwe potoki `filter`, `transform`, `take` . . . 199
13.10 Pozostałe przydatne adaptory C++20 . . . 199
13.11 Materializacja wyniku i koszt przetwarzania . . . 200
13.12 Własność danych, czas życia i `borrowed_range` . . . 201
13.13 `std::span`, `std::string_view` i `subrange` . . . 202
13.14 Własny widok za pomocą `view_interface` . . . 203
13.15 Różnica między `transform` i `views::transform` . . . 204
13.16 Projektowanie API z wykorzystaniem Ranges . . . 204
13.17 Najczęstsze błędy i praktyczne reguły . . . 205
13.18 Której konstrukcji użyć? . . . 205

14 Współbieżność i asynchroniczność 207

14.1 Motywacja i podstawowe pojęcia . . . 207
14.2 Uruchamianie wątków: `std::thread` . . . 208
14.3 Wyścig danych, warunek wyścigu i relacja *happens-before* . . . 208
14.4 Ochrona danych: `mutex` i blokady RAII . . . 209
14.5 Oczekiwanie na stan: `condition_variable` . . . 210
14.6 Operacje atomowe: `std::atomic` . . . 212
14.7 Model pamięci C++: formalne podstawy synchronizacji . . . 214
14.8 Wynik przyszłego obliczenia: `future` i `promise` . . . 218
14.9 Uruchamianie zadań: `std::async` . . . 220
14.10 `std::packaged_task`: funkcja z wynikiem future . . . 222
14.11 `std::jthread` i kooperacyjne zatrzymywanie (C++20) . . . 223
14.12 Inne przydatne mechanizmy biblioteki standardowej . . . 223
14.13 Zadania, pule wątków i planowanie pracy . . . 225
14.14 Skalowanie i koszt współbieżności . . . 228
14.15 Przykład numeryczny: równoległe sumowanie . . . 229
14.16 OpenMP: programowanie równoległe przez dyrektywy . . . 229
14.17 Porównanie i zasady projektowania . . . 238
14.18 Kompilacja, sprawdzanie i diagnostyka . . . 239
Podsumowanie rozdziału . . . 239
Literatura i dokumentacja . . . 240

15 Korutyny, zadania asynchroniczne i `std::async` 241

15.1 Po co C++ potrzebuje korutyn? . . . 241
15.2 Różnica między zawieszeniem a zablokowaniem wątku . . . 242
15.3 Ramka korutyny i jej cykl życia . . . 242
15.4 Uchwyt `std::coroutine_handle` i własność ramki . . . 243
15.5 Własny generator: `co_yield` od podstaw . . . 244
15.6 Protokół `co_await`: `await_ready`, `await_suspend`, `await_resume` . . . 245
15.7 Własne `Task<T>`: leniwe obliczenie a asynchroniczność . . . 247
15.8 Korutyna a `std::async`: fundamentalne porównanie . . . 248
15.9 Rzeczywista asynchroniczność: korutyna i własny executor . . . 250
15.10 Obsługa wyjątków, anulowanie i czas życia danych . . . 252
15.11 Pułapki synchronizacji i planowania . . . 253
15.12 Koszty, optymalizacje i pomiary . . . 254
15.13 Związek z OpenMP, `thread` i `future` . . . 254
15.14 Najczęstsze błędy projektowe . . . 255
Podsumowanie rozdziału 15 . . . 255
Literatura i dokumentacja rozdziału 15 . . . 256

16 Zaawansowana współbieżność, model pamięci i algorytmy bez blokad 257
16.1 Cel rozdziału i powiązanie z podstawami . . . 257
16.2 Formalny porządek zdarzeń w wielowątkowym programie . . . 257
16.3 Testy modelu pamięci: wyniki, które zaskakują . . . 258
16.4 Porządki atomowe i ich dokładne zastosowania . . . 260
16.5 CAS: porównaj i zamień, pętle ponawiania i porządek niepowodzenia . . . 261
16.6 Bariery pamięci i `std::atomic_thread_fence` . . . 262
16.7 Gwarancje postępu: *blocking*, *lock-free* i *wait-free* . . . 263
16.8 Problem ABA: poprawne CAS nie wystarcza . . . 264
16.9 Kolejka SPSC: praktyczny algorytm bez mutexa . . . 265
16.10 Atomowa publikacja niezmiennych wersji danych . . . 267
16.11 Planowanie zadań: pule wątków, *work stealing* i *backpressure* . . . 268
16.12 Anulowanie, sygnały stopu i punkty oczekiwania . . . 270
16.13 Fałszywe współdzielenie (*false sharing*) i cache coherence . . . 271
16.14 Linearizowalność i niezmienniki współdzielonych struktur . . . 272
16.15 Współbieżność a parallel STL, Ranges i OpenMP . . . 273
16.16 Diagnostyka: kompilatory, sanitizatory, testy przeciążeniowe . . . 274
16.17 Typowe błędy i zalecenia projektowe . . . 275
Podsumowanie rozdziału . . . 276
Literatura i dokumentacja . . . 276

17 Kompilacja, linkowanie, ABI i moduły C++20 277

17.1 Dlaczego trzeba rozumieć proces budowania? 277
17.2 Od pliku źródłowego do programu 277
17.3 Nagłówki, makra i zasada jednej definicji 279
17.4 Symbole, łączność i typowe błędy linkera 282
17.5 Biblioteki statyczne 283
17.6 Biblioteki współdzielone i dynamiczne ładowanie 284
17.7 ABI — Application Binary Interface 286
17.8 System budowania: CMake i projekt wieloplikowy 288
17.9 Moduły C++20: nowy mechanizm organizacji kodu 290
17.10 Optymalizacja procesu budowania 293
17.11 Praktyczny przewodnik diagnostyczny 294
Podsumowanie rozdziału 295
Literatura i dokumentacja 295

Rozdział 1

Kontenery STL i algorytmy

1.1 Cel i zakres rozdziału

Biblioteka standardowa C++ udostępnia gotowe struktury danych, których wybór ma istotne konsekwencje dla złożoności operacji, lokalności pamięci, bezpieczeństwa wyjątkowego i poprawności iteratorów. Samo rozróżnienie między `std::vector`, `std::list` i `std::map` nie wystarcza: trzeba rozumieć również *własność elementów*, *unieważnianie iteratorów*, wymagania dotyczące porządku i haszowania oraz relację między kontenerami a algorytmami. W dalszej części domyślnie używamy C++20.

Główna zasada projektowa. Najpierw ustalamy rzeczywiste operacje: czy potrzebujemy dostępu przez indeks, wyszukiwania po kluczu, zachowania kolejności, stabilnych adresów, czy częstych wstawek i usunięć. Dopiero potem wybieramy kontener. Prosty `std::vector` pozostaje często najlepszym rozwiązaniem, nawet gdy jego teoretyczna złożoność niektórych operacji jest gorsza, ponieważ dane leżą w sąsiadujących komórkach pamięci.

1.2 Klasyfikacja kontenerów i podstawowe pojęcia

Biblioteka standardowa rozróżnia m.in. kontenery sekwencyjne (`array`, `vector`, `deque`, `list`, `forward_list`), uporządkowane asocjacyjne (`map`, `set` i ich warianty `multi`) oraz nieuporządkowane asocjacyjne (`unordered_map`, `unordered_set` i warianty `multi`). `stack`, `queue` i `priority_queue` są *adapterami kontenerów*: oferują inny interfejs dostępu do wewnętrznej struktury danych. `std::string` jest wyspecjalizowaną strukturą sekwencyjną, ale semantykę tekstu omawiamy oddzielnie.

Kontener	Dostęp	Wstawienie / usunięcie	Typowe zastosowanie
`vector`	Indeks $O(1)$	Koniec: zamortyzowane $O(1)$; środek: $O(n)$	Dane zwarte, iteracja, obliczenia
`deque`	Indeks $O(1)$	Oba końce: $O(1)$; środek: $O(n)$	Kolejka dwustronna
`list`	Iteracja $O(n)$	$O(1)$ przy znanym iteratorze	Stabilne węzły, `splice`
`forward_list`	Iteracja $O(n)$	$O(1)$ za poprzednikiem	Lista jednokierunkowa
`map` / `set`	Szukanie $O(\log n)$	$O(\log n)$	Klucze w ustalonej kolejności
`unordered_map`	Średnio $O(1)$	Średnio $O(1)$	Słownik, szybkie wyszukiwanie

W tabeli n oznacza liczbę elementów, a koszty zależą od wymagań poszczególnych operacji.

`unordered_map` może w najgorszym przypadku wykonywać operacje w czasie $O(n)$ (np. przy wielu kolizjach). Sformułowanie „*zamortyzowane* $O(1)$" oznacza, że pojedyncza operacja może być kosztowna, ale średni koszt w długiej sekwencji operacji jest ograniczony stałą.

Wartości, wskaźniki, referencje. Kontenery zwykle przechowują *własne obiekty elementów*. `std::vector<T>` nie jest kontenerem referencji do obiektów `T`; może za to przechowywać np. `std::reference_wrapper<T>`, `T*` lub `std::unique_ptr<T>`. Wybór wpływa na własność obiektów, kopiowanie, czas życia i poprawność dostępu.

1.3 `std::vector`: domyślny kontener sekwencyjny

`std::vector<T>` przechowuje elementy *ciągle* w pamięci, co pozwala przekazywać je do interfejsów oczekujących tablicy (np. przez `data()` lub `std::span`). Obiekt wektora zarządza własną pamięcią i niszczy elementy po zakończeniu swojego czasu życia.

```
#include <cassert>
#include <vector>

int main() {
    std::vector<int> v{4, 8, 15};
    v.push_back(16);           // kopiowanie lub przenoszenie argumentu
    v.emplace_back(23);        // konstruowanie nowego elementu

    assert(v.size() == 5);
    assert(v[0] == 4);
    assert(v.at(4) == 23);
    assert(v.front() == 4 && v.back() == 23);

    v.pop_back();
    assert(v.size() == 4);
    v.clear();
    assert(v.empty());
}
```

Listing 1.1: Podstawowe operacje na vector (C++11); kompletny program.

`operator[]` *nie sprawdza* granic: użycie indeksu spoza zakresu prowadzi do niezdefiniowanego zachowania. Metoda `at()` sprawdza zakres i zgłasza `std::out_of_range`. Metody `front()`, `back()` i `pop_back()` wymagają niepustego kontenera.

1.3.1 `size`, `capacity`, `reserve` i `resize`

`size()` oznacza liczbę *istniejących* elementów, a `capacity()` liczbę elementów, dla których przydzielono miejsce bez konieczności realokacji. `reserve(k)` zmienia rezerwację pamięci, nie tworzy elementów; `resize(k)` zmienia ich liczbę.

```
#include <cassert>
#include <vector>

int main() {
    std::vector<int> data;
    data.reserve(100);
    assert(data.size() == 0);
    assert(data.capacity() >= 100);

    data.push_back(7);
    data.resize(4);          // dopisuje trzy elementy o wartosci 0
```

```
    assert(data.size() == 4);
    assert(data[0] == 7 && data[3] == 0);
}
```

Listing 1.2: Rezerwacja pamięci nie zwiększa rozmiaru (C++11); kompletny program.

Nie należy wywoływać `reserve(size()+1)` przed każdym `push_back()`: wymuszenie niemal liniowego wzrostu pojemności może zniweczyć politykę geometrycznego powiększania bufora i pogorszyć wydajność. `shrink_to_fit()` jest tylko żądaniem, a nie gwarancją zmniejszenia pojemności.

1.3.2 Realokacja i unieważnianie iteratorów

Po realokacji `vector` zmienia adresy wszystkich elementów: tracą ważność iteratory, referencje i wskaźniki do elementów, również poprzednia wartość `data()`. Bez realokacji `push_back()` unieważnia iterator `end()`, lecz nie unieważnia iteratorów i referencji do wcześniej istniejących elementów. Wstawienie w środku bez realokacji unieważnia iteratory i referencje do pozycji wstawienia oraz wszystkich późniejszych pozycji (włącznie z dawnym `end()`).

```
#include <cassert>
#include <vector>

int main() {
    std::vector<int> v{1, 3, 4};
    auto pos = v.begin() + 1;
    pos = v.insert(pos, 2);   // przyjmuje zwrocony iterator
    assert(*pos == 2);
    assert((v == std::vector<int>{1, 2, 3, 4}));
}
```

Listing 1.3: Bezpieczne ponowne pobranie iteratora po wstawieniu; kompletny program.

Przenoszenie elementów i bezpieczeństwo wyjątkowe. Podczas realokacji `vector` może przenosić lub kopiować elementy. Dla pewnych typów i operacji warunki silnej gwarancji wyjątkowej zależą od tego, czy konstruktor przenoszący jest `noexcept` oraz czy dostępne jest kopiowanie. Zagadnienie łączy się bezpośrednio z semantyką przenoszenia omówioną w rozdziale 6.

1.3.3 `push_back` kontra `emplace_back`

`emplace_back(args...)` przekazuje argumenty bezpośrednio do konstruktora nowego elementu. Nie oznacza to jednak automatycznie „zawsze szybciej”: jeśli mamy już obiekt, `push_back(std::move(x))` bywa równie naturalne; przy realokacji nadal konieczne może być przeniesienie innych elementów. Obie funkcje mogą zgłaszać wyjątki.

```
#include <cassert>
#include <string>
#include <utility>
#include <vector>

struct Person {
    std::string name;
    int age;
    Person(std::string n, int a) : name(std::move(n)), age(a) {}
};

int main() {
    std::vector<Person> people;
    people.emplace_back("Ada", 31);
```

```
    people.push_back(Person{"Alan", 42});
    assert(people[1].age == 42);
}
```

Listing 1.4: Konstruowanie elementu z argumentow (C++17); kompletny program.

1.4 `std::array`, `std::deque` i listy

1.4.1 `std::array<T,N>`: rozmiar znany w kompilacji

`std::array` przechowuje dokładnie N elementów. Nie dokonuje osobnej dynamicznej alokacji dla swojej zawartości; pamięć elementów stanowi część samego obiektu `array`. Można używać iteratorów, `size()`, `data()` i algorytmów, analogicznie do wektora.

```
#include <algorithm>
#include <array>
#include <cassert>
#include <tuple>

int main() {
    std::array<int, 4> x{5, 1, 4, 2};
    std::sort(x.begin(), x.end());
    assert((x == std::array<int, 4>{1, 2, 4, 5}));
    static_assert(std::tuple_size_v<decltype(x)> == 4);
}
```

Listing 1.5: std::array i algorytmy standardowe (C++11); kompletny program.

1.4.2 `std::deque`: dwa tanie końce

`deque` udostępnia dostęp indeksowany w czasie $O(1)$, lecz zwykle nie jest jednym ciągłym blokiem pamięci. Wstawianie i usuwanie na początku i końcu ma stałą złożoność; w przeciwieństwie do `vector` ma operacje `push_front()` i `pop_front()`. `deque` nie udostępnia `data()` dla wszystkich elementów.

```
#include <cassert>
#include <deque>

int main() {
    std::deque<int> q{2, 3};
    q.push_front(1);
    q.push_back(4);
    assert(q[0] == 1 && q[3] == 4);
    q.pop_front();
    assert(q.front() == 2);
}
```

Listing 1.6: Kolejka dwustronna (C++11); kompletny program.

Wstawienie na końcu `deque` może unieważnić *iteratory*, ale zachowuje referencje do istniejących elementów. Szczegółowe reguły różnią się dla wstawień i usunięć w środku oraz na końcach; nie należy mechanicznie przenosić reguł `vector` na `deque`.

1.4.3 `std::list` i `std::forward_list`

`list` jest listą dwukierunkową. Wstawianie i usuwanie przy znanym iteratorze odbywa się w stałym czasie; iteratory i referencje do pozostałych elementów pozostają ważne. Wyszukanie elementu przez wartość wymaga jednak przejścia listy $O(n)$, podobnie jak uzyskanie elementu o zadanym indeksie. W przeciwieństwie do `vector`, `list` nie oferuje operatora `[]` ani iteratorów dostępu swobodnego. Do sortowania używamy `list::sort()`, a nie `std::sort()`.

```
#include <cassert>
#include <iterator>
#include <list>

int main() {
    std::list<int> source{1, 2, 3};
    std::list<int> target{10};
    auto it = std::next(source.begin()); // wskazuje 2

    target.splice(target.end(), source, it);
    assert(*it == 2); // iterator pozostaje wazny
    assert(source.size() == 2 && target.size() == 2);
    target.sort();
    assert(target.front() == 2);
}
```

Listing 1.7: Przeniesienie wezla przez list::splice (C++11); kompletny program.

`forward_list` przechowuje węzły listy jednokierunkowej; wstawianie i usuwanie wykonuje *za* wskazanym poprzednikiem (`insert_after`, `erase_after`), a `before_begin()` udostępnia pozycję przed pierwszym elementem. Kontener nie posiada metody `size()` (rozmiar można policzyć liniowo przez `std::distance`).

Kiedy lista jest rzeczywiście potrzebna? Lista bywa przydatna, gdy musimy często *przenosić węzły bez kopiowania obiektów* i wymagamy stabilnych iteratorów. Dla częstej iteracji i obliczeń numerycznych `vector` z reguły zapewnia lepszą lokalność pamięci. Wybór należy jednak potwierdzić pomiarem.

1.5 Kontenery uporządkowane: `set` i `map`

`std::set<Key>` przechowuje unikalne klucze, a `std::map<Key,T>` unikalne pary klucz–wartość. Elementy są iterowane w kolejności zdefiniowanej przez komparator; wyszukiwanie, wstawianie i usuwanie są logarytmiczne. Warianty `multiset`/`multimap` dopuszczają powtarzające się klucze.

1.5.1 `map::operator[]`, `at` i `find`

Odwołanie `m[key]` do nieistniejącego klucza *wstawia* nowy element z wartością inicjalizowaną domyślnie; może zatem modyfikować strukturę danych. `at()` zgłasza wyjątek, jeżeli klucz nie istnieje. `find()` zwraca iterator, a `contains()` (C++20) zwraca wartość logiczną bez wstawiania.

```
#include <cassert>
#include <map>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> words{"kot", "pies", "kot"};
    std::map<std::string, int> count;
```

```
    for (const auto& word : words) ++count[word];

    assert(count.at("kot") == 2);
    assert(count.contains("pies"));
    assert(!count.contains("lis"));
    assert(count.find("lis") == count.end());
}
```

Listing 1.8: Liczenie czestosci slow z map (C++20); kompletny program.

1.5.2 `insert`, `try_emplace`, `insert_or_assign`

`insert` podejmuje próbę dodania gotowej pary. `try_emplace(key,args...)` (C++17) konstruuje wartość w miejscu z przekazanych argumentów tylko wtedy, gdy klucz nie występuje. Ważne zastrzeżenie: *wyrażenia będące argumentami wywołania* są obliczane jeszcze przed wejściem do `try_emplace`; nie należy więc podawać niepotrzebnie kosztownego wyrażenia tworzącego tymczasowy obiekt. `insert_or_assign` dodaje lub zastępuje wartość.

```
#include <cassert>
#include <map>
#include <memory>
#include <string>

struct Heavy {
    static inline int constructions = 0; // C++17
    int value;
    explicit Heavy(int x) : value(x) { ++constructions; }
};

int main() {
    std::map<std::string, Heavy> m;
    auto [it, added] = m.try_emplace("a", 3);
    assert(added && it->second.value == 3);

    auto [again, added_again] = m.try_emplace("a", 4);
    assert(!added_again && again->second.value == 3);
    assert(Heavy::constructions == 1);

    m.insert_or_assign("a", Heavy{9});
    assert(m.at("a").value == 9);

    std::map<int, std::unique_ptr<int>> owned;
    owned.try_emplace(1, std::make_unique<int>(10));
    assert(*owned.at(1) == 10);
}
```

Listing 1.9: Rozne operacje modyfikacji map (C++17); kompletny program.

1.5.3 Własny komparator i ścisły słaby porządek

Porównanie musi spełniać warunek *strict weak ordering*; w szczególności komparator powinien być irrefleksyjny ($\neg\,\mathrm{comp}(x,x)$) i przechodni. Jeżeli dwie wartości a, b nie spełniają ani `comp(a,b)`, ani `comp(b,a)`, kontener uznaje je za równoważne jako klucze — nawet gdy `operator==` zwraca `false`. Wymagania tego porządku są istotne również dla `sort` i `lower_bound`.

```
#include <cassert>
#include <cctype>
#include <map>
#include <string>
```

```

struct CaseInsensitiveLess {
    bool operator()(const std::string& a,
                    const std::string& b) const {
        auto ai = a.begin();
        auto bi = b.begin();
        while (ai != a.end() && bi != b.end()) {
            auto x = static_cast<unsigned char>(*ai++);
            auto y = static_cast<unsigned char>(*bi++);
            int lx = std::tolower(x);
            int ly = std::tolower(y);
            if (lx != ly) return lx < ly;
        }
        return a.size() < b.size();
    }
};

int main() {
    std::map<std::string, int, CaseInsensitiveLess> m;
    m.emplace("ABC", 1);
    m.emplace("abc", 2); // rownowazny klucz
    assert(m.size() == 1 && m.begin()->second == 1);
}
```

Listing 1.10: Mapowanie bez rozrozniania wielkosci liter: prosty komparator; kompletny program.

Przykład porównuje bajty w podstawowym środowisku znakowym; `std::tolower` z bieżącą lokalizacją C nie realizuje ogólnego Unicode case folding. Do pełnego porównywania napisów w Unicode trzeba zastosować osobne, świadomie wybrane reguły normalizacji i kolacji.

1.5.4 Wyszukiwanie heterogeniczne

Jeżeli komparator jest *transparentny* (np. `std::less<>`), `std::map<std::string,T,std::less<>>` umożliwia wyszukiwanie przez `std::string_view` bez konieczności tworzenia tymczasowego `std::string`. Heterogeniczne przeciążenia `find` są dostępne od C++14, a `contains` od C++20.

```
#include <cassert>
#include <functional>
#include <map>
#include <string>
#include <string_view>

int main() {
    std::map<std::string, int, std::less<>> m{{"alpha", 3}};
    std::string_view key = "alpha";
    auto it = m.find(key);
    assert(it != m.end() && it->second == 3);
    assert(m.contains(key));
}
```

Listing 1.11: Heterogeniczne wyszukiwanie w map (C++20); kompletny program.

1.5.5 Zmiana klucza przez uchwyt węzła (C++17)

Kluczy w `map` nie wolno modyfikować bezpośrednio przez iterator. Uchwyt `node_handle` uzyskany przez `extract()` pozwala wyjąć węzeł, zmienić jego klucz i wstawić z powrotem. Trzeba pamiętać, że ponowne wstawienie może się nie udać z powodu kolizji kluczy.

```
#include <cassert>
#include <map>
```

```cpp
#include <string>

int main() {
    std::map<std::string, int> m{{"old", 7}};
    auto node = m.extract("old");
    assert(!node.empty());
    node.key() = "new";
    auto result = m.insert(std::move(node));
    assert(result.inserted && m.at("new") == 7);
}
```

Listing 1.12: Zmiana klucza przez extract (C++17); kompletny program.

1.6 Kontenery haszujące: `unordered_map` i `unordered_set`

Kontener `unordered_map<Key,T,Hash,Eq>` rozmieszcza elementy w kubełkach. Funkcja `Hash` przekształca klucz w wartość typu `std::size_t`; `Eq` rozstrzyga równość kluczy. Wymagany jest warunek zgodności:

$$\mathrm{Eq}(a, b) \Longrightarrow \mathrm{Hash}(a) = \mathrm{Hash}(b).$$

Warunek odwrotny nie zachodzi: kolizje haszy są dozwolone i rozstrzygane przez porównanie kluczy. Iteracja po kontenerze nie ma stabilnego porządku i nie należy z niego korzystać jako z interfejsu logicznego programu.

```cpp
#include <cassert>
#include <cstddef>
#include <functional>
#include <unordered_map>

struct Point {
    int x, y;
    friend bool operator==(const Point&, const Point&) = default;
};

struct PointHash {
    std::size_t operator()(const Point& p) const noexcept {
        std::size_t h1 = std::hash<int>{}(p.x);
        std::size_t h2 = std::hash<int>{}(p.y);
        return h1 ^ (h2 + std::size_t{0x9e3779b9}
                        + (h1 << 6) + (h1 >> 2));
    }
};

int main() {
    std::unordered_map<Point, int, PointHash> m;
    m[{2, 5}] = 42;
    assert(m.at(Point{2, 5}) == 42);
}
```

Listing 1.13: Wlasny hash i porownanie kluczy (C++20); kompletny program.

Ta funkcja mieszająca służy demonstracji składni, a nie zastosowaniom kryptograficznym. Dla publicznie sterowanych kluczy przy projektowaniu systemów wystawionych na ataki kolizyjne należy rozważyć również odporność na złośliwe dane wejściowe.

1.6.1 `load_factor`, `reserve` i `rehash`

`load_factor()` to stosunek liczby elementów do liczby kubełków. `max_load_factor()` podaje lub ustawia pożądany maksymalny współczynnik. `reserve(n)` żąda przygotowania na n elementów, a

`rehash(k)` na przynajmniej k kubełków, z uwzględnieniem limitów wynikających z współczynnika wypełnienia.

```cpp
#include <cassert>
#include <unordered_set>

int main() {
    std::unordered_set<int> seen;
    seen.reserve(1000);
    for (int i = 0; i < 1000; ++i) seen.insert(i);
    assert(seen.size() == 1000);
    assert(seen.contains(17));
}
```

Listing 1.14: Rezerwowanie kubelek przed licznymi wstawkami (C++11); kompletny program.

Rehashing unieważnia iteratory, lecz *nie unieważnia* wskaźników i referencji do elementów. Wstawienie może unieważnić iteratory, jeśli wywoła rehash. Usunięcie unieważnia iteratory i referencje do usuniętych elementów. Nie wolno modyfikować kluczy w sposób zaburzający ich hash lub równość podczas przebywania elementów w kontenerze.

1.7 Adaptery: `stack`, `queue`, `priority_queue`

`stack` udostępnia schemat LIFO, `queue` — FIFO, a `priority_queue` — dostęp do elementu o najwyższym priorytecie według komparatora (domyślnie do największego). Adaptery zwykle nie zapewniają bezpośredniej iteracji po elementach.

```cpp
#include <cassert>
#include <functional>
#include <queue>
#include <stack>
#include <vector>

int main() {
    std::stack<int> s;
    s.push(1); s.push(2);
    assert(s.top() == 2);
    s.pop();

    std::queue<int> q;
    q.push(1); q.push(2);
    assert(q.front() == 1);
    q.pop();

    std::priority_queue<int, std::vector<int>, std::greater<int>> pq;
    pq.push(7); pq.push(2); pq.push(5);
    assert(pq.top() == 2); // kopiec minimum
}
```

Listing 1.15: Trzy adaptery standardowe (C++11); kompletny program.

`priority_queue` zwykle opiera się na kopcu: `top()` ma koszt $O(1)$, a `push()` i `pop()` $O(\log n)$. Nie jest to kolejka FIFO ani struktura z aktualizacją priorytetu dowolnego elementu przez gotowy publiczny interfejs.

1.8 Iteratory, stabilność adresów i unieważnianie

Iterator nie jest ogólnie tożsamy ze wskaźnikiem. Może przechowywać stan niezbędny do poruszania się po drzewie lub kubełkach. Po operacji modyfikującej trzeba wiedzieć, czy iterator, referencja i wskaźnik do elementu pozostają ważne. Typowe reguły podsumowano w tabeli.

Kontener	Wstawienie	Usunięcie
`vector`	Przy realokacji: wszystkie iteratory i referencje; bez realokacji: pozycja wstawienia i dalej	Pozycja usunięcia i dalej (także dawny `end()`)
`deque`	Na końcach: iteratory tracą ważność, referencje do elementów zwykle pozostają; w środku: także referencje	Zależy od pozycji; usunięcie w środku unieważnia wszystkie iteratory i referencje
`list`	Istniejące iteratory i referencje zachowane	Tylko usunięte elementy
`map`, `set`	Istniejące iteratory i referencje zachowane	Tylko usunięte elementy
`unordered_*`	Iteratory mogą utracić ważność przy rehash; referencje zachowane	Tylko usunięte elementy

Dla `deque` szczególnie ostrożnie trzeba traktować `end()`, którego ważność zależy od rodzaju usunięcia. W tabeli podano reguły najważniejsze dla praktyki, a nie kompletną specyfikację każdej przeciążonej operacji. Po operacjach zmieniających strukturę bezpieczną praktyką jest korzystanie z iteratora zwróconego przez metodę (np. `erase`, `insert`).

1.8.1 Bezpieczne usuwanie podczas iteracji

```cpp
#include <cassert>
#include <map>

int main() {
    std::map<int, int> m{{1, 2}, {2, 3}, {3, 4}};
    for (auto it = m.begin(); it != m.end();) {
        if (it->second % 2 == 0) it = m.erase(it);
        else ++it;
    }
    assert(m.size() == 1 && m.contains(2));
}
```

Listing 1.16: Usuwanie elementow map podczas iteracji (C++11); kompletny program.

Szczególny przypadek `vector<bool>`. `std::vector<bool>` jest specjalizacją oszczędzającą pamięć, która może reprezentować wartości w postaci bitów. Jej `operator[]` nie musi zwracać `bool&`, tylko specjalny obiekt proxy. Nie należy oczekiwać normalnej tablicy obiektów `bool`, ciągłego przechowywania pojedynczych `bool` ani bezpiecznej modyfikacji odrębnych bitów z różnych wątków bez synchronizacji.

1.9 Algorytmy standardowe i wymagania wobec iteratorów

Algorytmy oddzielają *operację* od *reprezentacji danych*. Wersje klasyczne przyjmują iteratory, a w C++20 wiele algorytmów w `std::ranges` można wywołać bezpośrednio na zakresie. Szczegóły kategorii iteratorów i widoków omówiliśmy w rozdziale 8; tutaj skupiamy się na doborze algorytmu i jego kontrakcie.

1.9.1 Wyszukiwanie, transformacje i agregacje

Do częstych operacji należą `find`, `find_if`, `count_if`, `transform`, `accumulate`, `all_of`, `any_of`, `none_of` i `for_each`. Algorytmy numeryczne znajdują się w nagłówku `<numeric>`, pozostałe głównie w `<algorithm>`.

```
#include <algorithm>
#include <cassert>
#include <numeric>
#include <vector>

int main() {
    std::vector<int> values{1, 2, 3, 4, 5};
    auto even = [](int x) { return x % 2 == 0; };

    assert(std::ranges::count_if(values, even) == 2);
    assert(std::ranges::all_of(values,
        [](int x) { return x > 0; }));
    assert(std::accumulate(values.begin(), values.end(), 0) == 15);

    std::vector<int> squares(values.size());
    std::ranges::transform(values, squares.begin(),
                           [](int x) { return x * x; });
    assert(squares[2] == 9);
}
```

Listing 1.17: Algorytmy klasyczne i Ranges (C++20); kompletny program.

1.9.2 Sortowanie i stabilność kolejności

`std::sort` wymaga iteratorów dostępu swobodnego i ma złożoność $O(n \log n)$ w najgorszym przypadku (w liczbie porównań). `stable_sort` zachowuje kolejność elementów, które komparator uznaje za równoważne, zwykle kosztem dodatkowych zasobów; jego szczegółowe gwarancje złożoności zależą od dostępnej pamięci. `nth_element` umieszcza wybrany element na pozycji, którą zajmowałby po sortowaniu, bez sortowania całej sekwencji; przeciętna liczba porównań jest liniowa.

```
#include <algorithm>
#include <cassert>
#include <string>
#include <vector>

struct Record {
    std::string name;
    int group;
};

int main() {
    std::vector<Record> v{{"A", 2}, {"B", 1}, {"C", 2}};
    std::ranges::stable_sort(v, {}, &Record::group);
    assert(v[0].name == "B");
    assert(v[1].name == "A" && v[2].name == "C");
}
```

Listing 1.18: Stabilne sortowanie rekordow po kluczu (C++20); kompletny program.

1.9.3 Wyszukiwanie binarne i porządek wejścia

`binary_search`, `lower_bound`, `upper_bound` i `equal_range` wymagają odpowiedniego uporządkowania (lub partycjonowania względem danego klucza) zakresu. Zastosowanie do nieuporządkowanych

danych daje wynik niespełniający kontraktu algorytmu. W wersjach na iteratorach nie-losowego dostępu liczba porównań może pozostać logarytmiczna, lecz przesuwanie iteratorów może być liniowe.

```cpp
#include <algorithm>
#include <cassert>
#include <vector>

int main() {
    std::vector<int> sorted{1, 3, 3, 7};
    auto it = std::lower_bound(sorted.begin(), sorted.end(), 3);
    assert(it == sorted.begin() + 1);

    auto upper = std::upper_bound(sorted.begin(), sorted.end(), 3);
    assert(upper == sorted.begin() + 3);
    assert(std::binary_search(sorted.begin(), sorted.end(), 7));
}
```

Listing 1.19: Wyszukiwanie miejsca wstawienia w posortowanym vector (C++11); kompletny program.

1.9.4 Idiom *erase–remove* i `std::erase_if`

`std::remove` nie usuwa elementów *z kontenera*: przestawia elementy pozostające i zwraca logiczny nowy koniec. Rozmiar kontenera zmniejsza dopiero wywołanie `erase`. Od C++20 wygodniej używać `std::erase` i `std::erase_if` dla wspieranych kontenerów.

```cpp
#include <algorithm>
#include <cassert>
#include <vector>

int main() {
    std::vector<int> a{1, 2, 3, 4, 5};
    auto last = std::remove_if(a.begin(), a.end(),
                               [](int x) { return x % 2 == 0; });
    a.erase(last, a.end());
    assert((a == std::vector<int>{1, 3, 5}));

    std::vector<int> b{1, 2, 3, 4, 5};
    auto removed = std::erase_if(b,
                                 [](int x) { return x % 2 == 0; });
    assert(removed == 2 && b == a);
}
```

Listing 1.20: Dwa sposoby usuwania z vector (C++20); kompletny program.

Dla kontenerów asocjacyjnych nie wolno stosować klasycznego `remove_if` tak jak dla wektora: ich elementy mają klucze, których pozycja jest regulowana przez strukturę danych. Stosujemy `erase` z iteratorem lub `std::erase_if` (C++20).

1.9.5 Algorytmy kopca i `std::priority_queue`

`make_heap`, `push_heap`, `pop_heap` i `sort_heap` pozwalają użyć własnego wektora jako kopca. `make_heap` działa w czasie liniowym, a operacje dopisania i usunięcia maksimum w $O(\log n)$. Są to operacje inne niż pełne sortowanie.

1.10 Przypadki projektowe: dobór kontenera do problemu

1.10.1 Histogram, deduplikacja i lista wyników

Załóżmy, że chcemy zebrać wartości, usunąć duplikaty i otrzymać wynik w kolejności rosnącej. Możemy użyć `std::set`, albo `vector` z `sort` i `unique`. Drugi sposób bywa korzystniejszy w przetwarzaniu wsadowym, ponieważ `vector` przechowuje dane ciągle i ogranicza liczbę alokacji węzłów.

```
#include <algorithm>
#include <cassert>
#include <vector>

int main() {
    std::vector<int> keys{5, 2, 5, 1, 2, 8};
    std::sort(keys.begin(), keys.end());
    keys.erase(std::unique(keys.begin(), keys.end()), keys.end());
    assert((keys == std::vector<int>{1, 2, 5, 8}));
}
```

Listing 1.21: Deduplikacja przez sort i unique (C++11); kompletny program.

1.10.2 Własność elementów w kontenerach polimorficznych

Przechowywanie obiektów klas pochodnych przez wartość w `vector<Base>` może powodować *object slicing*. Jeśli potrzebujemy polimorfizmu dynamicznego, naturalne jest np. `vector<unique_ptr<Base»`; gdy rodzina typów jest zamknięta, można rozważyć `vector<variant<...»` (zob. rozdział 12).

```
#include <cassert>
#include <memory>
#include <vector>

struct Shape {
    virtual ~Shape() = default;
    virtual int area() const = 0;
};
struct Square : Shape {
    int side;
    explicit Square(int x) : side(x) {}
    int area() const override { return side * side; }
};

int main() {
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Square>(3));
    assert(shapes.front()->area() == 9);
}
```

Listing 1.22: Kontener wlasciwych obiektow polimorficznych (C++14); kompletny program.

1.10.3 Kontenery a współbieżność

Samo wybranie kontenera standardowego nie czyni jego operacji modyfikujących bezpiecznymi dla współbieżnego wykonania. Równoczesne modyfikowanie wspólnego `vector` przez niezależne `push_back()` bez synchronizacji jest wyścigiem danych. Można natomiast równolegle odczytywać elementy kontenera, jeśli żaden wątek ich nie modyfikuje, a czas życia kontenera jest zachowany. Dla wielu kontenerów można także modyfikować *różne istniejące* elementy równolegle, jeśli nie dochodzi

do konfliktu pamięci; wyjątek stanowi m.in. specjalizacja `vector<bool>`. Zmiany strukturalne wymagają osobnej analizy i synchronizacji (zob. rozdział 1).

1.11 Złożoność, lokalność pamięci i pomiar wydajności

Teoretyczna złożoność to jeden z elementów oceny. Istotne są również: liczba alokacji, liczba odwołań do rozproszonych adresów, rozmiar elementu, przewidywalność gałęzi i zachowanie cache procesora. Dla wielu zastosowań `vector` z wyszukiwaniem liniowym może być szybszy od struktury drzewiastej przy małej liczbie kluczy. To nie przeczy złożoności asymptotycznej: stałe i charakter obciążenia są częścią kosztu rzeczywistego programu.

1.11.1 Koszt sumowania ciągłych danych

```cpp
#include <chrono>
#include <cstdint>
#include <iostream>
#include <list>
#include <numeric>
#include <vector>

int main() {
    constexpr int N = 200000;
    std::vector<int> v(N, 1);
    std::list<int> l(v.begin(), v.end());

    auto measure = [](const auto& xs) {
        auto begin = std::chrono::steady_clock::now();
        std::int64_t sum = std::accumulate(xs.begin(), xs.end(),
                                           std::int64_t{0});
        auto end = std::chrono::steady_clock::now();
        auto ns = std::chrono::duration_cast<
            std::chrono::nanoseconds>(end - begin).count();
        return std::pair{sum, ns};
    };

    auto [sum_v, time_v] = measure(v);
    auto [sum_l, time_l] = measure(l);
    std::cout << "vector: " << sum_v << ", " << time_v << " ns\n";
    std::cout << "list:   " << sum_l << ", " << time_l << " ns\n";
}
```

Listing 1.23: Prosty szkielet pomiaru vector i list (C++20); kompletny program.

To *ilustracja sposobu pomiaru*, a nie miarodajny benchmark. Jednorazowy wynik bywa zdominowany przez rozgrzanie cache, optymalizacje kompilatora, scheduler i zakłócenia systemowe. Poprawniejszy eksperyment powtarza pomiary, rozdziela przygotowanie danych od mierzonej operacji, uwzględnia pamięć, różne rozmiary danych, medianę i rozrzut pomiarów. Dla wiarygodnych porównań warto użyć np. Google Benchmark i profilera.

1.11.2 Przybliżone reguły wyboru

Wymaganie	**Rozsądny wybór początkowy**
Dużo iteracji, obliczeń, dostęp indeksowany	`vector`; przy stałym rozmiarze `array`
Częste dodawanie i usuwanie z obu końców	`deque`
Utrzymanie kolejności kluczy, wyszukiwanie przedziałowe	`map` / `set`
Wyszukiwanie po kluczu bez wymogu kolejności	`unordered_map` / `unordered_set`
Przenoszenie istniejących węzłów i stabilne iteratory	`list`
FIFO / LIFO	`queue` / `stack`
Wielokrotne pobieranie największego lub najmniejszego	`priority_queue`

1.12 Najczęstsze błędy i dobre praktyki

1. Używanie `vector::operator[]` bez pewności, że indeks jest poprawny; w kodzie granicznym warto stosować `at()`.
2. Traktowanie `reserve` jako tworzenia elementów (mylenie `size` z `capacity`).
3. Korzystanie z iteratorów lub referencji po unieważnieniu przez realokację, `erase` lub `rehash`.
4. Zakładanie, że `emplace_back` zawsze jest szybsze od `push_back`.
5. Używanie `map::operator[]` do samego sprawdzania istnienia klucza, z niezamierzonym wstawieniem nowej wartości.
6. Pisanie komparatorów naruszających *strict weak ordering*; komparator `a <= b` nie jest poprawną relacją porządkującą.
7. Używanie `std::sort` na `std::list` lub `std::lower_bound` na danych bez odpowiedniego porządku.
8. Wnioskowanie o kolejności elementów `unordered_map` na podstawie przypadkowego przebiegu programu.
9. Niepotrzebne stosowanie `list` z przekonaniem, że wstawienie $O(1)$ zawsze wygrywa z `vector`.
10. Wprowadzanie równoczesnych modyfikacji kontenera z wielu wątków bez uzasadnienia braku konfliktu i odpowiedniej synchronizacji.

1.13 Pytania kontrolne

1. Jaka jest różnica między `reserve` a `resize`, i które z nich tworzy nowe elementy?
2. Kiedy `push_back` na `vector` unieważnia wszystkie referencje, a kiedy zachowuje referencje do wcześniejszych elementów?
3. Dlaczego `map::operator[]` może być niepożądany podczas sprawdzania istnienia klucza?
4. Jakie wymagania muszą spełniać `Hash` i `Eq` w `unordered_map`?
5. Dlaczego `std::remove` nie zmienia rozmiaru `vector`?
6. Jak różnią się koszty i kontrakty `sort`, `stable_sort`, `nth_element` i `lower_bound`?
7. W jakich okolicznościach `list::splice` zapewnia korzyść niedostępną dla zwykłego `vector`?

1.14 Podsumowanie rozdziału

Dobry kod C++ wybiera kontener zgodnie z potrzebnymi operacjami i własnościami pamięci, a nie jedynie na podstawie pojedynczego rzędu złożoności. `vector` to często najlepszy punkt wyjścia; `deque`, kontenery drzewiaste, haszujące i listy rozwiązują bardziej szczegółowe problemy. Praktycznie równie ważne jak złożoności są reguły unieważniania iteratorów, wymagania komparatorów, bezpieczeństwo czasu życia i świadome wykorzystywanie algorytmów standardowych zamiast ręcznie pisanych pętli.

Literatura i dokumentacja

- https://en.cppreference.com/w/cpp/container — kontenery i zestawienie reguł unieważniania iteratorów.
- https://en.cppreference.com/w/cpp/algorithm — algorytmy biblioteki standardowej i ich kontrakty.
- https://en.cppreference.com/w/cpp/container/vector — `std::vector` i zarządzanie pojemnością.
- https://en.cppreference.com/w/cpp/container/unordered_map — struktury haszujące.
- Scott Meyers, *Effective Modern C++*, O'Reilly, 2014.
- Nicolai M. Josuttis, *The C++ Standard Library*, wydanie drugie, Addison-Wesley, 2012.

Rozdział 2

Model obiektowy C++ i polimorfizm

2.1 Po co poznawać model obiektowy?

Klasy w C++ są jednocześnie narzędziem modelowania danych, organizowania interfejsów i budowania różnych rodzajów polimorfizmu. Dwie implementacje tego samego interfejsu mogą jednak mieć bardzo różne własności:

- mogą rozstrzygać wywołanie już w trakcie kompilacji albo dopiero w czasie wykonania;
- mogą wymagać alokacji dynamicznej lub działać bez niej;
- mogą przechowywać obiekty bezpośrednio, przez referencję albo przez wskaźnik zarządzający czasem życia;
- mogą zachowywać tożsamość dynamicznego typu obiektu lub ją utracić wskutek *object slicing*.

W tym rozdziale rozdzielamy *gwarancje standardu C++* od typowych technik używanych przez kompilatory, takich jak tablice funkcji wirtualnych (*vtable*). Pozwala to czytać kod zarówno od strony języka, jak i możliwych kosztów wykonania.

2.2 Obiekty, podobiekty i tożsamość typu

Obiekt zajmuje pewien obszar pamięci, ma określony typ i czas życia. Obiekt klasy może zawierać *podobiekty składowych* oraz *podobiekty klas bazowych*. Dziedziczenie nie oznacza zatem „zamiany” klasy pochodnej na bazową: obiekt pochodny zawiera w sobie odpowiedni podobiekt bazowy.

```cpp
#include <cassert>
#include <string>

struct Entity {
    int id = 0;
};

struct Employee : Entity {
    std::string name;
};

int main() {
    Employee e;
    e.id = 7;
    e.name = "Ada";
    Entity& base = e;  // odniesienie do podobiektu Entity
    assert(&base == static_cast<Entity*>(&e));
    assert(base.id == 7);
```

```
19 }
```

Listing 2.1: Klasa pochodna zawiera podobiekt bazowy (C++11); kompletny program.

Typ statyczny a typ dynamiczny. *Typ statyczny* wyrażenia wynika z reguł języka i jest znany kompilatorowi. Gdy wyrażenie wskazuje na obiekt klasy polimorficznej (czyli klasy posiadającej lub dziedziczącej funkcję wirtualną), *typ dynamiczny* obiektu może być klasą pochodną.

```
1 struct Animal { virtual ~Animal() = default; };
2 struct Dog : Animal {};
3
4 Dog dog;
5 Animal& ref = dog;  // statyczny typ: Animal&; obiekt: Dog
6 Animal* ptr = &dog; // statyczny typ: Animal*; obiekt: Dog
```

Listing 2.2: Statyczny i dynamiczny typ obiektu; fragment.

W C++ nie ma wymogu, aby podobiekt klasy bazowej zajmował *pierwsze* bajty obiektu pochodnego. Nie należy również zakładać, że wskaźnik do całego obiektu i wskaźnik do dowolnej jego bazy mają tę samą reprezentację liczbową. Odpowiednie konwersje wykonuje język.

2.2.1 Składnia dziedziczenia i kontrola dostępu

```
1  class Base {
2  public:
3      int a = 1;
4  protected:
5      int b = 2;
6  private:
7      int c = 3;
8  };
9
10 class PublicDerived    : public Base {};
11 class ProtectedDerived : protected Base {};
12 class PrivateDerived   : private Base {};
```

Listing 2.3: Dziedziczenie publiczne, chronione i prywatne; fragment.

Przy dziedziczeniu publicznym publiczne elementy interfejsu bazowego pozostają publiczne, a chronione pozostają chronione. Przy dziedziczeniu chronionym oba rodzaje stają się chronione, natomiast przy prywatnym oba stają się prywatne dla użytkowników klasy pochodnej. Prywatne elementy bazy pozostają niedostępne *bezpośrednio* wewnątrz klasy pochodnej niezależnie od rodzaju dziedziczenia.

Sama dostępność elementu nie zmienia jego faktycznego istnienia w podobiekcie bazowym. Publiczne dziedziczenie najczęściej wyraża relację *is-a*: obiekt pochodny może być używany tam, gdzie oczekiwany jest interfejs bazowy.

2.3 Przeciążanie, przesłanianie i funkcje wirtualne

Przeciążanie (*overloading*) polega na deklarowaniu kilku funkcji o tej samej nazwie, różniących się listą parametrów. *Przesłanianie nazwy* (*name hiding*) zachodzi, gdy deklaracja w klasie pochodnej zasłania nazwę z klasy bazowej. *Nadpisywanie* (*overriding*) oznacza dostarczenie implementacji funkcji wirtualnej z bazy przez klasę pochodną.

```
1 #include <iostream>
```

```
2
3  struct Shape {
4      virtual ~Shape() = default;
5      virtual double area() const = 0;
6      virtual void print() const {
7          std::cout << "Shape\n";
8      }
9  };
10
11 struct Square final : Shape {
12     explicit Square(double side) : side_(side) {}
13     double area() const override { return side_ * side_; }
14     void print() const override {
15         std::cout << "Square: " << area() << '\n';
16     }
17 private:
18     double side_;
19 };
20
21 int main() {
22     Square square{3.0};
23     Shape& shape = square;
24     shape.print(); // Square: 9
25     std::cout << shape.area() << '\n';
26 }
```

Listing 2.4: Dynamiczne wiązanie wywołania przez virtual (C++11); kompletny program.

Klasa `Shape` jest *abstrakcyjna*, ponieważ jej funkcja `area()` jest czysto wirtualna, co oznaczamy `= 0`. Nie można skonstruować obiektu `Shape`, lecz można utworzyć referencję lub wskaźnik do jej podobiektu w obiekcie konkretnym.

Reguła wyboru funkcji. Dla wywołania `base.f()` najpierw rozstrzygane są dostępność nazwy, przeciążenia i poprawność wywołania według *typu statycznego*. Jeśli wybrana funkcja jest wirtualna, a wywołanie nie wyłącza mechanizmu wirtualnego przez kwalifikację nazwy, uruchamiany jest odpowiedni *final overrider* dla dynamicznego typu obiektu.

2.3.1 Dlaczego stosować `override` i `final`?

`override` oznacza, że dana metoda ma nadpisywać metodę wirtualną. Jeśli sygnatura nie pasuje, kompilator zgłosi błąd. `final` uniemożliwia dalsze nadpisywanie metody albo dziedziczenie po klasie.

```
1  struct Base {
2      virtual void draw() const {}
3      virtual ~Base() = default;
4  };
5
6  struct Good : Base {
7      void draw() const override {} // poprawnie
8  };
9
10 // BLEDNE -- celowo niekompilowalne:
11 // struct Bad : Base {
12 //     void draw() override {} // brak const
13 // };
```

Listing 2.5: override wykrywa niezgodność sygnatur; fragment.

Przy nadpisywaniu znaczenie mają między innymi kwalifikatory `const`/`volatile` metody i kwalifikator referencyjny `&`/`&&`. Specyfikacja wyjątków też podlega regułom zgodności: nadpisanie metody `noexcept` metodą potencjalnie zgłaszającą wyjątek jest niedozwolone. Zwykłego typu wyniku nie można zmienić, z wyjątkiem określonych *kowariantnych* typów zwracanych.

```
struct Service {
    virtual int read() const & noexcept { return 1; }
    virtual ~Service() = default;
};

struct FastService : Service {
    int read() const & noexcept override { return 2; }
};
```

Listing 2.6: noexcept i kwalifikatory referencyjne w nadpisywaniu; fragment.

2.3.2 Przesłanianie przeciążeń i `using`

```
#include <cassert>

struct Printer {
    int print(int) const { return 1; }
    int print(double) const { return 2; }
};

struct FancyPrinter : Printer {
    using Printer::print;
    int print(const char*) const { return 3; }
};

int main() {
    FancyPrinter p;
    assert(p.print(10) == 1);
    assert(p.print(2.5) == 2);
    assert(p.print("text") == 3);
}
```

Listing 2.7: Odsłanianie przeciążeń klasy bazowej; kompletny program.

Bez deklaracji `using Printer::print;` nazwa `print` z klasy pochodnej przesłaniałaby zestaw przeciążeń o tej nazwie z klasy bazowej. Mechanizm przesłaniania występuje również dla funkcji *niewirtualnych*.

2.3.3 Argumenty domyślne i wiązanie statyczne

Wartości domyślnych argumentów nie wybiera się polimorficznie: są związane ze statycznym typem wyrażenia, mimo że sama implementacja funkcji może być wybrana dynamicznie.

```
#include <cassert>

struct Base {
    virtual int calculate(int x = 10) const { return x; }
    virtual ~Base() = default;
};

struct Derived : Base {
    int calculate(int x = 20) const override { return 2 * x; }
};

```

```
int main() {
    Derived object;
    Base& base = object;
    assert(base.calculate() == 20); // 2 * 10
    assert(object.calculate() == 40); // 2 * 20
}
```

Listing 2.8: Domyślny argument pochodzi z deklaracji bazy (C++11); kompletny program.

Z tego powodu w praktycznych interfejsach polimorficznych warto unikać różnych domyślnych argumentów dla nadpisywanych funkcji.

2.4 Destruktor wirtualny i reguły czasu życia

Jeżeli obiekt klasy pochodnej ma być niszczony przez wskaźnik do klasy bazowej, destruktor bazy powinien być `public virtual` (alternatywnie projekt może zabraniać usuwania przez bazę, np. za pomocą `protected` destruktora niewirtualnego).

```
#include <iostream>
#include <memory>

struct Resource {
    virtual ~Resource() { std::cout << "~Resource\n"; }
};

struct FileResource : Resource {
    ~FileResource() override {
        std::cout << "~FileResource\n";
    }
};

int main() {
    std::unique_ptr<Resource> object =
        std::make_unique<FileResource>();
    // Najpierw ~FileResource, potem ~Resource.
}
```

Listing 2.9: Bezpieczne usuwanie obiektu przez wskaźnik bazowy (C++14); kompletny program.

Jeżeli bazowy destruktor nie jest wirtualny, standardowe `delete base_ptr;` przy dynamicznym typie różnym od statycznego może prowadzić do niezdefiniowanego zachowania; istnieją szczególne reguły związane z tzw. *destroying delete*, ale nie należy traktować ich jako zwykłego substytutu destruktora wirtualnego.

2.4.1 Kolejność konstrukcji i destrukcji

Dla obiektu klasy pochodnej kolejność konstrukcji jest następująca:

1. wirtualne podobiekty bazowe (przez najbardziej pochodny obiekt);
2. niewirtualne podobiekty bazowe w kolejności ich deklaracji;
3. składowe klasy w kolejności *deklaracji pól*, niezależnie od kolejności w liście inicjalizacyjnej;
4. ciało konstruktora klasy.

Destrukcja przebiega w odpowiedniej kolejności odwrotnej. W prostym pojedynczym dziedziczeniu można to zilustrować następująco.

```cpp
#include <iostream>

struct A {
    A()  { std::cout << "A+ "; }
    ~A() { std::cout << "A- "; }
};
struct B {
    B()  { std::cout << "B+ "; }
    ~B() { std::cout << "B- "; }
};
struct C : A {
    B member;
    C()  { std::cout << "C+ "; }
    ~C() { std::cout << "C- "; }
};

int main() {
    C object;
    // A+ B+ C+ C- B- A-
}
```

Listing 2.10: Kolejność konstruowania podobiektów (C++11); kompletny program.

2.4.2 Funkcje wirtualne w konstruktorach i destruktorach

Podczas wykonywania konstruktora `Base` wywołanie wirtualne nie trafia do implementacji z `Derived`, której część nie została jeszcze skonstruowana. Analogicznie podczas wykonywania destruktora `Base` część pochodna została już zniszczona.

```cpp
#include <cassert>

struct Base {
    Base() { assert(code() == 1); }
    virtual ~Base() = default;
    virtual int code() const { return 1; }
};
struct Derived : Base {
    int code() const override { return 2; }
};
int main() {
    Derived object;
    Base& b = object;
    assert(b.code() == 2);
}
```

Listing 2.11: Dynamiczne wiązanie podczas konstrukcji (C++11); kompletny program.

Nie należy wywoływać funkcji *czysto wirtualnej* przez mechanizm wirtualny z konstruktora albo destruktora klasy, na której poziomie funkcja jest czysto wirtualna: takie wywołanie jest niezdefiniowanym zachowaniem. Preferuj dwufazową konstrukcję kontrolowaną przez fabrykę tylko wtedy, gdy naprawdę jest potrzebna, a w typowych sytuacjach inicjalizację kompletnych podobiektów przez konstruktory i RAII.

2.5 Object slicing: utrata części pochodnej

Object slicing nie jest dynamicznym rzutowaniem. Gdy obiekt pochodny kopiujemy *przez wartość* do obiektu bazy, powstaje osobny obiekt bazy, zawierający tylko część bazową.

```cpp
#include <cassert>

struct Base {
    virtual ~Base() = default;
    virtual int value() const { return 1; }
};
struct Derived : Base {
    int value() const override { return 2; }
};

int by_value(Base object) { return object.value(); }
int by_reference(const Base& object) {
    return object.value();
}

int main() {
    Derived d;
    assert(by_value(d) == 1);      // nowy obiekt Base
    assert(by_reference(d) == 2);  // oryginalny Derived
}
```

Listing 2.12: Kopiowanie przez wartość powoduje slicing (C++11); kompletny program.

Zwykle przechowujemy heterogeniczne obiekty polimorficzne jako `std::unique_ptr<Base>` (własność wyłączna), `std::shared_ptr<Base>` (współdzielona własność) albo nieposiadające obiektu `Base&`/`Base*` (widoki). Bezpośredni `std::vector<Base>` przechowuje wyłącznie obiekty `Base` i nie zachowuje typów pochodnych przy kopiowaniu.

2.5.1 Bezpieczny kontener polimorficzny

```cpp
#include <cassert>
#include <memory>
#include <vector>

struct Operation {
    virtual ~Operation() = default;
    virtual int apply(int x) const = 0;
};
struct Add : Operation {
    explicit Add(int delta) : delta_(delta) {}
    int apply(int x) const override { return x + delta_; }
private:
    int delta_;
};
struct Multiply : Operation {
    explicit Multiply(int factor) : factor_(factor) {}
    int apply(int x) const override { return x * factor_; }
private:
    int factor_;
};

int main() {
    std::vector<std::unique_ptr<Operation>> operations;
    operations.push_back(std::make_unique<Add>(2));
    operations.push_back(std::make_unique<Multiply>(3));

    int result = 4;
    for (const auto& operation : operations)
        result = operation->apply(result);
    assert(result == 18); // (4 + 2) * 3
}
```

Listing 2.13: Własność obiektów pochodnych w kontenerze unikalnych wskaźników (C++14); kompletny program.

Warto pamiętać, że unikalny wskaźnik na typ pochodny można przenieść do unikalnego wskaźnika na bazę tylko wtedy, gdy spełnione są wymagania dotyczące konwersji wskaźników i deletera. Domyślny deleter w typowym przypadku usuwa obiekt przez `Base*`, dlatego destruktor polimorficznej bazy musi być poprawnie zaprojektowany.

2.6 RTTI: `dynamic_cast` oraz `typeid`

RTTI (*run-time type information*) pozwala w pewnych sytuacjach sprawdzić typ dynamiczny i wykonać sprawdzoną konwersję w hierarchii klas polimorficznych.

2.6.1 `dynamic_cast` — sprawdzony downcast

```cpp
#include <cassert>
#include <typeinfo>

struct Animal { virtual ~Animal() = default; };
struct Dog : Animal { int bark() const { return 10; } };
struct Cat : Animal {};

int main() {
    Dog dog;
    Animal* animal = &dog;

    if (auto* ptr = dynamic_cast<Dog*>(animal)) {
        assert(ptr->bark() == 10);
    }
    assert(dynamic_cast<Cat*>(animal) == nullptr);

    Animal& reference = dog;
    assert(&dynamic_cast<Dog&>(reference) == &dog);
    try {
        (void)dynamic_cast<Cat&>(reference);
        assert(false); // nie powinno zostac wykonane
    } catch (const std::bad_cast&) {
        // Nieudany dynamic_cast do referencji rzuca bad_cast.
    }
}
```

Listing 2.14: Sprawdzona konwersja w dół hierarchii (C++11); kompletny program.

W typowym *downcast* źródłowy typ klasy musi być polimorficzny. Nieudane rzutowanie do wskaźnika daje `nullptr`, a do referencji zgłasza `std::bad_cast`. Konwersja zwykłego wskaźnika `Derived*` do dostępnej, jednoznacznej bazy `Base*` nie potrzebuje RTTI ani `dynamic_cast`.

`static_cast` nie sprawdza typu dynamicznego

```cpp
struct Base { virtual ~Base() = default; };
struct Derived : Base { int only_derived = 7; };

Base base;
Base* ptr = &base;
// Kompilator moze zaakceptowac ponizsze rzutowanie,
// ale wskazywany obiekt nie jest obiektem Derived:
// Derived* wrong = static_cast<Derived*>(ptr);
```

```
// int x = wrong->only_derived; // niezdefiniowane zachowanie
```

Listing 2.15: Niebezpieczny downcast; fragment celowo błędny.

Stosuj `static_cast` przy downcast tylko wtedy, gdy poprawny typ dynamiczny wynika z udowodnionego niezmiennika. Jeżeli architektura wymaga bardzo wielu wywołań `dynamic_cast`, rozważ lepszy interfejs wirtualny, visitor albo `std::variant`.

2.6.2 `typeid` i typ dynamiczny

```
#include <cassert>
#include <typeinfo>

struct Base { virtual ~Base() = default; };
struct Derived : Base {};

int main() {
    Derived object;
    Base& base = object;
    assert(typeid(base) == typeid(Derived));
    assert(typeid(object) == typeid(Derived));
}
```

Listing 2.16: Odczyt informacji o typie dynamicznym (C++11); kompletny program.

Dla wyrażenia typu klasy polimorficznej, będącego *glvalue*, `typeid(expression)` zwraca informację o typie dynamicznym. W innych sytuacjach zazwyczaj uwzględnia typ statyczny wyrażenia. Nazwy zwracane przez `type_info::name()` są zależne od implementacji; nie należy używać ich jako stabilnego identyfikatora w zapisywanych plikach lub w protokołach sieciowych. Dla `typeid(*ptr)` z zerowym wskaźnikiem do typu polimorficznego może zostać zgłoszony `std::bad_typeid`.

2.7 Wielodziedziczenie i wirtualne klasy bazowe

C++ pozwala dziedziczyć po wielu klasach. Jest to przydatne np. dla kilku niezależnych interfejsów, ale wymaga zrozumienia niejednoznaczności oraz struktury podobiektów bazowych.

```
#include <cassert>

struct Readable {
    virtual ~Readable() = default;
    virtual int read() const = 0;
};
struct Writable {
    virtual ~Writable() = default;
    virtual void write(int x) = 0;
};
struct Buffer : Readable, Writable {
    int value = 0;
    int read() const override { return value; }
    void write(int x) override { value = x; }
};

int main() {
    Buffer buffer;
    Writable& out = buffer;
    Readable& in = buffer;
    out.write(17);
    assert(in.read() == 17);
}
```

Listing 2.17: Dwie niezależne bazy-interfejsy (C++11); kompletny program.

2.7.1 Problem diamentu

Załóżmy, że `Left` i `Right` dziedziczą po `Base`, a `Diamond` po obu. Przy zwykłym dziedziczeniu istnieją *dwa różne* podobiekty `Base`. Przy dziedziczeniu wirtualnym najbardziej pochodny obiekt zawiera jedną współdzieloną wirtualną bazę `Base`.

```
#include <cassert>

struct Root {
    explicit Root(int n) : value(n) {}
    int value;
};
struct Left : virtual Root {
    Left() : Root(1) {} // ignorowane jako inicjalizacja
};                          // bazy w obiekcie Diamond
struct Right : virtual Root {
    Right() : Root(2) {}
};
struct Diamond : Left, Right {
    Diamond() : Root(42), Left(), Right() {}
};

int main() {
    Diamond d;
    Root* from_left = static_cast<Left*>(&d);
    Root* from_right = static_cast<Right*>(&d);
    assert(from_left == from_right);
    assert(d.value == 42);
}
```

Listing 2.18: Jedna wirtualna baza we wzorcu diamentu (C++11); kompletny program.

Najbardziej pochodna klasa `Diamond` odpowiada za konstrukcję wirtualnej bazy `Root`. W implementacjach wielodziedziczenie i wirtualne bazy mogą wymagać korekty wskaźników oraz dodatkowych informacji o położeniu podobiektów. Nie należy jednak utożsamiać modelu języka z konkretną reprezentacją pamięci w danym ABI.

2.8 Kowariantne typy zwracane

W szczególnym przypadku nadpisująca funkcja wirtualna może zwracać wskaźnik lub referencję do typu bardziej pochodnego, zamiast wskaźnika lub referencji do klasy bazowej.

```
#include <cassert>

struct Base {
    virtual ~Base() = default;
    virtual Base* self() { return this; }
};
struct Derived : Base {
    Derived* self() override { return this; }
    int payload = 5;
};

int main() {
    Derived d;
```

```
    Base& b = d;
    Base* a = b.self(); // typ statyczny Base*
    Derived* x = d.self(); // typ statyczny Derived*
    assert(a == x);
    assert(x->payload == 5);
}
```

Listing 2.19: Kowariantny typ wyniku metody wirtualnej (C++11); kompletny program.

Kowariancja nie obejmuje dowolnych szablonów: na przykład `std::unique_ptr<Derived>` nie jest kowariantnym typem zwracanym nadpisującym metodę zwracającą `std::unique_ptr<Base>`. Dlatego metody `clone()` zwracające `unique_ptr` w klasach pochodnych zwykle zachowują wspólny typ wyniku.

2.9 Klasy abstrakcyjne jako kontrakty i wzorzec NVI

Dobrą praktyką może być rozdzielenie publicznego, niewirtualnego interfejsu kontrolującego niezmienniki od prywatnej implementacji wirtualnej (*non-virtual interface*, NVI).

```
#include <cassert>
#include <stdexcept>

class Encoder {
public:
    virtual ~Encoder() = default;

    int encode(int value) const {
        if (value < 0)
            throw std::invalid_argument("negative input");
        return do_encode(value);
    }

private:
    virtual int do_encode(int value) const = 0;
};

class DoubleEncoder final : public Encoder {
private:
    int do_encode(int value) const override {
        return 2 * value;
    }
};

int main() {
    DoubleEncoder encoder;
    Encoder& api = encoder;
    assert(api.encode(21) == 42);
}
```

Listing 2.20: NVI: publiczny kontrakt i prywatna metoda wirtualna (C++11); kompletny program.

Wersja publiczna może centralnie realizować walidację, logowanie, kontrolę błędów i niezmienników. Prywatna funkcja wirtualna w klasie bazowej *może* zostać nadpisana przez klasę pochodną, mimo że nie jest z niej bezpośrednio wywoływalna przez zwykłą nazwę; w przykładzie nadpisanie jest prywatne.

2.9.1 Polimorficzne kopiowanie przez `clone`

Gdy chcemy uzyskać niezależną kopię obiektu o zachowanym typie dynamicznym, możemy zastosować wirtualną fabrykę `clone()`.

```
#include <cassert>
#include <memory>

struct Node {
    virtual ~Node() = default;
    virtual std::unique_ptr<Node> clone() const = 0;
    virtual int value() const = 0;
};
struct Number final : Node {
    explicit Number(int x) : x_(x) {}
    std::unique_ptr<Node> clone() const override {
        return std::make_unique<Number>(*this);
    }
    int value() const override { return x_; }
private:
    int x_;
};

int main() {
    std::unique_ptr<Node> a = std::make_unique<Number>(42);
    std::unique_ptr<Node> b = a->clone();
    assert(a.get() != b.get());
    assert(a->value() == b->value());
}
```

Listing 2.21: Kopiowanie polimorficzne (C++14); kompletny program.

Takie podejście przywraca semantykę kopiowania do hierarchii, której obiekty zwykle są trzymane przez wskaźniki. Trzeba jednak jawnie zdefiniować, co oznacza *głęboka kopia*, zwłaszcza jeśli obiekt współdzieli inne zasoby.

2.10 Model implementacyjny: *vptr*, *vtable* i ABI

Standard C++ określa *zachowanie*, ale nie nakazuje użycia żadnej konkretnej struktury wewnętrznej do realizacji funkcji wirtualnych. Popularne ABI używają mniej więcej następującego modelu:

- obiekt polimorficzny przechowuje ukryty wskaźnik *vptr*;
- wskaźnik ten prowadzi do tablicy *vtable*, która zawiera informacje potrzebne do wywołań wirtualnych i często do RTTI;
- wywołanie wirtualne może wymagać odczytania `vptr`, odpowiedniego wpisu i wykonania wywołania pośredniego.

To *opis typowej implementacji*, nie kontrakt języka. W szczególności liczba i położenie wskaźników `vptr`, układ tablic oraz narzut pamięci są zależne od ABI i kompilatora. Wielodziedziczenie może komplikować ten schemat, ponieważ niektóre wywołania wymagają korekty wskaźnika `this`.

Schemat poglądowy (nie jest układem wymaganym przez standard)

Obiekt `Derived`		**Tablica wirtualna**
`vptr`	$\longrightarrow$	adres implementacji `draw()`
`Base::id`		adres odpowiedniego destruktora
`Derived::payload`		metadane zależne od ABI

2.10.1 `sizeof`, wyrównanie i brak gwarancji binarnego układu

```cpp
#include <iostream>

struct Plain { int x = 0; };
struct Poly {
    int x = 0;
    virtual ~Poly() = default;
};

int main() {
    std::cout << sizeof(Plain) << '\n';
    std::cout << sizeof(Poly) << '\n';
}
```

Listing 2.22: Empiryczny rozmiar klasy polimorficznej (C++11); kompletny program.

Nie wpisujemy w skrypcie stałych wartości tych rozmiarów: na wielu platformach typ `Poly` będzie większy, ale rozmiary wynikają z implementacji i obowiązujących reguł wyrównania. Sama obecność `virtual` nie gwarantuje określonego liczbowo narzutu ani ustalonej reprezentacji binarnej. Nie należy serializować surowych bajtów obiektu polimorficznego jako trwałego formatu danych.

2.10.2 Koszty i devirtualizacja

Wywołanie wirtualne może być droższe od zwykłego z powodu pośrednictwa oraz potencjalnie trudniejszego *inliningu*. W praktyce jednak koszt zależy od rozkładu typów dynamicznych, lokalności pamięci, optymalizacji międzyjednostkowej i architektury CPU. Kompilator może zastosować *devirtualizację*: jeśli potrafi ustalić docelową implementację, wywołanie może stać się bezpośrednie, a następnie zostać rozwinięte inline.

```cpp
struct Interface {
    virtual int eval(int) const = 0;
    virtual ~Interface() = default;
};
struct Square final : Interface {
    int eval(int x) const override { return x * x; }
};

int evaluate(const Square& x, int value) {
    return x.eval(value); // cel wywolania jest znany
}
```

Listing 2.23: final może ułatwiać ustalenie celu wywołania; fragment.

Nie oznacza to, że `final` zawsze poprawia wydajność. Porównania należy prowadzić na przykładach reprezentatywnych i przy realistycznych opcjach optymalizacji. Wąskim gardłem może być w istocie rozproszony układ obiektów w pamięci, a nie sam koszt wywołania funkcji wirtualnej.

2.11 Alternatywa: polimorfizm statyczny przez szablony i CRTP

Polimorfizm *statyczny* wykorzystuje dedukcję typów i instancjowanie szablonów, dzięki czemu kompilator zna konkretny typ w miejscu użycia. Pozwala to generować kod wyspecjalizowany bez konieczności utrzymywania wspólnej bazy wirtualnej.

2.11.1 Szablon funkcji i concepts (C++20)

```cpp
#include <cassert>
#include <concepts>

struct Circle {
    double radius;
    double area() const { return 3.141592653589793 * radius * radius; }
};
struct Rectangle {
    double a, b;
    double area() const { return a * b; }
};

template<class T>
concept HasArea = requires(const T& x) {
    { x.area() } -> std::convertible_to<double>;
};

template<HasArea T>
double area_of(const T& shape) {
    return shape.area();
}

int main() {
    assert(area_of(Rectangle{2, 3}) == 6.0);
    assert(area_of(Circle{2}) > 12.5);
}
```

Listing 2.24: Polimorfizm statyczny przez concept (C++20); kompletny program.

Powyższy kod nie wymaga wspólnej klasy bazowej. Ograniczeniem jest to, że w typowym kodzie szablonowym zestaw typów musi być znany przy kompilacji miejsca instancjowania. Nie otrzymujemy automatycznie heterogenicznego `std::vector<T>` dla wielu różnych typów.

2.11.2 CRTP (*Curiously Recurring Template Pattern*)

```cpp
#include <cassert>

template<class Derived>
struct Printable {
    int twice() const {
        const auto& self = static_cast<const Derived&>(*this);
        return 2 * self.value();
    }
};

struct Number : Printable<Number> {
    int data;
    explicit Number(int x) : data(x) {}
    int value() const { return data; }
};

int main() {
    Number n{21};
    assert(n.twice() == 42);
}
```

Listing 2.25: CRTP z metodą dostarczaną przez bazę (C++11); kompletny program.

CRTP nie jest bezpośrednim zamiennikiem wszystkich interfejsów wirtualnych: wyrażenie `Printable<A>*` nie jest uniwersalnym wskaźnikiem do dowolnego `Printable<B>`. Może natomiast umożliwiać *static dispatch*, implementację domieszek (*mixins*) i wykrywanie błędów już przy instancjowaniu

szablonów.

2.12 Polimorfizm bez dziedziczenia: `std::variant`

Jeżeli lista możliwych typów jest zamknięta i znana przy kompilacji, `std::variant` (C++17) pozwala przechowywać jedną z kilku alternatyw *przez wartość*. Zamiast wirtualnego wywołania używa się zwykle `std::visit`.

```
#include <cassert>
#include <variant>

struct Add { int x; };
struct Multiply { int x; };
using Instruction = std::variant<Add, Multiply>;

int execute(const Instruction& instruction, int input) {
    return std::visit([input](const auto& action) -> int {
        using T = std::decay_t<decltype(action)>;
        if constexpr (std::is_same_v<T, Add>)
            return input + action.x;
        else
            return input * action.x;
    }, instruction);
}

int main() {
    assert(execute(Add{2}, 4) == 6);
    assert(execute(Multiply{3}, 4) == 12);
}
```

Listing 2.26: Zamknięty zbiór typów i std::visit (C++17); kompletny program.

To podejście ułatwia przechowywanie wartości bez osobnych alokacji i ma inną charakterystykę wydajnościową. Ceną jest konieczność jawnego wyliczenia alternatywnych typów w definicji `variant` oraz aktualizowania odwiedzających, gdy pojawiają się nowe alternatywy. W przykładzie użyto też `std::decay_t` oraz `std::is_same_v` z nagłówka `<type_traits>`.

2.13 Polimorfizm dynamiczny i statyczny — porównanie

Cecha	**`virtual`**	**Szablon/CRTP**	**`variant`**
Zbiór implementacji	Otwarty w granicach hierarchii	Ustalany przy instancjowaniu	Zamknięty zestaw alternatyw
Wspólny uchwyt	`Base&`, `Base*`	Zależny od typu	Jeden typ sumaryczny
Mechanizm wywołania	Zwykle dynamiczny	Zwykle statyczny	Wybór aktywnej alternatywy
Przechowywanie	Często wskaźniki do obiektów	Zazwyczaj konkretne obiekty	Wartość wewnątrz `variant`
Rozszerzanie o nowy typ	Zwykle bez zmiany wywołującego	Nowa instancjacja	Zmiana listy typów
Typowe koszty	Dyspozycja i lokalność danych	Rozmiar kodu po instancjacjach	Rozmiar największej alternatywy i odwiedzanie

Wybór zależy przede wszystkim od architektury programu:

- **Interfejsy pluginów i otwarte hierarchie**: zwykle polimorfizm dynamiczny, z uwzględnieniem stabilności ABI.

- **Algorytmy generyczne na znanych typach**: zwykle szablony i concepts.
- **Zamknięta rodzina wariantów wartości**: często `std::variant`.
- **Wiele małych obiektów w intensywnej pętli**: przy wyborze należy zmierzyć także koszty alokacji, lokalność danych i wynik optymalizacji kompilatora.

2.14 Reguła zera/pięciu w klasach polimorficznych

Wprowadzenie destruktora wirtualnego powoduje, że automatyczne generowanie operacji przenoszących nie działa tak samo jak w prostej klasie agregującej pola. Dodatkowo publiczne kopiowanie samego podobiektu bazowego może umożliwiać przypadkowe obcięcie obiektu.

W wielu hierarchiach interfejsowych rozsądne jest zabronienie kopiowania i świadome zarządzanie obiektami przez smart pointery:

```
class Interface {
public:
    Interface() = default;
    virtual ~Interface() = default;
    Interface(const Interface&) = delete;
    Interface& operator=(const Interface&) = delete;
    Interface(Interface&&) = delete;
    Interface& operator=(Interface&&) = delete;

    virtual void run() = 0;
};
```

Listing 2.27: Niekopiowalna klasa bazowa interfejsu (C++11); fragment.

Nie jest to reguła uniwersalna: klasy reprezentujące wartości mogą być kopiowalne, a wybrane hierarchie wspierają jawne `clone()`. Istotne, aby polityka kopiowania, przenoszenia i własności była świadomym elementem kontraktu.

2.15 Typowe błędy i pytania kontrolne

1. **Brak wirtualnego destruktora bazy**, mimo usuwania obiektów pochodnych przez wskaźnik do bazy.
2. **Brak `override`** i przypadkowe utworzenie innej metody przez pominięcie `const`, zmianę parametrów albo kwalifikatora `&`/`&&`.
3. **Slicing** podczas przekazania przez wartość lub wstawiania do kontenera bazowych wartości.
4. **Bezpodstawny `static_cast` w dół hierarchii** bez gwarancji rzeczywistego typu obiektu.
5. **Wywołanie funkcji czysto wirtualnej** podczas konstrukcji lub destrukcji na etapie, gdy implementacja pochodna nie jest aktywna.
6. **Mylenie implementacyjnej vtable z gwarancją standardu**.
7. **Zależność od domyślnych argumentów metod wirtualnych** różniących się między bazą a klasą pochodną.
8. **Nadmierna hierarchia klas**, gdy prostsza jest kompozycja, szablon lub `std::variant`.

Pytania, które warto umieć rozstrzygnąć samodzielnie:

1. Czy `virtual` oznacza, że *każde* wywołanie metody danej klasy musi być realizowane pośrednio?

2. Dlaczego `Base value = derived;` może dać inny wynik niż `Base& ref = derived;`?
3. Co stanie się z funkcją wirtualną nadpisaną w `Derived`, jeśli wywoła ją konstruktor `Base`?
4. W jakiej sytuacji `dynamic_cast<Derived&>` zgłasza `std::bad_cast`?
5. Kiedy użyć `std::variant`, a kiedy interfejsu wirtualnego?

2.16 Wskazówki dotyczące diagnostyki i pomiarów

Przy analizie klas polimorficznych przydatne są ostrzeżenia `-Wall -Wextra -Wpedantic -Woverloaded-virtual` i, zależnie od kompilatora, `-Wsuggest-override`. Błędy czasu życia mogą być wykrywane przez AddressSanitizer i UBSan (w szczególności odpowiednie kontrole vptr, jeśli są wspierane).

```
g++ -std=c++20 -O2 -Wall -Wextra -Wpedantic \
    -Woverloaded-virtual example.cpp -o example
./example

# Testy diagnostyczne (zalezne od platformy):
g++ -std=c++20 -O1 -g -fsanitize=address,undefined \
    -fno-omit-frame-pointer example.cpp -o example_san
./example_san
```

Listing 2.28: Kompilacja przykładowego programu w GCC lub Clangu.

Należy porównywać z *realnym* obciążeniem, a nie zakładać, że jeden mechanizm jest zawsze szybszy od drugiego. Przy pomiarach uwzględniamy optymalizację, lokalność danych, rozmiar próby, rozrzut pomiarów i to, czy kompilator nie wyeliminował pracy.

2.17 Podsumowanie rozdziału

Kluczowe jest rozdzielenie trzech kwestii: *jakie typy obiektów* występują, *kto odpowiada za ich czas życia* oraz *kiedy wybierana jest implementacja operacji*. `virtual` umożliwia polimorfizm dynamiczny, lecz wymaga rozważenia destruktorów i własności obiektów. Szablony i CRTP oferują polimorfizm statyczny, a `std::variant` jest wygodną alternatywą dla zamkniętej rodziny typów. Praktyczna wydajność zależy nie tylko od kosztu wywołania, ale również od reprezentacji i rozmieszczenia danych.

Literatura i dalsze źródła

- ISO C++ Standard: model obiektowy, klasy, dziedziczenie, funkcje wirtualne i RTTI (normatywne źródło reguł języka).
- https://en.cppreference.com/w/cpp/language/virtual — funkcje wirtualne i reguły nadpisywania.
- https://en.cppreference.com/w/cpp/language/dynamic_cast — sprawdzane konwersje w hierarchiach.
- https://en.cppreference.com/w/cpp/language/derived_class — dziedziczenie i wirtualne klasy bazowe.
- Bjarne Stroustrup, *The C++ Programming Language*, wydanie czwarte, Addison-Wesley, 2013.

Rozdział 3

Specyfikatory i kwalifikatory deklaracji w C++

3.1 Wprowadzenie: czym właściwie są `static`, `const` i `inline`?

Słowa kluczowe `static`, `extern`, `inline`, `const`, `constexpr` i podobne nie są jednym rodzajem "modyfikatorów". Mogą określać różne, w znacznej mierze niezależne własności deklaracji:

- *czas przechowywania* obiektu (*storage duration*), czyli jak długo istnieje jego pamięć;
- *łączność* nazwy (*linkage*), czyli czy ta sama nazwa może oznaczać ten sam byt w różnych jednostkach translacji;
- *modyfikowalność* obiektu lub dostępu przez dany typ;
- *możliwość obliczenia wyrażenia podczas kompilacji*;
- *reguły wielokrotnego definiowania* funkcji lub zmiennej;
- *własności funkcji i metod*, np. zakaz niejawnej konwersji albo gwarancje dotyczące wyjątków.

Jednostka translacji. W uproszczeniu plik `.cpp` po rozwinięciu dyrektyw `#include` tworzy jednostkę translacji (*translation unit*, TU). Plik nagłówkowy bywa włączany do wielu jednostek translacji, dlatego umieszczanie w nim definicji wymaga uwagi.

Czas przechowywania. Rozróżniamy w szczególności:

- *automatyczny* — zwykle zmienne lokalne, tworzone przy wejściu w odpowiedni zakres i niszczone przy jego opuszczaniu;
- *statyczny* — obiekt istnieje przez czas działania programu;
- *wątkowy* — oddzielny obiekt istnieje dla każdego wątku;
- *dynamiczny* — czas życia kontrolowany jawnie lub przez obiekty zarządzające zasobem (np. `std::unique_ptr`).

Łączność. Nazwa może nie mieć łączności (typowo zmienna lokalna), mieć *łączność wewnętrzną* (*internal linkage*: dostępna jako ten sam byt tylko w obrębie TU) albo *łączność zewnętrzną* (*external linkage*: może odnosić się do jednego bytu w wielu TU). C++20 wprowadza również łączność modułową; w tym rozdziale zajmujemy się głównie klasycznym modelem plików nagłówkowych i źródłowych.

3.2 `const`: zakaz modyfikacji przez dany dostęp

Deklaracja `const` kwalifikuje typ. Dla obiektu oznacza, że nie wolno zmieniać jego wartości przez dostęp, w którym typ jest `const` (z uwzględnieniem szczególnych zasad np. `mutable`). Samo `const` *nie gwarantuje* obliczenia wartości w czasie kompilacji.

```
int read_value();

void example() {
    const int a = 10;
    // a = 11;              // error: a is const

    const int b = read_value();  // OK: runtime initialization
    int c = read_value();
    c += b;                      // OK: c is mutable
}
```

3.2.1 Wskaźniki, referencje i pozycja `const`

Różnica między kwalifikacją wskaźnika i wskazywanego obiektu jest fundamentalna:

```
int a = 1;
int b = 2;

const int* p = &a;  // pointer to const int
p = &b;             // OK: pointer can change
// *p = 3;          // error: pointee is const through p

int* const q = &a;  // const pointer to int
*q = 3;             // OK: a can change
// q = &b;          // error: pointer is const

const int* const r = &a;  // both are const through r
const int& ref = a;      // read-only access through ref
```

Argumenty i metody. Dla dużych obiektów często używamy `const T&`, aby uniknąć kopiowania i zadeklarować brak modyfikacji przez referencję. Kwalifikator `const` za listą parametrów metody odnosi się do obiektu `*this`:

```
#include <string>

class Person {
    std::string name_;
public:
    explicit Person(std::string name) : name_(name) {}

    const std::string& name() const { return name_; }
    void rename(std::string s) { name_ = s; }
};
```

Kiedy stosować? Używaj `const` dla wartości, wskaźników, referencji i metod, jeśli modyfikacja przez dany interfejs nie powinna być dozwolona. Nie używaj go jako zamiennika `constexpr`: zwykła stała zainicjalizowana wynikiem funkcji może być wyznaczona dopiero podczas wykonywania programu.

3.3 `constexpr`: stałe i funkcje dostępne dla obliczeń stałych

`constexpr` dla zmiennej wymaga inicjalizacji nadającej się do obliczenia jako wyrażenie stałe; taka zmienna jest kwalifikowana `const`. Dla funkcji `constexpr` oznacza *możliwość* wywołania w kontekście wymagającym wyniku obliczanego w czasie kompilacji, a nie obowiązek obliczenia każdego wywołania podczas kompilacji.

```
#include <array>

constexpr int square(int x) {
    return x * x;
}

int read_value();

void example() {
    constexpr int n = square(5);    // constant expression
    std::array<int, n> data{};     // valid array extent

    int x = read_value();
    int y = square(x);             // runtime call is valid

    // constexpr int z = square(x);  // error: x not constant
    (void)data;
    (void)y;
}
```

Ważne rozróżnienie. Zwykle `const int n = 5;` także może uczestniczyć w wyrażeniach stałych, bo spełnia odpowiednie reguły języka; nie oznacza to jednak, że *każda* zmienna `const` jest stała czasu kompilacji. `constexpr` wyraża ten zamiar jednoznacznie i jest sprawdzany przez kompilator.

Kiedy stosować? Używaj `constexpr`, gdy wartość ma być wykorzystywana jako stała czasu kompilacji, np. do rozmiarów tablic `std::array` i argumentów szablonów. Nadaje się także do `static_assert`. Oznaczaj nim również funkcje, które naturalnie nadają się zarówno do obliczeń stałych, jak i zwykłych wywołań.

3.3.1 `if constexpr` (od C++17)

To osobna konstrukcja sterująca, typowo stosowana w szablonach. W odróżnieniu od zwykłego `if` może odrzucić gałąź podczas instancjacji szablonu:

```
#include <type_traits>

template<class T>
const char* category() {
    if constexpr (std::is_integral_v<T>) {
        return "integral";
    } else {
        return "non-integral";
    }
}
```

3.4 `consteval` i `constinit` (od C++20)

3.4.1 `consteval`: wywołanie wymagające stałej ewaluacji

Funkcja `consteval` jest funkcją *natychmiastową* (*immediate function*). Wywołanie, które nie należy do szczególnego kontekstu natychmiastowego, musi być wyrażeniem stałym. W typowym kodzie oznacza to, że nie można przekazać jej danych znanych dopiero podczas działania programu.

```
consteval int cube(int x) {
    return x * x * x;
}

int read_value();

void example() {
    constexpr int a = cube(3);  // OK
    int b = cube(4);            // OK: call evaluated as constant
    int x = read_value();
    // int c = cube(x);         // error: x not constant
    (void)b;
    (void)x;
}
```

`consteval` wybieraj wtedy, kiedy wywołanie *musi* być obliczone w kontekście stałym; nie jest to ogólny przepis na szybsze wykonywanie programu.

3.4.2 `constinit`: gwarancja inicjalizacji statycznej

`constinit` dotyczy zmiennych o statycznym lub wątkowym czasie przechowywania. Wymusza inicjalizację statyczną, a więc wyklucza inicjalizację dynamiczną. *Nie czyni zmiennej stałą*:

```
constexpr int twice(int x) { return 2 * x; }

constinit int global_count = twice(10);

void update() {
    global_count += 1;        // OK: mutable variable
}

int read_value();
// constinit int bad = read_value();  // error: dynamic init
```

Nie wolno łączyć `constinit` i `constexpr` w tej samej deklaracji. `constinit` ma sens np. dla mutowalnego stanu globalnego, który musi być inicjalizowany bez inicjalizacji dynamicznej. Nie rozwiązuje natomiast problemów synchronizacji dostępu wielu wątków.

3.5 `static`: trzy różne zastosowania

Znaczenie `static` zależy od miejsca deklaracji. Jest to jedno z najczęstszych źródeł nieporozumień.

3.5.1 Nazwa w przestrzeni nazw: łączność wewnętrzna

W przestrzeni nazw (w tym globalnej) `static` nadaje nazwie łączność wewnętrzną. Definicja umieszczona w dwóch plikach `.cpp` opisuje wtedy dwa różne byty.

```
// file: first.cpp
```

```
static int cache_size = 100;
static int helper(int x) { return x + 1; }

// file: second.cpp
static int cache_size = 200;  // a different variable
```

We współczesnym C++ często preferuje się *anonimową przestrzeń nazw* dla lokalnych szczegółów implementacji:

```
// implementation.cpp
namespace {
    int cache_size = 100;
    int helper(int x) { return x + 1; }
}
```

Czego unikać? Nie definiuj `static` zmiennej w nagłówku z zamiarem stworzenia *jednego* wspólnego obiektu. Każda TU otrzyma swoją instancję. Jeżeli celem jest wspólny obiekt w nagłówku, rozważ `inline` (od C++17), opisane dalej.

3.5.2 Zmienna lokalna: czas przechowywania statyczny

Wewnątrz funkcji `static` nie nadaje nazwie łączności zewnętrznej ani wewnętrznej: lokalna nazwa nadal nie ma łączności. Zmienia natomiast czas przechowywania obiektu. Inicjalizacja lokalnej zmiennej statycznej, jeśli nie została wykonana wcześniej, następuje przy pierwszym przejściu sterowania przez jej deklarację.

```
int next_id() {
    static int counter = 0;
    return ++counter;
}

// next_id() returns 1, then 2, then 3, ...
```

Od C++11 jednorazowa inicjalizacja lokalnej zmiennej statycznej jest bezpieczna wobec współbieżności: tylko jeden wątek przeprowadza inicjalizację, a pozostałe odpowiednio czekają. *Nie wynika z tego*, że późniejsze modyfikacje są bezpieczne. Przy wywołaniach równoległych powyższe `++counter` może powodować wyścig danych.

```
#include <atomic>

int next_id_thread_safe() {
    static std::atomic<int> counter{0};
    return counter.fetch_add(1, std::memory_order_relaxed) + 1;
}
```

Kiedy stosować? Lokalna `static` jest dobra dla stanu należącego do funkcji lub obiektu tworzonego leniwie przy pierwszym użyciu. Unikaj jej, jeśli miałaby ukrywać zależności, utrudniać testy lub tworzyć współdzielony mutowalny stan bez synchronizacji.

3.5.3 Składowa klasy: wspólna dla wszystkich obiektów

Statyczne pola klasy należą do klasy, a nie do każdej instancji oddzielnie. Statyczna metoda nie ma wskaźnika `this`; dlatego nie może bezpośrednio odwoływać się do niestatycznych pól.

```cpp
#include <cstddef>

struct Config {
    inline static int attempts = 0;          // C++17
    static constexpr std::size_t limit = 256; // inline since C++17

    static void record_attempt() {
        ++attempts;
    }
};

void example() {
    Config::record_attempt();
    auto n = Config::limit;
    (void)n;
}
```

Przed C++17 zwykłe statyczne pole niestałe deklarowano w klasie, a definiowano raz poza nią:

```cpp
// counter.hpp
struct Counter {
    static int value;
};

// counter.cpp
#include "counter.hpp"
int Counter::value = 0;
```

Od C++17 `inline static` pozwala na definicję z inicjalizacją bezpośrednio w klasie. Pole `static constexpr` jest od C++17 niejawnie `inline` i nie wymaga osobnej definicji poza klasą. W starszych standardach dodatkowa definicja bywała potrzebna przy *odr-use*, np. przy pobieraniu adresu.

3.6 `extern`: deklaracja bytu zdefiniowanego gdzie indziej

Typowy wzorzec to deklaracja w pliku nagłówkowym oraz *jedna* definicja w pliku źródłowym. `extern` w takiej deklaracji wskazuje, że nazwa może odnosić się do bytu z łącznością zewnętrzną; nie alokuje nowego obiektu.

```cpp
// config.hpp
#pragma once
extern int connection_count;
extern const int max_connections;

// config.cpp
#include "config.hpp"
int connection_count = 0;
const int max_connections = 64;

// main.cpp
#include "config.hpp"
void connect() {
    if (connection_count < max_connections)
        ++connection_count;
}
```

Ponieważ deklaracja w `config.cpp` zawiera wcześniejsze `extern`, definicja tej zmiennej ma łączność zewnętrzną. Natomiast samodzielna definicja zmiennej typu `const int` na poziomie przestrzeni nazw ma domyślnie łączność wewnętrzną (w klasycznym kodzie bez modułów).

Uwaga na definicje. Zapis `extern int x = 5;` *jest definicją*, gdy występuje na poziomie przestrzeni nazw: inicjalizator zmienia znaczenie deklaracji. Dlatego nie umieszczamy takiego zapisu w zwykłym nagłówku dołączanym do wielu TU.

3.6.1 Dlaczego `extern const` nie zastąpi `constexpr`?

```
// constants.hpp
extern const int array_size;

// constants.cpp
#include "constants.hpp"
const int array_size = 16;

// client.cpp
#include "constants.hpp"
// std::array<int, array_size> a;  // invalid: not constant here
```

Inna TU nie widzi inicjalizatora, więc nie może wykorzystywać tej zmiennej jako znanej stałej szablonowej. Jeśli wartość musi być stała czasu kompilacji i wspólna w wielu TU, od C++17 zwykle wybieramy `inline constexpr` w nagłówku.

3.6.2 `extern "C"`: łączność językowa

Jest to inne zastosowanie słowa `extern`: określa *łączność językową*, np. przy współpracy z biblioteką C.

```
extern "C" int c_library_function(int);
```

Nie oznacza ono, że wywołanie funkcji ma semantykę języka C; przede wszystkim dotyczy zgodności łączenia i typu funkcji.

3.7 `inline`: dozwolone definicje w wielu TU

Historycznie `inline` sugerowało wstawienie ciała funkcji w miejsce wywołania, ale *nie wymusza* takiej optymalizacji. Najważniejsza współczesna rola polega na odpowiednich regułach jednej definicji (*One Definition Rule*, ODR).

3.7.1 Funkcje `inline`

Definicja funkcji `inline` może znaleźć się w wielu TU, np. przez dołączenie nagłówka, jeśli spełnione są wymagania ODR, w szczególności zgodność definicji. Dla funkcji `inline` o łączności zewnętrznej wszystkie takie definicje odnoszą się do jednej funkcji (w tym do tego samego adresu).

```
// math.hpp
#pragma once
inline int add(int a, int b) {
    return a + b;
}
```

Gdybyśmy usunęli `inline` i dołączyli nagłówek do dwóch `.cpp`, mielibyśmy niedozwolone wielokrotne definicje zwykłej funkcji o łączności zewnętrznej.

Funkcje zdefiniowane w obrębie definicji klasy są zazwyczaj niejawnie `inline` (z zastrzeżeniem szczególnych reguł modułów C++20). Funkcje `constexpr` i `consteval` też są niejawnie `inline`

przy pierwszej deklaracji.

3.7.2 Zmienne `inline` (od C++17)

Zmienna `inline` zdefiniowana w nagłówku może oznaczać *jeden wspólny obiekt* w wielu TU:

```
// state.hpp
#pragma once
#include <atomic>

inline std::atomic<int> requests{0};
inline constexpr int buffer_size = 4096;

inline int register_request() {
    return requests.fetch_add(1, std::memory_order_relaxed) + 1;
}
```

To praktyczna podstawa bibliotek *header-only*. Na poziomie przestrzeni nazw zmienna `inline constexpr` ma zazwyczaj łączność zewnętrzną. Sama zmienna `constexpr` ma zaś domyślnie łączność wewnętrzną.

Ważne: `static inline` to nie to samo. Na poziomie przestrzeni nazw `static inline int x = 0;` ma łączność wewnętrzną: każda TU posiada własne `x`. Dodanie `inline` nie zmienia tego w jeden globalny obiekt.

3.7.3 Reguła ODR w praktyce

- Zwykła funkcja lub zmienna o łączności zewnętrznej: zasadniczo jedna definicja w całym programie.
- Funkcja `inline`, zmienna `inline`, a także odpowiednio zdefiniowane szablony: dozwolone definicje w wielu TU, z dodatkowymi wymaganiami co do ich identyczności.
- Sama deklaracja w nagłówku jest zwykle bezpieczna; problem dotyczy definicji.

3.8 Łączenie specyfikatorów: najważniejsze przypadki

Poniższe zapisy nie są wzajemnie zamienne:

```
const int a = 7;                // const; namespace: internal linkage
constexpr int b = 7;            // compile-time const; internal linkage
inline constexpr int c = 7;     // C++17: shared across TUs
static int d = 7;               // namespace: internal linkage
static constexpr int e = 7;     // compile-time; internal linkage
inline int f = 7;               // C++17: one shared object
extern int g;                   // declaration, definition elsewhere
extern const int h;             // declaration, definition elsewhere
constinit int i = 7;            // C++20: static init, mutable
static constinit int j = 7;     // C++20: internal linkage
```

W powyższym przykładzie zakładamy, że wszystkie deklaracje znajdują się na poziomie przestrzeni nazw, poza modułami nazwanymi. W rzeczywistym projekcie `g` i `h` muszą mieć definicje, jeśli są odpowiednio używane; nie należy też kopiować wielu definicji `i` do nagłówków.

Konstrukcja	Najważniejsza cecha	Typowe zastosowanie
`const`	Brak modyfikacji przez dany dostęp	Parametry, referencje, stałe wartości
`constexpr`	Możliwość użycia wyniku jako wyrażenia stałego	Parametry szablonów, rozmiary, funkcje obliczeniowe
`consteval`	Wymagana stała ewaluacja wywołania	Walidacja i generowanie stałych przy kompilacji
`constinit`	Zakaz inicjalizacji dynamicznej	Mutowalny stan o inicjalizacji statycznej
`static` lokalne	Jeden obiekt o statycznym czasie przechowywania	Leniwy stan funkcji
`static` globalne	Łączność wewnętrzna	Szczegóły implementacji w `.cpp`
`static` w klasie	Składowa wspólna dla instancji	Liczniki, konfiguracja klasy
`extern`	Deklaracja nazwy odnoszącej się do innej definicji	Podział `.hpp`/`.cpp`
`inline`	Szczególne reguły wielokrotnych definicji	Funkcje i zmienne w nagłówkach

3.9 `thread_local`: osobny obiekt dla każdego wątku

`thread_local` oznacza wątkowy czas przechowywania. Każdy wątek posiada własną instancję obiektu, więc modyfikacje tej instancji nie są współdzielone z innymi wątkami.

```
int next_local_id() {
    thread_local int id = 0;
    return ++id;
}
```

Różnica względem `static int id = 0;` jest istotna: `static` daje jedną instancję współdzieloną przez wątki, `thread_local` — po jednej na wątek.

Na poziomie przestrzeni nazw można użyć `static thread_local` (dla łączności wewnętrznej) albo `extern thread_local` (w deklaracji obiektu zdefiniowanego w innym TU).

Kiedy stosować? Dla stanu rzeczywiście związanego z wątkiem, np. lokalnego generatora lub bufora roboczego. Nie jest to ogólny zamiennik blokad i nie nadaje się do danych, które muszą być wspólne między wątkami.

3.10 `mutable` i `volatile`: dwa inne rodzaje kwalifikacji

3.10.1 `mutable`: kontrolowana modyfikacja wewnątrz obiektu stałego

`mutable` dotyczy niestatycznych pól klasy. Pozwala modyfikować takie pole nawet w metodzie `const`. Typowy przypadek to blokada, która jest szczegółem technicznym i nie stanowi logicznego stanu obiektu:

```
#include <mutex>

class SafeCounter {
    mutable std::mutex mutex_;
    int value_ = 0;

public:
```

```
    void increment() {
        std::lock_guard<std::mutex> lock(mutex_);
        ++value_;
    }

    int value() const {
        std::lock_guard<std::mutex> lock(mutex_);
        return value_;
    }
};
```

Nie stosuj `mutable` tylko po to, aby obchodzić prawidłowo zaprojektowany interfejs `const`. Sam `mutable` nie zapewnia bezpieczeństwa wątkowego.

3.10.2 `volatile`: nie jest mechanizmem synchronizacji

`volatile` służy do określonych sytuacji, w których dostępy do obiektu mają być obserwowalnymi operacjami (np. w pewnych zastosowaniach niskopoziomowych). Nie zapewnia atomowości, porządkowania pamięci ani braku wyścigów danych w C++.

```
// Not a thread-safe counter:
volatile int wrong_counter = 0;

// For concurrent increments, use e.g.:
#include <atomic>
std::atomic<int> correct_counter{0};
```

Dla synchronizacji wątków stosuj `std::atomic`, muteksy i inne narzędzia współbieżności, a nie `volatile`.

3.11 `auto` i `decltype(auto)`: dedukcja typu

We współczesnym C++ `auto` w deklaracji zmiennej z inicjalizatorem włącza dedukcję typu. Dedukcja przez wartość odrzuca najwyższy poziom `const` oraz referencyjność wyrażenia.

Natomiast `decltype(auto)` zachowuje semantykę `decltype`.

```
int& shared_value() {
    static int n = 10;
    return n;
}

void example() {
    const int k = 4;
    auto a = k;             // int (copy)
    const auto b = k;       // const int

    auto x = shared_value();           // int: copy
    decltype(auto) y = shared_value(); // int&: reference
    y = 20;                            // updates shared state
    (void)a;
    (void)b;
    (void)x;
}
```

Dedukcja typu z `auto` poprawia czytelność, zwłaszcza przy iteratorach i rozbudowanych typach. Nie należy jednak niezauważenie tracić referencji, gdy kod wymaga operowania na oryginalnym obiekcie. Wtedy używaj np. `auto&` lub `const auto&`.

3.12 Powiązane specyfikatory metod i funkcji

Nie są to specyfikatory czasu przechowywania, lecz często występują obok poprzednich słów kluczowych:

```cpp
#include <cstddef>

struct Base {
    virtual ~Base() = default;
    virtual int evaluate() const noexcept { return 1; }
};

struct Derived final : Base {
    int evaluate() const noexcept override { return 2; }
};

struct Index {
    explicit Index(std::size_t x) : value(x) {}
    std::size_t value;
};

[[nodiscard]] int compute_result() { return 42; }
```

- `explicit` blokuje wybrane niejawne konwersje przez konstruktory i operatory konwersji;
- `virtual` umożliwia dynamiczny polimorfizm, `override` pozwala sprawdzić, czy metoda rzeczywiście nadpisuje metodę bazową, a `final` zabrania dalszego nadpisania metody lub dziedziczenia po klasie;
- `noexcept` deklaruje funkcję nierzucającą wyjątków (naruszenie tej gwarancji prowadzi do `std::terminate`);
- `[[nodiscard]]` pozwala kompilatorowi ostrzegać, gdy wynik został zignorowany.

  Atrybut `[[maybe_unused]]` pozwala ograniczyć ostrzeżenia o nieużywanych bytach.

3.13 Praktyczna organizacja plików: przykład całościowy

Rozważmy licznik wspólny dla programu, stała kompilacyjna oraz funkcję dostępną z kilku TU:

```cpp
// statistics.hpp
#pragma once
#include <atomic>
#include <cstddef>

inline constexpr std::size_t block_size = 1024;
extern std::atomic<unsigned> total_jobs;

inline unsigned completed_jobs() {
    return total_jobs.load(std::memory_order_relaxed);
}
```

```cpp
// statistics.cpp
#include "statistics.hpp"

std::atomic<unsigned> total_jobs{0};
```

```cpp
// worker.cpp
#include "statistics.hpp"
```

```
3
4  void do_one_job() {
5      total_jobs.fetch_add(1, std::memory_order_relaxed);
6  }
```

```
1  // main.cpp
2  #include "statistics.hpp"
3  #include <iostream>
4
5  void do_one_job();
6
7  int main() {
8      do_one_job();
9      std::cout << block_size << " " << completed_jobs() << '\n';
10 }
```

Kompilacja przykładowa:

```
1  g++ -std=c++20 -Wall -Wextra -pedantic \
2      main.cpp worker.cpp statistics.cpp -o program
```

Mamy tu jedną definicję `total_jobs` w `statistics.cpp`, jedną stałą `block_size` udostępnioną przez `inline constexpr`, a także legalną definicję funkcji `completed_jobs` w nagłówku. Użycie atomików sprawia, że sam licznik może być bezpiecznie aktualizowany przez wiele wątków; `memory_order_relaxed` wystarcza tutaj wyłącznie do atomowego zliczania, bez dodatkowej synchronizacji innych danych.

3.14 Najczęstsze błędy i reguły wyboru

3.14.1 Błędy

1. **Definicja zwykłej zmiennej w nagłówku:** zapis `int counter = 0;` powoduje wielokrotne definicje, jeśli nagłówek włączymy do kilku TU. Użyj `extern` i jednej definicji w `.cpp` albo `inline` (C++17).
2. **`static` w nagłówku jako wspólny stan:** w rzeczywistości tworzy osobny stan w każdej TU.
3. **Utożsamianie `const` z `constexpr`:** obiekt `const` może być inicjalizowany w runtime.
4. **Utożsamianie `constexpr` z `consteval`:** zwykłą funkcję `constexpr` wolno wywoływać w runtime.
5. **Utożsamianie `inline` z optymalizacją:** kompilator może wstawić lub nie wstawić ciała dowolnej funkcji, zgodnie z dozwolonymi optymalizacjami.
6. **Utożsamianie `static` z synchronizacją:** bezpieczna inicjalizacja lokalnego statyka nie zabezpiecza późniejszych zapisów.
7. **Używanie `volatile` zamiast `atomic`:** `volatile` nie eliminuje wyścigów danych.
8. **Nadmierne korzystanie z globalnego mutowalnego stanu:** nawet poprawny pod względem ODR program może być trudny do testowania oraz podatny na problemy kolejności inicjalizacji.

3.14.2 Kolejność inicjalizacji obiektów globalnych

Inicjalizacja dynamiczna obiektów o statycznym czasie przechowywania w różnych TU może mieć nieoczywistą kolejność. Jest to źródło *static initialization order fiasco*. Jedną z praktycznych technik jest przeniesienie zależności do lokalnego obiektu statycznego, inicjalizowanego przy pierwszym wywołaniu:

```
#include <string>

const std::string& application_name() {
    static const std::string name = "MyApplication";
    return name;
}
```

Gdy możliwe jest *stałe inicjalizowanie*, preferuj proste `constexpr` lub `constinit`. Pamiętaj jednak, że `constinit` wyklucza inicjalizację dynamiczną, lecz nie narzuca niemutowalności obiektu.

3.14.3 Zalecenia w skrócie

- Potrzebujesz niemutowalnego dostępu? Użyj `const`.
- Potrzebujesz wyrażenia stałego? Użyj `constexpr`.
- Wywołanie musi być stałe? Rozważ `consteval`.
- Mutowalny obiekt ma wymagać inicjalizacji statycznej? Rozważ `constinit`.
- Chcesz ukryć szczegół implementacji w `.cpp`? Rozważ anonimową przestrzeń nazw.
- Potrzebujesz stanu zachowywanego między wywołaniami? Rozważ lokalną `static`, pamiętając o wątkach.
- Potrzebujesz jednego symbolu zdefiniowanego w `.cpp`? Deklaruj go w nagłówku przez `extern`.
- Potrzebujesz definicji w nagłówku współdzielonej przez TU? Użyj `inline`, jeśli jest odpowiednie; dla stałych kompilacyjnych `inline constexpr`.
- Potrzebujesz oddzielnego stanu wątku? Użyj `thread_local`.
- Potrzebujesz komunikacji i synchronizacji między wątkami? Użyj atomików lub blokad, a nie `volatile`.

Konkluzja. Najważniejsze pytanie nie brzmi "które słowo kluczowe jest szybsze?", lecz: czy deklaracja ma określać *czas życia*, *łączność*, *niemutowalność*, *stałą ewaluację*, czy *dopuszczalność wielu definicji*? Dopiero wtedy wybór specyfikatora jest jednoznaczny.

Rozdział 4

Wskaźniki, zarządzanie pamięcią i inteligentne wskaźniki w C++

4.1 Po co istnieją wskaźniki?

Wskaźnik (ang. *pointer*) przechowuje wartość pozwalającą odwołać się do obiektu albo funkcji. Typowy wskaźnik do obiektu ma postać `T*`. W odróżnieniu od obiektu typu `T`, wskaźnik może wskazywać na inny obiekt, może być pusty, a samo wskazywanie *nie oznacza posiadania wskazywanego obiektu*.

Wskaźniki wykorzystuje się między innymi do:

- przekazywania i modyfikowania istniejących obiektów bez ich kopiowania;
- opisywania opcjonalnej relacji "obiekt istnieje albo go nie ma";
- implementowania dynamicznych struktur danych, takich jak listy, drzewa i grafy;
- realizowania polimorfizmu i dynamicznego wiązania;
- współpracy z interfejsami C, sterownikami i bibliotekami systemowymi;
- kontrolowania czasu życia zasobów za pomocą `std::unique_ptr`, `std::shared_ptr` i `std::weak_ptr`.

Najważniejsza zasada. W nowoczesnym C++ wskaźnik `T*` traktujemy przede wszystkim jako *nieposiadający dostęp* do obiektu (*observer*). Odpowiedzialność za czas życia obiektu powinna być reprezentowana osobno i jednoznacznie. Najczęściej wystarcza obiekt przechowywany bezpośrednio, kontener standardowy albo `std::unique_ptr`.

4.2 Podstawy: adres, dereferencja i `nullptr`

4.2.1 Deklarowanie wskaźników

Operator `&` zastosowany do obiektu daje jego adres, natomiast operator jednoargumentowy `*` służy do dereferencji (dostępu do wskazywanego obiektu):

```
1  #include <iostream>
2
3  int main() {
4      int x = 10;
5      int* p = &x;            // p points to x
6
```

```
    std::cout << x << '\n';  // 10
    std::cout << *p << '\n'; // 10

    *p = 25;                 // modifies x
    std::cout << x << '\n';  // 25
}
```

W deklaracji `int* p` gwiazdka należy do deklaratora, nie jest operacją dereferencji. Nie należy też utożsamiać wskaźnika z samym obiektem: `p` jest osobnym obiektem o własnej wartości i adresie.

```
int x = 7;
int* p = &x;
int** pp = &p;  // pointer to a pointer

**pp = 9;       // changes x
```

Zapis `int* a, b;` deklaruje wskaźnik `a` oraz zwykłą zmienną `b`. Dla czytelności lepiej deklarować je oddzielnie.

4.2.2 Wskaźnik pusty i poprawna dereferencja

Pusty wskaźnik zapisujemy za pomocą `nullptr` (od C++11). Nie wolno go dereferencjonować. Samo porównanie z `nullptr` jest bezpieczne:

```
int* p = nullptr;

if (p != nullptr) {
    *p = 5;
}

if (p) {                 // equivalent non-null test
    *p = 6;
}
```

`nullptr` ma osobny typ `std::nullptr_t`; w odróżnieniu od historycznych zapisów `0` i `NULL` nie jest zwykłą liczbą całkowitą. Pomaga to przy przeciążaniu funkcji:

```
void f(int);
void f(int*);

f(0);         // calls f(int)
f(nullptr);   // calls f(int*)
```

Uwaga o wskaźnikach niezainicjalizowanych. Lokalne `int* p;` bez inicjalizacji ma nieokreśloną wartość; korzystanie z niej może prowadzić do niezdefiniowanego zachowania. Zawsze inicjalizuj wskaźnik poprawnym adresem lub `nullptr`.

4.2.3 Dostęp do składowych: . i ->

Operator `.` wybiera składową obiektu, natomiast `->` wybiera składową przez wskaźnik:

```
#include <string>

struct User {
    std::string name;
    int score = 0;
};
```

```
7
8  User u{"Anna", 12};
9  User* p = &u;
10
11 u.score = 13;
12 p->score = 14;       // same as (*p).score = 14
```

Nawiasy w `(*p).score` są potrzebne ze względu na priorytety operatorów. Zapis `*p.score` nie oznacza tego samego.

4.3 Wskaźniki, referencje i kwalifikator `const`

4.3.1 Kiedy wskaźnik, a kiedy referencja?

Referencja `T&` jest zwykle odpowiednia, gdy obiekt musi istnieć i parametr nie jest opcjonalny. Wskaźnik `T*` nadaje się, gdy brak obiektu jest dopuszczalny lub gdy chcemy zmieniać obiekt, na który wskazujemy.

```
1  void increase(int& x) { ++x; }
2
3  void increase_if_present(int* x) {
4      if (x) { ++*x; }
5  }
6
7  int n = 3;
8  increase(n);              // n == 4
9  increase_if_present(&n); // n == 5
10 increase_if_present(nullptr); // no operation
```

Referencja również może stać się *wiszącą*, gdy obiekt, do którego się odnosi, przestanie istnieć. Typ `T&` nie gwarantuje samodzielnie, że czas życia obiektu jest wystarczająco długi.

4.3.2 Co jest stałe: wskaźnik czy obiekt?

```
1  int a = 1;
2  int b = 2;
3
4  const int* p = &a;       // read-only access through p
5  p = &b;                  // OK: p can point elsewhere
6  // *p = 3;               // error
7
8  int* const q = &a;       // q cannot be reassigned
9  *q = 3;                  // OK: a is mutable
10 // q = &b;               // error
11
12 const int* const r = &a; // neither reassignment nor write
13 int const* s = &a;       // same type as const int*
```

Reguła praktyczna: `const` po `*` dotyczy samego wskaźnika, a przed `*` — wskazywanego typu.

Pozorna niemutowalność. Wskaźnik `const T*` nie pozwala modyfikować obiektu *przez ten wskaźnik*. Nie oznacza, że obiekt nie może ulec zmianie inną drogą:

```
1  int x = 10;
2  const int* read_only = &x;
3  x = 20;                  // allowed
4  // *read_only = 30;      // not allowed
```

4.4 Czas życia obiektu i wskaźniki wiszące

4.4.1 Adres nie oznacza prawa dostępu

Wskaźnik może przechowywać adres miejsca, w którym obiekt już nie istnieje. Jest wtedy wskaźnikiem *wiszącym* (ang. *dangling pointer*). Dereferencja takiego wskaźnika prowadzi do niezdefiniowanego zachowania (*undefined behavior*, UB).

```
int* bad() {
    int local = 42;
    return &local;       // WRONG: local dies at return
}

// int x = *bad();       // UB: dangling pointer
```

Prawidłowo można zwrócić samą wartość:

```
int good() {
    int local = 42;
    return local;        // safe value return
}
```

4.4.2 Unieważnianie wskaźników przez kontenery

Nie każdy wskaźnik staje się wiszący przez wyjście z funkcji. Relokacja elementów `std::vector` także może unieważnić wskaźniki, referencje i iteratory do tych elementów:

```
#include <vector>

std::vector<int> values{10, 20};
int* p = &values[0];

values.reserve(100);     // may reallocate storage
// *p = 50;              // unsafe if reallocation occurred
p = &values[0];          // reacquire a valid pointer
```

Po zmianie pojemności wymagającej realokacji adresy elementów się zmieniają. W ogólności po operacji mogącej unieważnić wskaźnik należy pobrać go ponownie, a nie próbować odgadnąć, czy nadal działa.

4.4.3 Trzy niezależne pytania

Przy każdym wskaźniku rozstrzygaj:

1. Czy wartość wskaźnika jest poprawna (np. nie jest niezainicjalizowana)?
2. Czy obiekt, na który wskazujemy, nadal istnieje?
3. Czy operacja jest dozwolona dla danego typu i zakresu pamięci (np. czy nie wychodzi poza tablicę)?

Sam test `p != nullptr` rozwiązuje co najwyżej fragment pierwszego problemu, ale *nie* wykrywa wiszących wskaźników.

4.5 Tablice, arytmetyka wskaźników i `std::span`

4.5.1 Arytmetyka wskaźników

Dla wskaźnika do elementu tablicy operacje `p + k` i `p[k]` odnoszą się do jej elementów, pod warunkiem że poruszamy się w dozwolonym zakresie. Można utworzyć wskaźnik *jeden za końcem* tablicy, ale nie wolno go dereferencjonować.

```
int a[]{4, 8, 12};
int* p = a;          // points to a[0]

int x = *(p + 1);    // 8
int y = p[2];       // 12
int* end = a + 3;   // valid one-past-the-end pointer
// int z = *end;     // UB
```

Dodawanie i odejmowanie wskaźników ma sens w obrębie *tej samej* tablicy (z uwzględnieniem pozycji jeden za końcem). Nie należy wykorzystywać takich działań do przemieszczania się między dowolnymi obiektami w pamięci.

4.5.2 Tablica to nie wskaźnik

Typ `int[3]` i typ `int*` są różne, choć w wielu wyrażeniach tablica jest automatycznie konwertowana na wskaźnik do pierwszego elementu (tzw. *array-to-pointer decay*).

```
int a[3] = {1, 2, 3};
int* p = a;

static_assert(sizeof(a) == 3 * sizeof(int));
// sizeof(p) is the size of a pointer, not of the array
```

Funkcja `void f(int* p)` nie zna długości tablicy. Jeśli trzeba pracować z ciągłym fragmentem pamięci o znanym rozmiarze, C++20 udostępnia `std::span`:

```
#include <span>
#include <vector>

int sum(std::span<const int> xs) {
    int result = 0;
    for (int x : xs) {
        result += x;
    }
    return result;
}

int a[]{1, 2, 3};
std::vector<int> v{4, 5, 6};

int s1 = sum(a);  // 6
int s2 = sum(v);  // 15
```

`std::span<T>` przechowuje widok ciągłych danych (z informacją o liczbie elementów), ale *nie jest właścicielem* pamięci. Gdy dane znikną albo przeniosą się w inne miejsce, `span` również może stać się niepoprawny. Dla większości dynamicznych tablic własnościowych preferuj `std::vector<T>`.

4.6 `void*`, konwersje i rzutowania wskaźników

4.6.1 `void*`: adres bez informacji o typie

Wskaźnik `void*` może przechowywać adres obiektu bez informacji o jego konkretnym typie. Bywa używany w interfejsach C i bibliotekach niskopoziomowych, ale nie pozwala na bezpośrednią dereferencję:

```
int x = 42;
void* opaque = &x;              // type erased
// *opaque = 7;                 // error: cannot dereference void*

int* p = static_cast<int*>(opaque);
*p = 7;                         // valid: original object is int
```

Przywrócenie błędnego typu i późniejsza próba użycia obiektu mogą prowadzić do niezdefiniowanego zachowania. Jeśli typ jest znany, zwykle lepiej przechowywać od razu `T*`.

4.6.2 `static_cast`, `dynamic_cast` i `reinterpret_cast`

- `static_cast<T*>` nadaje się m.in. do poprawnych konwersji pomiędzy `void*` a wskaźnikiem obiektowym, jeśli rzeczywisty typ i wyrównanie na to pozwalają.
- `dynamic_cast<T*>` służy do sprawdzanych w czasie wykonania konwersji w hierarchiach klas polimorficznych. Niepowodzenie konwersji wskaźnika zwraca `nullptr`.
- `reinterpret_cast<T*>` umożliwia niskopoziomową reinterpretację reprezentacji wskaźnika, ale sama konwersja nie uprawnia do dowolnej dereferencji ani nie tworzy obiektu odpowiedniego typu.

```
struct Base {
    virtual ~Base() = default;
};

struct Derived : Base {
    int value = 12;
};

Derived d;
Base* base = &d;

if (Derived* p = dynamic_cast<Derived*>(base)) {
    p->value = 30;          // checked downcast succeeded
}
```

Przy przekazywaniu parametrów funkcji nie używaj rzutowań jako sposobu na obejście `const`, sprawdzania czasu życia ani zasad własności. Dobrze zaprojektowany interfejs zazwyczaj w ogóle nie wymaga rzutowania wskaźników.

4.7 Pamięć dynamiczna: `new`, `delete` i ich ograniczenia

4.7.1 Klasyczny mechanizm

Operator `new` tworzy obiekt w pamięci przydzielonej dynamicznie i zwraca wskaźnik. Odpowiadający mu `delete` niszczy obiekt i zwalnia pamięć:

```cpp
int* p = new int(42);
// use *p

delete p;
p = nullptr;           // avoids this one stale alias
```

Dla tablic obowiązuje para `new[]` i `delete[]`:

```cpp
int* a = new int[5]{}; // zero-initialized elements
// use a[0], ..., a[4]

delete[] a;
```

Nie wolno łączyć `new[]` z `delete` ani zwykłego `new` z `delete[]`. Ponowne `delete` tego samego żywego przydziału również jest błędem. `delete nullptr;` jest natomiast bezpieczne.

4.7.2 Dlaczego ręczne zarządzanie pamięcią jest niebezpieczne?

```cpp
void fragile() {
    int* p = new int(42);
    operation_that_may_throw();
    delete p;   // skipped if the operation throws
}
```

Jeśli funkcja zgłosi wyjątek, pamięć zostaje utracona (*memory leak*). Analogiczny problem powstaje przy wielu instrukcjach `return`, kopiowaniu wskaźników i niespójnym przekazywaniu odpowiedzialności za `delete`.

Zalecenie. W typowym kodzie aplikacyjnym nie pisz bezpośrednio `new`/`delete`. Użyj obiektu lokalnego, `std::vector<T>` albo inteligentnego wskaźnika. Ręczne operacje alokowania pozostają potrzebne w implementacjach niskopoziomowych, alokatorach i przy integracji z określonym API.

4.8 RAII i semantyka własności

4.8.1 RAII: zasób należy do obiektu

Resource Acquisition Is Initialization (RAII) oznacza powiązanie przejęcia zasobu z utworzeniem obiektu zarządzającego, a zwolnienia zasobu z jego destruktorem. Stosuje się je nie tylko do pamięci, ale też do plików, blokad, gniazd sieciowych i uchwytów systemowych.

```cpp
#include <fstream>
#include <string>

void save(const std::string& path) {
    std::ofstream file(path);
    file << "example\n";
} // file is closed by its destructor
```

Destruktor uruchamia się podczas normalnego opuszczania zakresu oraz podczas odwijania stosu po wyjątku. Stąd RAII upraszcza bezpieczeństwo wyjątkowe.

4.8.2 Trzy rodzaje relacji z obiektem

Własność wyłączna (*unique ownership*). Dokładnie jeden obiekt zarządzający odpowiada za zniszczenie zasobu: najczęściej `std::unique_ptr<T>`.

Własność współdzielona (*shared ownership*). Wielu właścicieli może utrzymywać jeden zasób przy życiu: `std::shared_ptr<T>`.

Dostęp bez własności (*non-owning access*). Użytkownik widzi obiekt, ale nie przedłuża jego życia: zwykły wskaźnik, referencja, `std::span<T>` lub `std::weak_ptr<T>` (ten ostatni obserwuje stan zarządzany przez `shared_ptr` i pozwala warunkowo uzyskać właściciela).

Dobór reprezentacji wynika z *kontraktu własności*, a nie z samego faktu, że obiekt znajduje się w pamięci dynamicznej.

4.9 `std::unique_ptr`: jeden właściciel

4.9.1 Tworzenie obiektu i automatyczne niszczenie

`std::unique_ptr<T>` (nagłówek `<memory>`, C++11) zarządza pojedynczym obiektem przez wyłączną własność. Od C++14 najczęściej tworzymy go przez `std::make_unique<T>`:

```
#include <memory>
#include <string>

#include <utility>

struct Person {
    explicit Person(std::string n) : name(std::move(n)) {}
    std::string name;
};

void example() {
    auto p = std::make_unique<Person>("Ala");
    p->name = "Ola";
    // no delete needed
} // destructor destroys Person
```

`unique_ptr` można traktować jako obiekt zarządzający: jego destruktor wywołuje odpowiednią funkcję usuwającą zasób (*deleter*). Domyślnie jest nią `std::default_delete<T>`.

4.9.2 Brak kopiowania i przenoszenie własności

Wskaźnik o własności wyłącznej nie może być kopiowany. Może natomiast zostać *przeniesiony* za pomocą `std::move`:

```
#include <memory>
#include <utility>

auto first = std::make_unique<int>(10);
// auto bad = first;                  // error: copy

auto second = std::move(first);      // transfers ownership
// first == nullptr
*second = 20;
```

`std::move` samo niczego nie zwalnia i nie przenosi fizycznie wskazywanego obiektu. Zwraca wyrażenie pozwalające wywołać konstruktor lub przypisanie przenoszące. W tym przypadku przenoszona jest *własność wskaźnika*, a obiekt typu `int` nie zmienia adresu.

4.9.3 Przekazywanie do funkcji: kto jest właścicielem?

Rozróżnijmy trzy interfejsy:

```
#include <memory>
#include <utility>

struct Widget { void run() {} };

void inspect(const Widget& w);             // borrows, non-null
void maybe_inspect(const Widget* w);       // borrows, nullable
void take(std::unique_ptr<Widget> w);      // takes ownership

void demo() {
    auto p = std::make_unique<Widget>();
    inspect(*p);                // p still owns the object
    maybe_inspect(p.get());     // p still owns the object
    take(std::move(p));         // ownership transferred
    // p is now empty
}
```

Jeżeli funkcja *jedynie używa* obiektu, przekazywanie przez `T&`, `const T&` lub `T*` jest zwykle czytelniejsze niż przekazywanie `unique_ptr` przez referencję. Parametr `unique_ptr<T>` przyjmowany *przez wartość* sygnalizuje przejęcie własności. Dla funkcji, która ma zastąpić obiekt będący własnością wywołującego, można użyć `std::unique_ptr<T>&`.

4.9.4 Metody `get`, `reset`, `release`, `swap`

```
#include <memory>
#include <utility>

auto p = std::make_unique<int>(7);
int* observer = p.get();       // borrow; do not delete

p.reset(new int(9));           // destroys old int
// observer is now dangling

auto q = std::make_unique<int>(11);
p.swap(q);                    // exchanges managed objects

int* raw = q.release();        // q becomes empty; raw owns now
// responsibility for delete raw is now explicit
delete raw;
```

`get()` zwraca dostęp bez własności. `reset()` zastępuje zarządzany obiekt, zwalniając dotychczasowy. `release()` oddaje surowy wskaźnik *bez zwolnienia obiektu*; dlatego jest potrzebne głównie przy przekazywaniu własności do interfejsu, który tego wymaga. Nie należy mylić `release()` z `reset()`.

4.9.5 Tablice: `unique_ptr<T[]>` a `vector<T>`

```
#include <memory>
#include <vector>

```

```
auto a = std::make_unique<int[]>(5); // array of 5 zeros
a[2] = 17;

std::vector<int> b(5);          // usually more convenient
b.at(2) = 17;                   // checked indexing
```

`unique_ptr<T[]>` dobiera `delete[]` i udostępnia `operator[]`, lecz sam nie przechowuje rozmiaru. Zwykle preferuj `std::vector<T>`, bo posiada rozmiar, iteratory i bogatszy interfejs. Dla tablicy o stałym rozmiarze określonym w kodzie rozważ `std::array<T, N>`.

4.9.6 Własny deleter: zasób inny niż pamięć `new`

Inteligentny wskaźnik może wywoływać dowolną funkcję zwalniającą zasób. Typowym przykładem jest plik w interfejsie C:

```
#include <cstdio>
#include <memory>

struct FileCloser {
    void operator()(std::FILE* file) const noexcept {
        if (file) { std::fclose(file); }
    }
};

using FilePtr = std::unique_ptr<std::FILE, FileCloser>;

void example() {
    FilePtr file(std::fopen("log.txt", "w"));
    if (!file) { return; }

    std::fputs("hello\n", file.get());
} // fclose is called automatically
```

W praktyce deleter bywa osobną strukturą, a jego typ wchodzi w skład typu `unique_ptr<T, Deleter>`. Mechanizm pozwala owinąć RAII również zasoby zwalniane przez `free()`, `fclose()` lub bibliotekę zewnętrzną.

4.9.7 Polimorfizm i wirtualny destruktor

```
#include <memory>

struct Shape {
    virtual ~Shape() = default;
    virtual double area() const = 0;
};

struct Circle : Shape {
    explicit Circle(double r) : radius(r) {}
    double area() const override { return 3.14159 * radius * radius; }
    double radius;
};

std::unique_ptr<Shape> make_shape() {
    return std::make_unique<Circle>(2.0);
}
```

Przy niszczeniu obiektu pochodnego przez wskaźnik do bazy zwykle potrzebny jest wirtualny destruktor bazy. Dotyczy to również przenoszenia własności z klasy pochodnej do bazowej przy

użyciu `unique_ptr` i domyślnego deletera.

4.10 `std::shared_ptr`: współdzielona własność

4.10.1 Jak działa licznik właścicieli?

`std::shared_ptr<T>` (C++11) udostępnia współdzieloną własność zasobu. Zwykle tworzymy go za pomocą `std::make_shared<T>`:

```
#include <memory>

auto a = std::make_shared<int>(10);
auto b = a;             // shared ownership

// a.use_count() == 2 (provided there are no other owners)
b.reset();              // releases b's ownership
// a.use_count() == 1
```

Gdy liczba *silnych właścicieli* spadnie do zera, zarządzany obiekt zostaje zniszczony. Implementacja utrzymuje tzw. *blok kontrolny* (*control block*) zawierający między innymi liczniki właścicieli, informacje potrzebne do zniszczenia zasobu i ewentualny deleter.

Schemat ideowy

```
shared_ptr a ---\
                +--> [ control block: strong = 2 ] --> [ T ]
shared_ptr b ---/           weak = 0
```

`use_count()` ma znaczenie przede wszystkim diagnostyczne. Nie powinno się na podstawie odczytanego licznika podejmować decyzji o bezpieczeństwie wątkowym ani o wyłącznej własności, która może zmienić się równolegle.

4.10.2 Tworzenie i przekazywanie `shared_ptr`

```
#include <memory>

struct Data { int x = 0; };

std::shared_ptr<Data> create() {
    return std::make_shared<Data>();
}

void keep(std::shared_ptr<Data> p) {
    // stores a copy of p somewhere, if needed
}

void read_only(const Data& data) {
    // does not require shared ownership
}

void demo() {
    auto p = create();
    read_only(*p); // borrow
    keep(p);       // creates another temporary shared owner
}
```

Parametr `shared_ptr<T>` przyjmowany przez wartość oznacza, że funkcja otrzymuje własny udział we własności na czas wywołania i ewentualnie może go przechować. Jeśli funkcja wyłącznie czyta

obiekt, zwykle wystarcza `const T&` lub `T*`. Samo przekazanie `const shared_ptr<T>&` unika inkrementacji licznika, ale bardziej wiąże interfejs z modelem własności.

4.10.3 `make_shared` a konstruktor ze wskaźnika

W typowych implementacjach `make_shared` umieszcza obiekt i blok kontrolny w jednym przydziale pamięci, co może ograniczać narzut alokacji. Natomiast konstrukcja `shared_ptr<T>(new T)` zwykle wymaga osobnych przydziałów.

```
auto preferred = std::make_shared<int>(42);
std::shared_ptr<int> also_valid(new int(42));
```

Nie wolno jednak tworzyć dwóch `shared_ptr` niezależnie z *tego samego* surowego wskaźnika:

```
int* raw = new int(42);
std::shared_ptr<int> a(raw);
// std::shared_ptr<int> b(raw); // WRONG: second control block
```

Dwa niezależne bloki kontrolne próbowałyby zwolnić ten sam obiekt. Poprawne współdzielenie powstaje przez *kopiowanie istniejącego* `shared_ptr`: `auto b = a;`.

Kiedy nie stosować `make_shared`? Zintegrowany przydział może zatrzymywać pamięć zajmowaną przez obiekt w przydzielonym bloku do momentu zniszczenia także ostatniego `weak_ptr`, nawet gdy destruktor obiektu już się wykonał. Przy dużych obiektach i długo żyjących słabych referencjach warto rozważyć osobny przydział. Innymi powodami mogą być specjalne wymagania alokacji lub konstruktory niedostępne dla `make_shared`.

4.10.4 Współdzielenie własności a współdzielenie mutacji

Sam `shared_ptr` nie synchronizuje dostępu do pól obiektu:

```
#include <memory>
#include <mutex>

struct Counter {
    std::mutex mutex;
    int value = 0;

    void increment() {
        std::lock_guard<std::mutex> lock(mutex);
        ++value;
    }
};

auto p = std::make_shared<Counter>();
// many threads may own copies of p, but must synchronize
// concurrent modifications of Counter::value
```

Blok kontrolny pozwala różnym egzemplarzom `shared_ptr` bezpiecznie współdzielić własność. Równoczesna modyfikacja *tego samego* egzemplarza wymaga jednak synchronizacji. Od C++20 można użyć `std::atomic<std::shared_ptr<T>`.

4.11 `std::weak_ptr`: obserwacja bez utrzymywania przy życiu

4.11.1 Problem cykli własności

Gdy obiekty wzajemnie posiadają się przez `shared_ptr`, liczniki właścicieli mogą nigdy nie spaść do zera:

```
#include <memory>

struct Node {
    std::shared_ptr<Node> next;
};

void leak_cycle() {
    auto a = std::make_shared<Node>();
    auto b = std::make_shared<Node>();
    a->next = b;
    b->next = a;          // cycle of strong ownership
} // nodes remain alive after both locals are destroyed
```

To nie jest problem *niezerowego wskaźnika*, lecz błędnego modelu własności: każdy węzeł utrzymuje przy życiu drugi.

4.11.2 Przerywanie cyklu i `lock()`

`std::weak_ptr<T>` obserwuje ten sam blok kontrolny, ale nie zwiększa liczby silnych właścicieli:

```
#include <memory>

struct Node {
    std::weak_ptr<Node> parent;         // non-owning
    std::shared_ptr<Node> child;        // owning
};

void example() {
    auto root = std::make_shared<Node>();
    auto leaf = std::make_shared<Node>();

    root->child = leaf;
    leaf->parent = root;

    if (auto parent = leaf->parent.lock()) {
        // parent is a temporary shared owner
        (void)parent;
    }
} // no ownership cycle
```

Metoda `lock()` atomowo próbuje uzyskać `shared_ptr<T>`. Jeżeli obiekt jeszcze istnieje, zwraca właściciela utrzymującego go przy życiu przez czas używania. W przeciwnym razie zwraca pusty `shared_ptr`.

```
std::weak_ptr<int> observer;
{
    auto p = std::make_shared<int>(8);
    observer = p;
    if (auto alive = observer.lock()) {
        *alive += 1;           // safe while alive owns it
    }
}

```

```
// observer.expired() is now true
// observer.lock() now returns an empty shared_ptr
```

W kodzie współbieżnym nie stosuj schematu `if (!w.expired()) ...` z późniejszym odwołaniem: między sprawdzeniem a użyciem obiekt może zniknąć. Preferuj jedną operację `lock()` i posługuj się otrzymanym `shared_ptr`.

4.12 `enable_shared_from_this` i wskaźnik `this`

Gdy metoda obiektu zarządzanego przez `shared_ptr` chce przekazać dalej *współdzieloną własność do siebie*, nie może wykonać `std::shared_ptr<T>(this)`: powstałby drugi, niezależny blok kontrolny.

Można zamiast tego dziedziczyć publicznie po `std::enable_shared_from_this<T>`:

```
#include <memory>

struct Session : std::enable_shared_from_this<Session> {
    std::shared_ptr<Session> self() {
        return shared_from_this();
    }
};

void example() {
    auto session = std::make_shared<Session>();
    auto second_owner = session->self();
    // both share the same control block
}
```

Metodę `shared_from_this()` wolno wywołać dopiero wtedy, gdy obiektem zarządza już `shared_ptr`. Jej wywołanie na obiekcie lokalnym albo w konstruktorze, zanim powstanie właściciel, zgłasza wyjątek `std::bad_weak_ptr`.

Od C++17 dostępna jest również metoda `weak_from_this()`.

4.13 Zaawansowany przykład: wskaźnik aliasujący `shared_ptr`

`shared_ptr` rozróżnia *obiekt zarządzany* (którego czas życia kontroluje blok kontrolny) i *przechowywany wskaźnik* (zwracany przez `get()`). Specjalny konstruktor aliasujący pozwala udostępnić adres składowej, jednocześnie utrzymując przy życiu cały obiekt:

```
#include <memory>

struct Pair {
    int first = 1;
    int second = 2;
};

void example() {
    auto whole = std::make_shared<Pair>();
    std::shared_ptr<int> member(whole, &whole->second);

    whole.reset();       // Pair is still alive
    *member = 20;        // accesses Pair::second safely
} // member destroys the last strong owner
```

Konstruktor aliasujący jest przydatny np. przy udostępnianiu podobiektów, ale wymaga prawidłowej relacji czasów życia. Nie należy go traktować jako sposobu na naprawienie wcześniej wiszącego

wskaźnika.

4.14 Wskaźniki w interfejsach funkcji i klas

4.14.1 Katalog typowych kontraktów

Interfejs	Znaczenie
`void f(const T&)`	Obiekt wymagany, tylko odczyt; brak przekazania własności.
`void f(T&)`	Obiekt wymagany, możliwa modyfikacja; brak przekazania własności.
`void f(const T*)`	Obiekt opcjonalny, tylko odczyt; brak własności.
`void f(T*)`	Obiekt opcjonalny, możliwa modyfikacja; brak własności.
`void f(unique_ptr<T>)`	Przejęcie wyłącznej własności.
`unique_ptr<T> f()`	Przekazanie wyłącznej własności wywołującemu.
`void f(shared_ptr<T>)`	Uzyskanie udziału we współdzielonej własności.
`void f(weak_ptr<T>)`	Możliwość obserwacji obiektu, którego życie kontrolują inni.
`void f(span<const T>)`	Widok na ciągłą sekwencję, bez własności.

W tabeli pominięto przestrzeń nazw `std::` dla czytelności. To *konwencje projektowania API*, a nie automatyczne gwarancje długowieczności surowych wskaźników i referencji.

4.14.2 Dlaczego nie zawsze używać `shared_ptr`?

Rozważmy funkcję, która wyświetla nazwę:

```
#include <iostream>
#include <memory>
#include <string>

struct Product { std::string name; };

void print(const Product& p) {
    std::cout << p.name << '\n';
}

void example() {
    Product local{"book"};
    print(local);                  // no allocation

    auto owned = std::make_unique<Product>(Product{"pen"});
    print(*owned);                 // still no extra owner
}
```

Funkcja `print` nie potrzebuje znać strategii przechowywania obiektu. Przekazanie referencji oddziela *sposób dostępu* od *sposobu zarządzania czasem życia*.

4.15 PImpl: ukrywanie implementacji przez `unique_ptr`

Technika *pointer to implementation* pozwala utrzymać szczegóły implementacji poza publicznym nagłówkiem. Typ implementacyjny może być niekompletny w nagłówku, ale miejsce wykonania domyślnego deletera musi widzieć jego definicję. Dlatego destruktor klasy definiujemy w pliku `.cpp`.

```
// widget.hpp
#pragma once
#include <memory>
```

```

class Widget {
public:
    Widget();
    ~Widget();                 // defined in .cpp

    Widget(Widget&&) noexcept;
    Widget& operator=(Widget&&) noexcept;

    Widget(const Widget&) = delete;
    Widget& operator=(const Widget&) = delete;

    void work();

private:
    struct Impl;
    std::unique_ptr<Impl> impl_;
};
```

```
// widget.cpp
#include "widget.hpp"
#include <memory>

struct Widget::Impl {
    int value = 0;
    void run() { ++value; }
};

Widget::Widget() : impl_(std::make_unique<Impl>()) {}
Widget::~Widget() = default;
Widget::Widget(Widget&&) noexcept = default;
Widget& Widget::operator=(Widget&&) noexcept = default;

void Widget::work() { impl_->run(); }
```

Ten przykład prezentuje semantykę przenoszenia. Po przeniesieniu źródłowy obiekt `Widget` ma pusty `impl_`; nie wolno wywoływać na nim `work()` bez odpowiedniego sprawdzenia. W produkcyjnym API można ten stan jawnie obsłużyć lub określić w kontrakcie.

4.16 Wskaźniki do funkcji i składowych klas

Wskaźniki nie muszą odnosić się do danych. Można wskazywać funkcje, a także składowe klas, choć te ostatnie wymagają innej składni.

4.16.1 Wskaźnik do funkcji

```
int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

using Operation = int(*)(int, int);

int apply(int x, int y, Operation op) {
    return op(x, y);
}

int a = apply(2, 3, &add); // 5
int b = apply(2, 3, &mul); // 6
```

Obiekty wywoływalne różnych typów można przechowywać za pomocą `std::function`. Prosty szablon lub lambda bywają jednak bardziej efektywne.

4.16.2 Wskaźnik do składowej

```
struct Record {
    int value = 0;
    int twice() const { return 2 * value; }
};

int Record::* field = &Record::value;
int (Record::* method)() const = &Record::twice;

Record r{7};
int x = r.*field;          // 7
int y = (r.*method)();     // 14

Record* p = &r;
int z = p->*field;         // 7
```

Wskaźnik do składowej nie jest zwykłym adresem gotowego obiektu. Dopiero w połączeniu z konkretną instancją klasy pozwala odwołać się do odpowiedniej składowej.

4.17 Wskaźniki i wielowątkowość

4.17.1 `shared_ptr` nie eliminuje wyścigów danych

Możliwość bezpiecznego utrzymania czasu życia obiektu nie oznacza, że jego pola można dowolnie zmieniać z wielu wątków. Jeśli dwa wątki zapisują do `*p`, potrzebna jest osobna synchronizacja (np. `std::mutex` albo atomiki).

Od C++20 dostępna jest specjalizacja `std::atomic<std::shared_ptr<T>>`, przydatna do atomowego publikowania nowej wersji współdzielonego obiektu:

```
#include <atomic>
#include <memory>

struct Config {
    int workers = 4;
};

std::atomic<std::shared_ptr<const Config>> current{
    std::make_shared<const Config>(Config{4})
};

void publish(int count) {
    current.store(std::make_shared<const Config>(Config{count}));
}

int read_workers() {
    auto snapshot = current.load(); // keeps snapshot alive
    return snapshot->workers;
}
```

Tu czytelnicy otrzymują stabilny, niemutowalny *snapshot*. Atomowość dotyczy odczytu i zapisu `shared_ptr`, a nie arbitralnych operacji na jego zawartości.

4.18 Najczęstsze błędy zebrane w jednym miejscu

1. **Dereferencja `nullptr` lub wskaźnika niezainicjalizowanego.** Zawsze sprawdzaj kontrakt i inicjalizuj wskaźniki.
2. **Wskaźnik wiszący.** `get()`, `span` i surowa referencja nie przedłużają życia zasobu.
3. **Podwójne `delete`.** Jeden przydział musi mieć jedną, jednoznaczną strategię zwalniania.
4. **Mieszanie `new[]` i `delete`.** Do tablic służy `delete[]`; zwykle lepiej wybrać kontener.
5. **Zwracanie adresu lokalnej zmiennej.** Jej czas życia kończy się przy wyjściu z zakresu.
6. **Przekazywanie surowego wskaźnika z `get()` do nowego `shared_ptr`.** Tworzy niezależny blok kontrolny.
7. **Nieuzasadnione `shared_ptr` wszędzie.** Współdzielenie własności kosztuje i zaciera odpowiedzialność.
8. **Cykl silnych właścicieli.** Przerywaj go przez `weak_ptr` lub przeprojektowanie relacji.
9. **Nadużywanie `release()`.** Po wywołaniu ktoś musi jawnie przejąć odpowiedzialność za zasób.
10. **Brak wirtualnego destruktora w bazie.** Usuwanie obiektu pochodnego przez wskaźnik bazowy z domyślnym deleterem może wówczas prowadzić do UB.
11. **Założenie, że `shared_ptr` chroni obiekt przed wyścigami.** Chroni zarządzanie własnością, nie samą mutację.
12. **Przechowywanie wskaźników do elementów `vector` po realokacji.** Przy operacjach zmieniających bufor pobieraj adresy ponownie.

4.19 Jak wybrać właściwy typ?

Sytuacja	Najczęstszy wybór
Pojedynczy obiekt o prostym czasie życia	Zwykła zmienna `T`
Dynamiczna lista elementów	`std::vector<T>`
Obiekt wymagany tylko do odczytu	`const T&`
Obiekt wymagany i modyfikowany	`T&`
Obiekt opcjonalny bez przekazania własności	`T*` lub `const T*`
Pojedynczy właściciel dynamicznego obiektu	`std::unique_ptr<T>`
Kilku niezależnych właścicieli	`std::shared_ptr<T>`
Obserwacja obiektu współdzielonego	`std::weak_ptr<T>`
Fragment istniejącej tablicy, bez własności	`std::span<T>`
Polimorficzna kolekcja obiektów	`std::vector` zawierający `unique_ptr`

4.19.1 Praktyczny przykład: kolekcja obiektów polimorficznych

```
#include <iostream>
#include <memory>
#include <vector>

struct Task {
    virtual ~Task() = default;
    virtual void run() const = 0;
};

struct PrintTask : Task {
```

```
    void run() const override { std::cout << "hello\n"; }
};

int main() {
    std::vector<std::unique_ptr<Task>> tasks;
    tasks.push_back(std::make_unique<PrintTask>());

    for (const auto& task : tasks) {
        task->run();
    }
} // vector and tasks destroy everything automatically
```

4.20 Podsumowanie i praktyczne reguły

- Preferuj *przechowywanie przez wartość*, jeśli obiekt nie wymaga odrębnej, dynamicznej tożsamości.
- Gdy obiekt ma jednego właściciela, używaj `std::unique_ptr`; własność przenoś przez `std::move`.
- Sięgaj po `std::shared_ptr` dopiero wtedy, gdy kilka części programu *rzeczywiście* musi niezależnie utrzymywać ten sam obiekt przy życiu.
- Jeśli obserwator nie powinien utrzymywać obiektu przy życiu, użyj `std::weak_ptr` i metody `lock()`.
- Do niewłasnościowego dostępu używaj `T&`, `const T&`, `T*` lub `std::span<T>` w zależności od kontraktu.
- Nie dereferencjonuj wskaźnika tylko dlatego, że jest różny od `nullptr`: obiekt może już nie istnieć.
- Preferuj `std::vector` i `std::array` zamiast ręcznie alokowanych tablic.
- Zasoby systemowe obejmuj RAII i odpowiednim deleterem; osobno analizuj bezpieczeństwo wątkowe wskazywanego obiektu.

Konkluzja. Najważniejsze pytanie brzmi nie “jak zdobyć adres?”, ale “*kto jest właścicielem obiektu, jak długo on istnieje i kto ma prawo z niego korzystać?*”. Właśnie te trzy kwestie rozstrzygają, czy w danym miejscu potrzebujemy referencji, surowego wskaźnika, `unique_ptr`, `shared_ptr` czy `weak_ptr`.

Rozdział 5

Semantyka przenoszenia, kategorie wartości i przekazywanie doskonałe

5.1 Dlaczego wprowadzono semantykę przenoszenia?

Rozważmy kontener przechowujący milion liczb. Skopiowanie go oznacza zwykle przydzielenie nowej pamięci i skopiowanie każdego elementu. Jeżeli dotychczasowy obiekt nie będzie już potrzebny, bardziej naturalna jest operacja *przekazania własności* jego zasobu zamiast kopiowania. Od C++11 język udostępnia *semantykę przenoszenia* (*move semantics*): programista może rozróżnić sytuację, w której obiekt źródłowy ma pozostać niezmieniony, od sytuacji, w której jego zasoby wolno wykorzystać ponownie.

```
#include <string>
#include <utility>
#include <vector>

std::vector<std::string> a(100'000, "data");
auto b = a;                // copy: a remains intact

auto c = std::move(a);     // move may transfer a's allocation
// c now owns its elements; a is still valid, but its state
// after this move should not be assumed without a guarantee.
```

Nie oznacza to, że każde przeniesienie jest szybsze niż kopiowanie. Dla typu `int` oba działania są w praktyce tożsame, a niektóre typy kopiują część małych danych również podczas przenoszenia. Kluczowa idea jest semantyczna: *przeniesienie pozwala zmienić stan źródła, a kopiowanie nie powinno go zmieniać.*

Zakres rozdziału. Zakładamy **C++20**. Kategorie wartości istnieją od początku języka, referencje do r-wartości i `std::move` wprowadzono w C++11, *guaranteed copy elision* w C++17, a wiele przykładów poniżej wykorzystuje koncepcje omówione w rozdziale 5. Wszystkie przykłady zawierają komentarze angielskie, aby łatwo przenosić je do projektów.

5.2 Typ wyrażenia a kategoria wartości

Typ i **kategoria wartości** są dwiema różnymi własnościami wyrażenia. Typ mówi, jaki rodzaj danych otrzymujemy; kategoria mówi m.in., czy wyrażenie identyfikuje istniejący obiekt i jak można wiązać je z referencjami. Dokładnie trzy podstawowe kategorie to: *lvalue*, *xvalue* i *prvalue*.

Kategoria	Intuicja i przykłady
`lvalue`	Wyrażenie identyfikujące obiekt lub funkcję; np. `x`, `*ptr`, `++x` dla wbudowanego `int`.
`xvalue`	*Expiring value*: wyrażenie identyfikujące obiekt, którego zasoby mogą być ponownie wykorzystane; np. `std::move(x)`.
`prvalue`	*Pure rvalue*: wyrażenie obliczające wartość lub inicjalizujące obiekt; np. `42`, `x + 1`, `std::string("hi")`.

Pozostałe dwie nazwy oznaczają sumy kategorii:

$$\underbrace{\text{glvalue}}_{\text{lvalue}\cup\text{xvalue}}, \qquad \underbrace{\text{rvalue}}_{\text{prvalue}\cup\text{xvalue}}.$$

Nie należy utożsamiać `rvalue` z tymczasowym obiektem ani `lvalue` z wyrażeniem znajdującym się po lewej stronie przypisania. Są to techniczne kategorie zdefiniowane przez język.

5.2.1 Kilka wyrażeń o zaskakujących kategoriach

```
#include <string>
#include <utility>

int x = 7;
int* p = &x;

// x                 : lvalue
// *p                : lvalue
// (x)               : lvalue
// 7                 : prvalue
// x + 1             : prvalue
// std::move(x)      : xvalue
// std::string{"abc"}: prvalue

int& lref = x;
int&& rref = 9;

// lref              : lvalue
// rref              : lvalue (!) - it has a name
// std::move(rref)   : xvalue
```

To fundamentalna zasada: **nazwa zmiennej jest wyrażeniem lvalue, nawet jeśli typ tej zmiennej to referencja rvalue `T&&`**. Samo zadeklarowanie `T&&` nie oznacza, że każde użycie takiej zmiennej będzie r-wartością.

5.2.2 Badanie kategorii za pomocą `decltype((expr))`

Dla prostego, nieujętego w nawiasy identyfikatora składnia `decltype(x)` zwraca deklarowany typ. Dla ogólnego wyrażenia `decltype((expr))` oddaje jego kategorię poprzez typ:

$$\text{decltype}((e)) = \begin{cases} T\&, & \text{gdy } e \text{ jest lvalue typu } T, \\ T\&\&, & \text{gdy } e \text{ jest xvalue typu } T, \\ T, & \text{gdy } e \text{ jest prvalue typu } T. \end{cases}$$

```
#include <type_traits>
#include <utility>

int x = 10;
```

```
5 static_assert(std::is_same_v<decltype(x), int>);
6 static_assert(std::is_same_v<decltype((x)), int&>);
7 static_assert(std::is_same_v<decltype((std::move(x))), int&&>);
8 static_assert(std::is_same_v<decltype((x + 1)), int>);
```

Uwaga. `decltype` stosuje szczególne reguły dla nieujętych w nawiasy nazw i dostępu do składowych. Dlatego `decltype(x)` oraz `decltype((x))` mogą zwrócić różne typy. Nie jest to sprzeczność, lecz zamierzona cecha języka.

5.2.3 Różnica między `auto` i `decltype(auto)`

Zwykłe `auto` w deklaracji zmiennej najczęściej dedukuje typ przez wartość i pomija referencję wynikającą z inicjalizatora. Natomiast `decltype(auto)` stosuje reguły `decltype` i może zachować kategorię wartości wyrażenia.

```
1  #include <type_traits>
2
3  int n = 5;
4  auto copy = (n);           // int: independent value
5  decltype(auto) alias = (n); // int&: refers to n
6
7  static_assert(std::is_same_v<decltype(copy), int>);
8  static_assert(std::is_same_v<decltype(alias), int&>);
9
10 auto&& first = n;          // deduces int&
11 auto&& second = 5;         // deduces int&&
12 static_assert(std::is_same_v<decltype(first), int&>);
13 static_assert(std::is_same_v<decltype(second), int&&>);
```

W typie wyniku funkcji zapis `decltype(auto)` wymaga ostrożności. Instrukcja `return (x);` może zwrócić referencję do zmiennej lokalnej, która przestanie istnieć. Zwykłe `auto` jest często bezpieczniejszym wyborem, gdy funkcja ma zwrócić własny obiekt przez wartość.

5.3 Referencje lvalue i rvalue

```
1  #include <string>
2
3  std::string name = "Ada";
4  std::string& a = name;            // binds to mutable lvalue
5  const std::string& b = name;      // binds to lvalue
6  const std::string& c = "temporary"; // can bind to temporary
7  std::string&& d = std::string{"x"}; // binds to rvalue
8
9  // std::string& e = std::string{"x"}; // error
10 // std::string&& f = name;           // error
```

Referencja `T&` umożliwia pracę z istniejącym, zwykle nazwanym obiektem; `T&&` może wiązać się z r-wartością. `const T&` wiąże się zarówno z lvalue, jak i z rvalue, nie dając prawa do modyfikowania obiektu poprzez tę referencję.

5.3.1 Wybór przeciążenia

```
1 #include <iostream>
2 #include <string>
```

```cpp
#include <utility>

void inspect(const std::string&) {
    std::cout << "const lvalue reference\n";
}
void inspect(std::string&&) {
    std::cout << "rvalue reference\n";
}

int main() {
    std::string text = "hello";
    inspect(text);             // const lvalue reference
    inspect(std::string{"x"}); // rvalue reference
    inspect(std::move(text));  // rvalue reference
}
```

Istnienie przeciążenia `f(T&&)` samo w sobie nie przesądza o rzeczywistym przeniesieniu czegokolwiek. Funkcja może tylko odczytać argument; *move* jest operacją wykonywaną przez konstruktor lub przypisanie, a nie przez mechanizm wiązania referencji.

5.3.2 Wydłużenie życia obiektu tymczasowego

W typowych deklaracjach lokalnych związanie obiektu tymczasowego bezpośrednio z referencją może wydłużyć jego życie do końca życia tej referencji:

```cpp
#include <string>

const std::string& a = std::string{"safe until a ends"};
std::string&& b = std::string{"safe until b ends"};
```

Nie wolno jednak uogólniać tej reguły na zwracanie referencji ani na dowolne przekazywanie referencji dalej. Przykładowo:

```cpp
#include <string>

const std::string& broken() {
    return std::string{"dangling"}; // dangling reference
}
```

Zwrócona referencja nie utrzymuje tego obiektu przy życiu. Jeżeli funkcja ma *wytworzyć* wynik, zwykle powinna zwracać go przez wartość.

5.4 Co naprawdę robi `std::move`?

Nazwa jest nieco myląca: `std::move(x)` **nie przenosi zasobów samodzielnie**. W przybliżeniu wykonuje rzutowanie do referencji rvalue odpowiedniego typu. Dopiero dalszy konstruktor, operator przypisania lub inna funkcja może potraktować wynik jako obiekt nadający się do przeniesienia.

```cpp
#include <type_traits>
#include <utility>

template<class T>
constexpr std::remove_reference_t<T>&&
my_move(T&& value) noexcept {
    return static_cast<std::remove_reference_t<T>&&>(value);
}
```

Powyższa implementacja ilustruje istotę działania `std::move`. W rzeczywistym kodzie korzystamy z funkcji standardowej z nagłówka `<utility>`.

```
#include <string>
#include <utility>

std::string first = "long text";
std::string second = std::move(first);
// A move constructor is selected for second.
// first is valid; do not assume its exact contents.

first = "reused";             // valid: assignment
first.clear();                // valid: clear()
```

Stan obiektu po przeniesieniu. Typy biblioteczne zwykle gwarantują stan *poprawny, lecz nieokreślony co do zawartości* (*valid but unspecified*), o ile kontrakt konkretnego typu nie obiecuje więcej. Takiego obiektu można bezpiecznie zniszczyć, przypisać mu nową wartość albo wykonać operację, której wymagania wstępne na to pozwalają. Nie należy jednak *zakładać*, że każdy przeniesiony `std::string` jest pusty. Dla niektórych typów, np. `std::unique_ptr`, stan po przeniesieniu określono dokładniej.

5.4.1 Problem obiektu `const`

```
#include <string>
#include <utility>

const std::string original = "cannot be emptied here";
std::string other = std::move(original);
// Typically: copy, not move (const string&& cannot bind to
// string's non-const move constructor).
```

`std::move(original)` ma w tym przypadku typ `const std::string&&`. Typowy konstruktor przenoszący przyjmuje `std::string&&`, więc nie może otrzymać argumentu `const`. Kompilator wybiera konstruktor kopiujący przyjmujący `const std::string&`. **Samo wystąpienie `std::move` w kodzie nie gwarantuje przeniesienia.**[1]

5.5 Konstruktor przenoszący i przenoszący operator przypisania

W klasie `T` konstruktor przenoszący ma zwykle sygnaturę `T(T&&)`, natomiast przenoszący operator przypisania: `T& operator=(T&&)`. Nie należy mylić ich z kopiowaniem:

```
struct Item {
    Item() = default;
    Item(const Item&);            // copy constructor
    Item& operator=(const Item&); // copy assignment
    Item(Item&&) noexcept;        // move constructor
    Item& operator=(Item&&) noexcept; // move assignment
    ~Item();                      // destructor
};
```

5.5.1 Przykład z obserwacją operacji

[1] Dla typów niekopiowalnych ten sam zapis może skończyć się błędem kompilacji.

```cpp
#include <iostream>
#include <utility>

struct Trace {
    Trace() { std::cout << "default\n"; }
    Trace(const Trace&) { std::cout << "copy ctor\n"; }
    Trace(Trace&&) noexcept { std::cout << "move ctor\n"; }
    Trace& operator=(const Trace&) {
        std::cout << "copy assign\n";
        return *this;
    }
    Trace& operator=(Trace&&) noexcept {
        std::cout << "move assign\n";
        return *this;
    }
};

int main() {
    Trace a;                    // default
    Trace b = a;                // copy ctor
    Trace c = std::move(a);     // move ctor
    b = c;                      // copy assign
    b = std::move(c);           // move assign
}
```

Konstruktor tworzy *nowy* obiekt, a operator przypisania zmienia już istniejący. W tym przykładzie `Trace` jedynie wypisuje komunikaty. W klasie zarządzającej zasobem konstruktor przenoszący powinien przekazać własność zasobu, a operator przypisania musi dodatkowo poprawnie pozbyć się zasobu posiadanego wcześniej przez cel.

5.5.2 Pełny przykład: klasa samodzielnie zarządzająca tablicą

To przykład *edukacyjny*. W rzeczywistym projekcie zwykle lepiej użyć standardowego kontenera, np. `std::vector<int>`.

```cpp
#include <algorithm>
#include <cstddef>
#include <utility>

class RawBuffer {
    std::size_t size_ = 0;
    int* data_ = nullptr;

public:
    explicit RawBuffer(std::size_t n)
        : size_(n), data_(n ? new int[n]{} : nullptr) {}

    ~RawBuffer() { delete[] data_; }

    RawBuffer(const RawBuffer& other)
        : RawBuffer(other.size_) {
        if (size_ != 0)
            std::copy_n(other.data_, size_, data_);
    }

    RawBuffer& operator=(const RawBuffer& other) {
        if (this != &other) {
            RawBuffer copy(other); // may throw; *this unchanged
            swap(copy);
        }
        return *this;
```

```
    }

    RawBuffer(RawBuffer&& other) noexcept
        : size_(std::exchange(other.size_, 0)),
          data_(std::exchange(other.data_, nullptr)) {}

    RawBuffer& operator=(RawBuffer&& other) noexcept {
        if (this != &other) {
            delete[] data_;
            size_ = std::exchange(other.size_, 0);
            data_ = std::exchange(other.data_, nullptr);
        }
        return *this;
    }

    void swap(RawBuffer& other) noexcept {
        using std::swap;
        swap(size_, other.size_);
        swap(data_, other.data_);
    }

    std::size_t size() const noexcept { return size_; }
    int& operator[](std::size_t i) { return data_[i]; }
    const int& operator[](std::size_t i) const { return data_[i]; }
};
```

`std::exchange(obj, value)` zwraca starą wartość `obj` i zastępuje ją wartością `value`. Konstruktor przenoszący przejmuje więc wskaźnik i rozmiar, a źródło ustawia w jawnie określony stan pusty. Operator przenoszącego przypisania usuwa dotychczasową tablicę celu, przejmuje nową i zabezpiecza przypadek `b = std::move(b)`. W implementacji kopiowania widać z kolei *silną gwarancję wyjątkową*: jeśli utworzenie kopii nie powiedzie się, pierwotny obiekt pozostaje bez zmian.

5.6 Reguła pięciu, reguła trzech i reguła zera

Klasy zarządzające zasobem często muszą kontrolować pięć operacji: (1) destruktor, (2) konstruktor kopiujący, (3) kopiujący operator przypisania, (4) konstruktor przenoszący oraz (5) przenoszący operator przypisania. To *reguła pięciu*.

Historyczna *reguła trzech* odnosiła się do destruktora, konstruktora kopiującego i kopiującego operatora przypisania, zanim język udostępnił move semantics.

Ważne: deklarowanie destruktora może blokować generowanie konstruktora przenoszącegc

```
#include <memory>
#include <type_traits>

struct WithDestructor {
    std::unique_ptr<int> p;
    ~WithDestructor() = default;
};

static_assert(!std::is_move_constructible_v<WithDestructor>);

struct WithMove {
    std::unique_ptr<int> p;
    ~WithMove() = default;
    WithMove() = default;
    WithMove(WithMove&&) noexcept = default;
    WithMove& operator=(WithMove&&) noexcept = default;
};
```

```
18 static_assert(std::is_move_constructible_v<WithMove>);
```

W `WithDestructor` użytkownik zadeklarował destruktor, więc kompilator nie deklaruje automatycznie operacji przenoszących. Kopiowanie jest niedostępne z powodu `unique_ptr`. Samo `= default` przy destruktorze nie usuwa tej reguły.

5.6.1 Najlepsza praktyka: reguła zera

Reguła zera mówi, że klasy, które nie są bezpośrednio odpowiedzialne za posiadanie surowych zasobów, zwykle nie powinny same definiować żadnej z pięciu operacji.

```
#include <string>
#include <vector>

struct Dataset {
    std::string title;
    std::vector<double> values;
};

Dataset original{"results", {1.0, 2.0, 3.0}};
Dataset copy = original;           // deep copy of vector data
Dataset moved = std::move(original); // member-wise move
```

To kontenery standardowe odpowiadają za przydział i zwalnianie pamięci. Automatycznie generowane operacje `Dataset` wykonują odpowiednie operacje na jego składowych. **Zalecenie:** zacznij od reguły zera; własny konstruktor przenoszący pisz jedynie wtedy, gdy zachowanie domyślne nie jest właściwe.

5.7 Przenoszenie a wyjątki: `noexcept`

Kontenery takie jak `std::vector<T>` przy realokacji muszą przenieść lub skopiować dotychczasowe elementy do nowego obszaru. Jeśli przenoszenie `T` może rzucać wyjątki, a `T` można kopiować, biblioteka może wybrać kopiowanie, by zachować silniejsze gwarancje wyjątkowe. Właśnie dlatego poprawne operacje przenoszące, które nie mogą rzucać wyjątków, warto deklarować `noexcept`.

```
#include <type_traits>
#include <utility>

struct Resource {
    Resource() = default;
    Resource(const Resource&) = default;
    Resource(Resource&&) noexcept = default;
};

static_assert(std::is_nothrow_move_constructible_v<Resource>);

// move_if_noexcept can provide an lvalue reference if
// moving might throw and a copy is available.
```

`std::move_if_noexcept(x)` zwraca odpowiednio `T&&` albo `const T&`, zależnie od własności konstruktorów danego typu. To narzędzie używane przy implementowaniu kontenerów, a nie uniwersalny zamiennik `std::move`.

Ostrożnie z `noexcept`. Wyjście wyjątku poza funkcję zadeklarowaną jako `noexcept` wywołuje `std::terminate`. Dlatego bezwyjątkowość musi wynikać z rzeczywistych operacji składowych.

5.8 Zwijanie referencji (*reference collapsing*)

Gdy referencje występują pośrednio poprzez aliasy lub podstawienie argumentów szablonu, stosuje się *zwijanie referencji*. Wszystkie przypadki przedstawia tabela:

Typ pośredni	Zapis	Wynik
`T& i &`	`T& &`	`T&`
`T& i &&`	`T& &&`	`T&`
`T&& i &`	`T&& &`	`T&`
`T&& i &&`	`T&& &&`	`T&&`

Krótko: jeżeli *choć jeden* składnik jest `&`, wynik jest `&`; tylko `&&` połączone z `&&` daje `&&`.

```
#include <type_traits>

using L = int&;
using R = int&&;

static_assert(std::is_same_v<L&, int&>);
static_assert(std::is_same_v<L&&, int&>);
static_assert(std::is_same_v<R&, int&>);
static_assert(std::is_same_v<R&&, int&&>);
```

Zwijanie referencji jest podstawą *perfect forwarding*: dzięki niemu jeden szablon może przyjmować zarówno lvalue, jak i rvalue i przekazywać tę różnicę dalej.

5.9 Referencje przekazujące i dedukcja `T&&`

Zapis `T&&` ma dwa znaczenia w zależności od kontekstu. Jeśli `T` jest dedukowanym, niekwalifikowanym parametrem typu *tego samego* szablonu funkcji, `T&&` to *referencja przekazująca* (*forwarding reference*). W pozostałych typowych kontekstach jest to po prostu referencja rvalue.

```
#include <string>

template<class T>
void forwarding(T&& x);         // forwarding reference

template<class T>
void not_forwarding(const T&& x); // rvalue reference

void fixed(std::string&& x);    // rvalue reference

template<class T>
struct Holder {
    void receive(T&& x);        // NOT forwarding: T comes
                                // from the class template

    template<class U>
    void relay(U&& x);          // forwarding: U is deduced
};
```

Dla wywołania `forwarding(x)` z lvalue typu `int` kompilator dedukuje `T = int&`, zatem parametr ma po zwinięciu typ `int&`. Dla `forwarding(5)` dedukuje `T = int`, a parametr ma typ `int&&`.

```
#include <type_traits>

template<class T>
void check(T&& x) {
```

```
    if constexpr (std::is_lvalue_reference_v<T>) {
        // Called with an lvalue.
    } else {
        // Called with an rvalue.
    }
}
```

Drobne wyjątki. `auto&&` zwykle również pozwala na przekazywanie kategorii, lecz dedukcja `auto` ma szczególne reguły dla list inicjalizacyjnych. `const T&&` nie jest referencją przekazującą.

5.10 Perfect forwarding i `std::forward`

Przez *perfect forwarding* rozumiemy przekazanie argumentu do następnej funkcji przy zachowaniu kategorii wartości, a także odpowiednich kwalifikatorów typu.

```
#include <iostream>
#include <utility>

void target(int&)  { std::cout << "lvalue\n"; }
void target(int&&) { std::cout << "rvalue\n"; }

template<class T>
void incorrect(T&& value) {
    target(value); // value is a named variable: always lvalue
}

template<class T>
void correct(T&& value) {
    target(std::forward<T>(value));
}

int main() {
    int x = 1;
    incorrect(2); // lvalue (!)
    correct(x);   // lvalue
    correct(2);   // rvalue
}
```

W przybliżeniu `std::forward<T>(value)` zwraca lvalue, jeżeli `T` wydedukowano jako referencję lvalue, i xvalue w przeciwnym razie. `std::move(value)` zawsze wymusza kategorię xvalue: w funkcji przekazującej byłoby to błędem, ponieważ niepotrzebnie pozwalałoby na przeniesienie zasobu z argumentu przekazanego jako lvalue.

5.10.1 Przykład: fabryka obiektów

```
#include <memory>
#include <string>
#include <utility>

struct Person {
    std::string name;
    int age;

    Person(std::string n, int a)
        : name(std::move(n)), age(a) {}
};

```

```
template<class T, class... Args>
std::unique_ptr<T> make_object(Args&&... args) {
    return std::unique_ptr<T>(
        new T(std::forward<Args>(args)...)
    );
}

int main() {
    std::string first = "Ada";
    auto p = make_object<Person>(first, 37); // first copied
    auto q = make_object<Person>(std::move(first), 38);
}
```

To edukacyjna wersja fabryki. W produkcyjnym kodzie `std::make_unique<T>(...)` robi to samo w prostszy sposób. W drugim wywołaniu użytkownik świadomie pozwolił na przeniesienie `first`; w pierwszym pozostaje on nienaruszony. `std::forward<Args>(args)...` rozwija pakiet argumentów omówiony w rozdziale 4.

5.10.2 Nie przekazuj tego samego zasobu dwa razy bez zastanowienia

```
#include <utility>

template<class T>
void bad_double_forward(T&& x) {
    consume(std::forward<T>(x));
    consume(std::forward<T>(x)); // may use moved-from object
}
```

Ten fragment jest *schematem błędu*: zakładamy, że istnieje funkcja `consume`. Jeśli pierwsze wywołanie wykorzysta argument rvalue przez przeniesienie, drugie otrzyma już obiekt po przeniesieniu. Referencję przekazującą traktuj zatem jako przeznaczoną zwykle do *jednego* przekazania dalej.

5.11 Perfect forwarding a przeciążenia i szablony klas

Konstruktor z referencją przekazującą potrafi być bardzo zachłanny podczas rozstrzygania przeciążeń. Może np. wygrać z konstruktorem kopiującym przy argumentach nie-`const`.

```
#include <string>
#include <utility>

struct Named {
    std::string name;

    Named(const Named&) = default;

    template<class U>
    explicit Named(U&& u)
        : name(std::forward<U>(u)) {}
};

// Named a{"text"};
// Named b(a);  // may select the constructor template,
//              // then fail to initialize std::string from Named&
```

Zwykle lepiej zacząć od zwykłych przeciążeń albo konstruktora przyjmującego konkretny typ. Jeżeli szablon jest potrzebny, można ograniczyć go koncepcją:

```cpp
#include <concepts>
#include <string>
#include <type_traits>
#include <utility>

struct SafeNamed {
    std::string name;

    SafeNamed(const SafeNamed&) = default;
    SafeNamed(SafeNamed&&) = default;

    template<class U>
      requires (!std::same_as<std::remove_cvref_t<U>, SafeNamed>
                && std::constructible_from<std::string, U&&>)
    explicit SafeNamed(U&& u)
        : name(std::forward<U>(u)) {}
};
```

Takie powiązanie rozdziału o perfect forwarding z koncepcjami z rozdziału 5 jest bardzo przydatne w bibliotekach generycznych.

5.12 Copy elision: kiedy nie ma ani kopii, ani przeniesienia?

Prosty model *zwrócenie przez wartość = kopia lub move* jest niepoprawny w nowoczesnym C++. Od C++17 pewne wyrażenia `prvalue` inicjalizują wynik *bezpośrednio w miejscu docelowym*. Nie istnieje dodatkowy obiekt, który trzeba by skopiować lub przenieść.

```cpp
struct NonMovable {
    NonMovable() = default;
    NonMovable(const NonMovable&) = delete;
    NonMovable(NonMovable&&) = delete;
};

NonMovable create() {
    return NonMovable{}; // guaranteed elision, C++17+
}

int main() {
    NonMovable x = create(); // OK in C++17+
}
```

Nie jest to zwykła optymalizacja, którą kompilator *może* wyłączyć: w pokazanym przypadku wynika to z semantyki języka.

5.12.1 NRVO a gwarantowana elizja

```cpp
#include <string>

std::string first() {
    return std::string{"hello"}; // mandatory direct construction
}

std::string second() {
    std::string s = "hello";
    return s;                    // NRVO may occur
}
```

Named Return Value Optimization (NRVO) dotyczy zwracania nazwanej lokalnej zmiennej. W

C++20 kompilator *może* pominąć utworzenie dodatkowego obiektu. Jeśli NRVO nie nastąpi, wyrażenie `s` w instrukcji `return` może uczestniczyć w niejawnej próbie przeniesienia.

Nie pisz automatycznie `return std::move(s);`

```
std::string make_text() {
    std::string s = "result";
    return s;             // allows NRVO
}

std::string less_good() {
    std::string s = "result";
    return std::move(s);  // inhibits NRVO; move may be required
}
```

`std::move(s)` zmienia wynik w xvalue i tym samym nie spełnia warunków NRVO dotyczących nazwy obiektu. Jeśli funkcja zwraca własną lokalną zmienną przez wartość, zazwyczaj najlepsze jest `return s;`.

5.13 Argumenty funkcji: kiedy kopiować, kiedy przenosić?

Wybór sposobu przyjmowania argumentu jest częścią kontraktu API. Nie chodzi jedynie o wydajność, ale przede wszystkim o własność i intencję.

Parametr	Znaczenie	Kiedy używać
`const T&`	Odczyt bez przejmowania własności.	Duże obiekty, gdy wystarczy tylko odczyt.
`T&`	Modyfikacja obiektu należącego do wywołującego.	Parametry wyjściowe i operacje in-place.
`T`	Własna wartość; może być skopiowana albo przeniesiona.	Małe typy i funkcje mające zachować wartość.
`T&&`	Dostęp do argumentu będącego r-wartością.	Przeciążenia zależne od kategorii oraz operacje konsumujące.
`T&&` z dedukcją `T`	Referencja przekazująca.	Opakowania i fabryki zachowujące kategorię argumentów.

5.13.1 Idiom *pass by value, then move*

```
#include <string>
#include <utility>

class User {
    std::string name_;
public:
    explicit User(std::string name)
        : name_(std::move(name)) {}
};

std::string a = "Alice";
User first(a);                    // copy into parameter, then move
User second(std::string{"Bob"});  // parameter from prvalue, then move
```

To dobre rozwiązanie, gdy funkcja *zawsze* potrzebuje własnej kopii/przeniesienia argumentu. Nie jest jednak magicznie najszybsze w każdym przypadku: dla lvalue może wymagać kopiowania i jeszcze jednego przeniesienia, a duże obiekty niemożliwe do przeniesienia mogą wymagać osobnego projektu API.

5.13.2 Przekazywanie własności `unique_ptr`

```cpp
#include <memory>
#include <utility>

void adopt(std::unique_ptr<int> p) {
    // Ownership transferred into this function.
}

int main() {
    auto p = std::make_unique<int>(42);
    adopt(std::move(p));
    // p == nullptr after moving this unique_ptr
}
```

Przekazanie `unique_ptr` przez wartość oznacza przejęcie własności. Jeżeli funkcja ma jedynie korzystać z obiektu, zwykle lepiej przyjąć `T&` lub `const T&`. Temat własności zasobów omówiono w rozdziale 3.

5.14 Rvalue-qualified methods i świadome udostępnianie zasobów

C++ pozwala ograniczać metodę do lvalue albo rvalue odbiorcy, dopisując `&` lub `&&` po liście parametrów. Można to wykorzystać np. do udostępnienia zawartości obiektu bez kopiowania, gdy całe opakowanie jest tymczasowe.

```cpp
#include <string>
#include <utility>

class Message {
    std::string data_;
public:
    explicit Message(std::string s) : data_(std::move(s)) {}

    const std::string& text() const & noexcept {
        return data_;              // lvalue owner: borrow
    }
    std::string text() && noexcept {
        return std::move(data_);   // rvalue owner: consume
    }
};

Message m{"hello"};
const std::string& borrowed = m.text();
std::string owned = Message{"world"}.text();
```

Wariant `&&` może bezpiecznie oddać wartość przez przeniesienie. Gdybyśmy zwracali w nim referencję do składowej tymczasowego obiektu, powstałaby referencja wisząca. Projektując takie API, warto rozważyć również wywołania na `const` r-wartościach i ich wymagane zachowanie.

5.15 Kontenery, iteratory przenoszące i wzorce użytkowe

5.15.1 Dodawanie elementów do `std::vector`

```cpp
#include <string>
#include <utility>
#include <vector>

std::vector<std::string> words;
std::string word = "one";

words.push_back(word);             // copy from lvalue
words.push_back(std::move(word));  // move from lvalue cast to xvalue
words.emplace_back(3, 'x');       // construct "xxx" in place
```

Metoda `emplace_back` konstruuje nowy element z podanych argumentów. Nie musi być szybsza od `push_back`. Gotowy obiekt można przekazać z pomocą `std::move`. Podczas realokacji kontener nadal może kopiować lub przenosić wcześniej dodane elementy.

5.15.2 Przenoszenie całego zakresu

```cpp
#include <algorithm>
#include <iterator>
#include <string>
#include <vector>

std::vector<std::string> source{"a", "b"};
std::vector<std::string> dest;

std::move(source.begin(), source.end(),
          std::back_inserter(dest)); // algorithm std::move

// source still contains two valid strings, but their
// contents are not assumed.
```

Nazwa `std::move` występuje również dla algorytmu z nagłówka `<algorithm>`. W przeciwieństwie do pojedynczego `std::move(x)` z nagłówka `<utility>`, algorytm przypisuje kolejno elementy z zakresu docelowego, korzystając z możliwości ich przenoszenia.

5.16 Typowe błędy i pułapki

5.16.1 Błąd 1: przekonanie, że `std::move` niszczy obiekt

`std::move` jest rzutowaniem, a nie destruktorem. Obiekt źródłowy nadal istnieje aż do końca własnego czasu życia. Może zmienić stan, jeśli na jego podstawie wykonano rzeczywistą operację przeniesienia.

5.16.2 Błąd 2: `std::move` z argumentu przekazanego jako `const T&`

```cpp
#include <string>
#include <utility>

std::string ineffective(const std::string& x) {
    return std::move(x); // copies for std::string
}
```

Jeżeli funkcja potrzebuje zwrócić niezależną kopię, czytelniej napisać `return x;`. Jeśli ma konsumować argument, powinna otrzymać wartość lub odpowiednią modyfikowalną referencję. Nie wolno usuwać `const` przez rzutowanie, by wymusić przeniesienie z oryginalnie stałego obiektu.

5.16.3 Błąd 3: użycie nazwanego `T&&` bez `std::move`

```
#include <memory>
#include <utility>

void take(std::unique_ptr<int>&& p) {
    // auto q = p;            // error: p is an lvalue
    auto q = std::move(p);    // explicit ownership transfer
}
```

5.16.4 Błąd 4: przechowywanie referencji do argumentu tymczasowego

```
#include <string>
#include <string_view>

std::string_view dangerous() {
    return std::string{"temporary"}; // dangling view
}
```

`std::string_view` jest widokiem, a nie właścicielem pamięci. Semantyka przenoszenia nie rozwiązuje problemów związanych z czasem życia. Gdy wynik funkcji ma być własnością wywołującego, zwracaj `std::string` przez wartość.

5.16.5 Błąd 5: przenoszenie nie gwarantuje zachowania wskaźników

Przeniesienie kontenera może spowodować unieważnienie wcześniej zachowanych iteratorów, referencji lub wskaźników zależnie od typu kontenera, operacji i alokatora. Nie zakładaj ogólnej zasady, że po `std::move` adresy danych zawsze pozostają takie same. Sprawdź dokumentację odpowiedniego typu.

5.16.6 Błąd 6: nadmierne stosowanie `std::forward`

`std::forward<T>` ma sens głównie tam, gdzie `T` pochodzi z dedukcji dla referencji przekazującej. Przy zwykłej lokalnej zmiennej przeznaczonej do przekazania własności należy używać `std::move`; przy zwracaniu lokalnej zmiennej przez wartość zazwyczaj żadnego z nich.

5.17 Przykład przekrojowy: kolejka zadań

Rozważmy prosty bufor zadań przechowujący funkcje do późniejszego wykonania. Użyjemy `std::unique_ptr`, aby zadanie jednoznacznie posiadało zasób. Poniższy przykład jest niezależny od wielowątkowości; *kolejka zadań* nie uruchamia niczego w osobnym wątku.

```
#include <functional>
#include <iostream>
#include <memory>
#include <utility>
#include <vector>

```

```cpp
class Tasks {
    std::vector<std::function<void()>> items_;
public:
    template<class F>
    void add(F&& f) {
        items_.emplace_back(std::forward<F>(f));
    }

    void run() {
        for (auto& f : items_) f();
        items_.clear();
    }
};

int main() {
    Tasks tasks;
    auto value = std::make_shared<int>(10);
    tasks.add([value] { std::cout << *value << '\n'; });
    tasks.run();
}
```

`std::function<void()>` w C++20 wymaga, aby przechowywany obiekt wywoływalny był kopiowalny. Dlatego w przykładzie lambda przechowuje `shared_ptr`. Gdybyśmy zamiast niego przechwycili przez przeniesienie `unique_ptr`, lambda stałaby się niekopiowalna i taki obiekt nie spełniłby wymagań `std::function` w C++20. C++23 wprowadza `std::move_only_function`, która rozwiązuje ten problem. To pokazuje, że prawidłowy wybór sposobu przenoszenia zależy od kontraktów pozostałych typów.

5.18 Praktyczna ściąga i zalecenia projektowe

Sytuacja	**Zalecenie**
Zwracam utworzony obiekt	Zwróć go przez wartość; korzystaj z elizji lub NRVO.
Przenoszę zasób z lokalnego obiektu	Użyj `std::move(x)` w miejscu przekazania własności.
Piszę wrapper o argumentach `Args&&...`	Wykorzystaj `std::forward<Args>(args)...`.
Klasa ma `vector/string` jako składowe	Preferuj regułę zera i generowane operacje.
Klasa posiada goły wskaźnik do alokacji	Preferuj zmianę projektu na RAII; jeśli zarządzasz nim ręcznie, rozważ wszystkie operacje specjalne.
Kontener przenosi elementy przy realokacji	Właściwy, bezpieczny `noexcept` w konstruktorze move pomaga w gwarancjach wyjątkowych.
Argument będzie tylko odczytywany	Zwykle `const T&` dla dużego `T`, a małe typy przez wartość.
Funkcja przejmuje `unique_ptr`	Przyjmij go przez wartość; wywołujący świadomie używa `std::move`.

Najważniejsze rozróżnienie.

1. `std::move` wyraża zgodę na potraktowanie obiektu jako źródła przeniesienia, ale samo zasobów nie przenosi.
2. `std::forward<T>` zachowuje kategorię argumentu dedukowanego w szablonie.

3. Konstruktor przenoszący to zwykle operacja `T(T&&)`, ale może wykonywać dowolną operację zgodną z kontraktem typu.
4. `T&&` jako typ parametru nie zawsze jest referencją przekazującą.
5. Nazwany parametr typu `T&&` jest wyrażeniem `lvalue`.
6. Zwracanie przez wartość jest często najlepszym sposobem przekazywania wyniku.
7. Kiedy tylko się da, stosuj regułę zera zamiast własnych pięciu funkcji specjalnych.

Pytania kontrolne.

1. Dlaczego wywołanie `g(x)` w funkcji `f(T&& x)` traktuje `x` jako lvalue?
2. Kiedy `std::move(x)` kończy się kopiowaniem zamiast przeniesienia?
3. Czym różnią się `return T{};` i `return local;` w C++20?
4. Dlaczego przy powiększaniu `vector` konstruktor move z `noexcept` ma znaczenie?
5. Kiedy użyć `std::forward<T>(x)`, a kiedy `std::move(x)`?
6. Dlaczego metoda zwracająca `const std::string&` nie powinna ujawniać referencji do składowej obiektu tymczasowego?

Dalsza lektura. *cppreference*: *Value categories*, *Move constructors*, *Reference declaration*, *Copy elision*, *std::move*, *std::forward*.

Rozdział 6

Wyjątki i bezpieczeństwo wyjątkowe w C++

6.1 Po co C++ ma wyjątki?

Mechanizm wyjątków oddziela kod realizujący zwykły przebieg algorytmu od kodu reagującego na niepowodzenia. Jeżeli funkcja nie może spełnić swojego kontraktu, może zgłosić wyjątek poleceniem `throw`. Wówczas wykonanie jest przekazywane do odpowiedniego bloku `catch`; po drodze niszczone są obiekty automatyczne, których czas życia dobiegł końca. To ostatnie zjawisko nazywamy *odwijaniem stosu* (*stack unwinding*).

Z perspektywy projektowania API warto odróżniać:

- *zwykły wynik* (w tym informację, że elementu nie znaleziono), dla którego często lepszy jest `std::optional`;
- *błąd operacji, który wywołujący powinien świadomie obsłużyć*, reprezentowany np. przez kod błędu lub, od C++23, `std::expected`;
- *niepowodzenie uniemożliwiające dotrzymanie kontraktu*, dla którego wyjątek bywa naturalnym sposobem propagacji.

Nie ma uniwersalnej reguły nakazującej używanie wyjątków wszędzie. Znaczenie mają kontrakty funkcji, konwencje projektu, ograniczenia środowiska oraz wymagania dotyczące opóźnień wykonania.

6.2 Składnia `try`, `throw` i `catch`

```cpp
#include <iostream>
#include <stdexcept>

double divide(double a, double b) {
    if (b == 0.0)
        throw std::invalid_argument("zero denominator");
    return a / b;
}

int main() {
    try {
        std::cout << divide(10.0, 2.0) << '\n';
        std::cout << divide(10.0, 0.0) << '\n';
        std::cout << "not executed\n";
```

```
    } catch (const std::invalid_argument& e) {
        std::cerr << "invalid input: " << e.what() << '\n';
    }
}
```

Instrukcja `throw E{...}` tworzy obiekt wyjątku. Obsługa szuka pasującej klauzuli `catch`; jeśli wyjątek nie zostanie obsłużony, program kończy działanie poprzez `std::terminate`. Nie należy zakładać, że nieobsłużony wyjątek zawsze spowoduje pełne odwinięcie stosu: szczegóły zakończenia programu mogą zależeć od implementacji.

6.2.1 Przechwytywanie przez referencję i kolejność klauzul

Przyjmujemy zwykle `const E&`: pozwala to uniknąć niepotrzebnego kopiowania i *object slicing* przy przekazywaniu obiektu typu pochodnego jako typu bazowego przez wartość.

```
#include <exception>
#include <iostream>
#include <stdexcept>

void handle() {
    try {
        throw std::out_of_range("bad index");
    } catch (const std::out_of_range& e) {
        std::cout << "out_of_range: " << e.what() << '\n';
    } catch (const std::exception& e) {
        std::cout << "other std::exception: " << e.what() << '\n';
    } catch (...) {
        std::cout << "unknown exception\n";
    }
}
```

Klauzule są sprawdzane w kolejności występowania: najpierw umieszcza się typy bardziej szczegółowe, a na końcu bazowe. Wyrażenie `catch (...)` przechwytuje wyjątek dowolnego typu, ale nie udostępnia bezpośrednio jego obiektu.

Ponowne zgłoszenie. `throw;` wewnątrz aktywnej obsługi ponownie zgłasza bieżący wyjątek z zachowaniem jego dynamicznego typu. Natomiast `throw e;` zgłasza nowe wyrażenie; w zależności od statycznego typu `e` może prowadzić do kopiowania i obcięcia do klasy bazowej.

```
void log_and_rethrow() {
    try {
        throw std::runtime_error("disk error");
    } catch (const std::exception& e) {
        std::cerr << e.what() << '\n';
        throw;                  // keeps original exception object
        // throw e;              // may slice to std::exception
    }
}
```

6.3 Odwijanie stosu i RAII

Najważniejsza reguła praktyczna brzmi: *zwalnianie zasobu powinno wynikać z destruktora obiektu zarządzającego zasobem*. Stanowi to rozwinięcie idei RAII poznanej w rozdziałach 3 i 10.

```
#include <iostream>
#include <stdexcept>
```

```cpp
#include <string>

struct Trace {
    std::string name;
    explicit Trace(std::string n) : name(std::move(n)) {
        std::cout << "create " << name << '\n';
    }
    ~Trace() noexcept {
        std::cout << "destroy " << name << '\n';
    }
};

void third() {
    Trace c{"C"};
    throw std::runtime_error("failure");
}
void second() {
    Trace b{"B"};
    third();
}
void first() {
    Trace a{"A"};
    second();
}
// try { first(); } catch (const std::exception&) {}
// Destruction order: C, B, A.
```

Wyjątek powoduje zakończenie odpowiednich zakresów, a obiekty z lokalnym automatycznym czasem przechowywania są niszczone w odwrotnej kolejności ich skonstruowania. Jeżeli konstruktor zgłosi wyjątek, destruktor *całego, nieutworzonego obiektu* nie jest wywoływany, ale niszczone są już w pełni skonstruowane podobiekty bazowe i składowe.

6.3.1 Zarządzanie plikiem przy pomocy RAII

```cpp
#include <cstdio>
#include <memory>
#include <stdexcept>

struct FileCloser {
    void operator()(std::FILE* file) const noexcept {
        if (file) std::fclose(file);
    }
};
using File = std::unique_ptr<std::FILE, FileCloser>;

File open_file(const char* name) {
    File file{std::fopen(name, "rb")};
    if (!file) throw std::runtime_error("cannot open file");
    return file;
}
```

Nawet gdy po otwarciu pliku dalsze operacje zgłoszą wyjątek, `unique_ptr` wykona deleter. Zauważmy, że `std::fclose` może sygnalizować błąd przy zamykaniu; prosty noexcept-deleter ignoruje ten kod błędu. Jeżeli informacja o błędzie zamknięcia jest istotna, należy zapewnić osobną, jawną operację finalizacji, której wynik można obsłużyć *przed* destrukcją.

6.4 Gwarancje bezpieczeństwa wyjątkowego

Bezpieczeństwo wyjątkowe nie oznacza tylko braku wycieków pamięci. Chodzi również o stan obiektów po niepowodzeniu:

Gwarancja	Znaczenie
No-throw / no-fail	Operacja nie zgłasza wyjątku; w wariancie no-fail dodatkowo musi się udać.
Silna (strong)	Jeśli pojawi się wyjątek, obserwowalny stan objęty kontraktem pozostaje taki jak przed operacją.
Podstawowa (basic)	Obiekty zachowują niezmienniki, zasoby nie wyciekają, ale stan może się zmienić.
Brak gwarancji	Po błędzie mogą zostać naruszone niezmienniki lub utracone zasoby.

Silna gwarancja nie oznacza automatycznie cofnięcia efektów zewnętrznych, np. wysłanego pakietu czy wpisu w bazie danych. Zakres transakcyjności trzeba sprecyzować w kontrakcie.

6.4.1 Silna gwarancja dzięki operacji „przygotuj, a potem zatwierdź”

```cpp
#include <utility>
#include <vector>

class Table {
    std::vector<int> data_;
public:
    explicit Table(std::vector<int> values)
        : data_(std::move(values)) {}

    void append_many(const std::vector<int>& more) {
        auto candidate = data_;          // may throw
        candidate.insert(candidate.end(),
                         more.begin(), more.end()); // may throw
        data_.swap(candidate);           // noexcept for std::vector<int>
    }

    const std::vector<int>& data() const noexcept { return data_; }
};
```

Jeżeli kopiowanie lub `insert` zawiedzie, `data_` pozostaje nietknięte. Dopiero bezwyjątkowe `swap` zatwierdza przygotowaną wersję. Kosztem jest dodatkowa pamięć i kopiowanie; silna gwarancja bywa więc świadomym wyborem projektowym, a nie darmową optymalizacją.

Uwaga na aliasing. W przykładzie argument `more` może nawet odnosić się do `data_` obiektu `Table`: kopia `candidate` powstaje przed modyfikacją oryginału. W kodzie operującym bezpośrednio na tym samym kontenerze trzeba dodatkowo analizować ważność iteratorów i referencji.

6.4.2 Konstruktor i niezmienniki klasy

Dobrze napisany konstruktor tworzy w pełni poprawny obiekt albo zgłasza wyjątek. Bezpiecznie zarządzane składowe automatycznie posprzątają po częściowo wykonanej inicjalizacji.

```cpp
#include <memory>
#include <stdexcept>
```

```
3
4  class Session {
5      std::unique_ptr<int[]> buffer_;
6      std::size_t size_;
7  public:
8      explicit Session(std::size_t n)
9          : buffer_(std::make_unique<int[]>(n)), size_(n) {
10         if (n == 0) throw std::invalid_argument("empty session");
11     }
12     std::size_t size() const noexcept { return size_; }
13 };
```

Nie należy zastępować RAII ręcznie pisanymi `catch`, w których sprząta się kolejne zasoby. Takie procedury są podatne na błędy, zwłaszcza gdy konstruktor zdobywa wiele zasobów po kolei.

6.5 `noexcept`: deklarowanie i sprawdzanie

`noexcept` w specyfikacji funkcji oznacza, że wyjątek nie może opuścić jej ciała. Naruszenie tego wymagania kończy się wywołaniem `std::terminate` (nie jest to „ciche zignorowanie” błędu).

```
1  void close_handle() noexcept {
2      // cleanup that must not let exceptions escape
3  }
4
5  void may_fail();
6  static_assert(noexcept(close_handle()));
7  static_assert(!noexcept(may_fail()));
```

W drugim użyciu `noexcept(expr)` jest *operatorem* zwracającym wartość logiczną w czasie kompilacji. Nie uruchamia on samego wyrażenia.

6.5.1 Warunkowe `noexcept`

```
1  #include <type_traits>
2  #include <utility>
3
4  template<class T>
5  void swap_values(T& a, T& b)
6      noexcept(std::is_nothrow_move_constructible_v<T> &&
7               std::is_nothrow_move_assignable_v<T>) {
8      T tmp(std::move(a));
9      a = std::move(b);
10     b = std::move(tmp);
11 }
```

Jest to przykład *warunkowej specyfikacji*. Nie oznacza, że przedstawiony algorytm daje silną gwarancję, jeżeli przenoszenie rzuca wyjątki; dla takich typów stan `a` lub `b` może się zmienić przed niepowodzeniem. W kodzie generycznym trzeba analizować nie tylko składnię, ale też gwarancje operacji typu `T`.

6.5.2 Dlaczego `noexcept` jest ważne dla `vector`?

Przy powiększaniu pojemności kontener może potrzebować przenieść lub skopiować stare elementy do nowej pamięci. Dla typów kopiowalnych konstrukcja kopiująca bywa bezpieczniejsza, jeśli konstruktor przenoszący może zgłosić wyjątek.

```cpp
#include <type_traits>
#include <utility>

struct Item {
    Item() = default;
    Item(const Item&) {}              // copying available
    Item(Item&&) noexcept(false) {}   // move may throw
};

static_assert(std::is_copy_constructible_v<Item>);
static_assert(!std::is_nothrow_move_constructible_v<Item>);

// std::move_if_noexcept(x) usually gives const Item& here.
```

`std::move_if_noexcept` zwraca `T&&`, jeżeli przenoszenie jest zadeklarowane jako bezwyjątkowe albo typ nie jest kopiowalny; w przeciwnym razie zwraca `const T&`. Zasada ta pomaga kontenerom utrzymywać silną gwarancję w wielu, ale nie we wszystkich przypadkach. Szczegóły są zależne od konkretnej operacji i własności typu elementów.

6.5.3 Destruktory i `std::terminate`

Domyślnie destruktor jest niewyrzucający, o ile destruktory jego składowych i klas bazowych nie wymagają innej specyfikacji. Szczególnie podczas odwijania stosu wypuszczenie kolejnego wyjątku z destruktora może spowodować `std::terminate`.

```cpp
struct SafeCleanup {
    ~SafeCleanup() noexcept {
        try {
            risky_cleanup();
        } catch (...) {
            // Log, record an error, or use another no-throw fallback.
            // Never let it escape this destructor.
        }
    }
    static void risky_cleanup();
};
```

Nie oznacza to, że każdą awarię należy ukryć. Jeśli operacja finalizacji może się nie udać i użytkownik musi znać wynik, projektujmy jawną metodę `close()` lub `commit()`, a destruktor niech pełni rolę bezpiecznej ścieżki awaryjnej.

6.6 Własne klasy wyjątków i hierarchia standardowa

W nagłówkach `<stdexcept>`, `<exception>`, `<new>` i `<system_error>` znajdziemy wiele gotowych typów. W typowych zastosowaniach najczęściej spotkamy `std::invalid_argument`, `std::out_of_range`, `std::runtime_error`, `std::bad_alloc` oraz `std::system_error`.

```cpp
#include <stdexcept>
#include <string>

class ParseError : public std::runtime_error {
    std::size_t position_;
public:
    ParseError(std::size_t pos, std::string message)
        : std::runtime_error(std::move(message)),
          position_(pos) {}

```

```
    std::size_t position() const noexcept { return position_; }
};

int parse_digit(char c, std::size_t pos) {
    if (c < '0' || c > '9')
        throw ParseError(pos, "expected decimal digit");
    return c - '0';
}
```

Własna klasa może przechowywać dodatkowe informacje, np. identyfikator operacji, pozycję w tekście lub kod błędu. Nie należy umieszczać tam niepotrzebnie ogromnych danych. Metoda `what()` klasy `std::exception` jest wirtualna i `noexcept`; jeśli ją przesłaniamy, musimy zachować zgodny kontrakt i poprawny czas życia zwracanego łańcucha.

Błędy systemowe. `std::system_error` umożliwia przenoszenie kodu błędu `std::error_code` wraz z kategorią, np. błędu systemowego, bez uzależniania całego API od przypadkowego tekstu `what()`.

6.7 Wyjątki w funkcjach szablonowych i neutralność wyjątkowa

Komponent generyczny powinien zwykle zachować niezmienniki i pozwolić wyjątkom operacji wywołanych na typie `T` przejść do klienta, zamiast pochłaniać je przez `catch (...)`.

```
#include <algorithm>
#include <vector>

template<class T, class Compare>
void sort_elements(std::vector<T>& xs, Compare comp) {
    std::sort(xs.begin(), xs.end(), comp);
    // An exception from comp may propagate to the caller.
}
```

Należy jednak odróżnić *neutralność wyjątkową* od silnej gwarancji. Jeśli komparator zgłosi wyjątek w trakcie sortowania, kontener pozostaje poprawnym obiektem, ale jego kolejność nie musi odpowiadać stanowi sprzed wywołania.

6.8 Przenoszenie wyjątków między funkcjami i wątkami

Typ `std::exception_ptr` pozwala zachować wyjątek poza blokiem `catch`, a później zgłosić go ponownie. Jest przydatny szczególnie w programowaniu współbieżnym.

```
#include <exception>
#include <stdexcept>
#include <thread>

std::exception_ptr error;
std::thread worker([&] {
    try {
        throw std::runtime_error("worker failed");
    } catch (...) {
        error = std::current_exception();
    }
});
worker.join();

if (error) {
```

```
    std::rethrow_exception(error);
}
```

Zmienna `error` nie jest tutaj współdzielona jednocześnie przez dwa aktywne dostępy: zapis zachodzi w wątku roboczym, a odczyt dopiero po `join()`, które zapewnia wymaganą synchronizację. Nie wolno dopuścić, aby wyjątek opuścił funkcję startową zwykłego `std::thread`: typowym skutkiem jest `std::terminate`.

6.8.1 `std::future` przechowuje wyjątek

Wspomniane w rozdziale o współbieżności `std::async` i `std::future` ułatwiają propagowanie niepowodzenia do odbiorcy wyniku.

```
#include <future>
#include <stdexcept>

auto fut = std::async(std::launch::async, []() -> int {
    throw std::runtime_error("calculation failed");
});
try {
    (void)fut.get();  // rethrows the stored exception
} catch (const std::runtime_error&) {
    // report the failure in the receiving thread
}
```

Przez `get()` wynik (lub wyjątek) trafia do wątku odbiorcy. W `std::future` wynik można pobrać tylko raz; inne własności tego mechanizmu omówiliśmy w rozdziale o asynchroniczności.

6.9 Wyjątki zagnieżdżone i dodatkowy kontekst

Gdy funkcja wyższego poziomu rozumie więcej niż warstwa niższa, może wzbogacić informację o błędzie i zachować oryginalną przyczynę.

```
#include <exception>
#include <stdexcept>

void load_config() {
    try {
        throw std::runtime_error("invalid token");
    } catch (...) {
        std::throw_with_nested(
            std::runtime_error("cannot load configuration"));
    }
}

void print_nested(const std::exception& e, int depth = 0) {
    std::cerr << std::string(depth * 2, ' ') << e.what() << '\n';
    try {
        std::rethrow_if_nested(e);
    } catch (const std::exception& inner) {
        print_nested(inner, depth + 1);
    } catch (...) {
        std::cerr << "nested non-standard exception\n";
    }
}
```

W tym fragmencie należy dodać nagłówki `<iostream>` i `<string>` przy samodzielnym uruchamianiu funkcji `print_nested`. Takie opakowywanie wyjątków nie jest tym samym co zgłoszenie `throw e;`:

nowy wyjątek przechowuje kontekst, a przyczynę można odzyskać przez `rethrow_if_nested`.

6.10 `std::uncaught_exceptions` i strażnicy zakresu

Funkcja `std::uncaught_exceptions()` (C++17) zwraca liczbę aktualnie nieobsługiwanych obiektów wyjątków w bieżącym wątku. Można jej użyć do wykrycia, czy wyjście z zakresu następuje wskutek odwijania stosu.

```
#include <exception>

class FailureGuard {
    int initial_ = std::uncaught_exceptions();
    int& failure_count_;
public:
    explicit FailureGuard(int& count) noexcept : failure_count_(count) {}
    FailureGuard(const FailureGuard&) = delete;
    FailureGuard& operator=(const FailureGuard&) = delete;

    ~FailureGuard() noexcept {
        if (std::uncaught_exceptions() > initial_) {
            ++failure_count_; // no-throw bookkeeping
        }
    }
};
```

Porównanie z *początkową liczbą*, a nie tylko z zerem, jest ważne przy zagnieżdżonych wyjątkach. Strażnik nie daje jednak pełnego, transakcyjnego mechanizmu cofania zmian. W projektach korzystających z C++20 można napisać własny scope guard, ale trzeba uważnie określić, czy przechowuje funkcję przez kopię, przez przeniesienie czy przez referencję i czy jej wywołanie może rzucać.

6.11 Wyjątki a alternatywne modele obsługi błędów

6.11.1 `std::optional`: brak wyniku nie musi być błędem

```
#include <optional>
#include <string_view>

std::optional<std::size_t> find_colon(std::string_view s) {
    const auto i = s.find(':');
    if (i == std::string_view::npos) return std::nullopt;
    return i;
}
```

Brak dwukropka może być normalnym scenariuszem. Zgłaszanie wyjątku przy każdym nieudanym wyszukiwaniu bywa niepotrzebne.

6.11.2 `std::expected<T,E>` — C++23, nie C++20

`std::expected` reprezentuje *albo wartość typu T, albo błąd typu E*. Został ustandaryzowany w C++23 i nie należy zakładać, że nagłówek `<expected>` jest dostępny w kompilacji C++20.

```
// This example requires C++23 and <expected>.
#include <expected>
#include <string>
#include <string_view>

```

```cpp
std::expected<int, std::string>
parse_positive(std::string_view text) {
    if (text == "1") return 1;
    if (text == "2") return 2;
    return std::unexpected("expected 1 or 2");
}

void use_expected() {
    auto result = parse_positive("3");
    if (!result) {
        const std::string& error = result.error();
        (void)error;
    }
}
```

Dzięki temu błędy oczekiwane przez API są jawne w sygnaturze. `std::expected` nie eliminuje wszystkich wyjątków: konstrukcja zawartego obiektu czy inne operacje nadal mogą się nie udać. W C++20 podobny kontrakt można zrealizować własnym typem `Result`, `std::variant<T,E>` lub odpowiednią biblioteką.

6.11.3 Kody błędów, wyjątki i wydajność

W konwencjonalnych implementacjach C++ obsługa sukcesu na ścieżce, na której żaden wyjątek nie wystąpił, może być bardzo tania, ale samo zgłoszenie i obsłużenie wyjątku zwykle kosztuje więcej niż zwrócenie prostego kodu błędu. Nie należy jednak zamieniać każdego `throw` na kod zwrotny tylko na podstawie intuicji: warto mierzyć rzeczywistą częstotliwość błędów, profilować aplikację i uwzględniać prostotę kontraktów.

6.12 Typowe błędy i dobre praktyki

1. **Nie rzucaj wskaźnikami do lokalnych obiektów.** Preferuj wyjątki będące obiektami, zwykle pochodnymi od `std::exception`.
2. **Przechwytuj wyjątki polimorficzne przez `const&`.** Unikaj obcinania obiektu (*slicing*).
3. **Do ponownego zgłoszenia używaj `throw;`.** Nie zastępuj go bez namysłu `throw e;`.
4. **Stosuj RAII.** Surowe wywołania `new`/`delete` rozdzielone operacjami rzucającymi wyjątki to częsta przyczyna wycieków pamięci.
5. **Nie wypuszczaj wyjątków z destruktorów.** Błędy finalizacji zgłaszaj jawnie w odpowiednio zaprojektowanym API.
6. **Nie oznaczaj funkcji `noexcept` bez analizy.** To obietnica z konsekwencją w postaci `std::terminate`.
7. **Nie ukrywaj błędów w pustym `catch (...)`.** Obsłuż wyjątek, zapisz informację diagnostyczną albo przekaż go wyżej.
8. **Określ gwarancję niepowodzenia.** Dla operacji mutujących dane rozważ silną gwarancję i strategię „przygotuj–zatwierdź”.
9. **Nie pozwalaj wyjątkom opuszczać funkcji startowej `std::thread`.** Użyj `std::exception_ptr`, `promise/future` lub odpowiedniego wyższego poziomu API.
10. **Wybierz właściwy model błędu.** Brak zwykłego wyniku, przewidywalna porażka operacji i naruszenie kontraktu to różne sytuacje.

6.13 Przykład końcowy: transakcyjna aktualizacja danych

Poniższy program ilustruje współpracę wyjątku, RAII i silnej gwarancji. Załóżmy, że aktualizacja dodaje wartości do rejestru, ale w razie błędu rejestr ma pozostać bez zmian.

```
#include <iostream>
#include <stdexcept>
#include <utility>
#include <vector>

class Register {
    std::vector<int> values_;
public:
    Register() : values_{1, 2, 3} {}

    void add_checked(const std::vector<int>& numbers) {
        std::vector<int> proposed = values_;
        for (int x : numbers) {
            if (x < 0) {
                throw std::invalid_argument("negative input");
            }
            proposed.push_back(x);
        }
        values_.swap(proposed);
    }

    const std::vector<int>& values() const noexcept {
        return values_;
    }
};

int main() {
    Register r;
    try {
        r.add_checked({4, -5, 6});
    } catch (const std::invalid_argument& e) {
        std::cerr << e.what() << '\n';
    }
    // Still {1, 2, 3}; no partial update.
    for (int x : r.values()) std::cout << x << ' ';
}
```

Najpierw powstaje kopia danych. Sprawdzanie i alokowanie nowej pamięci mogą zgłaszać wyjątki, lecz działają na kopii. Po pomyślnym wykonaniu następuje zatwierdzenie przez `swap`. Jest to wzorzec, który można stosować również w rozbudowanych klasach, przy świadomym uwzględnieniu kosztów i własności ich składowych.

6.14 Podsumowanie rozdziału

W praktyce mechanizm wyjątków jest najbardziej użyteczny wtedy, gdy łączy się go z dobrym modelem własności zasobów i z precyzyjnie określonymi gwarancjami operacji. `throw`, `catch` i `noexcept` to tylko składnia; o poprawności projektu decydują RAII, niezmienniki obiektów, semantyka transakcyjna oraz świadome oddzielanie błędów wyjątkowych od zwykłych wariantów wyniku.

Rozdział 7

Organizacja pamięci, wyrównanie i wydajność niskopoziomowa w C++

7.1 Po co analizować organizację pamięci?

W rozdziałach o wskaźnikach (rozdział 3), semantyce przenoszenia (rozdział 6) i iteratorach (rozdział 8) interesowało nas głównie *poprawne zarządzanie obiektami*. Teraz przyjrzymy się temu, jak obiekty są reprezentowane w pamięci oraz w jaki sposób wybory programisty wpływają na liczbę alokacji i lokalność dostępu do danych.

Rozróżniajmy trzy poziomy opisu:

1. **Gwarancje języka C++:** rozmiar obiektu, wymagane wyrównanie, czas życia obiektu, legalność dostępu i zasady alokowania pamięci.
2. **Szczegóły implementacji:** rozmieszczenie pól, rozmiary wskaźników, zachowanie konkretnego alokatora czy układ binarny ABI.
3. **Właściwości sprzętu:** linie cache, hierarchia pamięci, prefetching, przepustowość i zjawisko *false sharing*.

Kod poprawny według standardu może być wolny; szybki kod działający na jednym komputerze może być powolny na innym. Dlatego optymalizacja wymaga najpierw *pomiaru*, a następnie weryfikacji hipotezy.

7.2 Obiekt, bajt, adres i czas życia

7.2.1 Pamięć nie jest tym samym co obiekt

W modelu C++ *obszar pamięci* może istnieć bez żywego obiektu typu `T`. Obiekt charakteryzują między innymi typ, rozmiar, czas życia i, dla pewnych klas, odpowiednio wywołane konstruktory i destruktory. Samo uzyskanie adresu wystarczająco dużego bloku nie oznacza, że wolno traktować ten blok jak dowolny obiekt C++.

```
#include <cstddef>
#include <type_traits>

static_assert(sizeof(char) == 1);
static_assert(sizeof(std::byte) == 1);
static_assert(sizeof(int) >= 1);

struct Point { double x, y; };
```

```
static_assert(sizeof(Point) >= 2 * sizeof(double));
```

`sizeof(T)` podaje rozmiar obiektu w *bajtach C++*. Jeden bajt nie musi mieć ośmiu bitów; ich liczbę określa `CHAR_BIT` z nagłówka `<climits>`. Rozmiar typu obiektowego jest dodatni. Każdy obiekt ma też wymagane *wyrównanie*, które omówimy dalej.

Czas przechowywania a czas życia. Należy odróżnić *storage duration* od *object lifetime*:

- obiekt lokalny ma zazwyczaj automatyczny czas przechowywania;
- obiekt z `static` ma statyczny czas przechowywania;
- obiekt `thread_local` ma czas przechowywania wątku;
- pamięć otrzymana przez `new` lub alokator jest pamięcią o dynamicznym czasie przechowywania.

Nie należy utożsamiać tych kategorii z gwarantowanymi fizycznymi obszarami „stos” i „sterta”: te nazwy opisują zwyczajowe implementacje, a nie pełny model abstrakcyjny języka.

```
#include <memory>

int global = 17;                 // static storage duration
thread_local int per_thread = 0; // thread storage duration

void demo() {
    int local = 5;               // automatic storage duration
    auto p = std::make_unique<int>(10);
    // Obiekt *p zyje w pamieci zarzadzanej dynamicznie.
}
```

7.2.2 Reprezentacja obiektu i bezpieczne oglądanie bajtów

C++ pozwala badać reprezentację bajtową obiektu przez `char`, `unsigned char` lub `std::byte`. Nie wolno jednak wyciągać z tego wniosku, że dowolny wskaźnik `T*` można przekonwertować na `U*` i odczytać dane jako `U`: takie postępowanie może naruszyć reguły wyrównania, czasu życia i dostępu przez glvalue odpowiedniego typu.

```
#include <array>
#include <cstddef>
#include <cstring>
#include <cstdint>

std::uint32_t x = 0x12345678;
std::array<std::byte, sizeof(x)> bytes{};
std::memcpy(bytes.data(), &x, sizeof(x));

std::uint32_t y = 0;
std::memcpy(&y, bytes.data(), sizeof(y));
// y == x
```

Kolejność bajtów zależy od *endianness*. Można ją rozpoznać przez `std::endian` (C++20, nagłówek `<bit>`), jednak do serializacji najlepiej jawnie ustalać porządek bajtów, format pól i wersję protokołu. Nie należy wysyłać całej struktury przez `write(fd, &object, sizeof(object))` jako przenośnego formatu: padding, reprezentacja liczb i ABI mogą się różnić.

7.2.3 `std::bit_cast` i ograniczenia rzutowań

`std::bit_cast<To>(from)` (C++20, `<bit>`) tworzy wartość `To` przez skopiowanie reprezentacji bitowej, o ile oba typy są trywialnie kopiowalne i mają równy rozmiar. Dodatkowo reprezentacja musi oznaczać poprawną wartość docelowego typu.

```
#include <bit>
#include <cstdint>

float f = 1.0f;
// Przyklad zaklada sizeof(float) == sizeof(uint32_t).
// Na typowej platformie wynik to kod bitowy IEEE 754.
if constexpr (sizeof(float) == sizeof(std::uint32_t)) {
    auto bits = std::bit_cast<std::uint32_t>(f);
    (void)bits;
}
```

Nie mylmy `bit_cast` z `reinterpret_cast`: pierwszy wytwarza *wartość*, drugi może tylko zmienić sposób interpretacji wskaźnika; sam nie tworzy obiektu docelowego.

7.3 Layout struktury: `sizeof`, `alignof`, padding

7.3.1 Skąd bierze się padding?

Każdy typ obiektowy ma wymaganie wyrównania. Kompilator może wstawić *padding*, czyli nieużywane bajty, aby kolejne pola miały odpowiednie adresy, a elementy tablicy struktur były poprawnie wyrównane. Rozmieszczenie pól zależy od typu klasy i reguł języka; konkretna liczba bajtów paddingu zależy od implementacji.

```
#include <cstddef>
#include <iostream>

struct Unordered {
    char flag;
    double value;
    int count;
};

struct Ordered {
    double value;
    int count;
    char flag;
};

std::cout << sizeof(Unordered) << '\n';
std::cout << sizeof(Ordered) << '\n';
std::cout << alignof(Ordered) << '\n';
```

Na częstym ABI, przy `sizeof(double)==8`, `alignof(double)==8` i `sizeof(int)==4`, pierwszy typ zajmuje zwykle 24 bajty, a drugi 16. **Nie są to wartości wymagane przez standard**. Samo przestawienie pól może zmniejszyć zapotrzebowanie na pamięć milionów rekordów, ale może też pogorszyć czytelność, stabilność ABI lub lokalność dostępu.

	Unordered (przykładowo)	**Ordered (przykładowo)**
Początek	`flag`: 1 B	`value`: 8 B
Dalej	padding: 7 B	`count`: 4 B
Dalej	`value`: 8 B	`flag`: 1 B
Koniec	`count`: 4 B + padding: 4 B	padding: 3 B
Razem	24 B	16 B

7.3.2 `offsetof`, standard-layout i ABI

Dla typów *standard-layout* makro `offsetof` pozwala poznać przesunięcie pola od początku obiektu. Używanie go z innymi typami klas od C++17 jest warunkowo wspierane, dlatego do analiz przenośnych przyjmujemy tutaj standard-layout.

```
#include <cstddef>
#include <cstdint>
#include <type_traits>

struct Header {
    std::uint32_t id;
    std::uint16_t version;
};
static_assert(std::is_standard_layout_v<Header>);
static_assert(offsetof(Header, version) >=
              sizeof(std::uint32_t));
```

W analizie ABI trzeba pamiętać o polach wirtualnych, dziedziczeniu, przestrzeniach adresowych i konwencjach wywołania. Zmiana kolejności pól publicznej struktury używanej w bibliotekach binarnych może zmienić jej ABI nawet wtedy, gdy program źródłowy nadal się kompiluje.

7.3.3 `alignof` i `alignas`

`alignof(T)` zwraca wyrównanie typu `T`; `alignas(N)` pozwala narzucić silniejsze wyrównanie. Należy zachować ograniczenia języka dotyczące dopuszczalnych wartości `N`. Wyrównanie może zwiększyć rozmiar obiektu, co jest istotne przy tablicach.

```
#include <atomic>
#include <cstdint>

struct alignas(64) AlignedCounter {
    std::atomic<std::uint64_t> value{0};
};
static_assert(alignof(AlignedCounter) >= 64);
```

Wyrównanie do 64 bajtów ma sens tylko wtedy, gdy wynika z wymagań sprzętu lub pomiarów. Nie jest to uniwersalna wielkość linii cache. Ponadto `alignas` nie czyni dostępu do obiektu atomowym, nie zastępuje `std::atomic` i samo nie zapewnia synchronizacji.

7.3.4 Czy warto „pakować” struktury?

Atrybuty kompilatora w rodzaju `__attribute__((packed))` lub `#pragma pack` nie stanowią przenośnych mechanizmów standardowego C++20. Zmniejszenie rozmiaru może wymusić niewyrównany dostęp, obniżyć wydajność, a na niektórych architekturach być problemem poprawności. W kodzie protokołów lepiej często dekodować pola jawnie z bufora bajtów i nie wiązać formatu z layoutem klasy.

7.4 Wyrównane alokacje i czas życia obiektów

7.4.1 Różnica między alokacją a konstruowaniem

Klasyczne `new T(args)` wykonuje logicznie dwa zadania: pozyskuje odpowiednio wyrównaną pamięć i inicjalizuje w niej obiekt. Analogicznie `delete` niszczy obiekt i zwalnia pamięć. Dla narzędzi niskopoziomowych warto rozdzielić te operacje.

```
#include <new>

struct Widget { explicit Widget(int) {} };

void* mem = ::operator new(sizeof(Widget));
Widget* w = nullptr;
try {
    w = ::new (mem) Widget(5); // placement new
} catch (...) {
    ::operator delete(mem);
    throw;
}
w->~Widget();                 // koniec zycia obiektu
::operator delete(mem);      // zwolnienie samej pamieci
```

Taka konstrukcja jest poprawna dla `Widget` o wyrównaniu obsługiwanym przez zwykłe `operator new`. Dla typów *over-aligned* należy korzystać z odpowiedniej alokacji obsługującej `std::align_val_t` lub ze standardowego alokatora, który uwzględnia wymagania wyrównania.

Dlaczego to nie jest zalecany idiom aplikacyjny? Obsługa wyjątków, wielokrotnych destruktorów, rozmiaru tablicy i zgodności alokacji z dealokacją łatwo prowadzi do błędów. W większości kodu wystarczają `std::vector`, `std::make_unique` i obiekty automatyczne. Rozdzielanie alokacji od konstruowania ma sens głównie przy implementowaniu kontenerów, pul pamięci i innych abstrakcji zasobów.

7.4.2 `std::construct_at` i `std::destroy_at` (C++20)

Od C++20 można użyć funkcji z nagłówka `<memory>` do jawnego rozpoczęcia i zakończenia czasu życia obiektu w odpowiednio przygotowanej pamięci.

```
#include <cstddef>
#include <memory>

struct Tracked {
    int value;
    explicit Tracked(int n) : value(n) {}
};

alignas(Tracked) std::byte buffer[sizeof(Tracked)];
Tracked* obj = std::construct_at(
    reinterpret_cast<Tracked*>(buffer), 42);
// ... korzystamy z *obj ...
std::destroy_at(obj);
```

Ważne: `std::destroy_at` kończy czas życia obiektu, ale nie zwalnia jego pamięci. Nie wolno wykonywać dwóch destrukcji tego samego żywego obiektu ani odwoływać się do obiektu po destrukcji. Ponadto rozmiar i wyrównanie bufora muszą odpowiadać konstruowanemu typowi; sam `reinterpret_cast` nie naprawia niewłaściwego wyrównania.

7.4.3 Wyrównany `operator new`

Dla typów o szczególnie silnym wyrównaniu istnieją przeciążenia alokacji i dealokacji wykorzystujące `std::align_val_t` (C++17). Zwykłe `new Overaligned` wybiera odpowiedni mechanizm automatycznie.

```
#include <new>

struct alignas(128) Block { char data[128]; };
Block* b = new Block{};
delete b;
```

Jeśli ręcznie używamy `::operator new` z argumentem `std::align_val_t{128}`, musimy dobrać zgodną operację `::operator delete` z parametrem wyrównania. Mieszanie tych rodzin funkcji jest błędem.

7.4.4 Ponowne użycie tej samej pamięci i `std::launder`

W tej samej pamięci można zakończyć życie jednego obiektu, a potem rozpocząć życie kolejnego, ale dostęp przez wcześniej utworzone wskaźniki podlega szczegółowym regułom języka. `std::launder` (C++17) ma zastosowanie w określonych sytuacjach, gdy nowy obiekt nie jest przezroczystym zamiennikiem poprzedniego; nie jest uniwersalnym remedium na aliasing, nieprawidłowy czas życia czy źle wyrównaną pamięć.

Reguła praktyczna. W kodzie aplikacyjnym nie sięgaj po `std::launder` bez konkretnego problemu z czasem życia i bez zrozumienia odpowiednich reguł. Preferuj własność zasobu wyrażoną typem RAII.

7.5 Alokatory: pamięć, obiekty i bezpieczeństwo wyjątkowe

7.5.1 Standardowy model alokatora

Kontenery standardowe używają *alokatorów* do uzyskiwania pamięci na swoje elementy. Alokator nie jest tym samym co funkcja `new` i nie musi budować obiektu w chwili rezerwacji pamięci. `std::allocator_traits` zapewnia jednolity interfejs do alokowania, konstruowania i niszczenia obiektów.

```
#include <memory>

struct Item { explicit Item(int v) : value(v) {} int value; };
std::allocator<Item> alloc;
Item* p = alloc.allocate(2); // pamiec na dwa obiekty

std::allocator_traits<decltype(alloc)>::construct(alloc, p, 5);
std::allocator_traits<decltype(alloc)>::construct(alloc, p + 1, 7);

// teraz dwa obiekty sa zywe
std::allocator_traits<decltype(alloc)>::destroy(alloc, p + 1);
std::allocator_traits<decltype(alloc)>::destroy(alloc, p);
alloc.deallocate(p, 2);
```

Przykład dla przejrzystości zakłada, że oba konstruktory nie rzucają wyjątków. **Pełna implementacja kontenera** musi zliczać pomyślnie skonstruowane obiekty i w razie wyjątku zniszczyć tylko je, a potem zwolnić pamięć. Właśnie z tego powodu lepiej opierać się na gotowych kontenerach niż

ręcznie pisać alokację elementów.

7.5.2 Rezerwacja pojemności i realokacja `vector`

`std::vector` przechowuje elementy w pamięci ciągłej, dzięki czemu zapewnia dobrą lokalność dostępu. `reserve` może ograniczyć liczbę realokacji, jeśli znamy przewidywaną wielkość kontenera.

```
#include <vector>

std::vector<int> values;
values.reserve(10'000);
for (int i = 0; i < 10'000; ++i)
    values.push_back(i);
```

`reserve(n)` nie zmienia `size()`, lecz zapewnia `capacity() >= n`, chyba że zgłoszony zostanie wyjątek. Realokacja unieważnia wskaźniki, referencje i iteratory do elementów. Nie jest dobrym pomysłem wywoływanie `reserve(size()+1)` przy każdym dodaniu elementu: można w ten sposób utracić korzystną politykę geometrycznego wzrostu pojemności.

7.5.3 Dlaczego własny alokator to nie pierwszy krok?

Własny alokator w szablonie kontenera wiąże się z wymaganiami dotyczącymi propagowania przy kopiowaniu, przenoszeniu i zamianie kontenerów oraz z kwestią porównywania alokatorów. Dla większości zastosowań wygodniejsza jest warstwa `std::pmr`, którą poznamy teraz.

7.6 `std::pmr`: polimorficzne zasoby pamięci

7.6.1 Alokator jako decyzja w czasie działania

`std::pmr` (C++17, `<memory_resource>`) oddziela *rodzaj kontenera* od *sposobu przydzielania pamięci*. `std::pmr::vector<T>` jest aliasem `std::vector<T, ...>` z alokatorem polimorficznym. Zasób `memory_resource` można wybierać w czasie działania programu.

```
#include <memory_resource>
#include <vector>

std::pmr::monotonic_buffer_resource arena;
std::pmr::vector<int> numbers{&arena};
for (int i = 0; i < 100; ++i)
    numbers.push_back(i);
```

Kontener przechowuje *wskaźnik do zasobu*, ale nie przejmuje własności zasobu pamięci. Zasób musi pozostać żywy wystarczająco długo, aby obsłużyć wszystkie powiązane alokacje i dealokacje. **Nie wolno niszczyć lub resetować areny, gdy żywe obiekty wciąż korzystają z jej pamięci**.

7.6.2 Arena monotoniczna: wiele alokacji, jedno zwolnienie

`std::pmr::monotonic_buffer_resource` wydaje kolejne fragmenty pamięci. Jej `deallocate` nie zwalnia pojedynczych bloków; odzyskanie następuje przy `release()` lub destrukcji zasobu. To pasuje do wielu obiektów budowanych razem i usuwanych w jednej fazie, np. drzewa parsowania pojedynczego dokumentu.

```
#include <cstddef>
#include <memory_resource>
```

```
#include <vector>

alignas(std::max_align_t) std::byte storage[4096];
std::pmr::monotonic_buffer_resource arena(
    storage, sizeof(storage),
    std::pmr::null_memory_resource());

{
    std::pmr::vector<int> data{&arena};
    for (int i = 0; i < 32; ++i) data.push_back(i);
} // kontener znika; jego deallocate nie uwalnia blokow
// arena jest teraz bezpieczna do zniszczenia lub release()
```

`null_memory_resource()` wymusza niepowodzenie alokacji, gdy lokalny bufor się wyczerpie (rzucane jest `std::bad_alloc`). Jeżeli zamiast niego użyjemy zwykłego zasobu nadrzędnego (*upstream*), arena może dobierać kolejne bufory.

Ważna konsekwencja. Gdy `vector` zwiększa pojemność, porzuca stary blok. W przypadku areny monotonicznej ten blok zwykle pozostaje zajęty aż do końca całej fazy. Arena nie oznacza zatem automatycznie najmniejszego zużycia pamięci; warto stosować `reserve`.

7.6.3 Zasoby pulowe: `unsynchronized_pool_resource`

Gdy obiekty są często alokowane i usuwane w różnych momentach, przydatne bywają pule przechowujące bloki o podobnych rozmiarach.

```
#include <memory_resource>
#include <vector>

std::pmr::unsynchronized_pool_resource pool;
std::pmr::vector<int> items{&pool};
items.resize(500);
```

`unsynchronized_pool_resource` nie zapewnia bezpieczeństwa jednoczesnego użycia z wielu wątków. Do współdzielonego użycia istnieje `synchronized_pool_resource`. Wybór zależy od profilu alokacji i pomiarów: pule nie są uniwersalnie szybsze.

7.6.4 Własny zasób: liczymy rzeczywiste alokacje

Wariant diagnostyczny może zliczać żądania pamięci i delegować je do `new_delete_resource()`. Trzy wirtualne funkcje `do_allocate`, `do_deallocate`, `do_is_equal` tworzą podstawę własnej implementacji.

```
#include <cstddef>
#include <memory_resource>

struct CountingResource : std::pmr::memory_resource {
    std::pmr::memory_resource* upstream =
        std::pmr::new_delete_resource();
    std::size_t allocations = 0;
    std::size_t bytes = 0;
private:
    void* do_allocate(std::size_t n,
                      std::size_t alignment) override {
        void* p = upstream->allocate(n, alignment);
        ++allocations;
        bytes += n;
        return p;
```

```
    }
    void do_deallocate(void* p, std::size_t n,
                       std::size_t alignment) override {
        upstream->deallocate(p, n, alignment);
    }
    bool do_is_equal(
        const std::pmr::memory_resource& other
    ) const noexcept override {
        return this == &other;
    }
};
```

```
CountingResource stats;
{
    std::pmr::vector<int> data{&stats};
    for (int i = 0; i < 100; ++i) data.push_back(i);
}
// stats.allocations i stats.bytes opisuja zadania
// przeslane do tego konkretnego zasobu.
```

Uwaga: nie są to wszystkie alokacje programu, a `bytes` oznacza sumę żądanych rozmiarów, nie największe jednoczesne zużycie pamięci. Warto liczyć także dealokacje, *peak usage* i rozkład rozmiarów bloków.

7.6.5 Zagnieżdżone kontenery i propagacja zasobu

```
#include <memory_resource>
#include <string>
#include <vector>

CountingResource stats;
std::pmr::vector<std::pmr::string> names{&stats};
names.emplace_back("alpha");
names.emplace_back("a rather long text that needs allocation");
```

`pmr::string` również korzysta z polimorficznego alokatora, ale dokładne zachowanie małych napisów zależy od implementacji (*small string optimization*). Nie należy zakładać, że każdy `emplace_back` przydziela pamięć ani że krótki napis wymaga alokacji dynamicznej.

7.7 Lokalność danych i hierarchia cache

7.7.1 Dlaczego asymptotyka nie wystarcza?

Dwa algorytmy o złożoności $O(n)$ mogą mieć bardzo różną wydajność, jeżeli pierwszy czyta sąsiadujące elementy tablicy, a drugi podąża za wskaźnikami do rozproszonych obiektów. Procesor pobiera dane porcjami (*cache lines*); dostęp sekwencyjny sprzyja prefetchingowi, zaś losowy dostęp bywa ograniczony opóźnieniem pamięci.

Nie ma jednego rozmiaru linii cache dla wszystkich procesorów ani jednej wartości kosztu trafienia i chybienia. Zależą one od sprzętu, aktualnego obciążenia i rodzaju dostępu.

```
#include <numeric>
#include <vector>

std::vector<int> v(100000, 1);
long long sum = std::accumulate(v.begin(), v.end(), 0LL);
// Dostep sekwencyjny do pamieci ciaglej.
```

Wskaźniki do osobno alokowanych małych węzłów mogą pogorszyć lokalność i powiększyć narzut pamięciowy. Nie oznacza to, że `std::vector` jest zawsze lepszy od `std::list`: wybór zależy od wymaganych operacji, unieważniania iteratorów i rzeczywistego profilu obciążenia.

7.7.2 AoS kontra SoA

W układzie *Array of Structures* (AoS) przechowujemy obok siebie wszystkie składowe kolejnych rekordów. W *Structure of Arrays* (SoA) przechowujemy osobne tablice poszczególnych składowych.

```
#include <vector>

struct PointAoS { float x, y, z, intensity; };
std::vector<PointAoS> aos;

struct PointSoA {
    std::vector<float> x, y, z, intensity;
};
PointSoA soa;
```

Gdy obliczamy wyłącznie sumę współrzędnych `x`, układ SoA umożliwia odczytywanie samych potrzebnych danych, a nie całych rekordów. Ułatwia to też wektoryzację. Jeśli natomiast w każdej operacji potrzebujemy wszystkich pól jednego rekordu, AoS może być korzystniejszy. Wydajność zależy od rozmiaru danych, kodu i sprzętu.

AoS: rekordy obok siebie	**SoA: kolumny obok siebie**
`[x1 y1 z1 i1] [x2 y2 z2 i2]`	`[x1 x2 ...] [y1 y2 ...]`
Wygodny dostęp do całego punktu	Wygodny dostęp do jednej cechy wszystkich punktów
Często mniej kontenerów i prostszy model	Często lepsze wykorzystanie cache w obliczeniach kolumnowych

7.7.3 Stride, skoki i przewidywanie dostępu

Przechodzenie po pamięci ze skokiem k oznacza, że kolejne operacje mogą dotykać odległych linii cache. Dla macierzy w układzie wierszowym zwykle korzystniej jest przechodzić po kolumnach *wewnątrz pojedynczego wiersza*, a następnie po wierszach. Zanim zmienimy kolejność pętli, należy uwzględnić zależności obliczeniowe, autowektoryzację i to, czy kompilator już dokonał danej transformacji.

```
#include <vector>

constexpr std::size_t rows = 100;
constexpr std::size_t cols = 200;
std::vector<double> matrix(rows * cols, 1.0);
double sum = 0.0;
for (std::size_t i = 0; i < rows; ++i)
    for (std::size_t j = 0; j < cols; ++j)
        sum += matrix[i * cols + j];
```

7.8 False sharing i współbieżne liczniki

7.8.1 Na czym polega false sharing?

Jeżeli dwa wątki wielokrotnie zapisują *różne* zmienne, ale zmienne znajdują się w tej samej linii cache, protokół spójności pamięci podręcznej może wymuszać kosztowną wymianę własności tej

linii między rdzeniami. Nazywamy to *false sharing*. Nie oznacza to *data race*, jeżeli poszczególne zmienne są atomowe lub inaczej poprawnie synchronizowane.

```
#include <atomic>
#include <cstdint>

struct NaiveCounters {
    std::atomic<std::uint64_t> first{0};
    std::atomic<std::uint64_t> second{0};
};
```

Dwa liczniki mogą być blisko siebie, jednak dokładny layout obiektu i mapowanie do linii cache nie są gwarantowane. Na niektórych platformach ich równoczesna modyfikacja będzie kosztowna.

7.8.2 Rozdzielenie liczników w pamięci

```
#include <atomic>
#include <cstdint>

struct alignas(64) IsolatedAtomic {
    std::atomic<std::uint64_t> value{0};
};
struct SeparatedCounters {
    IsolatedAtomic first, second;
};
```

Tu użyliśmy 64 bajtów wyłącznie jako *przykładowej* wartości. Od C++17 w nagłówku `<new>` istnieją dwie stałe opisujące zalecane odstępy dla danych często modyfikowanych przez różne wątki (*destructive interference*) oraz dla danych, które powinny pozostawać blisko siebie (*constructive interference*). Pełną nazwę pierwszej z nich pokazuje kod poniżej. Wartości stałych zależą od implementacji; nie stanowią pomiaru rozmiaru linii cache w uruchomionym procesorze.

```
#include <atomic>
#include <new>

struct alignas(
    std::hardware_destructive_interference_size
) Counter {
    std::atomic<unsigned> value{0};
};
```

Rozdzielenie liczników zwiększa rozmiar struktury. Przy milionie liczników koszt pamięciowy może przewyższyć korzyść. Technika ma największy sens, gdy oddzielne wątki często zapisują niewielką liczbę niezależnych pól.

7.8.3 False sharing a semantyka atomowa

`std::memory_order_relaxed` wystarczy do niektórych niezależnych liczników, gdy zależy nam wyłącznie na atomowym zliczaniu; nie gwarantuje jednak synchronizacji innych danych. Usunięcie false sharing nie zastępuje poprawnego modelu pamięci współbieżnej. Szczegółowe zasady operacji atomowych były omawiane w rozdziale 1 o współbieżności.

7.9 Jak rzetelnie mierzyć wydajność?

7.9.1 Zasady pomiarów

Najważniejsze pytanie brzmi nie „czy ten kod wygląda szybciej?”, lecz „czy w rzeczywistym scenariuszu uzyskujemy powtarzalną poprawę?”. Doświadczenie powinno uwzględniać:

1. kompilację zoptymalizowaną (np. `-O2` lub `-O3`);
2. *warm-up* i wielokrotne uruchomienia;
3. jednakowe dane oraz taki sam rezultat dla obu wariantów;
4. ochronę przed usunięciem testowanej pracy przez optymalizator;
5. pomiar rozkładu wyników, a nie pojedynczego odczytu;
6. ograniczenie wpływu alokacji, I/O, schedulerów i skalowania CPU;
7. kontrolę zużycia pamięci, a nie tylko czasu;
8. narzędzia profilujące (np. Linux `perf`, odpowiedniki sprzętowe lub profilery danego środowiska).

7.9.2 Minimalna funkcja pomiarowa

```
#include <chrono>
#include <utility>

template<class F>
auto measure(F&& f) {
    auto start = std::chrono::steady_clock::now();
    auto result = std::forward<F>(f)();
    auto stop = std::chrono::steady_clock::now();
    return std::pair{result, stop - start};
}
```

Taka funkcja przydaje się do nauki, ale *nie jest kompletna* biblioteka benchmarkowa. Dla zbyt krótkich fragmentów narzut pomiaru może przeważać nad czasem pracy. Dla liczb zmiennoprzecinkowych porównywanie różnych kolejności sumowania wymaga ostrożności, ponieważ zmiana kolejności działań może zmienić zaokrąglenia.

7.9.3 Sanitizatory i wydajność

Sanitizatory, np. AddressSanitizer i UndefinedBehaviorSanitizer, są narzędziami do badania poprawności pamięci i innych błędów. Wprowadzają narzut, dlatego *pomiarów porównawczych wydajności* nie należy bezrefleksyjnie wykonywać na buildach sanitizowanych. Bezpieczny cykl pracy wygląda następująco: test poprawności, profilowanie zwykłego buildu, zmiana jednej rzeczy, ponowny test poprawności i ponowny pomiar.

7.10 Przykładowy projekt: szybkie przetwarzanie wielu punktów

Przyjmijmy, że program przechowuje milion punktów (x, y, z, I) i dla każdego oblicza tylko prosty filtr zależny od x i I. Możemy rozważyć dwa modele:

1. **AoS:** `vector<Point>` — prostszy interfejs, wygodny dostęp do pełnego punktu.

2. **SoA:** osobne `vector<float>` na x, y, z, I — możliwy mniejszy transfer danych potrzebnych do filtru.

Następnie oceniamy, czy warto użyć `reserve` przed zapełnieniem kontenerów; jeśli rekordy są tworzone na potrzeby jednego zapytania, możemy sprawdzić arenę monotoniczną. Trzeba jednak rozróżnić *koszt wypełnienia struktur* od *kosztu przetwarzania*: arena zmienia zachowanie alokacji, nie gwarantuje poprawy pracy procesora nad danymi.

Dla kodu numerycznego duży wpływ może mieć też wybór kolejności wymiarów macierzy, reprezentacji sąsiedztwa w siatce czy sposobu obliczania operatora na konkretnym punkcie zamiast konstruowania dużych obiektów pośrednich. Właściwa reprezentacja danych potrafi dać większy zysk niż drobne optymalizacje pojedynczych instrukcji.

7.11 Jak dobrać technikę?

Problem	**Pierwszy sensowny krok**
Za dużo dynamicznych alokacji	`reserve`, ograniczenie obiektów pośrednich, pomiar alokacji
Wiele obiektów znika razem	`pmr::monotonic_buffer_resource` po kontroli czasu życia
Dużo małych alokacji o różnej żywotności	`pmr::...pool_resource` i pomiar
Praca nad jedną kolumną wielu rekordów	Porównanie AoS z SoA
Losowe skoki po pamięci	Sprawdzenie lokalności, zmian formatu danych i blokowania pętli
Niezależne atomiki spowalniają się nawzajem	Analiza false sharing i separacja intensywnie zapisywanych pól
Trzeba uzyskać silne wyrównanie	`alignas`, odpowiednia alokacja i kontrola `alignof`
Trzeba napisać kontener lub pulę	`allocator_traits`, zarządzanie czasem życia, RAII
Potrzeba przenośnej serializacji	Jawny format bajtowy, bez kopiowania całych struktur

7.12 Typowe błędy i pułapki

1. **Wniosek „`sizeof(T)` to suma pól”.** Padding może zwiększyć rozmiar, a reguły klas mogą być złożone.
2. **Uznanie layoutu z jednej kompilacji za standard.** ABI, wyrównanie i rozmiary typów różnią się między platformami.
3. **Nieprawidłowe `reinterpret_cast`.** Sama zmiana typu wskaźnika nie legalizuje dostępu do danych.
4. **Mylenie pamięci z czasem życia obiektu.** `allocate` nie zastępuje konstruowania obiektów klasowych.
5. **Zwalnianie pamięci bez destruktora.** Dla nietrywialnych typów może wyciekać zasób zarządzany przez obiekt.
6. **Niezgodna para alokacja–dealokacja.** Szczególnie ważne przy over-alignment i ręcznej alokacji.
7. **Pozostawienie kontenera po zniszczeniu areny `pmr`.** Jego pamięć staje się niepoprawna; wynik to niezdefiniowane zachowanie.

8. **Wywołanie `release()` przy żywych obiektach.** Arena może unieważnić ich pamięć bez wywoływania destruktorów.
9. **Założenie, że `monotonic_buffer_resource` oszczędza pamięć.** Zwolnienie pojedynczych bloków zwykle nie następuje.
10. **Współdzielenie `unsynchronized_pool_resource` między wątkami bez ochrony.** Zasób nie jest thread-safe.
11. **Optymalizacja poprzez mniejszą strukturę bez pomiaru.** Mniejszy `sizeof` nie gwarantuje krótszego czasu wykonania.
12. **Rzekoma optymalizacja przez `volatile`.** `volatile` nie zapewnia synchronizacji między wątkami.
13. **Pomiar jednego przebiegu.** Scheduler, częstotliwości zegara i cache mogą silnie zniekształcić wynik.
14. **Użycie sanitizatora jako miary szybkości.** Narzut instrumentacji zmienia charakterystykę programu.

Pytania kontrolne.

1. Czym różnią się `sizeof(T)` i `alignof(T)`?
2. Dlaczego zmiana kolejności pól może zmienić rozmiar struktury?
3. Czy placement `new` przydziela pamięć?
4. Jaka jest różnica między `destroy_at` a `deallocate`?
5. Kiedy arena monotoniczna może zużywać więcej pamięci niż oczekujemy?
6. Dlaczego `pmr::vector<T>` nie jest właścicielem zasobu?
7. Kiedy SoA może przyspieszyć obliczenia na rekordach?
8. Czy false sharing i data race są tym samym problemem?
9. Dlaczego `std::memory_order_relaxed` nie rozwiązuje wszystkich problemów synchronizacji?
10. Jak odróżnić realną poprawę wydajności od szumu pomiarowego?

Dalsza lektura. Dokumentacja *cppreference* obejmuje hasła *Object*, *Object lifetime*, `alignof`, `alignas`, `std::construct_at` oraz `allocator_traits`. Osobno warto przeczytać opis zasobów `std::pmr` i własności *hardware interference size*. W praktyce polecam analizę śladów alokacji, profilery wydajności i *Compiler Explorer* do porównywania wygenerowanego kodu maszynowego.

Rozdział 8

Wyrażenia lambda, obiekty funkcyjne i abstrakcje wywoływalne

8.1 Po co istnieją obiekty funkcyjne?

W C++ funkcję można reprezentować nie tylko przez jej nazwę lub wskaźnik. Można również utworzyć *obiekt wywoływalny* (*callable object*): obiekt, który daje się zastosować w wyrażeniu przypominającym wywołanie funkcji. Pozwala to przekazywać zachowanie jako parametr algorytmu, zapamiętywać stan pomiędzy wywołaniami oraz tworzyć elastyczne API (np. sortowanie z własnym porządkiem, filtrowanie, obsługa zdarzeń).

```
#include <algorithm>
#include <vector>

std::vector<int> numbers{5, 2, 8, 1};
std::sort(numbers.begin(), numbers.end(),
          [](int a, int b) { return a > b; });
// numbers == {8, 5, 2, 1}
```

Trzeci argument `std::sort` to reguła porównywania. Zamiast pisać osobną funkcję o globalnej nazwie, definiujemy ją dokładnie w miejscu użycia. Wyrażenie `[](int a, int b) { return a > b; }` jest *lambdą*: tworzy obiekt unikatowego typu domknięcia (*closure type*).

Zakres i wymagania. Przyjmujemy **C++20**. Lambdy wprowadzono w C++11, przechwytywanie inicjalizujące i generyczne lambdy w C++14, przechwytywanie kopii `*this` w C++17, a jawne listy parametrów szablonu lambdy w C++20. Wykorzystamy semantykę przenoszenia z rozdziału 6, szablony wariadyczne z rozdziału 4 i koncepcje z rozdziału 5.

8.2 Trzy podstawowe postacie kodu wywoływalnego

8.2.1 Funkcja i wskaźnik do funkcji

Zwykła funkcja nie przechowuje kontekstu wywołania. Jej adres można przypisać wskaźnikowi o odpowiednim typie:

```
int twice(int x) { return 2 * x; }

int (*fp)(int) = &twice;
int a = fp(7);       // 14
int b = (*fp)(7);    // also 14
```

W deklaracji `int (*fp)(int)` nawiasy są istotne. Sam wskaźnik przechowuje adres funkcji; nie ma miejsca na dodatkowy stan, taki jak współczynnik mnożenia ustalany w czasie działania programu.

8.2.2 Funktor: klasa z `operator()`

```
struct Multiply {
    int factor;

    int operator()(int x) const {
        return factor * x;
    }
};

Multiply times_three{3};
int result = times_three(7); // 21
```

`Multiply` jest klasą, a `times_three` jej egzemplarzem. Dzięki polu `factor` każdy egzemplarz może reprezentować inne zachowanie. Operator wywołania może być przeciążany, szablonowy, opatrzony kwalifikatorami `const` lub `&&` i mieć złożoną implementację.

8.2.3 Lambda jako wygodny zapis funktora

```
int factor = 3;
auto times_three = [factor](int x) {
    return factor * x;
};
int result = times_three(7); // 21
```

Kompilator tworzy odrębny, nienazwany typ klasy odpowiadający lambdzie. Uproszczony model jej działania jest następujący:

```
struct ClosureModel {
    int factor;

    int operator()(int x) const {
        return factor * x;
    }
};
```

Jest to *model pojęciowy*, nie obiecana przez standard reprezentacja pamięci. Każde wystąpienie wyrażenia lambda ma własny typ, nawet jeżeli zapis kodu dwóch lambd wygląda identycznie.

8.3 Budowa wyrażenia lambda

Najważniejsze elementy zapisu przedstawia schemat:

```
[captures](parameters) mutable noexcept -> ResultType {
    // body
}
```

W C++20 można również jawnie określić parametry szablonu, koncepcje oraz klauzulę `requires`. Typ wyniku zwykle jest wywnioskowany z instrukcji `return`; podanie `-> ResultType` bywa przydatne, gdy chcemy wymusić konkretny typ lub zwracać referencję.

```
auto square = [](int x) { return x * x; };
auto half = [](double x) -> double { return x / 2; };
auto truth = [] { return true; }; // () may be omitted

int a = square(4);  // 16
double b = half(5); // 2.5
```

Najważniejsze zastrzeżenie. Lambda nie jest wskaźnikiem do funkcji. Zwykle tworzy obiekt z `operator()`; jedynie pewne lambdy bez przechwyceń mają konwersję na wskaźnik do funkcji o pasującej sygnaturze.

8.4 Przechwytywanie zmiennych (*captures*)

8.4.1 Przechwycenie przez wartość

```
int threshold = 10;
auto above = [threshold](int x) {
    return x > threshold;
};
threshold = 100;

bool first = above(20);  // true: saved value is 10
```

`[threshold]` inicjalizuje składową obiektu domknięcia wartością `threshold` w chwili tworzenia lambdy. Późniejsza zmiana zmiennej zewnętrznej nie modyfikuje tej kopii. Jeśli kopiowany obiekt ma kosztowne kopiowanie, samo utworzenie lub skopiowanie lambdy może także mieć koszt.

8.4.2 Przechwycenie przez referencję

```
int total = 0;
auto add = [&total](int value) {
    total += value;
};
add(4);
add(8); // total == 12
```

`[&total]` uzyskuje dostęp do tej samej zmiennej, nie do jej kopii. Lambda **nie wydłuża czasu życia** obiektu przechwyconego przez referencję. Dlatego jest wygodna, gdy callback wykonywany jest synchronicznie, ale potencjalnie niebezpieczna, jeśli jest zapamiętywany albo wykonywany później.

8.4.3 Przechwycenia domyślne i mieszane

```
int a = 2;
int b = 3;
int count = 0;

auto first  = [=]  { return a + b; }; // by copy
auto second = [&]  { ++count; };      // by reference
auto third  = [=, &count] {
    ++count;
    return a + b;
};
```

`[=]` i `[&]` obejmują te użyte w treści zmienne lokalne, które podlegają niejawnemu przechwyceniu; nie oznaczają automatycznego skopiowania *wszystkich* zmiennych z otoczenia. W większym kodzie zwykle lepiej wypisać przechwycenia jawnie: dzięki temu widać zależności i łatwiej ocenić bezpieczeństwo czasu życia.

8.4.4 Przechwytywanie inicjalizujące (C++14)

W nawiasach kwadratowych można zdefiniować nową nazwę i sposób jej inicjalizacji. Dzięki temu lambdy mogą przechowywać wartości pochodne, a także zasoby, których nie wolno kopiować:

```
#include <memory>
#include <utility>

auto p = std::make_unique<int>(42);
auto read = [q = std::move(p)] {
    return *q;
};
// p == nullptr; ownership is now in the closure
int answer = read(); // 42
```

Taka lambda jest *move-only*, ponieważ zawiera `std::unique_ptr`. Nie można jej skopiować. To istotne przy wyborze miejsca przechowywania callbacków: w C++20 `std::function` wymaga kopiowalnego obiektu docelowego.

8.4.5 Niebezpieczne przechwytywanie: wisząca referencja

```
// WRONG: do not call the returned lambda!
auto broken() {
    int local = 5;
    return [&local] { return local; };
}

// Correct: capture the value.
auto safe() {
    int local = 5;
    return [local] { return local; };
}
```

`broken()` zwraca obiekt z dostępem do zmiennej, która przestaje istnieć wraz z wyjściem z funkcji. Późniejsze wywołanie prowadziłoby do niezdefiniowanego zachowania (*undefined behavior*). Przechwycenie przez kopię rozwiązuje ten konkretny problem.

8.5 Przechwytywanie `this` i `*this`

W metodzie klasy można korzystać z jej pól wewnątrz lambdy. Trzeba jednak rozróżnić przechwycenie wskaźnika `this` od skopiowania bieżącego obiektu.

```
#include <string>

struct Label {
    std::string text;

    auto by_pointer() const {
        return [this] { return text; };
    }

```

```
    auto by_copy() const {
        return [*this] { return text; }; // C++17
    }
};
```

Lambda zwrócona przez `by_pointer()` odwołuje się do oryginalnego obiektu. Nie może być bezpiecznie wywoływana po jego zniszczeniu. Lambda z `[*this]` przechowuje kopię obiektu (jeśli da się go skopiować); dla składowych zarządzanych przez RAII jest to zwykle bardziej przewidywalne, ale może być kosztowne.

Uwaga: zapis `[=]` użyty w metodzie nie kopiuje automatycznie całego obiektu. Niejawne przechwytywanie `this` przez `[=]` jest w C++20 przestarzałe (*deprecated*); zapisujmy `[this]` jawnie albo `[*this]`, gdy potrzebujemy kopii.

8.6 Domyślnie stały operator wywołania i `mutable`

Dla typowej lambdy bez słowa `mutable` operator wywołania ma kwalifikator `const`. Oznacza to, że przechwyconych przez wartość składowych nie wolno w nim zwyczajnie modyfikować:

```
int counter = 0;

// ERROR: captured counter is not writable in this closure.
// auto bad = [counter] { ++counter; };

auto next = [counter]() mutable {
    return ++counter;
};

int a = next(); // 1
int b = next(); // 2
// outer counter is still 0
```

`mutable` modyfikuje zachowanie *kopii wewnątrz lambdy*, nie powoduje przechwycenia oryginalnej zmiennej przez referencję. Różne kopie obiektu domknięcia mogą przechowywać różny stan:

```
auto a = [n = 0]() mutable { return ++n; };
auto b = a; // independent copy of state

a(); // 1
a(); // 2
b(); // 1
```

Przy przechwyceniu przez referencję do modyfikacji zewnętrznego obiektu `mutable` nie jest potrzebne: stałość metody `operator()` nie zmienia stałości obiektu osiąganego przez referencję.

8.7 Typ lambdy, kopiowanie i konwersja na wskaźnik

```
#include <type_traits>

inline constexpr auto f = [](int x) { return x + 1; };

auto a = [](int x) { return x; };
auto b = [](int x) { return x; };
static_assert(!std::is_same_v<decltype(a), decltype(b)>);

int (*pf)(int) = +[](int x) { return x + 1; };
```

Dwie osobno zapisane lambdy mają różne typy, nawet jeśli robią to samo. W C++20 typ domknięcia lambdy bez przechwyceń jest domyślnie konstruowalny; dostępność różnych operacji kopiowania/przypisywania zależy od tego, czy lambda przechwytuje zmienne. Dla lambd z przechwyceniami obowiązują też ograniczenia narzucone przez typy pól (na przykład niekopiowalność `unique_ptr`).

`+[]` jest znanym idiomem wymuszającym konwersję nieprzechwytującej żadnej zmiennej lambdy na wskaźnik do funkcji. Nie działa dla lambd przechwytujących:

```
int offset = 10;
auto with_state = [offset](int x) { return x + offset; };
// int (*bad)(int) = with_state;  // ERROR
```

8.8 Lambdy generyczne i własne parametry szablonu

8.8.1 Parametry `auto` (C++14)

```
auto print_twice = [](const auto& value) {
    return value + value;
};

auto x = print_twice(7);                 // 14
auto s = print_twice(std::string("ab")); // "abab"
```

Parametr `auto` sprawia, że `operator()` obiektu domknięcia jest szablonem funkcji. Przy wywołaniu kompilator dedukuje odpowiednie typy tak jak przy zwykłej funkcji szablonowej.

8.8.2 Jawne parametry szablonu w lambdzie (C++20)

```
#include <concepts>

auto equal_types = []<class T>(const T& a, const T& b) {
    return a == b;
};

auto sum_integrals = []<std::integral... Ts>(Ts... values) {
    return (0 + ... + values);
};

bool yes = equal_types(3, 3);
int total = sum_integrals(1, 2, 3, 4); // 10
```

W przypadku `equal_types` oba argumenty muszą dać ten sam dedukowany typ `T`. W drugim przykładzie łączymy *concepts* z pakietem parametrów i wyrażeniem fold znanym z rozdziałów 4–5.

8.8.3 Uniwersalny wrapper z perfect forwarding

```
#include <functional>
#include <utility>

auto invoke_once = []<class F, class... Args>(
    F&& f, Args&&... args) -> decltype(auto) {
    return std::invoke(std::forward<F>(f),
                       std::forward<Args>(args)...);
};
```

`F&&` i `Args&&...` są tu referencjami przekazującymi, a `decltype(auto)` zachowuje typ wyniku, w tym jego referencyjność. W odróżnieniu od `std::move`, `std::forward` nie wymusza kategorii rvalue dla każdego argumentu. Dzięki `std::invoke` wrapper działa nie tylko dla zwykłych funkcji i lambd, lecz także dla wskaźników do składowych.

8.9 Wymuszanie sygnatury: `noexcept`, `requires`, zwracanie referencji

```
#include <concepts>
#include <utility>

auto absolute = [](int x) noexcept {
    return x < 0 ? -x : x;
};

auto increment = []<std::integral T>(T x) -> T {
    return x + 1;
};

auto first = []<class T>(T& x) -> T& {
    return x;
};

int value = 3;
first(value) = 17; // modifies value
```

Ważne są dwie subtelności. Po pierwsze, `noexcept` jest obietnicą programisty; jeśli wyjątek wydostanie się z funkcji oznaczonej `noexcept`, wywołane zostanie `std::terminate`. Po drugie, domyślna dedukcja wyniku lambdy deklarowanej bez `decltype(auto)` lub jawnego `-> T&` zwykle usuwa referencję. Właściwe zwracanie referencji wymaga zatem świadomego określenia typu wyniku i zapewnienia jego poprawnego czasu życia.

8.10 `std::invoke`: jeden mechanizm wywołania

`std::invoke` z nagłówka `<functional>` wywołuje funkcję, funktor lub lambdę, ale obsługuje także wskaźnik do metody i pola.

```
#include <functional>

struct Box {
    int value = 10;
    int add(int x) const { return value + x; }
};

Box b;
int a = std::invoke([](int x) { return x * 2; }, 4);
int c = std::invoke(&Box::add, b, 7);      // 17
int& r = std::invoke(&Box::value, b);
r = 25;                                    // b.value == 25
```

To podstawowa cegiełka generycznych algorytmów. W nagłówku `<type_traits>` zdefiniowano m.in. `std::invoke_result_t`, `std::is_invocable_v` i `std::is_nothrow_invocable_v`, które pozwalają sprawdzić w czasie kompilacji poprawność wywołania.

```
#include <type_traits>

```

```
static_assert(std::is_invocable_v<decltype(absolute), int>);
static_assert(!std::is_invocable_v<decltype(absolute),
                                   std::string>);
static_assert(std::is_nothrow_invocable_v<decltype(absolute),
                                          int>);
```

8.11 `std::function`: jeden typ dla wielu zachowań

Lambdy o różnych typach nie mogą bezpośrednio zastępować się w jednej zmiennej zadeklarowanej przez `auto`. Jeżeli chcemy przechowywać *dowolny* kopiowalny obiekt wywoływalny o zadanej sygnaturze, możemy użyć `std::function<R(Args...)>`.

```
#include <functional>
#include <string>

std::function<int(int)> operation =
    [](int x) { return 2 * x; };

int factor = 5;
operation = [factor](int x) { return factor * x; };
int result = operation(3); // 15

operation = nullptr;
if (!operation) {
    // no callable stored
}
```

`std::function` stosuje *type erasure*: ukrywa rzeczywisty typ obiektu docelowego za wspólnym interfejsem wywołania. Może przechowywać funkcje, funktory i lambdy z przechwyceniami. Nie jest odpowiednikiem `auto`: deklaracja `auto` zachowuje konkretny typ i pozwala zwykle kompilatorowi łatwiej zoptymalizować wywołanie.

8.11.1 Pusty `std::function` i wyjątek

```
#include <functional>

std::function<void()> callback;
if (callback) {
    callback();
}
// Calling callback() while empty throws std::bad_function_call.
```

Sprawdzenie `if (callback)` jest zatem standardową częścią obsługi opcjonalnych callbacków. Nie należy przechowywać `std::function` bez potrzeby, jeśli konkretny typ wywoływalny jest znany w czasie kompilacji.

8.11.2 Koszty i ograniczenie kopiowalności

Zwykle `std::function` wymaga pośredniego wywołania. Przechowywanie większego domknięcia może też wymagać dynamicznej alokacji (zależy to od implementacji i rozmiaru obiektu). To koszt elastyczności API, nie argument przeciw używaniu `std::function` w każdym przypadku.

W C++20 przechowywany obiekt docelowy musi być kopiowalny, dlatego następujący kod nie jest poprawny:

```cpp
#include <functional>
#include <memory>

// ERROR in C++20: the closure is not copy-constructible.
// std::function<int()> f =
//     [p = std::make_unique<int>(9)] { return *p; };
```

Od **C++23** biblioteka udostępnia `std::move_only_function`, która służy do przechowywania nienadających się do kopiowania obiektów wywoływalnych. W C++20 można natomiast przechowywać taki obiekt bezpośrednio jako `auto` lub przekazać go do szablonu, który nie wymaga kopiowania.

8.12 Co wybrać: `auto`, szablon, wskaźnik czy `std::function`?

Forma	Zalety	Ograniczenia
`auto f = lambda`	Konkretny typ, bez konieczności wymazywania typu, naturalna obsługa stanu	Zmienna ma ustalony typ domknięcia
`template<class F>`	Działa z wieloma typami, umożliwia inline i forwarding	Definicja szablonu zwykle dostępna w nagłówku; możliwe powiększanie kodu
Wskaźnik do funkcji	Prosty nieposiadający stanu interfejs	Nie przyjmuje lambd z przechwyceniami
`std::function`	Jedna sygnatura, zmienne zachowanie w czasie wykonania	Type erasure; koszt pośredniego wywołania; wymóg kopiowalności celu w C++20

Praktyczna reguła. Jeżeli callback wykorzystujemy natychmiast, wybierajmy zwykle szablon lub `auto`. Jeżeli musimy przechowywać i później podmieniać różne callbacki o jednej sygnaturze, `std::function` jest często czytelniejsza niż ręczne modelowanie polimorfizmu.

8.13 Lambdy ze standardowymi algorytmami

8.13.1 `find_if`, `count_if`, `transform`

```cpp
#include <algorithm>
#include <vector>

std::vector<int> xs{1, 2, 3, 4, 5, 6};

int limit = 3;
auto it = std::find_if(xs.begin(), xs.end(),
                       [limit](int x) { return x > limit; });
// *it == 4

auto even_count = std::count_if(xs.begin(), xs.end(),
                                [](int x) { return x % 2 == 0; });
// even_count == 3

std::vector<int> squares(xs.size());
std::transform(xs.begin(), xs.end(), squares.begin(),
               [](int x) { return x * x; });
```

Dla `std::find_if` trzeba przed dereferencją sprawdzić, czy wynik jest różny od `xs.end()`. Dla `std::transform` zbiór docelowy musi mieć wystarczającą pojemność na zapisywane elementy (alternatywą jest iterator `std::back_inserter`).

8.13.2 `std::ranges` i projekcje (C++20)

```cpp
#include <algorithm>
#include <ranges>
#include <string>
#include <vector>

struct Person {
    std::string name;
    int age;
};

std::vector<Person> people{{"Ada", 31}, {"Bob", 18}, {"Eve", 25}};

std::ranges::sort(people, {}, &Person::age);
// sorted by age ascending

auto adults = people | std::views::filter(
    [](const Person& p) { return p.age >= 18; });
```

Trzeci argument `ranges::sort` to *projekcja*: `&Person::age` pobiera klucz sortowania, a domyślny komparator porównuje klucze. `views::filter` nie buduje automatycznie nowego wektora; jest leniwym widokiem, który odwołuje się do danych źródłowych. Przy widokach szczególnie uważnie trzeba kontrolować czas życia kontenera i przechwyceń predykatu.

8.14 `std::ref`, `std::bind_front` i częściowe wiązanie argumentów

```cpp
#include <functional>

int add(int x, int y) { return x + y; }

auto add_ten = std::bind_front(add, 10); // C++20
int x = add_ten(5);                      // 15
```

`std::bind_front` przechowuje funkcję i ustalone argumenty, a pozostałe przyjmuje podczas wywołania. W wielu zastosowaniach równoważna lambda jest czytelniejsza:

```cpp
auto add_ten_lambda = [](int x) { return add(10, x); };
```

Domyślnie argumenty przechowywane przez `bind_front` są przechowywane przez wartość, z uwzględnieniem przenoszenia. Jeśli potrzebujemy odwołania do oryginalnego obiektu, stosujemy `std::ref` (a dla dostępu tylko do odczytu `std::cref`), pamiętając o czasie jego życia:

```cpp
#include <functional>

struct Accumulator {
    int total = 0;
    void add(int n) { total += n; }
};

Accumulator acc;
auto update = std::bind_front(&Accumulator::add, std::ref(acc));
update(7); // acc.total == 7
```

Starsze `std::bind` jest dostępne, ale ma bardziej skomplikowane reguły dotyczące argumentów i podstawiania. W nowym kodzie zwykle preferujemy lambdy lub `std::bind_front`.

8.15 Praktyczny projekt: rejestr callbacków

Rozważmy klasę powiadamiającą o zmianie wartości. Do przechowywania procedury obsługi zdarzenia użyjemy `std::function`. Dzięki temu można ją zastąpić inną nawet po utworzeniu obiektu.

```
#include <functional>
#include <utility>

class Counter {
public:
    using Callback = std::function<void(int)>;

    void on_change(Callback callback) {
        callback_ = std::move(callback);
    }

    void increment() {
        ++value_;
        if (callback_) callback_(value_);
    }

    int value() const noexcept { return value_; }

private:
    int value_ = 0;
    Callback callback_;
};

Counter counter;
counter.on_change([](int value) {
    // report the new value
});
counter.increment();
```

Kod w pojedynczym wątku jest prosty i wystarczający dla wielu interfejsów zdarzeniowych. W programie wielowątkowym równoczesna zmiana `callback_` oraz wykonywanie `increment()` wymaga dodatkowej synchronizacji. `std::function` nie zapewnia automatycznie ochrony wewnętrznego stanu przed wyścigami danych.

8.15.1 Callback a czas życia właściciela

```
#include <memory>

struct Worker : std::enable_shared_from_this<Worker> {
    int total = 0;

    auto callback() {
        std::weak_ptr<Worker> weak = shared_from_this();
        return [weak](int n) {
            if (auto self = weak.lock()) {
                self->total += n;
            }
        };
    }
};
```

Przykład zakłada, że obiekt `Worker` jest już zarządzany przez `std::shared_ptr` w chwili wywołania `callback()`. `weak_ptr` pozwala sprawdzić, czy obiekt wciąż istnieje, bez przedłużania jego życia przez przechwycenie silnego wskaźnika. To ważne również dla unikania cykli własności, np. gdy sam obiekt przechowuje callback odnoszący się do niego.

8.16 Rekurencyjne lambdy bez `std::function`

Lambda zwykle nie zna bezpośrednio własnej nazwy podczas definiowania. Można jednak przekazywać obiekt wywoływalny jako argument:

```
auto factorial = [](auto&& self, unsigned n) -> unsigned {
    return n <= 1 ? 1 : n * self(self, n - 1);
};

unsigned x = factorial(factorial, 5); // 120
```

To odmiana wzorca *self-parameter*. Dla realnych zastosowań rekurencyjnych warto ponadto zwrócić uwagę na przepełnienie liczb całkowitych i maksymalną głębokość stosu. W C++20 metoda ta pozwala uniknąć pośredniej alokacji i wymazywania typu, których mogłaby wymagać rekurencja oparta na `std::function`.

8.17 Lambdy w metaprogramowaniu: `std::visit` i `std::apply`

Dwa przydatne zastosowania łączą lambdy z mechanizmami z rozdziału 4. `std::visit` wybiera wywołanie na podstawie aktywnego typu `std::variant`, natomiast `std::apply` rozwija argumenty z krotki lub podobnej struktury.

8.17.1 Zestaw przeciążeń oparty na dziedziczeniu

```
#include <string>
#include <variant>

template <class... Fs>
struct overloaded : Fs... {
    using Fs::operator()...;
};

std::variant<int, std::string> value = std::string{"Hello"};

auto stringify = overloaded{
    [](int x) { return std::to_string(x); },
    [](const std::string& x) { return x; }
};

std::string result = std::visit(stringify, value); // "Hello"
```

Każda lambda dostarcza jeden `operator()`. Dziedziczenie wariadyczne i deklaracja `using Fs::operator()...` tworzą zbiór przeciążeń, spośród których kompilator wybiera odpowiednią funkcję. W tym przykładzie wszystkie warianty wywołania zwracają ten sam typ, `std::string`, czego wymaga tu pojedyncze wywołanie `std::visit`. Konstrukcja jest przydatna przy przetwarzaniu stanów programu i heterogenicznych danych.

8.17.2 Rozwijanie krotki przez `std::apply`

```cpp
#include <tuple>

std::tuple<int, int, int> data{2, 3, 5};
int sum = std::apply(
    [](auto... values) { return (0 + ... + values); },
    data);
// sum == 10
```

Tutaj `std::apply` bierze elementy krotki i wywołuje lambdę jak zwykłą funkcję trójargumentową. Sama lambda wykorzystuje *fold expression*, a więc nie trzeba pisać rekurencyjnego kodu.

8.18 Pakiety przechwyceń i obliczenia odroczone (C++20)

C++20 pozwala stosować *pack expansion* również w przechwytywaniu inicjalizującym. Możemy w ten sposób zbudować jeden obiekt domknięcia, który przechowuje cały pakiet wartości do późniejszego przetworzenia:

```cpp
#include <concepts>

template <std::integral... Ts>
auto delayed_sum(Ts... xs) {
    return [...stored = xs] {
        return (0 + ... + stored);
    };
}

auto job = delayed_sum(2, 3, 5);
int result = job(); // 10
```

Lambdę można zwrócić z funkcji, ponieważ wartości są przechowywane wewnątrz domknięcia. Przechwycenie przez referencję miałoby inne wymagania dotyczące czasu życia. W rzeczywistych fabrykach callbacków pakiet może zawierać obiekty niekopiowalne; wtedy należy zadbać o przekazanie własności (np. przez `std::move` i `std::forward`) oraz o wymagania interfejsu, który callback przechowuje.

8.19 Lambdy w obliczeniach stałych

Jeśli ciało lambdy dopuszcza obliczanie stałe, jej wywołanie może być częścią wyrażenia stałego. Od C++17 odpowiedni operator wywołania jest domyślnie `constexpr`, gdy spełnia wymagania takiej funkcji:

```cpp
constexpr auto cube = [](int x) { return x * x * x; };
static_assert(cube(3) == 27);

// C++20: each call must be evaluated at compile time.
auto fourth_power = [](int x) consteval {
    return x * x * x * x;
};
static_assert(fourth_power(2) == 16);
```

`consteval` jest mocniejszą deklaracją niż `constexpr`: wymaga, aby właściwe wywołania funkcji natychmiastowej spełniały reguły obliczania stałego. Nie należy traktować tego jako gwarancji, że każda zwykła lambda zostanie wykonana podczas kompilacji.

8.20 Koncepcje dla obiektów wywoływalnych

W C++20 można ograniczać typy wywoływalne m.in. następującymi koncepcjami:

- `std::invocable` – poprawność wywołania z danymi argumentami;
- `std::regular_invocable` – dodatkowe wymagania semantyczne;
- `std::predicate` – poprawnie wywoływalny predykat z wynikiem dającym się traktować jako wartość logiczną.

Koncepcje nie dowodzą wszystkich własności semantycznych. Przykładowo kompilator nie zweryfikuje matematycznie stabilności wyników predykatu.

```
#include <concepts>
#include <functional>
#include <utility>

template <class F, class T>
    requires std::predicate<F&, const T&>
bool test(const T& value, F&& predicate) {
    return std::invoke(std::forward<F>(predicate), value);
}

bool ok = test(8, [](int x) { return x % 2 == 0; });
```

Istotne rozróżnienie: koncepcja weryfikuje przede wszystkim własności typów i poprawność wyrażeń. Nie dowodzi matematycznie własności predykatu, np. że comparator tworzy ścisły słaby porządek. Programista nadal odpowiada za kontrakt semantyczny.

8.21 Typowe błędy i trudniejsze przypadki

8.21.1 Referencja przechwycona do operacji asynchronicznej

```
#include <future>

std::future<int> start() {
    int x = 7;
    // WRONG: the task could outlive x.
    // return std::async(std::launch::async, [&x] { return x; });
    return std::async(std::launch::async, [x] { return x; });
}
```

W zadaniach asynchronicznych preferuj przechwycenia przez wartość lub świadomie zarządzaną własność, dopóki nie mamy ścisłej gwarancji, że obiekt referowany przetrwa wykonanie zadania. Sama biblioteka `std::async` nie wydłuża życia lokalnych zmiennych funkcji uruchamiającej.

8.21.2 Predykat sortowania nie może naruszać porządku

```
// WRONG: <= is not a strict ordering relation.
// std::sort(xs.begin(), xs.end(),
//           [](int a, int b) { return a <= b; });

std::sort(xs.begin(), xs.end(),
          [](int a, int b) { return a < b; });
```

Komparator do `sort` musi definiować *ścisły słaby porządek*. Przykładowo `comp(x,x)` ma dawać `false`. Naruszenie wymagań algorytmu może prowadzić do niezdefiniowanego zachowania, mimo że typy argumentów się zgadzają.

8.21.3 Pomylenie kopii z oryginalnym stanem

```
int sum = 0;
std::for_each(xs.begin(), xs.end(), [sum](int x) mutable {
    sum += x; // modifies only the closure's copy
});
// outer sum remains 0

std::for_each(xs.begin(), xs.end(), [&sum](int x) {
    sum += x;
});
```

Nie należy zakładać, że `mutable` zmienia zewnętrzną zmienną. Należy też pamiętać, że algorytmy mogą kopiować obiekt funkcyjny; nie opierajmy semantyki algorytmu na nieudokumentowanych szczegółach kopiowania predykatu.

8.21.4 Przechwycenie wskaźnika nie przedłuża życia obiektu

```
int* p = /* pointer to some object */ nullptr;
auto f = [p] {
    if (p) return *p;
    return 0;
};
```

Wartość wskaźnika może zostać skopiowana do lambdy, ale wskazywany obiekt nadal musi istnieć w chwili dereferencji. Przechwycenie `shared_ptr` przenosi własność współdzieloną, a `weak_ptr` pozwala ją warunkowo odzyskać. To nie jest to samo co skopiowanie wskaźnika surowego.

8.22 Zestawienie: kiedy stosować którą konstrukcję?

Potrzeba	**Zalecany wybór**
Krótki predykat do algorytmu	Lambda z jawnymi przechwyceniami
Wywołanie bez stanu w API C	Zwykła funkcja albo zgodna lambda bez przechwyceń jako wskaźnik do funkcji
Wydajny callback o typie znanym w czasie kompilacji	Parametr szablonowy `F`, ewentualnie `std::invocable`
Callback z możliwością późniejszej podmiany	`std::function` (C++20, kopiowalny cel)
Przechowanie `unique_ptr` wewnątrz lambdy	Init-capture i przeniesienie; nie przechowuj takiej lambdy w `std::function` w C++20
Callback żyjący dłużej niż zmienne lokalne	Kopiuj potrzebny stan lub zarządzaj życiem przez RAII i smart pointery
Uniwersalne wywołanie dowolnego callable	`std::invoke` i `std::forward`
Częściowe podstawienie kilku argumentów	Lambda lub `std::bind_front`
Bezpieczny predykat dla szablonu	`std::predicate` plus kontrola jego semantyki

Pytania kontrolne.

1. Dlaczego dwie lambdy o identycznym kodzie mają różne typy?
2. Jaka jest różnica między `[x]`, `[&x]` i `[x]() mutable`?
3. Kiedy bezpiecznie zwrócić lambdę przechwytującą zmienną lokalną?
4. Dlaczego lambda posiadająca `unique_ptr` nie może być celem `std::function` w C++20?
5. Jak `std::invoke` wywołuje wskaźnik do metody klasy?
6. Czym różnią się `[this]` i `[*this]`?
7. Co może pójść źle, gdy `std::views::filter` korzysta z lokalnej zmiennej przez referencję?
8. Kiedy interfejs przyjmujący szablon `F` jest lepszy od `std::function`?
9. Dlaczego komparator `a <= b` nie jest poprawny dla `std::sort`?
10. W jakiej sytuacji przechwycenie `weak_ptr` jest lepsze od `shared_ptr`?

Dalsza lektura. W dokumentacji *cppreference* warto przeczytać hasła *Lambda expressions*, *Function objects* i *Ranges*. Dokładne opisy narzędzi znajdują się również pod hasłami:

- `std::invoke` oraz `std::function`;
- `std::bind_front` oraz `std::predicate`.

Rozdział 9

Wymazywanie typów i typy sumaryczne

9.1 Problem: jak reprezentować różne rodzaje wartości?

W programowaniu generycznym często chcemy posługiwać się różnymi typami przez wspólny interfejs. C++ oferuje kilka mechanizmów realizacji tego celu, ale mechanizmy te nie są zamienne. Istotne są co najmniej cztery pytania:

1. Czy zestaw dozwolonych typów jest *zamknięty* (ustalony z góry), czy *otwarty* (może zostać rozszerzony o nowy typ)?
2. Czy chcemy przechować *wartość*, czy jedynie referencję/uchwyt do istniejącego obiektu?
3. Czy wywoływane operacje mają być znane już podczas kompilacji, czy wybierane dynamicznie w czasie działania programu?
4. Czy dopuszczamy alokację dynamiczną, kopiowanie oraz koszt pośredniego wywołania funkcji?

Typ sumaryczny (ang. *sum type*) reprezentuje alternatywę: wartość należy do jednego z kilku przypadków. W C++17 taką rolę pełni `std::variant<Ts...>`; `std::optional<T>` opisuje wariant „brak wartości albo wartość `T`". Natomiast *type erasure* (*wymazywanie typu*) pozwala ukryć konkretny typ obiektu za interfejsem nieparametryzowanym przez ten typ. Standardowe przykłady to `std::any` i `std::function`.

Mechanizm	Informacja o typie	Zasadnicze zastosowanie
`optional<T>`	Jedna możliwość `T` lub brak	Wynik opcjonalny, wartość nieobecna
`variant<Ts...>`	Zamknięty zbiór alternatyw	Zdarzenia, komendy, model stanu
`any`	Typ ustalany w czasie wykonania	Heterogeniczne wartości bez wspólnego interfejsu operacji
`function<R(Args...)>`	Dowolny zgodny obiekt wywoływalny	Przechowywany callback, funkcja jako wartość
`unique_ptr<Base>`	Otwarta hierarchia klas	Polimorfizm dynamiczny i tożsamość obiektu
Szablon/`concept`	Typ znany przy instancjowaniu	Polimorfizm statyczny, specjalizacja kodu

Przykład motywujący. Niech polecenie aplikacji będzie albo prośbą o zapisanie pliku, albo prośbą o dodanie dwóch liczb, albo poleceniem zakończenia programu. Jeśli znamy te trzy możliwości, użycie `variant` dobrze wyraża model danych. Gdy tworzymy system rozszerzeń umożliwiający

niezależnym modułom dodawanie nowych rodzajów poleceń, lepszy bywa interfejs polimorficzny lub ręcznie zbudowany mechanizm type erasure.

9.2 `std::optional<T>`: wartość, której może nie być

Typ `std::optional<T>` (C++17, nagłówek `<optional>`) przechowuje bezpośrednio obiekt typu `T`, jeżeli jest on obecny, lub reprezentuje stan pusty. Jest to pełnoprawna *wartość*, a nie wskaźnik. W szczególności pusty `optional` nie oznacza wskaźnika zerowego, a niepusty nie wymaga automatycznie osobnej alokacji dynamicznej.

```
#include <cassert>
#include <optional>
#include <string>

int main() {
    std::optional<int> x;          // stan pusty
    assert(!x.has_value());
    assert(x == std::nullopt);

    x = 42;
    assert(x.has_value());
    assert(*x == 42);
    assert(x.value() == 42);
    assert(x.value_or(-1) == 42);

    x.reset();
    assert(x.value_or(-1) == -1);
    x.emplace(7);
    assert(*x == 7);

    std::optional<std::string> name{std::in_place, 3, 'a'};
    assert(*name == "aaa");
}
```

Listing 9.1: Najważniejsze operacje optional (C++17); kompletny program.

Dostęp przez `*x` i `x->...` wymaga obecności wartości: *użycie pustego obiektu w taki sposób jest niezdefiniowanym zachowaniem*. Metoda `value()` wykonuje sprawdzenie i dla stanu pustego zgłasza `std::bad_optional_access`. Warto pamiętać, że `value_or(default)` przyjmuje wartość domyślną jako argument funkcji: wyrażenie tworzące argument jest obliczane również wtedy, gdy `optional` zawiera wartość. Może ponadto następować kopiowanie lub przeniesienie przechowywanego obiektu.

9.2.1 Praktyczny przykład: parsowanie liczby

Funkcja parsująca dane wejściowe może zwrócić `optional`, jeśli interesuje nas tylko informacja „udało się/nie udało się”. `std::from_chars` unika wyjątków i nie wymaga tworzenia dodatkowego łańcucha zakończonego zerem. W przykładzie wymagamy, aby *cały* łańcuch był prawidłową liczbą.

```
#include <cassert>
#include <charconv>
#include <optional>
#include <string_view>
#include <system_error>

std::optional<int> parse_int(std::string_view text) {
    int result = 0;
    auto [ptr, ec] = std::from_chars(
        text.data(), text.data() + text.size(), result);
    if (ec != std::errc{} || ptr != text.data() + text.size())
```

```
        return std::nullopt;
    return result;
}

int main() {
    assert(parse_int("123") == 123);
    assert(parse_int("123x") == std::nullopt);
    assert(parse_int("") == std::nullopt);
    assert(parse_int("99999999999999999999") == std::nullopt);
}
```

Listing 9.2: Parsowanie int do optional (C++17); kompletny program.

9.2.2 Własność, referencje i koszt

`optional<T>` posiada swoją wartość i zwykle zajmuje więcej pamięci niż sam `T`, ponieważ musi przechowywać informację, czy wartość istnieje. Rozmiar zależy od typu i implementacji. Standard C++20 nie udostępnia `std::optional<T&>`. Jeśli wynik ma wskazywać na istniejący obiekt, można użyć wskaźnika nieposiadającego własności albo `std::optional<std::reference_wrapper<T>`. W obu przypadkach programista odpowiada za czas życia wskazywanego obiektu.

```
#include <functional>
#include <optional>
#include <string>

std::optional<std::reference_wrapper<const std::string>>
find_name(bool exists, const std::string& name) {
    if (exists) return std::cref(name);
    return std::nullopt;
}
// Zwracana referencja jest poprawna tylko gdy `name` nadal istnieje.
```

Listing 9.3: Opcjonalna referencja: brak przenoszenia własności; fragment.

Gdy brak wartości jest sytuacją normalną, `optional` najczęściej czytelniej wyraża intencję niż wyjątkowe zakończenie funkcji. Jeżeli konieczna jest informacja *dlaczego* nie uzyskano wyniku, należy zaprojektować osobny typ błędu, użyć `variant` lub (w C++23) `std::expected<T,E>`.

9.3 `std::variant<Ts...>`: zamknięta alternatywa typów

`std::variant<Ts...>` (C++17, nagłówek `<variant>`) przechowuje *jedną aktywną alternatywę* z listy typów `Ts...`. Nie jest odpowiednikiem `std::tuple<Ts...>`: krotka zawiera *wszystkie* składowe, a wariant dokładnie jeden z dopuszczonych przypadków (poza szczególnym stanem `valueless_by_exception()`).

```
#include <cassert>
#include <string>
#include <variant>

int main() {
    using Value = std::variant<int, std::string>;
    Value x = 7;
    assert(x.index() == 0);
    assert(std::holds_alternative<int>(x));
    assert(std::get<int>(x) == 7);

    x = std::string("hello");
    assert(x.index() == 1);
```

```
    assert(std::get_if<int>(&x) == nullptr);
    assert(std::get_if<std::string>(&x) != nullptr);

    x.emplace<0>(19);
    assert(std::get<0>(x) == 19);
}
```

Listing 9.4: Tworzenie i badanie aktywnej alternatywy (C++17); kompletny program.

Niepoprawne `std::get<T>(x)` (gdy aktywny jest inny typ) zgłasza `std::bad_variant_access`. Alternatywa `std::get_if<T>(&x)` zwraca `nullptr`, a zatem nadaje się do sprawdzania bez wyjątku. `std::get<T>` i `std::holds_alternative<T>` wymagają, aby `T` występował *dokładnie raz* w liście alternatyw; dla powtarzających się typów używamy indeksów. Nie należy także zapominać o możliwym niepowodzeniu operacji konwersji przy konstruowaniu wariantu.

9.3.1 Domyślna alternatywa i `std::monostate`

Domyślnie konstruowany wariant próbuje skonstruować pierwszą alternatywę. Jeżeli chcemy jawnie reprezentować stan pusty, można umieścić na początku `std::monostate`; zapewnia on także domyślną konstruowalność wariantu, gdy inny pierwszy typ nie jest domyślnie konstruowalny.

```
#include <cassert>
#include <string>
#include <variant>

int main() {
    using State = std::variant<std::monostate, int, std::string>;
    State s;
    assert(std::holds_alternative<std::monostate>(s));
    s = std::string("ready");
    assert(std::get<std::string>(s) == "ready");
}
```

Listing 9.5: Jawny stan początkowy monostate; kompletny program.

9.3.2 Odwiedzanie wariantu: `std::visit`

Samo wielokrotne sprawdzanie `holds_alternative` szybko prowadzi do rozbudowanych instrukcji `if`. Mechanizm `std::visit(visitor, variant)` wywołuje odwiedzającego z aktywną wartością odpowiedniego typu. Gdy odwiedzający jest lambdą generyczną, kompilator instancjuje jej operator wywołania dla różnych alternatyw.

```
#include <cassert>
#include <string>
#include <type_traits>
#include <variant>

int main() {
    std::variant<int, double, std::string> v = "hi";

    const std::size_t size = std::visit([](const auto& x) {
        using T = std::decay_t<decltype(x)>;
        if constexpr (std::is_same_v<T, std::string>)
            return x.size();
        else
            return sizeof(x);
    }, v);

    assert(size == 2);
```

```
}
```

Listing 9.6: Jeden odwiedzający generyczny; kompletny program.

Do osobnej obsługi poszczególnych typów wygodny jest wzorzec `overloaded`, znany również z rozdziału o lambdach. Wykorzystuje on dziedziczenie po kilku obiektach funkcyjnych oraz ujawnienie wszystkich operatorów wywołania za pomocą `using`.

```
#include <cassert>
#include <string>
#include <variant>

template<class... Fs>
struct overloaded : Fs... {
    using Fs::operator()...;
};
template<class... Fs> overloaded(Fs...) -> overloaded<Fs...>;

int main() {
    using Data = std::variant<int, double, std::string>;
    Data data = std::string("abc");

    auto length = std::visit(overloaded{
        [](int) { return std::size_t{1}; },
        [](double) { return std::size_t{1}; },
        [](const std::string& s) { return s.size(); }
    }, data);
    assert(length == 3);
}
```

Listing 9.7: overloaded oraz std::visit (C++17); kompletny program.

W przypadku odwiedzania zwracającego wartość wymagany jest zgodny typ wyniku wszystkich możliwych wywołań (w praktyce należy zapewnić identyczny typ i kategorię wartości zwracanych wyrażeń, z wyjątkiem szczególnych reguł dla `void`). Przy wielu wariantach `std::visit` uwzględnia *kombinacje* alternatyw. To daje czytelny kod przy małej liczbie przypadków, ale może zwiększyć liczbę instancjacji szablonów.

```
#include <cassert>
#include <variant>

int main() {
    std::variant<int, double> a = 3;
    std::variant<int, double> b = 2.5;
    const double sum = std::visit(
        [](const auto& x, const auto& y) -> double {
            return static_cast<double>(x) + static_cast<double>(y);
        }, a, b);
    assert(sum == 5.5);
}
```

Listing 9.8: Odwiedzenie dwóch wariantów naraz; kompletny program.

9.3.3 Modyfikacja stanu i kwalifikacja `const`

`std::visit` respektuje typ i kategorię wartości przekazanego wariantu. Odwiedzając obiekt przez `variant&`, możemy modyfikować aktywną alternatywę; przez `const variant&` uzyskujemy referencję stałą. Przy przekazaniu `std::move(v)` odwiedzający może otrzymać referencję rvalue i przenieść przechowywaną wartość. Przeniesienie nie musi zmieniać aktywnego indeksu wariantu, ale zawartość przeniesionego obiektu może zostać zmieniona zgodnie z semantyką typu.

```cpp
#include <cassert>
#include <variant>

int main() {
    std::variant<int, double> v = 10;
    std::visit([](auto& x) { x += 1; }, v);
    assert(std::get<int>(v) == 11);
}
```

Listing 9.9: Modyfikacja aktywnej alternatywy; kompletny program.

9.3.4 Stan `valueless_by_exception`

`variant` może w wyjątkowych okolicznościach utracić aktywną alternatywę wskutek nieudanego przypisania lub rekonstrukcji typu, którego operacja zgłasza wyjątek. Stan ten identyfikujemy przez `valueless_by_exception()`, a `index()` zwraca w nim `std::variant_npos`. W takim stanie `std::visit` zgłasza `std::bad_variant_access`. Nie należy jednak traktować `valueless_by_exception()` jako zwykłej „pustej” alternatywy; przeznaczony do tego jest `std::monostate`.

Ważne ograniczenia. `variant` nie przechowuje typów referencyjnych ani `void`; nie można także dodać dowolnego nowego typu bez zmiany listy alternatyw. Duży wariant może zajmować pamięć odpowiadającą największej alternatywie plus informację o aktywnym przypadku i ewentualne wyrównanie; dokładny rozmiar nie jest stałą wielkością określoną przez standard.

9.4 Przykład projektowy: interpreter prostych poleceń

Rozpatrzmy miniaturowy model poleceń. Każdy rodzaj polecenia ma inne pola, ale wszystkie należą do zamkniętej rodziny. `variant` oddziela *dane* od *operacji*, a `visit` pozwala jednoznacznie obsłużyć każdy przypadek.

```cpp
#include <cassert>
#include <string>
#include <variant>
#include <vector>

struct Add  { int a, b; };
struct Save { std::string filename; };
struct Quit {};
using Command = std::variant<Add, Save, Quit>;

struct Execute {
    int& total;
    bool& stopped;

    void operator()(const Add& c) const {
        total += c.a + c.b;
    }
    void operator()(const Save& c) const {
        if (c.filename.empty()) stopped = true;
    }
    void operator()(const Quit&) const {
        stopped = true;
    }
};

int main() {
```

```
    std::vector<Command> commands{
        Add{2, 3}, Save{"result.txt"}, Add{4, 5}, Quit{}
    };
    int total = 0;
    bool stopped = false;
    for (const auto& command : commands) {
        if (stopped) break;
        std::visit(Execute{total, stopped}, command);
    }
    assert(total == 14);
    assert(stopped);
}
```

Listing 9.10: Modelowanie poleceń za pomocą variant; kompletny program.

Dodanie nowego typu polecenia, np. `Delete`, wymaga rozszerzenia `Command` i obsługi nowego przypadku. W dobrze skonstruowanym odwiedzającym brakująca obsługa będzie błędem kompilacji. To zaleta typów sumarycznych przy modelowaniu *zamkniętego* katalogu zdarzeń.

9.5 `std::any`: wartość o nieznanym typie

`std::any` (C++17, nagłówek `<any>`) może przechowywać obiekt dowolnego typu spełniającego wymaganie kopiowalności i pozwala później odzyskać dokładny typ przez `std::any_cast`. To klasyczna implementacja *type erasure*: użytkownik obiektu `any` nie zna szablonowego typu przechowywanej wartości.

```
#include <any>
#include <cassert>
#include <string>

int main() {
    std::any value = 17;
    assert(value.has_value());
    assert(std::any_cast<int>(value) == 17);

    if (auto ptr = std::any_cast<int>(&value)) {
        *ptr += 1;
    }
    assert(std::any_cast<int>(value) == 18);

    value.emplace<std::string>("text");
    assert(std::any_cast<int>(&value) == nullptr);
    assert(std::any_cast<const std::string&>(value) == "text");

    value.reset();
    assert(!value.has_value());
}
```

Listing 9.11: std::any i bezpieczne wydobycie typu (C++17); kompletny program.

`std::any_cast<T>(value)` zgłasza `std::bad_any_cast`, jeśli aktywny typ nie odpowiada żądanemu. Wariant wskaźnikowy `std::any_cast<T>(&value)` zwraca `nullptr`; jest zatem wygodny do ostrożnego sprawdzania typu. `type()` udostępnia `std::type_info` opisujący aktualnie przechowywany typ, a dla pustego obiektu zwraca `typeid(void)`.

```
#include <any>
#include <cassert>
#include <typeinfo>

int main() {
```

```
    std::any value = 42;
    assert(value.type() == typeid(int));
    assert(std::any_cast<double>(&value) == nullptr);
    assert(std::any_cast<int>(&value) != nullptr);
}
```

Listing 9.12: `any_cast` nie jest konwersją numeryczną; kompletny program.

`any` przechowuje *wartość*, nie udostępnia jednak uniwersalnego operatora `+`, porównania czy drukowania tej wartości. Aby wykonać operację, trzeba znać typ albo dostarczyć odrębny mechanizm dyspozycji. Ponadto przechowywany typ musi być kopiowalny: `std::any` nie nadaje się do bezpośredniego przechowywania `std::unique_ptr<T>`. Niektóre implementacje wykorzystują *small-object optimization*, ale standard nie obiecuje uniknięcia alokacji dla konkretnego małego typu.

Kiedy używać? `std::any` ma sens m.in. w luźno typowanych zasobnikach kontekstu, gdy autor zasobnika nie zna listy możliwych typów, a odbiorca umie jawnie odtworzyć typ właściwego wpisu. Jeśli rodzaje wartości są znane, `variant` daje z reguły silniejszą kontrolę statyczną i czytelniejszą obsługę przypadków. Unikaj `any` jako sposobu na ukrycie źle określonych kontraktów interfejsu.

9.6 `std::function`: wymazywanie typu obiektu wywoływalnego

`std::function<R(Args...)>` (C++11, nagłówek `<functional>`) przechowuje kopiowalny obiekt wywoływalny zgodny z podaną sygnaturą. Może to być wskaźnik do funkcji, funktor, wyrażenie lambda, a także obiekt związany z metodą składową. Konkretny typ obiektu zostaje ukryty, ale interfejs jego wywołania jest znany: `R(Args...)`.

```
#include <cassert>
#include <functional>

int times_two(int x) { return 2 * x; }
struct PlusThree {
    int operator()(int x) const { return x + 3; }
};

int main() {
    std::function<int(int)> f = times_two;
    assert(f(4) == 8);

    f = PlusThree{};
    assert(f(4) == 7);

    int delta = 5;
    f = [delta](int x) { return x + delta; };
    assert(f(4) == 9);
}
```

Listing 9.13: Jeden typ std::function dla różnych obiektów wywoływalnych; kompletny program.

9.6.1 Pusty obiekt i koszt abstrakcji

Domyślnie skonstruowany `std::function` jest pusty. Jego `operator bool()` informuje, czy istnieje obiekt wywoływalny; próba wywołania pustego obiektu zgłasza `std::bad_function_call`. W porównaniu z bezpośrednim wywołaniem szablonu `std::function` może wymagać pośredniego wywołania i dodatkowego przechowywania stanu, a czasami także alokacji dynamicznej. Nie jest jednak poprawne twierdzenie, że każdy `std::function` *zawsze* wykonuje alokację.

```cpp
#include <cassert>
#include <functional>

int main() {
    std::function<void()> callback;
    assert(!callback);
    callback = [] {};
    assert(static_cast<bool>(callback));
    callback();
    callback = nullptr;
    assert(!callback);
}
```

Listing 9.14: Pusty std::function i jawne sprawdzanie; kompletny program.

9.6.2 Ograniczenie kopiowalności i czas życia przechwyceń

W C++20 `std::function` wymaga kopiowalności typu przechowywanego obiektu wywoływalnego. Dlatego lambda zawierająca *move-only* `std::unique_ptr` nie może być bezpośrednio umieszczona w `std::function`. W C++23 pojawił się `std::move_only_function` dla takiego zastosowania.

Przechwytywanie przez referencję wymaga ponadto, aby obiekt odniesienia pozostawał żywy podczas każdego późniejszego wywołania. Sam `std::function` kontroluje czas życia *przechowywanej lambdy*, ale nie przedłuża automatycznie czasu życia obiektów, do których lambda odwołuje się przez referencje lub nieposiadające własności wskaźniki.

```cpp
#include <cassert>
#include <functional>
#include <memory>

int main() {
    auto state = std::make_shared<int>(10);
    std::function<int()> get_value = [state] {
        return *state;
    };
    state.reset();
    assert(get_value() == 10);
}
```

Listing 9.15: Callback utrzymujący obiekt przy życiu przez `shared_ptr`; kompletny program.

Trwałe przechwycenie `shared_ptr` może z kolei utworzyć cykl własności, np. gdy obiekt przechowuje callback, a callback przechwytuje `shared_ptr` do tego samego obiektu. Wówczas często lepiej przechwycić `weak_ptr` i uzyskać tymczasową własność przez `lock()`.

```cpp
#include <cassert>
#include <functional>
#include <memory>
#include <optional>

int main() {
    auto state = std::make_shared<int>(42);
    std::weak_ptr<int> weak = state;

    std::function<std::optional<int>()> read = [weak]()
        -> std::optional<int> {
        if (auto shared = weak.lock()) return *shared;
        return std::nullopt;
    };

```

```
    assert(read() == 42);
    state.reset();
    assert(!read().has_value());
}
```

Listing 9.16: Callback niewymuszający przedłużania życia obiektu; kompletny program.

9.6.3 Szablon, wskaźnik do funkcji czy `std::function`?

Jeżeli funkcja korzysta z obiektu wywoływalnego tylko podczas bieżącego wywołania, często najprościej użyć szablonu. Pozwala to kompilatorowi znać konkretny typ i daje okazję do *inliningu*. Wskaźnik do funkcji jest lekki, ale nie przechowa samodzielnie stanu przechwycenia lambdy. `std::function` ułatwia natomiast przechowywanie i zastępowanie heterogenicznych callbacków o wspólnej sygnaturze.

```
#include <cassert>
#include <functional>
#include <utility>

template<class F, class... Args>
decltype(auto) call_now(F&& f, Args&&... args) {
    return std::invoke(std::forward<F>(f),
                       std::forward<Args>(args)...);
}

int main() {
    int factor = 3;
    auto f = [factor](int x) { return factor * x; };
    assert(call_now(f, 7) == 21);
}
```

Listing 9.17: Wywołanie generyczne bez type erasure; kompletny program.

9.7 Własny mechanizm type erasure: `Printable`

Pokażemy teraz, jak samodzielnie zbudować typ o stałym interfejsie, który może przechowywać różne wartości drukowalne do strumienia. Celowo nie wykorzystamy dziedziczenia typów użytkownika po wspólnej klasie bazowej. Użytkownik powinien móc przekazać `int`, `std::string` albo własny typ, dla którego działa `operator«`. Wymagamy przy tym *semantyki wartości*: kopiowanie `Printable` ma kopiować przechowywany obiekt.

Idea implementacji. Wewnątrz opakowania znajdują się:

1. abstrakcyjny interfejs `Concept` z operacjami `print` i `clone`;
2. szablonowy model `Model<T>`, który przechowuje konkretną wartość i realizuje interfejs;
3. wskaźnik `unique_ptr<Concept>`, ukrywający konkretny typ w miejscu użycia `Printable`.

```
#include <cassert>
#include <concepts>
#include <memory>
#include <ostream>
#include <sstream>
#include <string>
#include <type_traits>
#include <utility>
```

```
9
10 class Printable {
11     struct Concept {
12         virtual ~Concept() = default;
13         virtual void print(std::ostream&) const = 0;
14         virtual std::unique_ptr<Concept> clone() const = 0;
15     };
16
17     template<class T>
18     struct Model final : Concept {
19         T value;
20         template<class U>
21         explicit Model(U&& x) : value(std::forward<U>(x)) {}
22         void print(std::ostream& os) const override {
23             os << value;
24         }
25         std::unique_ptr<Concept> clone() const override {
26             return std::make_unique<Model<T>>(value);
27         }
28     };
29
30     std::unique_ptr<Concept> self_;
31
32 public:
33     Printable() = default;
34
35     template<class T>
36       requires (!std::same_as<std::remove_cvref_t<T>, Printable> &&
37                 std::copy_constructible<std::decay_t<T>> &&
38                 requires(std::ostream& os,
39                          const std::decay_t<T>& value) {
40                     { os << value } -> std::same_as<std::ostream&>;
41                 })
42     Printable(T&& value)
43         : self_(std::make_unique<Model<std::decay_t<T>>>(
44               std::forward<T>(value))) {}
45
46     Printable(const Printable& other)
47         : self_(other.self_ ? other.self_->clone() : nullptr) {}
48
49     Printable(Printable&&) noexcept = default;
50
51     Printable& operator=(const Printable& other) {
52         if (this != &other) {
53             Printable tmp(other); // najpierw bezpieczna kopia
54             swap(tmp);            // potem nie-rzucajaca zamiana
55         }
56         return *this;
57     }
58
59     Printable& operator=(Printable&&) noexcept = default;
60
61     void swap(Printable& other) noexcept {
62         self_.swap(other.self_);
63     }
64
65     void print(std::ostream& os) const {
66         if (self_) self_->print(os);
67         else os << "<empty>";
68     }
69 };
70
71 int main() {
72     Printable a = 17;
```

```
    Printable b = std::string("hello");
    Printable c = b; // gleboka kopia wartosci
    b = 7.5;

    std::ostringstream stream;
    a.print(stream);
    c.print(stream);
    assert(stream.str() == "17hello");
}
```

Listing 9.18: Własne kopiowalne type erasure (C++20); kompletny program.

9.7.1 Analiza własności i kosztów

Ten interfejs jest *otwarty*: każdy nowy typ spełniający wymagania może zostać opakowany bez modyfikowania klasy `Printable`. Użytkownik nie musi dziedziczyć po klasie bazowej, co jest ważną zaletą w porównaniu ze zwykłym polimorfizmem opartym na `Base`. `Model<T>` jest instancjonowany w czasie kompilacji, natomiast `print()` jest wywoływane dynamicznie przez ukryty interfejs. Kopiowanie przez `clone()` zachowuje rzeczywisty typ wartości.

Wersja edukacyjna wykorzystuje osobną alokację dynamiczną przy przechowywaniu niepustej wartości i kolejną alokację przy głębokim kopiowaniu. Nie wykorzystuje *small-buffer optimization*, które mogłoby przechowywać niewielkie obiekty bezpośrednio wewnątrz opakowania. W produkcyjnym type erasure trzeba dodatkowo przemyśleć *wyrównanie*, przenoszenie, bezpieczne niszczenie, usuwanie typów niekopiowalnych, kwalifikatory `const`, obsługę wyjątków i stabilność ABI.

Gwarancja silna przypisania kopiującego. Tworzenie obiektu `tmp(other)` może zakończyć się wyjątkiem. Wówczas obiekt docelowy pozostaje niezmieniony. Jeśli kopia się uda, `swap(tmp)` jest operacją `noexcept`; stara zawartość zostanie zniszczona razem z `tmp`. Jest to klasyczny idiom *copy-and-swap*. Alternatywą jest niekopiowalne opakowanie, które usuwa operacje kopiowania i przechowuje tylko obiekty przenoszalne.

9.8 Type erasure a dziedziczenie i polimorfizm statyczny

9.8.1 Wspólna baza klas

Gdy typy zostały zaprojektowane jako część wspólnej hierarchii, `std::unique_ptr<Base>` pozwala przechowywać obiekty o różnych typach dynamicznych i wywoływać ich operacje wirtualne. Implementacje muszą dziedziczyć po `Base`, a tworzenie kopii polimorficznej wymaga własnego protokołu (np. `clone()`). Z drugiej strony `unique_ptr` jasno wyraża własność i nie kopiuje obiektów wskutek zwykłego kopiowania uchwytu (ponieważ kopiowanie jest zabronione).

```
#include <cassert>
#include <memory>
#include <string>
#include <vector>

struct Shape {
    virtual ~Shape() = default;
    virtual int area() const = 0;
};
struct Square final : Shape {
    int side;
    explicit Square(int s) : side(s) {}
    int area() const override { return side * side; }
```

```
};
struct Rectangle final : Shape {
    int width, height;
    Rectangle(int w, int h) : width(w), height(h) {}
    int area() const override { return width * height; }
};

int main() {
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Square>(3));
    shapes.push_back(std::make_unique<Rectangle>(2, 5));
    assert(shapes[0]->area() + shapes[1]->area() == 19);
}
```

Listing 9.19: Polimorfizm przez bazę i `unique_ptr`; kompletny program.

9.8.2 Kiedy nie wymazywać typu?

Jeżeli algorytm potrzebuje konkretnego typu wyłącznie podczas jednego wywołania, parametr szablonowy z `concept` jest często prostszy i pozwala unikać alokacji. Type erasure ma sens przede wszystkim na granicy interfejsów, gdy wartości o różnych typach muszą być *przechowywane w jednej strukturze*, przekazywane przez stabilny interfejs albo zastępowane w trakcie działania programu.

Rozwiązanie	**Zalety**	**Ograniczenia**
`variant<Ts...>`	Typy znane statycznie, sprawdzanie przypadków, lokalne przechowywanie wartości	Zamknięty katalog typów, złożoność odwiedzania i rozmiar największej alternatywy
`any`	Dowolna kopiowalna wartość, wspólny typ pojemnika	Trzeba odzyskać dokładny typ; brak wspólnego interfejsu operacji
`function`	Jedna sygnatura dla różnych callbacków	Kopiowalność (C++20), pośrednictwo, możliwe alokacje
Własne type erasure	Otwarty katalog modeli bez wspólnej bazy typów klienta	Złożona implementacja, zarządzanie pamięcią i wyjątkami
`unique_ptr<Base>`	Jawna hierarchia, dziedziczone operacje, kontrola własności	Wymaga wspólnej bazy; operacje wirtualne
Szablon z `concept`	Typ znany kompilatorowi, możliwość optymalizacji	Wiele instancjacji; zwykle nie przechowa różnych typów w jednej kolekcji

9.9 Wydajność: układ pamięci i koszty dynamicznej dyspozycji

Nie ma uniwersalnej odpowiedzi na pytanie, który mechanizm jest najszybszy. Poniżej wymieniamy dominujące koszty; faktyczne zachowanie zależy od rozmiaru typów, optymalizacji kompilatora, struktury programu i rozkładu typów w danych.

- `optional<T>` dodaje znacznik obecności; dostęp do zawartości może wymagać sprawdzenia warunku.
- `variant<Ts...>` przechowuje wystarczającą ilość miejsca na największy typ i indeks aktywnej alternatywy; odwiedzający może być implementowany przez instrukcję wyboru lub tablicę funkcji, a szczegóły zależą od kompilatora.
- `any` przechowuje metadane typu oraz wartość, potencjalnie w osobno alokowanym buforze.
- `function` może wymagać pośredniego wywołania i przechowania stanu lambdy; czasami da się uniknąć alokacji.

- Własna implementacja z `unique_ptr<Concept>` wykonuje alokację dynamiczną i pośrednie wywołanie wirtualne.

Przede wszystkim trzeba mierzyć *rzeczywisty* przypadek użycia. Jednorazowy koszt konstruowania callbacku może być nieistotny, gdy jego wykonanie trwa długo, ale może dominować w pętli składającej się z milionów krótkich operacji.

9.9.1 Test bez mylenia czasu kompilacji z wykonaniem

Rzetelne porównanie wymaga rozgrzania kodu, wykorzystania odpowiedniego zestawu danych, uwzględnienia optymalizacji i sprawdzenia, czy kompilator nie usunął mierzonej pracy. Warto rozdzielać koszt *tworzenia* opakowania od kosztu *wywołania* i analizować nie tylko średni czas, ale też rozkład wyników oraz liczbę alokacji. Benchmarku porównującego dwa mechanizmy nie traktujemy jako dowodu ich przewagi we wszystkich zastosowaniach.

9.10 Granice bezpieczeństwa: wyjątki, czas życia i współbieżność

Wyjątki. `optional::value()`, `variant::get`, `any_cast` i wywołanie pustego `std::function` mogą zgłaszać odpowiednie wyjątki. Konstrukcja zawartości i przypisanie także mogą rzucać, ponieważ wykonują operacje typu przechowywanego. Należy rozróżniać *brak wartości* (normalny stan `optional`) od *niepowodzenia operacji*, któremu warto nadać osobną reprezentację.

Czas życia. `variant`, `optional` i `any` posiadają własne przechowywane wartości. Można jednak umieścić w nich wskaźniki, `reference_wrapper` albo inne nieposiadające własności uchwyty; wtedy odpowiedzialność za czas życia wskazywanych danych pozostaje po stronie programisty. To samo dotyczy referencji przechwyconych przez lambdę umieszczoną w `std::function`.

Wątki. Żaden z opisanych typów nie czyni dowolnych współbieżnych modyfikacji tego samego obiektu bezpiecznymi. Przykładowo równoczesny zapis do jednego `variant` i odczyt jego aktywnej zawartości bez synchronizacji mogą powodować wyścig danych. Gdy callback jest uruchamiany równolegle, trzeba osobno określić zasady synchronizacji stanu przechwyconego przez lambdę.

9.11 Nowości po C++20 i świadomy wybór standardu

W całym rozdziale przykłady są zgodne z C++20. W nowszych standardach pojawiają się udogodnienia, które warto znać, ale których *nie wolno* zakładać przy kompilacji z `-std=c++20`:

- **C++23:** `std::expected<T,E>` — wynik albo jawna wartość błędu;
- **C++23:** `std::move_only_function` — opakowanie dla obiektów wywoływalnych, które nie muszą być kopiowalne;
- **C++23:** monadyczne operacje `optional::and_then`, `optional::transform` i `optional::or_else`;
- **C++26:** `std::function_ref` — niewłaścicielski widok na obiekt wywoływalny, którego czas życia musi zostać zapewniony przez wywołującego.

Dostępność funkcji bibliotecznych w konkretnym kompilatorze i jego bibliotece standardowej może być opóźniona w stosunku do daty opublikowania standardu.

9.12 Zasady projektowania i typowe błędy

1. Wybieraj `optional<T>`, gdy brak wyniku jest normalny i nie potrzebujesz szczegółowego typu błędu.
2. Wybieraj `variant`, gdy zestaw przypadków jest znany i zamknięty; obsługuj je przez `visit`, jeśli pozwala to uniknąć łańcucha ręcznych kontroli typu.
3. Stosuj `any` tylko tam, gdzie świadomie rezygnujesz ze statycznie określonego zestawu typów i masz jasny protokół odzyskiwania ich wartości.
4. Używaj `std::function` do *przechowywanych*, kopiowalnych callbacków; dla funkcji wykonywanej natychmiast rozważ parametr szablonowy.
5. Nie zakładaj, że `std::function` zawsze alokuje ani że `std::variant` zawsze jest szybszy. Zmierz przypadek zastosowania.
6. Nie pozostawiaj referencji lub wskaźników do obiektów, których czas życia zakończy się przed użyciem przechowywanego callbacku.
7. Unikaj `std::get` i `any_cast` w sytuacjach, w których nie masz gwarancji typu; do sprawdzania wykorzystuj `get_if` i wskaźnikowy `any_cast`.
8. Nie używaj `std::any` do bezpośredniego opakowania typów przenoszalnych, ale niekopiowalnych, np. `unique_ptr`.
9. Rozdziel decyzję o *otwartym zbiorze implementacji* od decyzji o *dynamicznym czasie życia i alokacji*.

Podsumowanie rozdziału

`optional` modeluje brak lub obecność wartości, a `variant` modeluje zamkniętą sumę przypadków. `any` ukrywa typ kopiowalnej wartości, nie narzucając jej operacji. `function` ukrywa typ obiektu wywoływalnego przy zachowaniu ustalonej sygnatury. Własne type erasure pozwala opakować otwartą rodzinę obcych typów bez wymagania wspólnej klasy bazowej, ale wymaga starannego określenia semantyki własności, kopiowania i wyjątków. Dobry projekt zaczyna się więc od określenia kontraktu: *które przypadki istnieją, kto posiada obiekty, jakie operacje udostępniamy oraz czy zbiór typów ma pozostać otwarty.*

Dokumentacja i literatura

- https://en.cppreference.com/w/cpp/utility/optional — `std::optional`.
- https://en.cppreference.com/w/cpp/utility/variant — `std::variant`.
- https://en.cppreference.com/w/cpp/utility/any — `std::any`.
- cppreference: `std::function` — dokumentacja opakowania wywołań.
- Bjarne Stroustrup, *A Tour of C++*, wyd. 3, Addison–Wesley, 2022.
- Klaus Iglberger, *C++ Software Design*, O'Reilly, 2022.

Rozdział 10

Szablony o zmiennej liczbie argumentów (*variadic templates*)

10.1 Idea i zastosowania

Zwykły szablon funkcji przyjmuje z góry ustaloną liczbę parametrów szablonu. *Variadic template* (szablon o zmiennej liczbie parametrów) może natomiast przyjmować *dowolną liczbę* argumentów szablonu, również zero. Mechanizmem, który to umożliwia, jest *pakiet parametrów* (ang. *parameter pack*).

```
template<class T>
void single(const T&) {}        // one type

template<class... Ts>
void many(const Ts&...) {}      // zero or more types

void demo() {
    many();                     // zero arguments
    many(10);                   // one argument
    many(10, 3.5, "abc");       // three arguments
}
```

Pakiety parametrów są ważną częścią nowoczesnego C++. Wykorzystują je m.in. klasy `std::tuple` i `std::variant` oraz fabryki `std::make_unique` i `std::make_shared`. Nie trzeba dzięki nim pisać przeciążeń dla każdej liczby argumentów.

Wymagane wersje języka. Pakiety parametrów i perfect forwarding pojawiły się w C++11. *Fold expressions* dodano w C++17, a koncepcje (`concepts`) i klauzulę `requires` w C++20. Dalej zakładamy C++20.

10.2 Trzy kropki: deklaracja pakietu a rozwinięcie

W różnych miejscach token `...` oznacza różne operacje. Rozróżnienie to stanowi podstawę poprawnego używania szablonów wariadycznych.

```
template<class... Ts>           // declare type pack Ts
void f(Ts... xs) {              // declare function parameter pack xs
    // ...
}
```

Deklaracja `class... Ts` wprowadza pakiet *typów*. Zapis `Ts... xs` rozwija pakiet typów i wprowadza odpowiadający mu pakiet parametrów funkcji. Wywołanie `f(1, 2.0, 'x')` tworzy w uproszczeniu funkcję o parametrach `(int, double, char)`.

10.2.1 Rozwinięcie pakietu (*pack expansion*)

Rozwinięcie składa się ze *wzorca* (*pattern*) oraz następującego po nim `...`. Dla każdego elementu pakietu kompilator podstawia kolejny element do całego wzorca:

```
template<class... Ts>
void demo(Ts... xs) {
    g(xs...);               // g(x1, x2, ..., xn)
    h((xs * 2)...);         // h(x1*2, x2*2, ..., xn*2)
}
```

W drugim wywołaniu wzorcem jest `(xs * 2)`, a *nie* pojedyncza zmienna `xs`. Ważne: `xs...` nie jest kontenerem ani tablicą; nie można po nim bezpośrednio iterować zwykłą pętlą `for`. To składnia kompilatora służąca do generowania listy wyrażeń.

Pakiet typów a pakiet wartości. W przykładzie `Ts` reprezentuje typy, natomiast `xs` to konkretne parametry funkcji. Jeden typ może wystąpić w pakiecie wielokrotnie, a każdy element pakietu wartości zachowuje swój właściwy typ.

10.2.2 Liczba elementów: `sizeof...`

Operator `sizeof...` daje liczbę elementów pakietu jako stałą czasu kompilacji. Nie należy go mylić z operatorem `sizeof` mierzącym rozmiar typu lub obiektu.

```
#include <cstddef>

template<class... Ts>
constexpr std::size_t count(Ts&&... xs) {
    static_assert(sizeof...(Ts) == sizeof...(xs));
    return sizeof...(xs);
}

static_assert(count() == 0);
static_assert(count(2, 3.5, 'a') == 3);
```

Nazwy `xs` nie używamy w ciele funkcji do odczytywania wartości; operacja liczenia jest czysto składniowa.

10.3 Pierwsze praktyczne funkcje

10.3.1 Wypisywanie dowolnej liczby argumentów

Najprostszy sposób wykorzystania pakietu to jego rozwinięcie w wywołaniu innej funkcji:

```
#include <iostream>
#include <string>

void output(int x, double y, const std::string& z) {
    std::cout << x << ' ' << y << ' ' << z << '\n';
}

```

```
template<class... Args>
void forward_to_output(Args... args) {
    output(args...);
}

int main() {
    forward_to_output(3, 2.5, std::string{"hello"});
}
```

Funkcja jest poprawna tylko dla takich wywołań, z których można utworzyć poprawne wywołanie `output`. Sama wariadyczność nie oznacza, że każda liczba i każdy rodzaj argumentów będą akceptowane przez funkcję docelową.

10.3.2 Przekazywanie przez wartość i przez referencję

```
template<class... Ts>
void by_value(Ts... xs);            // may copy/move

template<class... Ts>
void by_const_ref(const Ts&... xs); // read-only; no copies

template<class... Ts>
void by_ref(Ts&... xs);             // needs lvalues
```

Wybór postaci parametrów ma te same skutki co w zwykłych funkcjach. W szczególności `Ts... xs` tworzy osobne parametry przez wartość, a `const Ts&... xs` zwykle pozwala uniknąć niepotrzebnego kopiowania. Przypadek `Ts&&... xs` omówimy dokładniej w sekcji o *perfect forwarding*.

10.4 Rekurencyjne przetwarzanie pakietu: metoda C++11

Przed C++17 często rozbijano pakiet na *pierwszy argument* i *pozostałe argumenty*, a następnie wywoływano funkcję rekurencyjnie. Potrzebny był osobny przypadek kończący rekurencję.

```
#include <iostream>

void print_recursive() {
    std::cout << '\n';        // base case
}

template<class First, class... Rest>
void print_recursive(const First& first,
                     const Rest&... rest) {
    std::cout << first << ' ';
    print_recursive(rest...); // one argument fewer
}

int main() {
    print_recursive(1, "two", 3.0);
}
```

Przy trzech argumentach instancjonowane są kolejno wersje z pakietami o długościach 3, 2, 1 oraz przypadek końcowy. Jest to *rekurencja szablonowa*; nie należy automatycznie utożsamiać jej z dynamiczną rekurencją wykonywaną w programie. Kompilator może wygenerować zwykłą sekwencję operacji.

Kiedy stosować? Rekurencyjne rozkładanie pakietu nadal bywa przydatne, jeśli kolejne kroki mają odrębne warunki lub należy budować typy rekurencyjne. Jeżeli wykonujemy tę samą operację dla wszystkich argumentów, w C++17 i nowszych zazwyczaj lepszym wyborem jest *fold expression*.

10.5 Fold expressions (C++17)

Fold expression redukuje pakiet wyrażeń z użyciem jednego operatora binarnego, np. `+`, `&&`, `||` lub operatora przecinka. Nie wymaga jawnego przypadku rekurencyjnego.

10.5.1 Cztery postacie składni

Niech `xs` oznacza pakiet, $\circ$ ustalony operator, a z wartość początkową. Istnieją cztery postacie składni:

Postać C++	Sposób łączenia dla trzech argumentów
`(xs op ...)`	$x_1 \circ (x_2 \circ x_3)$ (prawostronne, bez inicjalizatora)
`(... op xs)`	$(x_1 \circ x_2) \circ x_3$ (lewostronne, bez inicjalizatora)
`(xs op ... op z)`	$x_1 \circ (x_2 \circ (x_3 \circ z))$ (prawostronne, z inicjalizatorem)
`(z op ... op xs)`	$((z \circ x_1) \circ x_2) \circ x_3$ (lewostronne, z inicjalizatorem)

W tabeli `op` jest jedynie oznaczeniem miejsca na rzeczywisty operator, np. `+`; nie jest słowem kluczowym języka. *Nawiasy całego fold expression są obowiązkowe.* Dwa operatory występujące w fold z inicjalizatorem muszą być takie same.

10.5.2 Suma, iloczyn i warunki logiczne

```
#include <concepts>

template<class... Ts>
    requires (std::integral<Ts> && ...)
constexpr auto sum(Ts... xs) {
    return (0 + ... + xs);       // left fold with identity 0
}

template<class... Ts>
constexpr bool all_true(Ts... xs) {
    return (true && ... && static_cast<bool>(xs));
}

template<class... Ts>
constexpr bool any_true(Ts... xs) {
    return (false || ... || static_cast<bool>(xs));
}

static_assert(sum() == 0);
static_assert(sum(2, 3, 4) == 9);
static_assert(all_true());
static_assert(!any_true());
static_assert(all_true(1, 2, 3));
```

Wyrażenie `(0 + ... + xs)` może być używane także dla pustego pakietu. Początkowe `0` wpływa jednak na typ wyniku i ewentualne promocje arytmetyczne; nie jest to uniwersalna implementacja sumy dowolnych typów liczbowych.

10.5.3 Lewostronne i prawostronne składanie nie zawsze są równe

Dla operatora odejmowania kolejność nawiasowania zmienia wynik:

```
template<class... Ts>
constexpr auto left_sub(Ts... xs) {
    return (100 - ... - xs);
}

template<class... Ts>
constexpr auto right_sub(Ts... xs) {
    return (xs - ... - 100);
}

static_assert(left_sub(10, 3) == 87);  // (100 - 10) - 3
static_assert(right_sub(10, 3) == 107); // 10 - (3 - 100)
```

Fold określa *strukturę nawiasów*, ale nie należy mylić tej własności z ogólną gwarancją kolejności wykonywania wszystkich podwyrażeń dla dowolnego operatora. Reguły ewaluacji wynikają również z semantyki danego operatora.

10.5.4 Pusty pakiet i element neutralny

Dla *jednoargumentowego* fold z pustym pakietem C++ dopuszcza zaledwie trzy operatory: `&&` zwraca `true`, `||` zwraca `false`, a `,` daje wyrażenie typu `void`. Pusty pakiet w `(... + xs)` nie jest poprawny. Stosując wersję z wartością początkową, możemy jawnie obsłużyć pusty pakiet:

```
template<class... Ts>
constexpr auto product(Ts... xs) {
    return (1 * ... * xs);
}

static_assert(product() == 1);
static_assert(product(2, 3, 4) == 24);
```

10.6 Operator przecinka i kolejność wykonywania

Operator przecinka jest wygodny, gdy dla każdego elementu pakietu chcemy wykonać instrukcję o efektach ubocznych. W odróżnieniu od kolejności ewaluacji zwykłych argumentów funkcji, jego wbudowana postać gwarantuje wykonanie lewej strony przed prawą.

```
#include <iostream>

template<class... Ts>
void print_all(const Ts&... xs) {
    ((std::cout << xs << ' '), ...);
    std::cout << '\n';
}

int main() {
    print_all(10, 2.5, "hello");
    print_all();                // only a newline
}
```

Zewnętrzne nawiasy tworzą fold, a wewnętrzne oddzielają wyrażenie wypisujące od operatora `,`.

Kolejność argumentów zwykłej funkcji

```
template<class... Ts>
void sink(Ts...);

// sink(change_a(), change_b(), change_c());
```

Samo rozwinięcie `...` w liście argumentów funkcji *nie* gwarantuje, że `change_a()` zostanie obliczone przed `change_b()`. W C++17 i nowszych wywołania te nie przeplatają swoich efektów, ale ich wzajemna kolejność nie jest określona. Gdy kolejność jest istotna, można użyć operatora przecinka lub innej konstrukcji dającej odpowiednie gwarancje sekwencjonowania.

10.7 Pakiety typów i metaprogramowanie

Wariadyczność może dotyczyć nie tylko funkcji, lecz także *szablonów klas*. W takim przypadku pakiet przechowuje listę typów, a instancjonowanie klasy odbywa się podczas kompilacji.

10.7.1 Własna lista typów

```
#include <cstddef>
#include <type_traits>

template<class... Ts>
struct type_list {};

template<class List>
struct list_size;

template<class... Ts>
struct list_size<type_list<Ts...>>
    : std::integral_constant<std::size_t, sizeof...(Ts)> {};

using Types = type_list<int, double, char>;
static_assert(list_size<Types>::value == 3);
static_assert(list_size<type_list<>>::value == 0);
```

Jest to przykład *częściowej specjalizacji* szablonu klasy. Wzorzec `type_list<Ts...>` dopasowuje dowolną listę typów, a `sizeof...(Ts)` oblicza jej długość.

10.7.2 Wydobywanie pierwszego typu

```
template<class List>
struct first_type;   // intentionally undefined for empty list

template<class T, class... Rest>
struct first_type<type_list<T, Rest...>> {
    using type = T;
};

using X = first_type<type_list<long, float>>::type;
static_assert(std::is_same_v<X, long>);
```

Rozkład na `T` oraz `Rest...` jest odpowiednikiem rekurencyjnego rozdzielenia argumentów funkcji na pierwszy i pozostałe. Dla pustej listy `type_list<>` nazwa `first_type<...>::type` celowo nie istnieje.

10.7.3 Czy wszystkie typy spełniają własność?

```cpp
#include <type_traits>

template<class... Ts>
inline constexpr bool all_integral_v =
    (std::is_integral_v<Ts> && ...);

template<class... Ts>
inline constexpr std::size_t count_integral_v =
    (std::size_t{0} + ... +
        (std::is_integral_v<Ts> ? 1u : 0u));

static_assert(all_integral_v<int, long, char>);
static_assert(!all_integral_v<int, double>);
static_assert(count_integral_v<int, double, char> == 2);
```

Jest to częsty wzorzec: pakiet typów łączymy z cechami typów (*type traits*) i operatorami logicznymi fold.

10.8 Pakiety wartości i parametry nietypowe

Pakiet może także zawierać wartości znane w czasie kompilacji, a nie typy. W szablonie klasy służą do tego *nietypowe parametry szablonu* (*non-type template parameters*).

```cpp
#include <array>
#include <cstddef>

template<int... Ns>
struct int_pack {
    static constexpr std::size_t size = sizeof...(Ns);
    static constexpr int sum = (0 + ... + Ns);
};

static_assert(int_pack<>::sum == 0);
static_assert(int_pack<2, 4, 6>::sum == 12);
```

Od C++17 można zadeklarować pakiet `auto...`:

```cpp
template<auto... Values>
struct value_pack {
    static constexpr std::size_t size = sizeof...(Values);
};

static_assert(value_pack<42, 'x', true>::size == 3);
```

Dla `auto...` dopuszczalność konkretnej wartości jako argumentu szablonu nadal podlega regułom parametrów nietypowych; nie można przekazać do niego dowolnej wartości z czasu wykonania.

10.9 Przekazywanie doskonałe (*perfect forwarding*)

Jednym z najważniejszych zastosowań pakietów jest budowanie *funkcji pośredniczących*, które przekazują argumenty do innej funkcji lub konstruktora, możliwie wiernie zachowując ich kategorię wartości (*value category*).

10.9.1 Różnica między lvalue a rvalue

```cpp
#include <iostream>

void use(int&)  { std::cout << "lvalue\n"; }
void use(int&&) { std::cout << "rvalue\n"; }

int main() {
    int x = 10;
    use(x);       // lvalue
    use(20);      // rvalue
}
```

Parametr funkcji, choćby zadeklarowany jako `T&&`, jest *nazwanym obiektem*; użycie jego nazwy w wyrażeniu ma kategorię *lvalue*. Dlatego samo `use(arg)` nie zachowa kategorii pierwotnego argumentu.

10.9.2 Referencje przekazujące i `std::forward`

```cpp
#include <utility>

template<class T>
void wrapper(T&& arg) {
    use(std::forward<T>(arg));
}

void forwarding_example() {
    int y = 0;
    wrapper(y);  // T = int&, forwarded as lvalue
    wrapper(30); // T = int, forwarded as rvalue
}
```

`T&&` jest *referencją przekazującą* (ang. *forwarding reference*), gdy `T` jest odpowiednio dedukowanym, niekwalifikowanym parametrem typu. Działa tu *składanie referencji*:

$$T\& \;\&\longrightarrow T\&, \qquad T\& \;\&\&\longrightarrow T\&, \qquad T\&\& \;\&\&\longrightarrow T\&\&.$$

W zapisie `std::forward<T>(arg)` jawny argument `T` jest konieczny, by odtworzyć pierwotną kategorię. `std::forward` nie wykonuje bezwarunkowego przeniesienia: przepuszcza lvalue jako lvalue, a rvalue jako rvalue.

10.9.3 Pakiet referencji przekazujących

```cpp
#include <functional>
#include <utility>

template<class F, class... Args>
decltype(auto) call(F&& f, Args&&... args) {
    return std::invoke(
        std::forward<F>(f),
        std::forward<Args>(args)...
    );
}
```

Tutaj rozwijany jest *cały wzorzec* `std::forward<Args>(args)`, a nie tylko nazwa `args`. Każdy element używa właściwego typu z pakietu `Args`. `std::invoke` potrafi wywoływać zwykłe funkcje,

obiekty funkcyjne i odpowiednio obsłużone wskaźniki do składowych. `decltype(auto)` zachowuje typ zwracany przez docelowe wywołanie, w tym referencję, jeżeli docelowa funkcja ją zwraca.

```
#include <string>

std::string join(std::string a, std::string b) {
    return a + b;
}

auto result = call(join, std::string{"ab"}, "cd");
// result == "abcd"
```

10.9.4 Czego nie robić: `std::move` i nazwy parametrów

```
template<class T>
void wrong_1(T&& arg) {
    use(arg);              // arg is always an lvalue here
}

template<class T>
void wrong_2(T&& arg) {
    use(std::move(arg));   // forces an rvalue even for lvalues
}
```

`std::move` to rzutowanie umożliwiające potraktowanie obiektu jak rvalue, a nie polecenie "zachowaj kategorię". Stosuj je, gdy rzeczywiście przekazujesz obiekt do operacji, która może pobrać z niego zasób. W uniwersalnym wrapperze używaj `std::forward`.

Ograniczenia żywotności. Referencje przekazujące *nie przedłużają automatycznie* żywotności obiektu do czasu dowolnego późniejszego wykonania. Przechowanie referencji do argumentu tymczasowego w strukturze, która przeżyje wywołanie, może prowadzić do wskaźników lub referencji wiszących. Ten sam problem dotyczy między innymi `std::forward_as_tuple`.

10.10 Tworzenie obiektów z dowolnych argumentów

10.10.1 Własna fabryka obiektów

```
#include <memory>
#include <utility>

template<class T, class... Args>
std::unique_ptr<T> create(Args&&... args) {
    return std::make_unique<T>(
        std::forward<Args>(args)...
    );
}

struct Point {
    int x, y;
    Point(int a, int b) : x(a), y(b) {}
};

auto point = create<Point>(4, 7);
```

`std::make_unique` już dostarcza taką funkcjonalność, więc `create` ma tu charakter dydaktyczny. Pokazuje jednak wzorzec używany w fabrykach oraz funkcjach `emplace` i `make`.

10.10.2 Tworzenie kontenera bez wymogu kopiowalności

```
#include <concepts>
#include <string>
#include <utility>
#include <vector>

template<class T, class... Args>
    requires (std::constructible_from<T, Args&&> && ...)
std::vector<T> make_vector(Args&&... args) {
    std::vector<T> result;
    result.reserve(sizeof...(Args));
    (result.emplace_back(std::forward<Args>(args)), ...);
    return result;
}

auto words = make_vector<std::string>("A", "BB", "CCC");
```

Operator przecinka daje tu jednoznaczną kolejność wywołań `emplace_back`. Funkcja przyjmuje także pusty pakiet argumentów. W przeciwieństwie do budowania wektora z listy inicjalizacyjnej pozwala łatwo przekazywać do kolejnych `emplace_back` argumenty przenoszalne, ale niekopiowalne. Dla elementów konstruowanych z więcej niż jednego parametru wymagałaby innego interfejsu grupowania argumentów.

10.11 `std::tuple`, `std::apply` i `std::index_sequence`

10.11.1 Heterogeniczna krotka `std::tuple`

`std::tuple<Ts...>` to kontener o strukturze opisanej listą typów znaną w czasie kompilacji. Każda pozycja może mieć inny typ:

```
#include <tuple>
#include <string>

std::tuple<int, std::string, double> t{7, "Ada", 3.5};

int n = std::get<0>(t);
const std::string& name = std::get<1>(t);
```

W przeciwieństwie do `std::vector<T>`, krotka nie przechowuje dowolnej liczby elementów jednego typu, lecz *ustaloną na etapie kompilacji* listę typów.

10.11.2 Rozwinięcie krotki do argumentów funkcji

Funkcja `std::apply` (C++17) wywołuje funkcję, przekazując elementy obiektu *tuple-like* jako osobne argumenty:

```
#include <iostream>
#include <tuple>

void apply_example() {
    std::apply([](const auto&... xs) {
        ((std::cout << xs << ' '), ...);
        std::cout << '\n';
    }, t);
}
```

`std::apply` oszczędza ręcznego konstruowania sekwencji indeksów, ale warto wiedzieć, jak taki mechanizm można zaimplementować.

10.11.3 Pakiety indeksów: `std::index_sequence`

```
#include <cstddef>
#include <iostream>
#include <tuple>
#include <utility>

template<class Tuple, std::size_t... I>
void print_tuple_impl(const Tuple& t,
                      std::index_sequence<I...>) {
    ((std::cout << std::get<I>(t) << ' '), ...);
    std::cout << '\n';
}

template<class... Ts>
void print_tuple(const std::tuple<Ts...>& t) {
    print_tuple_impl(t, std::index_sequence_for<Ts...>{});
}

void tuple_example() {
    print_tuple(std::make_tuple(1, "abc", 3.0));
}
```

`std::index_sequence<0,1,2>` jest specjalizacją szablonu przyjmującego pakiet indeksów całkowitych. `std::index_sequence_for<Ts...>` generuje indeksy $0, 1, \ldots, n-1$ dla pakietu długości n. Wzorzec `std::get<I>(t)...` rozwija wyrażenie osobno dla każdego indeksu.

10.11.4 Równoległe rozwijanie pakietów

W jednym wzorcu można użyć kilku pakietów. Muszą mieć *tę samą długość*, jeśli są rozwijane równolegle:

```
#include <tuple>
#include <type_traits>
#include <utility>

template<class A, class B>
struct zipped_types;

template<class... Ts, class... Us>
struct zipped_types<std::tuple<Ts...>, std::tuple<Us...>> {
    static_assert(sizeof...(Ts) == sizeof...(Us));
    using type = std::tuple<std::pair<Ts, Us>...>;
};

using Z = zipped_types<
    std::tuple<int, char>,
    std::tuple<double, bool>
>::type;

static_assert(std::is_same_v<
    Z, std::tuple<std::pair<int, double>,
                  std::pair<char, bool>>>);
```

Elementy `Ts` i `Us` są łączone pozycjami: $(T_1, U_1), \ldots, (T_n, U_n)$. Nie jest to iloczyn kartezjański pakietów.

10.12 Dziedziczenie wariadyczne i `std::visit`

Pakiet `Bases...` można rozwinąć na liście baz. Z kolei zapis `using Bases::operator()...` wprowadza ich operatory wywołania do jednej klasy. W ten sposób otrzymujemy zestaw przeciążeń.

```
#include <iostream>
#include <string>
#include <variant>

template<class... Fs>
struct overloaded : Fs... {
    using Fs::operator()...;
};

template<class... Fs>
overloaded(Fs...) -> overloaded<Fs...>; // C++17 deduction guide

int main() {
    std::variant<int, std::string> value = "hello";

    std::visit(overloaded{
        [](int n) { std::cout << "int: " << n << '\n'; },
        [](const std::string& s) {
            std::cout << "string: " << s << '\n';
        }
    }, value);
}
```

Jest to standardowy idiom C++17: każda lambda definiuje osobny `operator()`, a dziedziczenie wariadyczne łączy je w jeden obiekt wywoływalny. Gdy stosujemy ten wzorzec w wielu miejscach, warto umieścić `overloaded` w niewielkim nagłówku narzędziowym.

10.13 Ograniczanie pakietów za pomocą `concepts` (C++20)

Nie każda operacja ma sens dla każdej możliwej kombinacji typów. W C++20 można czytelnie określać wymagania dla *wszystkich* elementów pakietu.

```
#include <concepts>
#include <type_traits>

template<class... Ts>
    requires (std::integral<Ts> && ...)
constexpr bool all_nonnegative(Ts... xs) {
    return ((xs >= 0) && ...);
}

static_assert(all_nonnegative(0, 2, 7));
static_assert(!all_nonnegative(0, -1, 7));

// all_nonnegative(3.14); // rejected by constraints
```

Możemy również ograniczać pakiet w odniesieniu do konkretnego typu docelowego:

```
#include <concepts>

template<class T, class... Args>
concept ConstructibleWith =
    std::constructible_from<T, Args...>;

static_assert(ConstructibleWith<std::string, const char*>);
```

```
static_assert(!ConstructibleWith<int, std::string>);
```

Pierwszy przykład stosuje fold nad *warunkami dla pojedynczych typów*; drugi sprawdza *jedną własność całej sekwencji typów*, tj. czy odpowiedni konstruktor przyjmuje argumenty `Args...` w tej kolejności. Tych dwóch mechanizmów nie należy utożsamiać.

10.13.1 `if constexpr` i struktura pakietu

```
#include <iostream>

template<class... Ts>
void report(const Ts&... xs) {
    if constexpr (sizeof...(Ts) == 0) {
        std::cout << "empty\n";
    } else {
        ((std::cout << xs << ' '), ...);
        std::cout << '\n';
    }
}
```

`if constexpr` pozwala w zależności od długości pakietu wybierać gałąź w czasie kompilacji. W dobrze sformułowanym szablonie odrzucona gałąź nie jest instancjonowana dla danej specjalizacji. To użyteczne przy przypadkach brzegowych, które nie mają sensu dla pustego pakietu.

10.14 Ograniczenia, pułapki i diagnostyka

10.14.1 Pakiet typów nie jest pakietem argumentów funkcji

```
template<class... Ts>
void f(Ts... args) {
    // sizeof...(Ts): number of types
    // sizeof...(args): number of function parameters
    // args[0];  // ERROR: args is not an array
}
```

Do wybierania argumentu na ustalonej pozycji można użyć `std::tuple` i `std::get` albo wywoływać funkcję przez `std::apply`. Sam pakiet `args...` nie jest indeksowanym kontenerem.

10.14.2 Brak zgodności typów po rozwinięciu

```
template<class... Ts>
auto add(Ts... xs) {
    return (0 + ... + xs);
}

// add(1, std::string{"x"}); // ERROR: unsuitable operator+
```

Pakiet nie omija systemu typów. Każde wyrażenie powstałe po jego rozwinięciu musi być poprawne, podobnie jak zwykły kod C++. Jeżeli szablon ma określone wymagania, najlepiej zapisać je za pomocą `requires`.

10.14.3 Niewłaściwe przekazywanie kategorii wartości

```cpp
template<class F, class... Args>
void bad_call(F&& f, Args&&... args) {
    std::forward<F>(f)(args...); // loses rvalue categories
}

// Correct:
// std::forward<F>(f)(std::forward<Args>(args)...);
```

Dla typów niekopiowalnych taki błąd często ujawnia się natychmiast podczas kompilacji. Dla typów kopiowalnych może spowodować jedynie dodatkowe kopie albo zmianę wybranego przeciążenia.

10.14.4 Wnioskowanie typów i `{...}`

```cpp
#include <initializer_list>

template<class... Args>
void foo(Args&&...) {}

void braced_example() {
    // foo({1, 2, 3}); // ERROR: cannot deduce Args
    foo(std::initializer_list<int>{1, 2, 3}); // OK
}
```

Sama lista ujęta w nawiasy klamrowe `{...}` nie ma typu, który dowolny pakiet `Args&&...` potrafiłby wydedukować. Potrzebny jest określony typ parametru, np. `std::initializer_list<int>`, lub jawnie utworzony obiekt.

10.14.5 Ograniczenia położenia pakietu

W *głównej definicji szablonu klasy* pakiet parametrów szablonu musi być ostatni. Dla szablonów funkcji istnieją bardziej elastyczne reguły, ale kolejne parametry muszą być dedukowalne lub mieć argumenty domyślne, stosownie do kontekstu:

```cpp
// ERROR for a primary class template:
// template<class... Ts, class U> struct invalid;

template<class U, class... Ts>
struct valid {};
```

10.14.6 Skomplikowany kod zamiast prostego rozwiązania

Szablon wariadyczny to narzędzie, a nie obowiązkowy styl. Jeżeli argumenty mają *jednolity typ i ich liczba zmienia się w czasie wykonania*, lepszy bywa `std::span<T>`, `std::vector<T>` lub para iteratorów. Pakiet ma liczbę elementów i strukturę typów ustalaną na etapie kompilacji; nie zastępuje kontenera.

10.15 Variadic templates a funkcje wariadyczne w stylu C

Historyczne funkcje z deklaracją `void f(int, ...)` korzystają z mechanizmu argumentów wariadycznych języka C, a nie z pakietu parametrów szablonu C++:

```cpp
#include <cstdio>

```

```
int main() {
    std::printf("%d %f\n", 7, 2.5); // C-style variadic call
}
```

W formacie C zgodność argumentów ze specyfikatorami formatowania jest obowiązkiem programisty (niektóre kompilatory potrafią wykrywać część pomyłek). W szablonach wariadycznych typy argumentów pozostają dostępne kompilatorowi, dzięki czemu można wymuszać ograniczenia i stosować przeciążanie oraz `std::forward`. Nie należy jednak wyciągać z tego wniosku, że dowolny szablon automatycznie bezbłędnie formatuje argumenty.

10.16 Pełny przykład: prosty rejestrator zdarzeń

Zastosujmy omówione mechanizmy do funkcji tworzącej tekstowy komunikat z dowolnej liczby argumentów możliwych do wypisania do strumienia. Zamiast ręcznie doklejać spacje w wielu przeciążeniach, użyjemy fold nad operatorem przecinka.

```
#include <iostream>
#include <sstream>
#include <string>

// Each argument must support insertion into std::ostream.
template<class... Args>
std::string make_message(const Args&... args) {
    std::ostringstream out;
    ((out << args), ...);
    return out.str();
}

template<class... Args>
void log_line(const Args&... args) {
    std::cout << "[LOG] " << make_message(args...) << '\n';
}

int main() {
    log_line("count=", 3, ", ratio=", 0.75);
    log_line("finished");
    log_line();
}
```

Analiza. Dla argumentów `("count=",3,", ratio=",0.75)` pakiet zawiera cztery elementy. Wyrażenie `((out « args), ...)` wstawia je kolejno do strumienia, a `make_message` zwraca powstały napis. Dla pustego pakietu fold z operatorem przecinka jest poprawny, a wynikowy napis jest pusty. W wersji produkcyjnej należałoby rozważyć wymagania dotyczące formatowania, zachowanie w obliczu wyjątków, współbieżność oraz ewentualne skorzystanie z narzędzia formatującego wspieranego przez używany kompilator.

10.17 Zestawienie: kiedy używać poszczególnych mechanizmów?

Cel	Typowe rozwiązanie
Dowolna liczba argumentów o różnych typach	`template<class... Ts>` i `Ts...`
Wykonanie tej samej operacji dla każdego argumentu	Fold z operatorem `,` (C++17)
Suma, iloczyn lub wspólny warunek	Fold z `+`, `*`, `&&` lub `||`
Przekazanie argumentów dalej z zachowaniem kategorii	`Args&&...`, `std::forward<Args>(args)...`
Zapis heterogenicznych wartości	`std::tuple<Ts...>`
Wywołanie funkcji elementami krotki	`std::apply` (C++17)
Rozwinięcie krotki przy pomocy indeksów	`std::index_sequence`
Lista typów do obliczeń podczas kompilacji	Wariadyczny szablon klasy i specjalizacje
Ograniczenie do odpowiednich typów	Fold z koncepcjami i `requires` (C++20)
Zmienna długość listy jednolitych danych w czasie wykonania	`std::vector` lub `std::span`, nie pakiet

10.18 Podsumowanie i zadania do samodzielnego sprawdzenia

1. `class... Ts` deklaruje *pakiet typów*, a `Ts... xs` wprowadza *pakiet parametrów funkcji*.
2. `sizeof...(Ts)` oblicza liczbę elementów pakietu podczas kompilacji.
3. `pattern...` rozwija cały *wzorzec*, nie tylko pojedynczy identyfikator.
4. Fold expressions (od C++17) zastępują wiele prostych rekurencyjnych implementacji z C++11.
5. Dla pustego pakietu istotne są element neutralny i reguły operatora; nie każda forma fold jest poprawna.
6. W wrapperach zachowuj kategorię wartości argumentów, używając referencji przekazujących i `std::forward`.
7. `std::tuple`, `std::apply` i `std::index_sequence` współpracują z pakietami, umożliwiając operacje na heterogenicznych danych.
8. W C++20 używaj `concepts`/`requires`, aby błędne wywołania odrzucać na granicy interfejsu.

Pytania kontrolne. Spróbuj przewidzieć wyniki poniższych wyrażeń bez uruchamiania programu. Następnie zweryfikuj je za pomocą `static_assert`:

```
// static_assert(sizeof...(int) == 1); // ERROR: int is not a pack

// For the functions defined above:
static_assert(product() == 1);
static_assert(left_sub(10, 3) == 87);
static_assert(right_sub(10, 3) == 107);
static_assert(count_integral_v<int, double, char> == 2);
```

Pierwsza linia jest *celowo niepoprawna*: operator `sizeof...` wymaga nazwy pakietu, a nie zwykłego typu.

Rozdział 11

Zaawansowane metaprogramowanie: szablony, cechy typów i obliczenia podczas kompilacji

11.1 Czym jest metaprogramowanie?

Metaprogramowanie oznacza pisanie kodu, który na etapie kompilacji wyznacza typy, własności typów, wartości lub strukturę później kompilowanego programu. Szablon nie jest gotową funkcją czy klasą: stanowi przepis, z którego kompilator tworzy odpowiednie *specjalizacje* (*instantiations*). W C++20 do dyspozycji mamy kilka komplementarnych narzędzi:

- **szablony klas i funkcji** — generowanie kodu dla różnych typów;
- **`type_traits`** — obliczanie własności i przekształcanie typów;
- **specjalizacje i SFINAE** — wybór poprawnych wersji szablonów;
- **`if constexpr`, `constexpr`, `consteval`** — wykonywanie obliczeń i selekcja kodu w czasie kompilacji;
- **`concepts`** — czytelne formułowanie wymagań wobec typów;
- **CRTP** — wykorzystanie dziedziczenia do polimorfizmu statycznego.

Metaprogramowanie ma praktyczne cele: eliminowanie powtarzalnego kodu, uzyskiwanie lepszych błędów kompilacji, wybór wydajnej implementacji oraz zapewnienie poprawności określonych operacji *przed uruchomieniem* programu. Nie oznacza automatycznie zwiększenia wydajności — kosztem mogą być dłuższa kompilacja, powiększenie kodu maszynowego i trudniejsze komunikaty diagnostyczne.

```
#include <type_traits>

template<class T>
T twice(T x) {
    static_assert(std::is_arithmetic_v<T>);
    return x + x;
}

int a = twice(21);       // T = int
// twice("text");       // static_assert fails
```

Związki z wcześniejszym materiałem. Wykorzystamy szablony wariadyczne (rozdział 4), koncepcje (rozdział 5), przenoszenie i referencje (rozdział 6) oraz lambdy (rozdział 7). Wszystkie oznaczone jako poprawne przykłady odnoszą się do **C++20**.

11.2 Instancjowanie szablonów, parametry i zależne nazwy

11.2.1 Parametry typowe, nietypowe i szablonowe

Szablon może przyjmować argumenty będące typami, wartościami stałymi lub innymi szablonami. Parametr wartości (*non-type template parameter*, NTTP) w C++20 może również mieć odpowiedni typ klasowy spełniający wymagania tzw. typu strukturalnego.

```
#include <array>
#include <cstddef>

template<class T, std::size_t N>
struct Buffer {
    std::array<T, N> data{};
};

Buffer<double, 4> samples; // T=double, N=4
```

`Buffer<double,4>` i `Buffer<double,8>` są różnymi typami. Parametr `N` jest znany podczas kompilacji i może wystąpić tam, gdzie język wymaga wyrażenia stałego, np. jako rozmiar `std::array`.

11.2.2 Parametr będący obiektem klasy (C++20)

Od C++20 odpowiednie *typy strukturalne* mogą występować jako parametry szablonu będące wartościami. Umożliwia to np. identyfikowanie szablonu przez napis znany podczas kompilacji:

```
#include <cstddef>

template<std::size_t N>
struct fixed_string {
    char data[N]{};
    constexpr fixed_string(const char (&s)[N]) {
        for (std::size_t i = 0; i < N; ++i)
            data[i] = s[i];
    }
};

template<fixed_string Name>
struct NamedCommand {
    static constexpr auto name = Name;
};

static_assert(NamedCommand<"run">::name.data[0] == 'r');
```

To przykład zaawansowanej techniki; zwykły obiekt `std::string` nie jest w C++20 strukturalnym argumentem szablonu. Rozmiar i zawartość napisu wpływają tutaj na tożsamość specjalizacji szablonu.

11.2.3 Nazwy zależne: `typename` i `template`

W definicji szablonu kompilator nie zawsze wie, czy nazwa zależna od parametru jest typem, czy wartością. W wielu kontekstach musimy to jednoznacznie zaznaczyć:

```cpp
#include <vector>

template<class Container>
auto first_copy(const Container& c) {
    typename Container::value_type x = c.front();
    return x;
}

template<class T>
struct Wrapper {
    template<class U>
    static constexpr bool same = false;
};

template<class T>
constexpr bool use_member_template() {
    return Wrapper<T>::template same<int>;
}
```

Słowo `typename` wskazuje, że zależna nazwa członkowska jest typem. Zapis `::template same<int>` informuje, że po operatorze `::` występuje szablon składowy, a znaki `<` i `>` są delimitatorami argumentów szablonu. Dokładne reguły zależą od kontekstu; obu słów nie należy dopisywać mechanicznie wszędzie.

11.3 Metafunkcje i `std::integral_constant`

Klasyczne metaprogramowanie szablonowe opierało się na klasach, które *przenosiły wyniki obliczenia w swoich składowych statycznych*. Dla przykładu silnię można wyznaczyć przez rekurencyjne instancjowanie szablonów:

```cpp
template<unsigned N>
struct Factorial {
    static constexpr unsigned value =
        N * Factorial<N - 1>::value;
};

template<>
struct Factorial<0> {
    static constexpr unsigned value = 1;
};

static_assert(Factorial<5>::value == 120);
```

Specjalizacja dla zera pełni rolę warunku stopu. Bez niej kompilator próbowałby tworzyć kolejne instancje, aż osiągnąłby limit głębokości. Obliczenie ma charakter rekursji *w strukturze typów*, a nie rekursji wywołań funkcji wykonywanych w czasie działania programu.

Standardowym fundamentem takich metafunkcji jest `std::integral_constant<T,v>`:

```cpp
#include <type_traits>

using TrueType = std::integral_constant<bool, true>;
static_assert(TrueType::value);
static_assert(std::bool_constant<true>::value);
static_assert(std::true_type::value);
static_assert(!std::false_type::value);

template<unsigned N>
struct PowerOfTwo :
```

```
    std::integral_constant<unsigned, 2 * PowerOfTwo<N-1>::value> {};

template<>
struct PowerOfTwo<0> : std::integral_constant<unsigned, 1> {};

static_assert(PowerOfTwo<5>::value == 32);
```

Predykaty takie jak `std::is_pointer` udostępniają wartość `::value`. Od C++17 można używać krótszej wersji z przyrostkiem `_v`, na przykład:

`std::is_pointer_v<T>`.

11.3.1 Metafunkcja szablonowa czy funkcja constexpr?

```
constexpr unsigned factorial(unsigned n) {
    unsigned result = 1;
    for (unsigned i = 2; i <= n; ++i)
        result *= i;
    return result;
}

static_assert(factorial(5) == 120);
unsigned x = 6;
unsigned y = factorial(x); // allowed at run time
```

Jeśli wynik jest zwykłą wartością, funkcja `constexpr` jest najczęściej prostsza niż rekurencyjna metafunkcja. Klasyczne metafunkcje pozostają użyteczne, gdy *wynikiem jest typ*, gdy korzystamy ze specjalizacji lub interoperujemy ze starszym API.

11.4 Specjalizacje: pełne i częściowe

Szablon podstawowy definiuje przypadek ogólny. *Specjalizacja pełna* dotyczy konkretnego zestawu argumentów, a *specjalizacja częściowa* obsługuje pewną ich rodzinę:

```
#include <type_traits>

template<class T>
struct is_raw_pointer : std::false_type {};

template<class T>
struct is_raw_pointer<T*> : std::true_type {};

static_assert(!is_raw_pointer<int>::value);
static_assert(is_raw_pointer<double*>::value);
static_assert(!is_raw_pointer<int* const>::value);
```

Ostatni wynik nie jest pomyłką: `int* const` jest *stałym wskaźnikiem*, nie typem postaci `T*`. Jeśli chcemy ignorować kwalifikatory zewnętrzne, możemy najpierw użyć `std::remove_cv_t<T>`.

```
template<class T>
inline constexpr bool is_any_raw_pointer_v =
    is_raw_pointer<std::remove_cv_t<T>>::value;

static_assert(is_any_raw_pointer_v<int* const>);
```

Ważne ograniczenie. Dla **szablonów funkcji nie ma specjalizacji częściowej**. W takich przypadkach wykorzystujemy przeciążanie, koncepcje, `if constexpr` lub specjalizację pomocniczego szablonu klasy. Pełna specjalizacja funkcji istnieje, ale jej interakcja z przeciążeniami bywa zaskakująca; w nowym kodzie zwykle preferujemy przeciążenia.

11.5 `type_traits`: pytania o typy

Biblioteka `<type_traits>` jest zbiorem standardowych metafunkcji. Dzielimy je na *predykaty* (wynik logiczny) i *transformacje* (wynik będący typem). Ich składnia opiera się na ustalonych konwencjach:

- `Trait<T>::value` lub `Trait_v<T>` dla predykatów;
- `typename Trait<T>::type` lub `Trait_t<T>` dla transformacji.

```cpp
#include <type_traits>
#include <string>

static_assert(std::is_integral_v<int>);
static_assert(std::is_floating_point_v<double>);
static_assert(std::is_same_v<int, int>);
static_assert(std::is_pointer_v<const char*>);
static_assert(std::is_constructible_v<std::string, const char*>);
static_assert(std::is_move_constructible_v<std::string>);

using A = std::remove_const_t<const int>; // int
using B = std::remove_reference_t<int&>; // int
using C = std::remove_cvref_t<const int&>; // int (C++20)
using D = std::add_pointer_t<int>;        // int*
static_assert(std::is_same_v<A, B>);
static_assert(std::is_same_v<C, int>);
static_assert(std::is_same_v<D, int*>);
```

Cecha	Co sprawdza lub wyznacza?
`is_same_v<A,B>`	Identyczność typów (wraz z istotnymi kwalifikatorami).
`is_convertible_v<A,B>`	Dostępność konwersji niejawnej `A` do `B`.
`is_constructible_v<T,Args...>`	Możliwość skonstruowania `T` z argumentów zadanych typów.
`is_nothrow_move_constructible_v<T>`	Deklarowana bezwyjątkowość konstrukcji przenoszącej.
`is_trivially_copyable_v<T>`	Własność pozwalająca na określone operacje z reprezentacją bajtową.
`remove_cvref_t<T>`	Usunięcie referencji i kwalifikatorów `const`/`volatile` z poziomu wierzchniego.
`common_type_t<A,B>`	Wspólny typ wynikowy dla danych typów (o ile istnieje).
`invoke_result_t<F,Args...>`	Typ wyniku wywołania poprzez `std::invoke`.

`remove_cvref` a `decay`. `std::decay_t<T>` oprócz usunięcia referencji i zewnętrznych kwalifikatorów może przekształcić tablicę w wskaźnik oraz typ funkcji w wskaźnik do funkcji. Nie są to zatem synonimy.

```cpp
using X = std::remove_cvref_t<int(&)[3]>; // int[3]
using Y = std::decay_t<int(&)[3]>;        // int*
static_assert(std::is_same_v<Y, int*>);
```

Czego predykat nie gwarantuje? `std::is_move_constructible_v<T>` może być prawdziwe również wtedy, gdy wywołanie z argumentem rvalue używa konstruktora kopiującego. Podobnie `is_trivially_copyable` nie oznacza, że każdy odczyt bajtów, rzutowanie lub serializacja do pliku będzie poprawna bądź przenośna. Cechy typów opisują wybrane własności języka, nie pełny kontrakt semantyczny.

11.5.1 Składanie predykatów z zachowaniem krótkiego obliczania

`std::conjunction`, `std::disjunction` i `std::negation` łączą predykaty, które mają składową `::value`. Dwa pierwsze narzędzia zatrzymują odczytywanie `::value` z kolejnych predykatów, kiedy wynik jest już znany. Trzeba jednak poprawnie sformować same argumenty typowe tych metafunkcji.

```
#include <type_traits>

template<class T>
using SmallSignedInteger = std::conjunction<
    std::is_integral<T>,
    std::is_signed<T>,
    std::bool_constant<(sizeof(T) <= sizeof(int))>>;

static_assert(SmallSignedInteger<short>::value);
static_assert(!SmallSignedInteger<unsigned>::value);
```

Dla prostych stałych logicznych zapis `(cond1 && cond2)` bywa czytelniejszy. `conjunction` jest użyteczne przede wszystkim wtedy, gdy poszczególne warunki są odrębnymi metafunkcjami, a ich instancjowanie może być kosztowne lub niepoprawne bez wcześniejszego warunku.

11.6 Metafunkcje zwracające typy i leniwy wybór

Tworząc własną transformację typu, zwyczajowo zapisujemy wynik jako `using type = ...` i udostępniamy wygodny alias:

```
template<class T>
struct storage_type {
    using type = std::remove_cvref_t<T>;
};

template<class T>
using storage_type_t = typename storage_type<T>::type;

static_assert(std::is_same_v<storage_type_t<const long&>, long>);
```

Ważnym narzędziem jest `std::conditional_t<B,T,F>`, wybierające jeden z dwóch typów w zależności od znanej w kompilacji wartości logicznej:

```
template<class T>
using accumulator_t = std::conditional_t<
    std::is_integral_v<T>, long long, long double>;

static_assert(std::is_same_v<accumulator_t<int>, long long>);
static_assert(std::is_same_v<accumulator_t<float>, long double>);
```

Pułapka eager instantiation. `conditional_t` wybiera typ spośród *już utworzonych argumentów typowych*. Nie chroni przed błędami powstającymi podczas formowania któregoś z nich:

```
// Incorrect for int: typename T::value_type is invalid.
```

```
// template<class T>
// using bad = std::conditional_t<
//     std::is_integral_v<T>, T, typename T::value_type>;
```

Aby uzyskać prawdziwie warunkowe tworzenie typu, można zastosować specjalizację pomocniczej klasy:

```
template<class T, bool = std::is_integral_v<T>>
struct value_or_self {
    using type = typename T::value_type;
};

template<class T>
struct value_or_self<T, true> {
    using type = T;
};

template<class T>
using value_or_self_t = typename value_or_self<T>::type;

static_assert(std::is_same_v<value_or_self_t<int>, int>);
```

Podstawowa definicja zawiera nazwę zależną, która jest sprawdzana przy instancjowaniu odpowiedniego przypadku; nie jest potrzebna dla `int`.

11.7 `decltype`, `declval` i konteksty bez obliczania

`decltype(expr)` pozwala odczytać typ wyrażenia bez wykonywania samego wyrażenia. Subtelność stanowi różnica między nieujętym w nawiasy identyfikatorem a ogólnym wyrażeniem:

```
int x = 5;
static_assert(std::is_same_v<decltype(x), int>);
static_assert(std::is_same_v<decltype((x)), int&>);
```

Wyrażenie `(x)` jest *lvalue*, więc `decltype((x))` zwraca `int&`. Reguła ta ma bezpośredni związek z kategoriami wartości (rozdział 6).

`std::declval<T>()` pozwala użyć argumentu typu `T` w *kontekście niewartościowanym*, nawet jeśli nie możemy utworzyć jego obiektu. Nie wolno rzeczywiście wykonywać wywołania `declval`.

```
#include <utility>
#include <type_traits>

template<class T>
using dereference_t = decltype(*std::declval<T&>());

static_assert(std::is_same_v<dereference_t<int*>, int&>);

struct NonDefault {
    NonDefault() = delete;
    int method() const;
};

using Result = decltype(std::declval<const NonDefault&>().method());
static_assert(std::is_same_v<Result, int>);
```

Typ wyniku wywołania można również wyznaczyć za pomocą gotowej metafunkcji:

`std::invoke_result_t<F, Args...>`.

11.8 SFINAE: błędne podstawienie to nie zawsze błąd kompilacji

SFINAE (*Substitution Failure Is Not An Error*) mówi, że niektóre błędy powstałe podczas podstawiania argumentów do szablonu *nie kończą kompilacji*: dana specjalizacja zostaje wyłączona z zestawu kandydatów lub odpowiednia specjalizacja częściowa nie pasuje. Dotyczy to określonych kontekstów bezpośredniego podstawiania; **nie** jest mechanizmem przechwytującym dowolne błędy w ciele funkcji ani skutki uboczne instancjowania innych typów.

11.8.1 `std::enable_if_t`

```
#include <type_traits>

template<class T>
constexpr std::enable_if_t<std::is_integral_v<T>, T>
doubled(T x) {
    return x + x;
}

static_assert(doubled(3) == 6);
// doubled(2.5); // no matching function
```

Dla typu `double` nie istnieje `type` w `std::enable_if<false,double>`; funkcja zostaje wyłączona z odpowiedniego zbioru kandydatów. Do konstruktorów, operatorów i różnych wariantów przeciążeń istnieją odmienne miejsca umieszczania `enable_if`; łatwo przy tym uzyskać przypadkowe powtórne deklaracje lub nieczytelne błędy.

11.8.2 `void_t` — detection idiom

Od C++17 wygodnie wykrywać istnienie nazw i poprawność wyrażeń przy użyciu `std::void_t`. Jeżeli wszystkie podane argumenty są prawidłowymi typami, rezultatem jest po prostu `void`:

```
#include <type_traits>
#include <vector>

template<class, class = void>
struct has_value_type : std::false_type {};

template<class T>
struct has_value_type<T, std::void_t<typename T::value_type>>
    : std::true_type {};

static_assert(has_value_type<std::vector<int>>::value);
static_assert(!has_value_type<int>::value);
```

W podstawieniu `T=int` wyrażenie `int::value_type` nie ma sensu; specjalizacja częściowa nie jest więc wybierana. Analogicznie można badać poprawność wyrażenia z `decltype`:

```
template<class, class = void>
struct has_preincrement : std::false_type {};

template<class T>
struct has_preincrement<T,
    std::void_t<decltype(++std::declval<T&>())>>
    : std::true_type {};

static_assert(has_preincrement<int>::value);
```

11.8.3 SFINAE kontra `requires` (C++20)

Konstrukcję wykrywającą operację możemy wyrazić bezpośrednio za pomocą koncepcji z rozdziału 5:

```
#include <concepts>

template<class T>
concept Preincrementable = requires(T& x) {
    { ++x } -> std::same_as<T&>;
};

template<Preincrementable T>
void advance_one(T& x) { ++x; }
```

Metafunkcja `has_preincrement<T>` sprawdza jedynie poprawność operacji `++x`. Z kolei koncepcja `Preincrementable` dodatkowo wymaga, aby jej *typ wyniku* odpowiadał dokładnie `T&`. Różnica ta jest zamierzona: ograniczenia należy definiować zgodnie z kontraktem, a nie tylko według dostępności pewnego wyrażenia.

11.9 `if constexpr`: wybór implementacji na podstawie typu

Od C++17 często zamiast rekurencyjnych specjalizacji wybieramy gałąź wewnątrz pojedynczego szablonu. Niebrana gałąź zależna od argumentu szablonu nie jest instancjowana dla danego zestawu argumentów:

```
#include <type_traits>

template<class T>
constexpr auto number_or_value(const T& x) {
    if constexpr (std::is_arithmetic_v<T>)
        return x;
    else
        return x.value();
}

struct Box {
    constexpr int value() const { return 7; }
};
static_assert(number_or_value(3) == 3);
static_assert(number_or_value(Box{}) == 7);
```

Dlaczego nie zwykłe `if`? W zwykłym `if` kompilator musi przeanalizować obie gałęzie pod kątem poprawności dla instancjowanego kodu. Dla `int` wyrażenie `x.value()` nie jest poprawne, a sam warunek `std::is_arithmetic_v<int>` tego nie zmieni.

Ważna subtelność. `if constexpr(false)` nie jest zamiennikiem `#if 0`: poza kontekstem szablonowym odrzucona gałąź nadal podlega standardowym kontrolom poprawności. Ponadto mechanizm nie usprawiedliwia konstrukcji zawsze niepoprawnych niezależnie od argumentów szablonu.

11.10 Pakiety typów, sekwencje indeksów i fold expressions

Metaprogramowanie dobrze współpracuje z pakietami parametrów (rozdział 4). Zamiast tworzyć klasyczne rekurencyjne listy typów, często możemy używać rozwinięcia pakietu i wyrażenia zwijającego.

```cpp
#include <type_traits>

template<class... Ts>
struct type_list {};

template<class T, class... Ts>
inline constexpr bool contains_v =
    (std::is_same_v<T, Ts> || ...); // empty pack -> false

static_assert(contains_v<int, char, int, double>);
static_assert(!contains_v<long, char, int, double>);
static_assert(!contains_v<int>);
```

Zamiast tworzyć specjalizacje dla każdego elementu pakietu, korzystamy z jednego `fold expression`. Gdy potrzebna jest reprezentacja indeksów `0,1,...,N-1`, biblioteka udostępnia `std::make_index_sequence<N>`:

```cpp
#include <tuple>
#include <utility>
#include <cstddef>

template<class Tuple, std::size_t... I>
constexpr auto reversed_impl(const Tuple& t,
                             std::index_sequence<I...>) {
    return std::make_tuple(std::get<sizeof...(I)-1-I>(t)...);
}

template<class... Ts>
constexpr auto reversed(const std::tuple<Ts...>& t) {
    return reversed_impl(t, std::index_sequence_for<Ts...>{});
}

static_assert(reversed(std::tuple{1, 2, 3}) ==
              std::tuple{3, 2, 1});
```

Czy zawsze potrzebujemy indeksów? Nie. W przypadku prostego wykonania funkcji na elementach krotki w C++17 często wystarczy `std::apply`; sekwencja indeksów jest potrzebna zwłaszcza wtedy, gdy chcemy *przekształcić pozycje* lub mieć indywidualny dostęp do każdego indeksu na etapie kompilacji.

11.11 `constexpr`, `consteval` i obliczenia na danych

Szablony nie są jedynym sposobem wykonania pracy w trakcie kompilacji. Od C++11 rozwijano możliwości `constexpr`, a w C++20 mamy również `consteval`. Różnica jest istotna:

- funkcja `constexpr` *może* uczestniczyć w obliczeniach stałych, a w dozwolonych wywołaniach działa też podczas wykonania;
- wywołanie funkcji `consteval` musi spełniać reguły właściwe obliczaniu w czasie kompilacji (*immediate function*).

```cpp
constexpr int square(int x) { return x * x; }
consteval int cube_ct(int x) { return x * x * x; }

static_assert(square(4) == 16);
static_assert(cube_ct(3) == 27);

int n = 4;
int a = square(n); // runtime evaluation possible
```

```
// int b = cube_ct(n); // error: n is not constant
```

Przykładem praktycznym jest haszowanie `std::string_view` w stałym wyrażeniu. Poniższy wariant FNV-1a nadaje się do zbudowania stałej identyfikującej np. nazwę polecenia. Nie jest to hash kryptograficzny i nie wyklucza kolizji.

```
#include <cstdint>
#include <string_view>

consteval std::uint64_t fnv1a(std::string_view text) {
    std::uint64_t h = 14695981039346656037ull;
    for (unsigned char c : text) {
        h ^= c;
        h *= 1099511628211ull;
    }
    return h;
}

static_assert(fnv1a("abc") == 0xe71fa2190541574bull);
```

`constinit` i `std::is_constant_evaluated`. `constinit` zapewnia statyczną inicjalizację odpowiedniej zmiennej, lecz sam nie czyni jej stałą. Funkcja `std::is_constant_evaluated()` (C++20) pozwala w pewnych przypadkach rozpoznać, czy bieżąca ewaluacja ma charakter stały; nadużywanie jej może jednak zaciemniać semantykę funkcji.

11.12 CRTP: polimorfizm statyczny

CRTP (*Curiously Recurring Template Pattern*) polega na tym, że klasa dziedziczy po specjalizacji szablonu, której argumentem jest ona sama:

```
template<class Derived>
struct Printable {
    void print() const {
        static_cast<const Derived&>(*this).print_impl();
    }
};

struct Number : Printable<Number> {
    int value = 0;
    void print_impl() const;
};
```

Mechanizm `static_cast` jest tutaj poprawny pod założeniem, że obiekt bazowy faktycznie stanowi podobiekt `Derived`. CRTP nie zapewnia dynamicznego sprawdzania typu ani wirtualnego wywołania — poprawność wynika z konstrukcji hierarchii. Wywołanie `print_impl()` może być statycznie związane i często łatwiej je wbudować w kod wynikowy (*inline*).

Bardziej praktycznym przykładem jest domieszka (*mixin*) dostarczająca wspólną operację, lecz działająca na danych konkretnej klasy pochodnej:

```
template<class Derived>
struct TranslationMixin {
    constexpr void translate(int dx, int dy) {
        auto& self = static_cast<Derived&>(*this);
        self.x += dx;
        self.y += dy;
    }
```

```
};

struct Point : TranslationMixin<Point> {
    int x{}, y{};
    constexpr Point(int a, int b) : x(a), y(b) {}
};

constexpr bool test_translation() {
    Point p{1, 2};
    p.translate(3, 4);
    return p.x == 4 && p.y == 6;
}
static_assert(test_translation());
```

Baza `TranslationMixin<Point>` jest instancjowana z typem `Point`, chociaż sam typ `Point` nie jest jeszcze kompletny w momencie deklarowania dziedziczenia. Odwołania do składowych `x` i `y` zależą jednak od `Derived` i są sprawdzane dopiero podczas instancjowania odpowiedniej funkcji. W rzeczywistym API warto opisać wymagane składowe koncepcją lub przenieść operację do jawnie wskazanego interfejsu.

CRTP a wirtualne funkcje. Klasyczny polimorfizm dynamiczny pozwala wybierać implementację na podstawie rzeczywistego typu obiektu przez interfejs klasy bazowej, także gdy typ jest nieznany podczas kompilacji kodu wywołującego. CRTP wymaga znajomości konkretnego typu podczas instancjowania. Nie jest ogólnym zamiennikiem klas abstrakcyjnych. W C++20 prostsze zastosowania domieszek i interfejsów statycznych można często oprzeć na `concepts` i funkcjach wolnostojących.

11.13 Policy-based design: wybór zachowania przez typ

Szablon może przyjąć *politykę*, której metody definiują wariant algorytmu. Umożliwia to wybór zachowania bez wirtualnych wywołań, a kompilator może całkowicie usunąć niepotrzebne operacje:

```
struct Checked {
    static int divide(int a, int b) {
        if (b == 0) throw std::invalid_argument("zero");
        return a / b;
    }
};

struct Fast {
    static int divide(int a, int b) { return a / b; }
};

template<class Policy>
struct Calculator {
    int divide(int a, int b) const {
        return Policy::divide(a, b);
    }
};

// Calculator<Checked> safe;
// Calculator<Fast> fast; // caller must ensure b != 0
```

Polityka `Fast` nie zabezpiecza dzielenia przez zero: wykonanie `a/b` dla `b==0` jest niezdefiniowanym zachowaniem dla liczb całkowitych. Dobór polityki jest częścią kontraktu API. W przykładzie wymagany jest nagłówek `<stdexcept>`.

W realnym kodzie szablon polityki można ograniczyć koncepcją, a jeśli liczba wariantów jest

niewielka, zwykłe `if constexpr` może być łatwiejsze w utrzymaniu niż rozbudowana hierarchia polityk.

11.14 Kompletny przykład: dobór typu akumulatora

Załóżmy, że chcemy obliczać sumę liczb, lecz wybierać typ akumulatora w zależności od argumentu: dla typów całkowitych `long long`, a dla zmiennoprzecinkowych `long double`. Rozwiązanie łączy `concepts`, `type_traits` i `conditional_t`:

```
#include <concepts>
#include <type_traits>
#include <vector>

template<class T>
concept Numeric = std::integral<T> || std::floating_point<T>;

template<Numeric T>
using sum_type_t = std::conditional_t<
    std::integral<T>, long long, long double>;

template<Numeric T>
sum_type_t<T> accumulate_values(const std::vector<T>& xs) {
    sum_type_t<T> result{};
    for (const T& x : xs) {
        result += static_cast<sum_type_t<T>>(x);
    }
    return result;
}

static_assert(std::is_same_v<sum_type_t<int>, long long>);
static_assert(std::is_same_v<sum_type_t<double>, long double>);
```

Ograniczenia. Przykład wyjaśnia selekcję typu, lecz *nie daje gwarancji braku przepełnienia*. Nawet `long long` ma ograniczony zakres, a suma zmiennoprzecinkowa może akumulować błędy zaokrągleń. Najpierw należy ustalić wymagania numeryczne, a dopiero potem projektować odpowiednią metafunkcję.

11.15 Czas kompilacji, diagnostyka i praktyczne ograniczenia

Metaprogramowanie przenosi koszt części pracy z czasu wykonania na czas kompilacji, ale samo w sobie nie oznacza wydajniejszego programu. Każda odmienna specjalizacja szablonu może wymagać osobnego instancjowania i wygenerowania kodu. Powstają wtedy m.in.:

- dłuższe kompilacje i większe zużycie pamięci kompilatora;
- *code bloat*, czyli powiększenie plików wynikowych przez wiele podobnych instancji;
- skomplikowane komunikaty błędów przy zagnieżdżonych metafunkcjach;
- problemy z zależnościami między plikami nagłówkowymi;
- ograniczenia głębokości instancjowania i kosztu ewaluacji stałej.

Pliki nagłówkowe i jawne instancjowanie. Definicje szablonów zazwyczaj umieszczamy w nagłówkach, ponieważ kompilator musi je znać przy instancjowaniu z konkretnymi argumentami.

Możliwe jest również *jawne instancjowanie* dla ustalonych typów, co pozwala kontrolować zestaw generowanych specjalizacji:

```
// In a header:
template<class T> T triple(T x);
extern template int triple<int>(int);

// In one .cpp file (definition must be visible here):
template<class T> T triple(T x) { return x + x + x; }
template int triple<int>(int);
```

Ten przykład przedstawia ideę rozdzielenia deklaracji i jawnego instancjowania; obie oznaczone części należy umieścić w odpowiednich plikach, a dla wywołań innych typów zapewnić dostęp do definicji. Warto pamiętać o regule ODR z rozdziału 2.

11.16 Jaką technikę wybrać?

Potrzeba	**Najczęściej odpowiedni mechanizm**
Sprawdzenie wymaganych operacji w nowym API	`concepts`, `requires` (C++20)
Obliczenie wartości dla stałych danych	Funkcja `constexpr`
Wymuszenie ewaluacji w czasie kompilacji	`consteval` (C++20)
Warunkowy kod zależny od typu	`if constexpr`
Ustalenie własności typu	`std::is_..._v`, inne `type_traits`
Zmiana lub wybór typu wynikowego	`..._t`, specjalizacja pomocniczej klasy
Wsparcie biblioteki sprzed C++20	SFINAE, `enable_if`, `void_t`
Wieloargumentowe cechy typów	Pakiety typów, `fold expressions`
Dostęp do pozycji elementów krotki	`index_sequence`, `std::apply`
Wielokrotne użycie zachowania klasowego	CRTP / mixin, rozważ też koncepcje
Wybór wariantu strategii w typie	Szablon polityki (policy-based design)

11.17 Typowe błędy i zasady projektowe

1. **Mylenie typu z wartością.** `typename T::value_type` to typ, a `Trait<T>::value` to wartość; nie są wymienne.
2. **Brak warunku stopu.** Rekurencyjny szablon klasy może prowadzić do nieograniczonego instancjowania i błędów głębokości.
3. **Założenie, że SFINAE przechwytuje wszystkie błędy.** Działa tylko w określonych kontekstach podstawiania.
4. **Nadużywanie `conditional_t`.** Oba argumenty typowe trzeba poprawnie sformować, zanim zostanie dokonany wybór.
5. **Próba wywołania `declval`.** Można go używać wyłącznie w odpowiednich kontekstach niewartościowanych.
6. **Mylenie `decltype(x)` z `decltype((x))`.** Dodatkowe nawiasy mogą zmienić wynik na typ referencyjny.
7. **Mylenie `constexpr` z obowiązkiem obliczenia podczas kompilacji.** Nie każde wywołanie funkcji `constexpr` jest ewaluowane w czasie kompilacji.
8. **Przenoszenie semantyki do traits.** Typ spełniający syntaktyczne wymaganie nie musi spełniać własności biznesowej, np. braku utraty precyzji czy bezpiecznej serializacji.

9. **Niepotrzebny CRTP.** Gdy wystarczy wolna funkcja z koncepcją, hierarchia klas często niczego nie upraszcza.
10. **Przedwczesne metaprogramowanie.** Najprostsze czytelne rozwiązanie jest zazwyczaj lepsze od dodatkowej warstwy abstrakcji.

Pytania kontrolne.

1. Czym różni się instancjowanie szablonu od wykonania funkcji?
2. Dlaczego `Factorial<0>` wymaga osobnej specjalizacji?
3. Kiedy `std::decay_t<T>` i `std::remove_cvref_t<T>` dają różne wyniki?
4. Dlaczego `std::conditional_t` nie rozwiązuje wszystkich problemów warunkowego tworzenia typów?
5. Co dokładnie sprawdza `void_t<decltype(++declval<T&>())>`?
6. Kiedy niepowodzenie podstawienia jest SFINAE, a kiedy zwykłym błędem kompilacji?
7. Dlaczego `if constexpr` nie zastępuje preprocesora?
8. Czym różnią się `constexpr` i `consteval`?
9. W jakich sytuacjach CRTP nie zastąpi funkcji wirtualnych?
10. Kiedy lepiej zastosować `concepts` niż `enable_if`?

Dalsza lektura. Warto sięgnąć do dokumentacji *cppreference* dotyczącej *Templates*, *Type traits* i *SFINAE*. Pomocne są również hasła `std::void_t`, `std::enable_if`, `if constexpr`, `consteval` oraz `std::integer_sequence`. Do diagnozowania złożonych błędów szablonów przydają się opcje `-Wall -Wextra -Wpedantic`, `-ftemplate-backtrace-limit=0` (GCC) oraz ograniczanie zasięgu instancjowanego kodu do minimalnego przykładu.

Rozdział 12

Koncepcje i ograniczenia szablonów (*concepts* i `requires`)

12.1 Po co wprowadzono koncepcje?

Szablony C++ pozwalają pisać kod niezależny od konkretnego typu. Tradycyjny szablon deklaruje jednak zwykle *co kompilator ma zrobić*, nie wyjaśniając wprost, *jakich własności* wymaga od argumentów. Jeśli operacja jest niepoprawna, błąd może zostać wykryty dopiero w ciele szablonu, a komunikat diagnostyczny bywa długi.

```
template<class T>
T twice(const T& x) {
    return x + x;           // requires operator+
}

// twice(7);                // OK
// twice(SomeType{});       // may fail inside function body
```

Od C++20 można umieścić wymagania na poziomie interfejsu:

```
#include <concepts>

template<class T>
concept Addable = requires(const T& a, const T& b) {
    { a + b } -> std::convertible_to<T>;
};

template<Addable T>
T twice(const T& x) {
    return x + x;
}
```

Koncepcja (*concept*) to nazwany predykat sprawdzany podczas kompilacji, opisujący wymagania wobec argumentów szablonu. Może dotyczyć istnienia operatorów, typów składowych, własności typów, kategorii iteratorów i wielu innych cech. *Constraints* (ograniczenia) wpływają zarówno na dopuszczalność konkretyzacji, jak i na wybór przeciążenia funkcji.

Wersja języka. W całym rozdziale zakładamy **C++20** oraz kompilację np. poleceniem `g++ -std=c++20 -Wall -Wextra -Wpedantic main.cpp`. Definicje koncepcji standardowych znajdują się przede wszystkim w nagłówkach `<concepts>`, `<iterator>` i `<ranges>`.

12.2 Pierwsza koncepcja: składnia i zastosowanie

Definicja nazwanej koncepcji ma postać `template<...> concept Nazwa = warunek;`. Warunek powinien być wyrażeniem stałym typu `bool`. Definicja koncepcji występuje na poziomie przestrzeni nazw; nie jest zwykłą funkcją.

```
#include <concepts>
#include <type_traits>

template<class T>
concept Integer = std::is_integral_v<T>;

static_assert(Integer<int>);
static_assert(Integer<bool>);    // bool is integral
static_assert(!Integer<double>);
```

Koncepcję można sprawdzić wyrażeniem `Integer<int>` lub `static_assert`, ale nie należy jej mylić ze zwykłą zmienną. W szczególności `concept` nie daje się jawnie specjalizować w sposób analogiczny do szablonu klasy.

12.2.1 Cztery podstawowe sposoby ograniczania funkcji

Poniższe deklaracje opisują ten sam warunek: funkcja jest przeznaczona dla typów całkowitych w sensie `std::integral`.

```
#include <concepts>

template<class T>
requires std::integral<T>
T square1(T x) { return x * x; }

template<class T>
T square2(T x) requires std::integral<T> {
    return x * x;
}

template<std::integral T>
T square3(T x) { return x * x; }

std::integral auto square4(std::integral auto x) {
    return x * x;
}
```

W ostatnim przykładzie ograniczony jest nie tylko typ argumentu, ale też *typ wyniku*. Sama dedukcja typu wyniku nadal odbywa się na podstawie wyrażenia `x * x`; koncept jest sprawdzany po jej przeprowadzeniu. Zapis `std::integral auto` w parametrze jest skrótem szablonu funkcji, tzw. *abbreviated function template*.

Kiedy którą postać wybrać? Dla jednego prostego ograniczenia zapis z ograniczonym parametrem szablonu, np. `template<std::integral T>`, jest krótki i czytelny. Jeśli zależność dotyczy kilku parametrów, najwygodniejsza bywa klauzula `requires`. Skrócona postać z `auto` przydaje się w niewielkich funkcjach i lambdach.

12.3 Dwie role słowa `requires`

Identyczny zapis słowny `requires` występuje w dwóch różnych konstrukcjach:

1. *requires-clause* – klauzula ograniczająca deklarację;
2. *requires-expression* – wyrażenie logiczne, które bada poprawność podanych wymagań.

```
#include <concepts>

template<class T>
concept Incrementable = requires(T value) { // requires-expression
    ++value;
};

template<class T>
requires Incrementable<T>                // requires-clause
void increment(T& x) {
    ++x;
}
```

Można je połączyć bez nadawania koncepcji osobnej nazwy:

```
template<class T>
requires requires(T x) { ++x; }
void increment_again(T& x) {
    ++x;
}
```

Pierwsze `requires` w ostatnim przykładzie zaczyna klauzulę; drugie wprowadza wyrażenie sprawdzające wymagania. To poprawna składnia, choć przy wielokrotnym użyciu lepiej utworzyć nazwaną koncepcję.

12.4 Cztery rodzaje wymagań w `requires-expression`

Wyrażenie `requires` umożliwia opisanie czterech rodzajów warunków. Parametry wpisane w nawiasach `requires(T x)` są jedynie *lokalnymi parametrami do sprawdzania wyrażeń*; nie tworzy się dla nich obiektów podczas działania programu.

12.4.1 Wymaganie proste (*simple requirement*)

```
#include <ostream>

template<class T>
concept Printable = requires(std::ostream& os, const T& x) {
    os << x;                    // must be well-formed
};
```

Wyrażenie `os « x` nie jest tutaj wykonywane; kompilator sprawdza jego poprawność. Proste wymaganie nie nakłada warunku na typ zwracany.

12.4.2 Wymaganie typu (*type requirement*)

```
#include <vector>

template<class T>
concept HasValueType = requires {
    typename T::value_type;    // nested type must exist
};

```

```
8 static_assert(HasValueType<std::vector<int>>);
9 static_assert(!HasValueType<int>);
```

Słowo `typename` jest tu składnikiem składni wymagania, a nie deklaracją nowego typu.

12.4.3 Wymaganie złożone (*compound requirement*)

```
1 #include <concepts>
2
3 template<class T>
4 concept AddableNoThrow = requires(const T& a, const T& b) {
5     { a + b } noexcept -> std::same_as<T>;
6 };
```

Zapis `{ wyrazenie } noexcept -> Koncepcja;` może sprawdzać jednocześnie: poprawność wyrażenia, to, że jest ono `noexcept`, i własność typu `decltype((wyrazenie))`. Obie części, `noexcept` oraz `-> Koncepcja`, są opcjonalne.

```
1 template<class T>
2 concept PlusReturnsT = requires(const T& a, const T& b) {
3     { a + b } -> std::same_as<T>;
4 };
5
6 static_assert(PlusReturnsT<int>);
7 // A type with a proxy return value may not satisfy this.
```

Uwaga na typ i konwersje. Koncepcja `std::same_as<T>` żąda *dokładnie* typu `T`, natomiast koncepcja `std::convertible_to<T>` dopuszcza konwersję. Dla `int` operator `+` daje `int`, ale np. dla małych typów całkowitych może zajść promocja do `int`.

12.4.4 Wymaganie zagnieżdżone (*nested requirement*)

```
1 #include <type_traits>
2
3 template<class T>
4 concept SmallTrivial = requires {
5     requires sizeof(T) <= 16;
6     requires std::is_trivially_copyable_v<T>;
7 };
8
9 static_assert(SmallTrivial<int>);
```

Wymaganie zagnieżdżone zaczyna się kolejnym `requires` i musi zawierać warunek stały. Jest wygodne, gdy wymaganie dotyczy własności typu, a nie określonego wyrażenia.

12.4.5 Przykład: połączenie różnych wymagań

```
1 #include <concepts>
2 #include <cstddef>
3
4 template<class T>
5 concept Indexable = requires(T& obj, std::size_t i) {
6     typename T::value_type;
7     { obj.size() } -> std::convertible_to<std::size_t>;
8     obj[i];
```

```
};
```

To przykład *minimalnego kontraktu składniowego*. Koncepcja nie dowodzi, że indeks `i` jest w zakresie, że operacja jest szybka ani że wynik indeksowania zachowuje się jak zwykła referencja. Takie własności wymagają dodatkowych założeń.

12.5 Standardowe koncepcje z biblioteki C++20

Przed napisaniem własnej koncepcji warto sprawdzić, czy istnieje gotowa w standardowej bibliotece.

12.5.1 Typy, konwersje i dziedziczenie

```
#include <concepts>

static_assert(std::same_as<int, int>);
static_assert(!std::same_as<int, long>);
static_assert(std::integral<unsigned>);
static_assert(std::signed_integral<int>);
static_assert(std::unsigned_integral<unsigned>);
static_assert(std::floating_point<double>);
static_assert(std::convertible_to<int, double>);

struct Base {};
struct Derived : Base {};
static_assert(std::derived_from<Derived, Base>);
```

`std::derived_from<D,B>` obejmuje publiczne i jednoznaczne dziedziczenie oraz przypadek tego samego typu. Różni się pod tym względem od samego `std::is_base_of_v`.

12.5.2 Konstruowalność, kopiowanie, przenoszenie i porównywanie

Przydatne są m.in. koncepcje wyrażające inicjalizowalność, konstruowalność, możliwość przenoszenia i kopiowania oraz porównywania:

```
default_initializable    constructible_from
move_constructible       copy_constructible
movable                  copyable
semiregular              regular
equality_comparable      totally_ordered
```

```
#include <concepts>
#include <memory>
#include <string>

static_assert(std::regular<int>);
static_assert(std::copyable<std::string>);
static_assert(std::movable<std::unique_ptr<int>>);
static_assert(!std::copyable<std::unique_ptr<int>>);
static_assert(std::totally_ordered<int>);
```

Koncepcje te opisują interfejs i zakładają także określone *prawa semantyczne* (np. sensowne porównywanie). Sam kompilator nie potrafi udowodnić dowolnej własności semantycznej implementacji.

12.5.3 Funkcje wywoływalne i predykaty

```cpp
#include <concepts>

struct IsPositive {
    bool operator()(int x) const { return x > 0; }
};

static_assert(std::invocable<IsPositive, int>);
static_assert(std::predicate<IsPositive, int>);
static_assert(!std::predicate<IsPositive, void*>);
```

`std::invocable<F,Args...>` sprawdza możliwość wywołania, natomiast `std::predicate` wymaga dodatkowo wyniku, którego można użyć jako warunku logicznego, i wiąże się z określonymi wymaganiami semantycznymi.

12.5.4 Iteratory i zakresy

Koncepcje iteratorów są udostępniane przez `<iterator>`, a koncepcje zakresów przez `<ranges>`.

```cpp
#include <ranges>
#include <vector>
#include <list>

static_assert(std::ranges::random_access_range<
              std::vector<int>>);
static_assert(std::ranges::bidirectional_range<
              std::list<int>>);
static_assert(!std::ranges::random_access_range<
              std::list<int>>);
```

W szczególności przydatne są koncepcje *input*, *forward*, *bidirectional*, *random access* i *contiguous range*. Pozwalają opisywać wymagania algorytmów bez uzależniania ich od konkretnego kontenera.

12.6 Łączenie koncepcji i definiowanie własnych kontraktów

Wymagania można łączyć operatorami logicznymi `&&` i `||`. Często lepiej budować niewielkie, nazwane koncepcje niż powtarzać jedno długie wyrażenie.

```cpp
#include <concepts>

template<class T>
concept Arithmetic = std::integral<T> || std::floating_point<T>;

template<class T>
concept NonBooleanInteger =
    std::integral<T> && !std::same_as<T, bool>;

template<Arithmetic T>
T cube(T x) { return x * x * x; }

static_assert(Arithmetic<double>);
static_assert(!NonBooleanInteger<bool>);
```

Dla połączenia `A && B` wymaganie `B` jest sprawdzane dopiero, gdy `A` jest spełnione; analogicznie dla `A || B` drugi składnik jest potrzebny dopiero, gdy pierwszy nie jest spełniony. Ta reguła chroni często przed podstawianiem wyrażeń, które dla danego typu nie mają sensu.

```cpp
#include <concepts>
```

```
#include <type_traits>

template<class T>
concept HasIntegralValueType =
    requires { typename T::value_type; } &&
    std::integral<typename T::value_type>;
```

W powyższym przykładzie odwołanie do `T::value_type` po prawej stronie jest istotne dopiero po potwierdzeniu, że typ składowy istnieje. Należy jednak pamiętać, że złożone reguły podstawiania i normalizacji koncepcji mają swoje szczegóły; nie każdą źle sformułowaną deklarację naprawi samo zwarcie logiczne.

12.7 Koncepcje z wieloma parametrami

Koncepcja nie musi opisywać własności jednego typu. Może także opisywać *relację między typami*.

```
#include <concepts>

template<class From, class To>
concept SameType = std::same_as<From, To>;

template<class Derived, class Base>
concept PublicChildOf = std::derived_from<Derived, Base>;

template<class T, class U>
concept AddableTogether = requires(const T& a, const U& b) {
    a + b;
};

static_assert(AddableTogether<int, double>);
```

Wymienione koncepcje `SameType` i `PublicChildOf` służą tu jedynie ilustracji składni. W praktyce używaj bezpośrednio standardowych `std::same_as` i `std::derived_from`, jeżeli nie dodajesz do nich istotnego znaczenia domenowego.

Wstawianie argumentu przy ograniczonym parametrze

```
struct Animal {};

template<std::derived_from<Animal> T>
void inspect(const T&) {}

// Equivalent constraint: std::derived_from<T, Animal>
```

W zapisie powyżej typ `T` jest podstawiany *na pierwszej pozycji* koncepcji, zatem chodzi o warunek z poniższego przykładu:

```
std::derived_from<T, Animal>
```

12.8 Przeciążanie funkcji i wybór bardziej szczegółowego ograniczenia

Ograniczenia uczestniczą w wyborze przeciążenia. Jeśli dwie funkcje są dopasowane do argumentów, kompilator może preferować tę, której ograniczenie jest bardziej szczegółowe (ang. *subsumption*).

```
#include <concepts>
```

```
#include <string_view>

template<class T>
requires std::integral<T>
constexpr std::string_view category(T) {
    return "integral";
}

template<class T>
requires std::signed_integral<T>
constexpr std::string_view category(T) {
    return "signed integral";
}

static_assert(category(12) == "signed integral");
static_assert(category(12u) == "integral");
```

`std::signed_integral` jest zbudowane na `std::integral` i dodaje dodatkowe wymaganie. Dzięki temu bardziej szczegółowe przeciążenie zostaje wybrane dla `int`.

12.8.1 Ważna pułapka: podobny zapis to nie to samo ograniczenie

Porządek ograniczeń nie jest ogólnym dowodzeniem implikacji matematycznych. Kompilator porównuje znormalizowane ograniczenia atomowe i ich *tożsamość*, a nie tylko sens pozornie podobnych wyrażeń.

```
#include <type_traits>

template<class T>
concept HasSize = requires(const T& x) { x.size(); };

template<class T>
concept HasSizeAndCopy =
    HasSize<T> && std::is_copy_constructible_v<T>;

template<class T> requires HasSize<T>
int classify(const T&) { return 1; }

template<class T> requires HasSizeAndCopy<T>
int classify(const T&) { return 2; }
```

Druga koncepcja *wykorzystuje tę samą* koncepcję `HasSize`, dlatego kompilator rozpoznaje wspólny składnik. Jeśli powtórzymy ręcznie nawet identycznie wyglądające wymaganie opisujące metodę `size()`, nie mamy automatycznej gwarancji identyczności ograniczeń atomowych.

Rekomendacja. Twórz proste nazwane koncepcje i buduj bardziej szczegółowe z ich wykorzystaniem. Ułatwia to konserwację kodu i przewidywanie wyboru przeciążeń.

12.9 Koncepcje dla szablonów klas i ich specjalizacji

Ograniczać można nie tylko funkcje, lecz również szablony klas.

```
#include <concepts>

template<std::integral T>
class Counter {
    T value_{};
```

```
public:
    void add(T delta) { value_ += delta; }
    T value() const { return value_; }
};

// Counter<int> a;       // OK
// Counter<double> b;    // ERROR: constraint not satisfied
```

Ograniczenia mogą też sterować wyborem częściowej specjalizacji.

```
#include <concepts>
#include <string_view>

template<class T>
struct TypeLabel {
    static constexpr std::string_view value = "other";
};

template<std::integral T>
struct TypeLabel<T> {
    static constexpr std::string_view value = "integer";
};

template<std::floating_point T>
struct TypeLabel<T> {
    static constexpr std::string_view value = "floating";
};

static_assert(TypeLabel<int>::value == "integer");
static_assert(TypeLabel<double>::value == "floating");
```

Nie używaj jednak złożonego systemu specjalizacji, jeżeli prosta funkcja przeciążona lub `if constexpr` lepiej wyraża zamysł.

12.10 Koncepcje i pakiety parametrów: kontynuacja rozdziału 4

Koncepcje bardzo dobrze współpracują z *variadic templates*. Możemy nakładać warunek na każdy element pakietu typów za pomocą *fold expression*.

```
#include <concepts>

template<class... Ts>
requires (std::integral<Ts> && ...)
constexpr auto sum_integers(Ts... xs) {
    return (0 + ... + xs);
}

static_assert(sum_integers() == 0);
static_assert(sum_integers(1, 2, 3) == 6);
// sum_integers(1, 2.5);    // ERROR: double not integral
```

Zapis `(P<Ts> && ...)` reprezentuje koniunkcję wymagań dla wszystkich elementów pakietu. Dla pustego pakietu taki fold logiczny daje `true`; nie oznacza to jednak, że każde ciało funkcji poradzi sobie z zerową liczbą argumentów. W przykładzie użyliśmy `0` jako elementu neutralnego sumowania.

12.10.1 Ograniczony pakiet typów i uniwersalne przekazywanie

```
#include <concepts>
```

```
#include <utility>

template<std::integral... Ts>
void accept_integers(Ts&&... xs) {
    ((void)std::forward<Ts>(xs), ...); // illustrative use
}
```

Zapis `std::integral... Ts` również ogranicza wszystkie elementy pakietu. Uwaga: przy dedukcji z przekazywanymi referencjami `Ts&&`, typ `Ts` może być *referencją*, gdy argument jest lvalue. W takim przypadku `std::integral<Ts>` może być fałszywe. Jeżeli chcemy akceptować obie kategorie wartości, często właściwsze jest użycie `std::remove_cvref_t`:

```
#include <concepts>
#include <type_traits>
#include <utility>

template<class... Ts>
requires (std::integral<std::remove_cvref_t<Ts>> && ...)
void accept_any_int(Ts&&... xs) {
    ((void)xs, ...); // demonstration; no ownership transfer
}
```

Ta różnica jest szczególnie ważna przy projektowaniu generycznych wrapperów, które mają przyjmować także obiekty `const` i referencje.

12.11 Concepts a `if constexpr`, `static_assert` i SFINAE

Są to narzędzia pokrewne, ale mają różne cele.

12.11.1 `if constexpr`: wybór implementacji

```
#include <concepts>
#include <string>
#include <string_view>

template<class T>
std::string describe(const T& value) {
    if constexpr (std::integral<T>) {
        return "an integer";
    } else if constexpr (std::floating_point<T>) {
        return "a floating-point value";
    } else {
        return "another type";
    }
}
```

`if constexpr` umożliwia wybranie gałęzi kodu *wewnątrz* szablonu. Nie tworzy samo przez się ograniczonego interfejsu. Z kolei `requires` może odrzucić niewłaściwe wywołanie jeszcze na etapie dobierania przeciążeń.

12.11.2 `static_assert`: własny komunikat

```
#include <concepts>

template<class T>
void save_number(const T& value) {
```

```
    static_assert(std::integral<T>,
                  "save_number expects an integral type");
    // ...
}
```

`static_assert` jest użyteczne, gdy potrzebujemy jednoznacznego komunikatu albo sprawdzamy niezmiennik wewnątrz implementacji. Nie zastępuje jednak ograniczenia uczestniczącego w rozstrzyganiu przeciążeń.

12.11.3 Przed C++20: `std::enable_if` i idiom wykrywania

```
#include <type_traits>

// C++11/14/17 technique:
template<class T,
         std::enable_if_t<std::is_integral_v<T>, int> = 0>
T old_square(T x) {
    return x * x;
}

// C++20:
#include <concepts>
template<std::integral T>
T new_square(T x) {
    return x * x;
}
```

Dla nowego kodu C++20 najczęściej korzystniej wybrać koncepcję: intencja staje się czytelna i spójna z mechanizmem porządkowania przeciążeń. `std::enable_if` nadal ma znaczenie w bibliotekach wspierających starsze wersje standardu.

12.12 Praktyczny przykład: średnia z zakresu liczb

Zamiast pisać funkcję tylko dla `std::vector<double>`, możemy określić dokładne wymagania wobec *zakresu*.

```
#include <concepts>
#include <cstddef>
#include <optional>
#include <ranges>
#include <vector>
#include <array>

template<class R>
concept NumericInputRange =
    std::ranges::input_range<R> &&
    std::convertible_to<std::ranges::range_reference_t<R>, double>;

template<NumericInputRange R>
std::optional<double> average(R&& range) {
    double sum = 0.0;
    std::size_t count = 0;
    for (auto&& item : range) {
        sum += static_cast<double>(item);
        ++count;
    }
    if (count == 0) return std::nullopt;
    return sum / static_cast<double>(count);
```

```
}

int main() {
    std::vector<int> v{2, 4, 6};
    std::array<double, 2> a{3.0, 7.0};
    auto x = average(v); // 4.0
    auto y = average(a); // 5.0
    auto z = average(std::vector<int>{}); // empty optional
    return !(x && y && !z && *x == 4.0 && *y == 5.0);
}
```

Funkcja działa dla różnych kontenerów i zakresów dostarczających iterację jednokierunkową. Warunek `input_range` wyraża strukturę danych, a `convertible_to` – oczekiwany rodzaj elementów. Zastosowaliśmy `std::optional<double>`, aby pusty zakres nie powodował dzielenia przez zero. Dla zastosowań numerycznych należy dodatkowo rozważyć stabilność sumowania, przepełnienia i znaczenie konwersji do `double`.

Kiedy `ranges`? Koncepcje standardowych zakresów są szczególnie użyteczne, gdy funkcja operuje na sekwencji, ale nie wymaga konkretnie `std::vector` czy `std::list`. Zamiast wymuszać silniejsze wymaganie `random_access_range`, wybieraj najsłabszą koncepcję, która wystarcza do działania algorytmu.

12.13 Koncepcje dla lambd i algorytmów

Funkcje anonimowe również mogą mieć ograniczone parametry.

```
#include <concepts>

int main() {
    auto twice = [](std::integral auto x) { return x + x; };
    return twice(5) != 10;
}
```

Można też tworzyć generyczne algorytmy przyjmujące funkcje spełniające `std::predicate`.

```
#include <concepts>
#include <cstddef>
#include <vector>

template<class T, class Pred>
requires std::predicate<Pred&, const T&>
std::size_t count_if_custom(const std::vector<T>& values,
                            Pred pred) {
    std::size_t count = 0;
    for (const T& x : values)
        if (pred(x)) ++count;
    return count;
}
```

Zwróć uwagę na argumenty typu w `std::predicate<Pred&, const T&>`: sprawdzają możliwość wywołania *konkretnego rodzaju* predykatu z *konkretnym rodzajem* argumentu. Kategoria wartości ma znaczenie i nie należy bezrefleksyjnie wstawiać tu samych nazw typów.

12.14 Granice gwarancji: najczęstsze pułapki

12.14.1 Concept nie sprawdza dowolnej własności semantycznej

```
#include <concepts>

struct Strange {
    int value{};
    bool operator==(const Strange&) const { return false; }
};

static_assert(std::equality_comparable<Strange>);
// Syntactically valid, but equality lacks reflexivity.
```

Kompilator akceptuje składnię porównania, lecz `a == a` jest zawsze fałszywe. Kod nie spełnia matematycznych oczekiwań związanych z relacją równości. To obowiązek autora typu, niezależny od samego mechanizmu `concepts`.

12.14.2 Koncepcje nie zamieniają błędów w poprawny kod

Wyrażenia `requires` pozwalają w kontekście zależnym od parametru szablonu traktować niepoprawne podstawienia jako niespełnione wymagania. Nie oznacza to jednak, że *każdy* błędny fragment C++ staje się automatycznie `false`. Niezależne od parametru, trwale niepoprawne konstrukcje mogą prowadzić do błędu kompilacji.

12.14.3 Różnica między `T`, `T&` i `const T&`

```
#include <concepts>
#include <type_traits>

template<class T>
concept PlainInteger =
    std::integral<std::remove_cvref_t<T>>;

static_assert(!std::integral<int&>);
static_assert(PlainInteger<int&>);
static_assert(PlainInteger<const int&>);
```

Przy dedukcji typu generycznego parametru `T&&` ta różnica często decyduje o tym, czy z pozoru poprawne wywołanie spełni koncepcję.

12.14.4 `std::same_as` czy `std::convertible_to`?

```
#include <concepts>

template<class T>
concept StrictAdd = requires(T a, T b) {
    { a + b } -> std::same_as<T>;
};

template<class T>
concept FlexibleAdd = requires(T a, T b) {
    { a + b } -> std::convertible_to<T>;
};

static_assert(!StrictAdd<short>);  // short + short -> int
```

```
14 static_assert(FlexibleAdd<short>); // int -> short allowed
```

Koncepcja `FlexibleAdd` nie oznacza *bezstratnej* konwersji; `int` można niejawnie przekształcić do `short`, tracąc informację. Nazwa koncepcji powinna odzwierciedlać rzeczywistą, a nie postulowaną gwarancję.

12.14.5 Funkcja ograniczona nie zawsze jest optymalniejsza

Koncepcje opisują poprawność i wybór instancji szablonów. Nie wymuszają *inline*, wektoryzacji, eliminacji kopii ani lepszego kodu maszynowego. Można je stosować bez obietnicy przyspieszenia wykonywania programu.

12.15 Zasady projektowania: kiedy używać, a kiedy nie?

Sytuacja	**Zalecane rozwiązanie**
Funkcja generyczna wymaga konkretnej kategorii typów	Nazwany `concept`, np. `std::integral`
Wymagania dotyczą istnienia operacji lub typu składowego	Własny `requires-expression`
Ważne są różne wersje algorytmu dla różnych typów	Ograniczone przeciążenia lub specjalizacje
Jedna implementacja ma kilka gałęzi zależnych od typu	`if constexpr`
Sprawdzasz założenie wewnątrz ciała funkcji	`static_assert` z komunikatem
Weryfikujesz jednakowy warunek dla całego pakietu	Fold `(Concept<Ts> && ...)`
Potrzebujesz iteracji po sekwencji	`std::ranges::input_range` lub silniejszy koncept
Funkcja działa dla jednego, konkretnego typu	Zwykła funkcja często jest prostsza niż szablon
Projekt musi działać także w C++17	`type_traits` i (w razie potrzeby) SFINAE
Wymagasz warunku wykonywania programu, np. `n>0`	Zwykłe sprawdzenie w czasie działania; koncept go nie zastępuje

12.16 Podsumowanie i pytania kontrolne

1. `concept` nazywa predykat nad parametrami szablonu, sprawdzany w czasie kompilacji.
2. `requires-clause` ogranicza deklarację; `requires-expression` jest wyrażeniem badającym wymagania.
3. Istnieją cztery formy wymagań: proste, typu, złożone i zagnieżdżone.
4. `std::same_as`, `std::convertible_to`, `std::integral`, `std::predicate` i koncepcje zakresów powinny być zwykle pierwszym wyborem.
5. Ograniczenia mogą wpływać na wybór przeciążenia, ale mechanizm porządkowania nie jest dowolnym dowodzeniem implikacji logicznej.
6. Nazwane koncepcje warto wykorzystywać ponownie przy budowaniu koncepcji bardziej szczegółowych.
7. `std::remove_cvref_t<T>` ma często znaczenie przy koncepcjach typów dedukowanych z referencji przekazujących.

8. Koncepcje opisują przede wszystkim wymagania składniowe, a dodatkowe prawa semantyczne muszą respektować autorzy typów.

Pytania do samodzielnego sprawdzenia. Zastanów się, co kompilator zaakceptuje i dlaczego:

```
#include <concepts>
#include <type_traits>

static_assert(std::integral<bool>);
static_assert(!std::integral<int&>);
static_assert(std::integral<std::remove_cvref_t<const int&>>);
static_assert(!std::same_as<int, long>);

// 1. Why does StrictAdd<short> fail?
// 2. Why can a concept be true for a logically broken ==?
// 3. How do requires-clause and requires-expression differ?
// 4. Which overload of category(5) is chosen?
// 5. Why is NumericInputRange more general than vector<double>?
```

Źródła i dalsza lektura. Oficjalne reguły języka określa standard ISO C++, a przystępne opisy i przykłady znajdują się w serwisie *cppreference*. Przydatne adresy:

```
en.cppreference.com/cpp/language/constraints
en.cppreference.com/cpp/language/requires
en.cppreference.com/cpp/concepts
```

Rozdział 13

Iteratory, zakresy i widoki: *Ranges* oraz *Views* w C++20

13.1 Dlaczego wprowadzono Ranges?

Klasyczne algorytmy STL operują na *parze iteratorów*: pierwszym wskazującym początek danych i drugim wyznaczającym koniec. Dzięki temu `std::sort` lub `std::find` mogą współpracować z wieloma kontenerami. Wymaga to jednak przekazywania obu granic i dodatkowej ostrożności, aby pochodziły z tego samego zakresu.

```
#include <algorithm>
#include <vector>

std::vector<int> xs{4, 1, 3, 2};
std::sort(xs.begin(), xs.end()); // STL before C++20
std::ranges::sort(xs);           // C++20
```

Nowy model pozwala przekazać *zakres* (*range*) jako jeden obiekt oraz formułować wymagania wobec niego poprzez *koncepcje* (*concepts*, rozdział 5). Oddzielną innowacją są *widoki* (*views*): lekkie reprezentacje danych, często przekształcane **leniwe**, to znaczy dopiero podczas ich przeglądania.

```
#include <ranges>

std::vector<int> xs{1, 2, 3, 4, 5, 6};
auto pipeline = xs
    | std::views::filter([](int x) { return x % 2 == 0; })
    | std::views::transform([](int x) { return x * x; });

// No vector of squares has been created.
// Iteration produces 4, 16, 36.
for (int x : pipeline) { /* use x */ }
```

Założenia. W całym rozdziale przyjmujemy standard **C++20** oraz nagłówki `<algorithm>`, `<iterator>`, `<ranges>`, `<concepts>`, `<vector>` i pozostałe nagłówki wymienione w przykładach. W dalszych sekcjach posłużymy się semantyką przenoszenia (rozdział 6) i lambdami (rozdział 7).

13.2 Iteratory: uogólnienie wskaźników

13.2.1 Operacje `begin`, `end`, `*` i `++`

Iterator jest obiektem umożliwiającym poruszanie się po elementach zakresu. Wskaźnik `int*` jest przykładem iteratora, ale iterator może także przechowywać indeks, wskaźnik do węzła listy lub stan wewnętrzny generatora. Podstawowe operacje przedstawia program:

```
#include <vector>

std::vector<int> v{10, 20, 30};
auto it = v.begin();     // points to 10
int a = *it;            // 10
++it;                   // points to 20
int b = *it;            // 20
bool ok = it != v.end();
```

Iterator `end()` zwykle oznacza pozycję *za ostatnim* elementem; nie wolno go dereferencjonować. Dzięki temu pusty zakres spełnia `begin()==end()`, a przechodzenie po elementach ma jednolity schemat:

```
for (auto it = v.begin(); it != v.end(); ++it) {
    // *it is a valid element
}
```

Pętla zakresowa. Zapis `for (auto& x : v)` jest wygodniejszą postacią iteracji, ale w dalszym ciągu wykorzystuje operacje początku, końca, porównania, zwiększania i dereferencji. Aby *modyfikować* elementy, stosujemy `auto&`; aby uniknąć kopiowania bez modyfikowania, `const auto&`; dla niewielkich wartości liczbowych często wystarczy `auto`.

13.2.2 Iteratory stałe i nieważność iteratorów

```
std::vector<int> v{1, 2, 3};
auto m = v.begin();   // can modify *m
auto c = v.cbegin();  // cannot modify *c
*m = 10;
// *c = 20;         // compilation error
```

`const_iterator` chroni elementy przed modyfikacją poprzez ten iterator; nie oznacza, że cały kontener jest niezmienny. Równie ważna jest *nieważność iteratorów* (*iterator invalidation*). Na przykład ponowna alokacja pamięci przez `std::vector::push_back` może unieważnić **wszystkie** iteratory, wskaźniki i referencje do jego elementów.

```
std::vector<int> v{1, 2, 3};
auto it = v.begin();
v.push_back(4);       // may reallocate storage
// *it;              // unsafe if reallocation happened
it = v.begin();      // obtain a fresh iterator
```

Reguły zależą od kontenera i operacji. W `std::list` wstawienie elementu nie unieważnia iteratorów do pozostałych elementów, ale nie oznacza to, że można dalej używać iteratora do elementu usuniętego.

13.3 Kategorie iteratorów i ich wymagania

Zestaw koncepcji C++20 opisuje operacje i własności iteratorów, a nie jedynie nazwy klas. Najważniejsze kategorie zestawia tabela.

Koncepcja	**Możliwości i przykłady**
`input_iterator`	Odczytywanie kolejnych wartości; przeglądanie może być jednokrotne (np. strumień wejściowy).
`output_iterator`	Zapisywanie kolejnych wartości do celu (np. `back_insert_iterator`).
`forward_iterator`	Wielokrotne przejścia, porównywanie iteratorów, krok do przodu.
`bidirectional_iterator`	Dodatkowo krok wstecz, czyli `-it`; np. `list`.
`random_access_iterator`	Indeksowanie i skoki `it+n` w stałym czasie; np. `deque`.
`contiguous_iterator`	Elementy leżą kolejno w pamięci; np. iterator `vector<int>` i surowy wskaźnik.

Hierarchia wymagań dla iteratorów odczytujących jest w przybliżeniu następująca (każda kolejna koncepcja jest silniejsza):

$$\texttt{input} \Rightarrow \texttt{forward} \Rightarrow \texttt{bidirectional} \Rightarrow \texttt{random_access} \Rightarrow \texttt{contiguous}.$$

Koncepcja `output_iterator` dotyczy osobnej rodziny wymagań. Nowe koncepcje nie są w każdym szczególe identyczne ze starymi kategoriami *LegacyIterator*. Na przykład model `forward_iterator` nie wymaga zwracania referencji przy dereferencji.

```
#include <concepts>
#include <iterator>
#include <list>
#include <vector>

static_assert(std::contiguous_iterator<int*>);
static_assert(std::random_access_iterator<
    std::vector<int>::iterator>);
static_assert(std::bidirectional_iterator<
    std::list<int>::iterator>);
static_assert(!std::random_access_iterator<
    std::list<int>::iterator>);
```

Dlaczego to ważne? `std::ranges::sort` wymaga iteratorów o dostępie swobodnym, więc można użyć go do `std::vector`, ale **nie** do `std::list`. Listę sortuje się metodą `list::sort()`. Wybór algorytmu jest zatem ograniczony kategorią iteratora.

13.4 Własny iterator i własny zakres

Zbudujemy iterator generujący kolejne liczby całkowite. Nie będzie on przechowywać ani wskaźnika, ani kontenera: całą potrzebną informacją jest bieżąca wartość. Zdefiniujemy *iterator do przodu* (`forward_iterator`).

```
#include <concepts>
#include <cstddef>
#include <iterator>
```

```cpp
#include <ranges>

struct CountingIterator {
    using value_type = int;
    using difference_type = std::ptrdiff_t;
    using iterator_concept = std::forward_iterator_tag;

    int current = 0;

    int operator*() const { return current; }

    CountingIterator& operator++() {
        ++current;
        return *this;
    }
    CountingIterator operator++(int) {
        auto old = *this;
        ++*this;
        return old;
    }
    bool operator==(const CountingIterator&) const = default;
};

struct CountingRange {
    int first = 0;
    int last = 0;

    CountingIterator begin() const { return {first}; }
    CountingIterator end() const { return {last}; }
};

static_assert(std::forward_iterator<CountingIterator>);
static_assert(std::ranges::forward_range<CountingRange>);
```

Teraz kod `for (int i : CountingRange{3, 7})` wypisze `3, 4, 5, 6`. Funkcje `begin()` i `end()` wyznaczają *półotwarty* zakres $[3, 7)$. Prawidłowe użycie zakłada `first <= last`; dla `first > last` nasz minimalny iterator nie zbiegnie do wartości końcowej.

Co trzeba zagwarantować? Samo przejście `static_assert` nie dowodzi wszystkich semantycznych własności iteratora. Dla `forward_iterator` ważna jest m.in. własność *multi-pass*: kopiowanie iteratora i przejście jedną kopią nie powinno zmieniać wartości zwracanych przez drugą. Nasz iterator ją spełnia, ponieważ każda kopia ma własne pole `current`.

13.5 Zwykłe *range* i *sentinel*

Zakres jest obiektem, dla którego można otrzymać początek `std::ranges::begin(r)` i koniec `std::ranges::end(r)`. Typ wyniku obu wywołań **nie musi być identyczny**. Obiekt wyznaczający koniec to *sentinel*; musi dać się porównać z iteratorem, ale sam nie musi obsługiwać jego pozostałych operacji.

```cpp
#include <ranges>
#include <vector>

std::vector<int> xs{1, 2, 3};
auto first = std::ranges::begin(xs);
auto last = std::ranges::end(xs);
while (first != last) { ++first; }
```

W tym przypadku obie granice mają ten sam typ: zakres spełnia koncepcję `common_range`. W innych zakresach koniec może wyznaczać osobny obiekt sentinela. Odpowiednia koncepcja `std::sentinel_for` określa warunki porównywania granic:

```
static_assert(std::ranges::common_range<decltype(xs)>);
static_assert(std::sentinel_for<
    decltype(last), decltype(first)>);
```

Przykładem osobnego typu końca jest `std::default_sentinel_t`.

Rozmiar a liczba kroków. Operacja `std::ranges::size(r)` jest dostępna dla zakresów spełniających `sized_range`. Nie każdy zakres ma tanią operację wyznaczenia rozmiaru. Funkcja `distance(r)` z przestrzeni `std::ranges` może wymagać *przejścia przez wszystkie elementy*. Dla niektórych jednokrotnych źródeł danych przejście oznacza również ich zużycie.

13.6 Koncepcje zakresów w praktyce

Własności zakresów sprawdzamy za pomocą koncepcji z rozdziału 5. Najważniejsze z nich to:

- `input_range` i `forward_range`,
- `bidirectional_range` i `random_access_range`,
- `contiguous_range`, `common_range` i `sized_range`.

```
#include <ranges>
#include <vector>

template <std::ranges::input_range R>
long long sum_integers(R&& range) {
    long long result = 0;
    for (auto value : range) result += value;
    return result;
}

std::vector<int> v{1, 2, 3};
long long s = sum_integers(v); // 6
```

Parametr `R&&` jest tutaj *referencją przekazującą* (*forwarding reference*), a nie żądaniem przejęcia własności. Funkcja obsługuje także źródła jednokrotne, gdyż przechodzi zakres tylko raz. Dla naprawdę uniwersalnej funkcji trzeba osobno wyrazić warunki na typ elementu i wykonywaną na nim operację dodawania.

13.7 Algorytmy `std::ranges` i projekcje

Funkcje z rodziny `std::ranges`, np. `sort`, `find`, `copy` i `transform`, przyjmują bezpośrednio zakres albo parę iterator–sentinel. Ich wykonanie jest na ogół *natychmiastowe* (w odróżnieniu od samych widoków `views::transform`).

```
#include <algorithm>
#include <iterator>
#include <vector>

std::vector<int> v{4, 2, 3, 1};
std::ranges::sort(v);  // modifies v immediately

```

```
auto pos = std::ranges::find(v, 3);
if (pos != v.end()) { *pos = 30; }

std::vector<int> out;
std::ranges::copy(v, std::back_inserter(out));
```

13.7.1 Projekcja: sortowanie po wskazanym polu

Algorytmy Ranges często posiadają argument *projekcji*: funkcję wywoływaną na elemencie przed porównaniem albo wyszukiwaniem. Dzięki temu zamiast pisać własny komparator, można wskazać pole struktury.

```
#include <string>
#include <vector>

struct Person {
    std::string name;
    int age;
};

std::vector<Person> people{
    {"Ada", 40}, {"Ewa", 25}, {"Jan", 32}
};

std::ranges::sort(people, {}, &Person::age);
// ascending ages: 25, 32, 40

auto it = std::ranges::find(people, 32, &Person::age);
// it->name == "Jan"
```

W wywołaniu `sort(people, {}, &Person::age)` puste nawiasy oznaczają domyślny komparator `std::ranges::less`, a wskaźnik do składowej to projekcja. Standard używa mechanizmu `std::invoke`, omówionego w rozdziale 7.

13.8 Widoki: *view* nie jest synonimem kontenera

Widok jest zakresem przystosowanym do taniego tworzenia i kopiowania lub przenoszenia *samego obiektu widoku*. Nie musi mieć własnych danych: może przechowywać referencję do kontenera, mały zestaw iteratorów albo parametry generowania wartości.

```
#include <ranges>
#include <vector>

std::vector<int> xs{10, 20, 30, 40};
auto first_two = xs | std::views::take(2);
// first_two refers to the original xs
```

Stworzenie widoku nie jest równoznaczne ze skopiowaniem elementów do nowego kontenera. Odwołanie do danych źródłowych wymaga, by pozostawały one ważne przez cały czas korzystania z widoku. Szczegółowe reguły własności omówimy dalej.

Dwa znaczenia nazwy `views`. `std::views` jest aliasem przestrzeni nazw z obiektami adaptującymi zakresy. `std::ranges::view` to natomiast *koncepcja* spełniana przez dany typ. Nie należy mylić ich z pojęciem `std::ranges::range`.

13.9 Leniwe potoki `filter`, `transform`, `take`

Najczęściej stosowane widoki łączymy za pomocą operatora `|`. Adaptory przekazują sobie zakresy bez potrzeby budowania pośrednich wektorów:

```
std::vector<int> xs{1,2,3,4,5,6,7,8,9,10};

auto pipeline = xs
    | std::views::filter([](int x) {
          return x % 2 == 0;
      })
    | std::views::transform([](int x) {
          return x * x;
      })
    | std::views::take(3);

// Iteration produces 4, 16, 36.
```

`filter` pomija elementy niespełniające predykatu, `transform` wyznacza nową wartość przy dereferencji, a `take(3)` ogranicza liczbę wyników. Algorytm `std::ranges::transform` działa od razu; natomiast utworzenie widoku `views::transform` nie przelicza wszystkich wyników.

13.9.1 Kiedy następują obliczenia?

```
int calls = 0;
auto squares = xs | std::views::transform([&](int x) {
    ++calls;
    return x * x;
});
// calls == 0 here
int first = *squares.begin();
// calls has increased (transform was evaluated)
```

W tym przykładzie pokazujemy moment obliczenia, a nie zalecany styl projektowania: **predykaty i transformacje powinny zasadniczo unikać efektów ubocznych**. Ponowne przejście widoku, ponowna dereferencja lub implementacyjnie dopuszczone buforowanie może powodować inne liczby wywołań, niż sugerowałaby pojedyncza pętla. Nie należy projektować logiki programu na podstawie rzekomej liczby wywołań funkcji przez widok.

Ważny szczegół `filter_view`. Dla pewnych kategorii zakresu `filter_view` może zapamiętywać pierwszy znaleziony element. Ponadto modyfikowanie elementu tak, aby przestał spełniać predykat filtrowania, narusza wymagania `filter_view` i może prowadzić do niezdefiniowanego zachowania. Predykat powinien być stabilny wobec przebiegu iteracji.

13.10 Pozostałe przydatne adaptory C++20

13.10.1 `iota`: wartości wyznaczane bez tablicy

```
auto ids = std::views::iota(1, 5);
// Iteration produces 1, 2, 3, 4.

// This is an unbounded range, limited by take(5):
auto first_five = std::views::iota(1)
                | std::views::take(5);
// Produces 1, 2, 3, 4, 5.
```

`iota` przydaje się do indeksowania lub generowania sekwencji. Dla nieograniczonego widoku trzeba pamiętać o warunku zakończenia; ponadto operacje arytmetyczne nie mogą przepełniać typu całkowitego ze znakiem.

13.10.2 `drop`, `take` i `reverse`

```
std::vector<int> data{1, 2, 3, 4, 5};
auto middle = data | std::views::drop(1)
                   | std::views::take(3);
// 2, 3, 4

auto backwards = data | std::views::reverse;
// 5, 4, 3, 2, 1
```

`reverse` wymaga co najmniej zakresu dwukierunkowego; `take` i `drop` są przydatne także dla sekwencji generowanych leniwie. Dla niektórych widoków koszt `begin()` lub dojścia do pierwszego wyniku może być większy niż stały.

13.10.3 `keys`, `values` i `split`

```
std::vector<std::pair<int, std::string>> pairs{
    {1, "one"}, {2, "two"}
};
for (int key : pairs | std::views::keys) { /* 1, 2 */ }
for (const auto& value : pairs | std::views::values) {
    /* "one", "two" */
}

std::string csv = "aa,b,cc";
for (auto part : csv | std::views::split(',')) {
    std::string piece(part.begin(), part.end());
    // Copy each part into an owning string if needed.
}
```

`split` nie tworzy automatycznie `std::string` dla każdego fragmentu; otrzymujemy zakresy wskazujące fragmenty źródła. Zachowując wynik na później, trzeba zapewnić odpowiedni czas życia źródła. Nie należy też zakładać, że splitowanie tekstu CSV obsługuje cudzysłowy i escapowanie pól zgodnie z pełnym formatem CSV.

Uwaga na wersję standardu. `std::views::zip` i `std::views::enumerate` są konstrukcjami **C++23**, podobnie jak `std::ranges::to`. Nie są dostępne w czystym C++20. W standardzie C++20 wynik można zebrać do kontenera np. algorytmem `ranges::copy` z iteratorem wstawiającym `std::back_inserter`.

13.11 Materializacja wyniku i koszt przetwarzania

Jeśli wynik ma być niezależny od źródła i przechowywany na później, możemy zamienić potok w kontener. W C++20 nie ma standardowego `std::ranges::to<std::vector>()`.

```
#include <algorithm>
#include <iterator>
#include <vector>

std::vector<int> data{1,2,3,4,5,6};
```

```
auto even = data | std::views::filter([](int x) {
    return x % 2 == 0;
});

std::vector<int> result;
std::ranges::copy(even, std::back_inserter(result));
// result == {2, 4, 6}
```

Taka materializacja wymaga pamięci proporcjonalnej do wielkości wyniku. Zaletą potoku jest unikanie pośrednich alokacji i często możliwość *łączenia* transformacji w jednym przebiegu. Nie jest to jednak gwarancja, że widoki zawsze będą szybsze: koszty iteracji, predykatów, przepływu sterowania i optymalizacji zależą od konkretnego kodu i kompilatora. W zadaniach krytycznych wykonuj pomiary.

13.12 Własność danych, czas życia i `borrowed_range`

To najważniejsza część praktycznego korzystania z Ranges. Widoki **nie zawsze posiadają** bazowe dane. W przypadku `std::views::all` przeprowadzamy istotne rozróżnienie:

```
std::vector<int> source{2, 4, 6};
auto borrowed = std::views::all(source);
// Refers to source: source must stay alive.

auto owned = std::views::all(std::vector<int>{3, 5, 7});
// In C++20, this can be an owning_view.
// The temporary vector is moved into the view.
```

Nie oznacza to, że *każdy* widok utworzony z wartości tymczasowej przejmuje własność, ani że wszystkie widoki zachowują się tak samo. Różne adaptory przechowują różne rodzaje obiektów bazowych.

13.12.1 Wskaźniki wiszące wewnątrz potoku

```
auto make_bad_view() {
    std::vector<int> local{1, 2, 3};
    return local | std::views::filter([](int x) {
        return x > 1;
    });
}
// Using the returned view would access destroyed local.
```

Ponieważ `local` jest *lvalue*, adapter przechowuje nieposiadający danych widok do lokalnego kontenera. Po wyjściu z funkcji `local` zostaje zniszczony, a zwrócony widok zawiera nieaktualne odwołanie. Bezpieczne wyjścia to utworzenie widoku z przekazanego na własność kontenera (np. obiektu tymczasowego lub `std::move(local)`) albo zwrócenie zwykłego kontenera wynikowego.

```
auto make_owning_view() {
    std::vector<int> local{1, 2, 3};
    return std::views::all(std::move(local))
         | std::views::filter([](int x) { return x > 1; });
}
// The returned view owns its vector through owning_view.
```

13.12.2 Co naprawdę mówi `borrowed_range`?

Koncepcja `borrowed_range<R>` oznacza w uproszczeniu, że iteratory otrzymane z zakresu typu `R` mogą pozostać ważne po zniszczeniu *samego obiektu zakresu* (o ile dane, do których odsyłają, nadal istnieją). Nie jest to synonim "widok nie posiada danych" ani "wszystkie wskaźniki są zawsze bezpieczne".

```
#include <concepts>
#include <ranges>
#include <vector>

static_assert(std::ranges::borrowed_range<
    std::ranges::ref_view<std::vector<int>>>);
static_assert(!std::ranges::borrowed_range<
    std::vector<int>>);
```

13.12.3 `std::ranges::dangling` jako zabezpieczenie

Algorytm zwracający iterator do *tymczasowego* kontenera nie może bezpiecznie zwrócić rzeczywistego iteratora do zniszczonych danych. Dlatego pewne algorytmy Ranges zwracają specjalny typ `std::ranges::dangling`.

```
#include <type_traits>

using Result = decltype(std::ranges::find(
    std::vector<int>{1, 2, 3}, 2));
static_assert(std::same_as<Result, std::ranges::dangling>);
```

Nie należy jednak oczekiwać, że mechanizm ten wykryje wszystkie problemy z czasem życia. Jeżeli zapisujemy nieposiadający danych widok do lokalnej zmiennej, kompilator może nie zauważyć, że docelowy kontener umrze wcześniej niż widok.

13.13 `std::span`, `std::string_view` i subrange

`std::span<T>` opisuje ciągły fragment pamięci jako wskaźnik i rozmiar (lub rozmiar znany w czasie kompilacji). Jest to lekka, **nieposiadająca danych** alternatywa dla pary `(T*, size)`.

```
#include <span>

void normalize(std::span<double> xs) {
    for (double& x : xs) x /= 100.0;
}

std::vector<double> v{100, 200, 300};
normalize(v);  // v becomes {1, 2, 3}
```

`std::span<const T>` umożliwia odczyt bez modyfikowania elementów, a `std::span<T>` pozwala modyfikować elementy, mimo że sam widok nie posiada pamięci.

```
int a[]{1, 2, 3, 4};
std::span<int> all{a};
auto head = all.first(2);
head[1] = 20; // a[1] == 20
```

`std::string_view` spełnia podobną rolę dla tekstu; nie alokuje pamięci na znaki (oraz nie musi kończyć się bajtem zero). Trzeba uważać, aby nie zapisać go po zniszczeniu źródłowego `std::string`.

```cpp
#include <string_view>

std::string text = "abcdef";
std::string_view view{text};
std::string_view prefix = view.substr(0, 3); // "abc"
// Do not use view after text is destroyed or invalidated.
```

`std::ranges::subrange<I, S>` grupuje iterator `I` i sentinel `S` w zakres bez kopiowania elementów. Także tutaj obowiązuje odpowiedzialność za czas życia wskazywanych danych.

```cpp
auto sub = std::ranges::subrange(v.begin(), v.end());
// sub refers to elements of v; it does not own v.
```

Ostrożnie z realokacją. `std::span` do elementów wektora, `string_view` do znaków łańcucha oraz `subrange` z iteratorów do kontenera mogą stać się niepoprawne po operacji zmieniającej miejsce przechowywania danych. Przenoszenie lub realokacja nie są automatycznie śledzone.

13.14 Własny widok za pomocą `view_interface`

Na końcu zbudujemy *własny widok* generujący kwadraty liczb całkowitych od zera do $n-1$. Użyjemy `std::ranges::view_interface`, które dostarcza wygodnych operacji opartych na `begin()` i `end()`. W klasie pozostają wyłącznie parametry generatora, więc nie ma alokacji pamięci na elementy sekwencji.

```cpp
#include <concepts>
#include <cstddef>
#include <iterator>
#include <ranges>

struct SquaresView : std::ranges::view_interface<SquaresView> {
    struct iterator {
        using value_type = int;
        using difference_type = std::ptrdiff_t;
        using iterator_concept = std::forward_iterator_tag;

        int index = 0;
        int operator*() const { return index * index; }
        iterator& operator++() {
            ++index;
            return *this;
        }
        iterator operator++(int) {
            auto old = *this;
            ++*this;
            return old;
        }
        bool operator==(const iterator&) const = default;
    };

    int n = 0;
    SquaresView() = default;
    explicit SquaresView(int count) : n(count) {}

    iterator begin() const { return {0}; }
    iterator end() const { return {n}; }
};

static_assert(std::ranges::view<SquaresView>);
```

```
35 static_assert(std::ranges::forward_range<SquaresView>);
```

```
1  SquaresView squares{6};
2  for (int x : squares) { /* 0, 1, 4, 9, 16, 25 */ }
3
4  // Our view works with standard adapters:
5  auto even_squares = squares | std::views::filter(
6      [](int x) { return x % 2 == 0; });
7  // Iteration yields 0, 4, 16.
```

Konstruktor z argumentem `count` powinien otrzymać wartość nieujemną. Ponieważ liczymy `index * index` w typie `int`, należy ponadto ograniczyć rozmiar zakresu, aby uniknąć przepełnienia. W rozwiązaniu produkcyjnym należałoby te warunki weryfikować oraz dopracować obsługę szerszych typów całkowitych.

Widok a adapter. `SquaresView` jest własnym *widokiem*, który można przekazać do standardowych adapterów. Nie jest jeszcze własnym *range adaptor closure*, czyli obiektem pozwalającym zapisać `some_range | squares_adaptor`. Implementacja takiego adaptera stanowi osobny poziom abstrakcji i nie jest konieczna do budowania własnych widoków.

13.15 Różnica między `transform` i `views::transform`

Poniższe operacje mogą prowadzić do takich samych liczb, ale różnią się *czasem wykonania* i *sposobem przechowywania wyniku*:

```
1  std::vector<int> source{1, 2, 3};
2  std::vector<int> eager;
3  std::ranges::transform(source,
4      std::back_inserter(eager),
5      [](int x) { return x * x; });
6  // eager owns {1, 4, 9} right now
7
8  auto lazy = source | std::views::transform(
9      [](int x) { return x * x; });
10 // lazy does not own a vector of squared values
```

Wariant pierwszy wymaga pamięci na trzy elementy i kończy obliczenia przed kolejną instrukcją. Wariant drugi może obliczać wartości dopiero przy dereferencji iteratorów. Po zmianie `source` kolejne odczyty widoku `lazy` mogą odzwierciedlać nowe dane.

13.16 Projektowanie API z wykorzystaniem Ranges

Jeżeli funkcja tylko przegląda elementy dowolnego źródła, preferuj parametr ograniczony koncepcją `input_range`. Dla wielokrotnych przejść korzystaj z `forward_range`. Jeśli potrzebujesz ciągłego bloku pamięci i znasz typ elementów, wygodnym wyborem jest `std::span<T>`.

```
1  template <std::ranges::input_range R>
2  void print_each(R&& r) {
3      for (auto&& x : r) {
4          // Print or consume x.
5      }
6  }
7
8  void process_buffer(std::span<const double> values) {
9      for (double value : values) {
```

```
10          // Read numeric values.
11      }
12  }
```

Zależnie od celu można zwracać widok albo kontener. Widok jest wygodny, gdy chcemy dalej budować potok i mamy kontrolę nad czasem życia źródła. Kontener jest często bezpieczniejszy, gdy zwracany wynik ma istnieć niezależnie od wszystkich argumentów.

13.17 Najczęstsze błędy i praktyczne reguły

1. **Dereferencja `end()`.** Iterator końcowy jest granicą, a nie elementem. Przed `*it` trzeba mieć `it != end`.
2. **Mylenie kategorii iteratorów.** Lista nie obsługuje skoków w stałym czasie. Nie można użyć dla niej algorytmu `ranges::sort`; należy wywołać metodę `list::sort`.
3. **Traktowanie widoku jak kontenera.** `views::filter` nie tworzy kopii wybranych elementów. Zmiany i zniszczenie źródła mogą wpływać na wynik.
4. **Wiszące referencje w lambdach.** Zwrócony widok z predykatem przechwytującym lokalny obiekt przez referencję może być niebezpieczny nawet wtedy, gdy sam kontener ma odpowiednio długi czas życia.
5. **Efekty uboczne w predykacie.** Liczba i kolejność wywołań w leniwym potoku nie są odpowiednią podstawą działania programu; predykaty filtrowania powinny zachować stabilną semantykę.
6. **Zakładanie stałego kosztu operacji.** Obliczenie `distance` lub pierwszego pasującego elementu może wymagać przejrzenia dużej części źródła.
7. **Mieszanie standardów.** `ranges::to`, `views::zip` i `views::enumerate` nie należą do C++20.
8. **Używanie nieważnych iteratorów.** Realokacja wektora lub usuwanie elementu może unieważnić iteratory także wtedy, gdy znajdują się wewnątrz wcześniej utworzonego widoku.

13.18 Której konstrukcji użyć?

Potrzeba	**Zalecany wybór**
Przejście po zwykłym kontenerze	Pętla `for (auto& x : v)`
Jednorazowe sortowanie albo wyszukiwanie	`std::ranges::sort`, `std::ranges::find`
Leniwe filtrowanie i przekształcanie	`std::views::filter`, `std::views::transform`
Generowanie sekwencji indeksów	`std::views::iota`
Odczyt lub modyfikacja ciągłego bufora	`std::span<T>` (bez własności)
Lekki odczyt fragmentu tekstu	`std::string_view` (bez własności)
Zakres wyznaczony iteratorami	`std::ranges::subrange`
Wynik niezależny od źródła	Kontener, np. `std::vector`
Własny generator wartości	Typ modelujący `std::ranges::view`
Wydajny ogólny algorytm generyczny	Szablon z `std::ranges::input_range` lub silniejszą koncepcją

Pytania kontrolne.

1. Dlaczego iterator `end()` nie jest elementem kolekcji?

2. Co odróżnia `forward_iterator` od `input_iterator`?
3. Czy `std::ranges::sort` działa dla `std::list`? Dlaczego?
4. Po co `std::ranges` dopuszcza sentinel innego typu niż iterator?
5. W którym momencie `views::transform` wykonuje funkcję?
6. Dlaczego `views::filter` nie powinien zmieniać elementów, od których zależy jego predykat?
7. Czym różni się `views::all(lvalue)` od `views::all(rvalue)` dla `std::vector`?
8. Co gwarantuje, a czego nie gwarantuje `borrowed_range`?
9. Dlaczego `ranges::find(std::vector{...}, x)` może zwrócić `dangling`?
10. Jak zebrać elementy `filter_view` do wektora w czystym C++20?
11. Czym różni się `std::span<const T>` od `std::vector<T>`?
12. Co należy zaimplementować, aby własny obiekt był `forward_range`?

Dalsza lektura. Dobrym punktem odniesienia jest dokumentacja *cppreference*: *Ranges library*, *Iterator library*, *Range adaptors*, *std::ranges::borrowed_range*, *std::ranges::dangling* oraz *std::ranges::view_interface*. Warto testować kod z opcjami kompilatora `-std=c++20` i `-Wall -Wextra -Wpedantic`, a także korzystać z AddressSanitizer przy badaniu czasu życia danych.

Rozdział 14

Współbieżność i asynchroniczność

14.1 Motywacja i podstawowe pojęcia

W programie jednowątkowym instrukcje wykonywane są w jednym *wątku sterowania*. Nie znaczy to, że procesor nie wykonuje w tle innych czynności; chodzi o model samego programu. W programie współbieżnym istnieje kilka niezależnych ciągów instrukcji, których wykonanie może się przeplatać. Zamiast jednego zadania rozpatrujemy np. odczyt danych, obliczenia i zapisywanie wyników.

Współbieżność (*concurrency*)
: Wiele zadań może robić postępy w nakładających się okresach czasu. Nie wymaga to, aby fizycznie wykonywały się w tej samej chwili.

Równoległość (*parallelism*)
: Co najmniej dwa zadania wykonują się fizycznie jednocześnie, na przykład na dwóch rdzeniach procesora.

Asynchroniczność (*asynchrony*)
: Rozpoczęcie operacji nie wymusza natychmiastowego oczekiwania na jej wynik. Wywołujący może kontynuować pracę, a wynik odebrać później.

Pojęcia te nie są synonimami. Można mieć współbieżność bez równoległości (jeden rdzeń i przełączanie zadań), a także interfejs asynchroniczny bez nowego wątku (np. opóźnione wykonanie funkcji).

Uwaga. Zegar i kolejność komunikatów w przykładach współbieżnych nie stanowią gwarantowanego wyniku programu. Planista systemu może przeplatać wątki inaczej przy każdym uruchomieniu.

14.1.1 Proces, wątek, zadanie i współdzielone dane

Proces zwykle posiada własną przestrzeń adresową. Wątki tego samego procesu współdzielą między innymi pamięć dynamiczną i zmienne globalne, ale mają własne stosy i własne wykonanie instrukcji. *Zadanie* to jednostka pracy, którą można przypisać wątkowi; nie każde zadanie musi oznaczać nowy wątek.

Dostęp do wspólnego obiektu wymaga szczególnej ostrożności, jeżeli co najmniej jeden wątek go modyfikuje. Sam fakt istnienia wielu wątków nie jest błędem; problem pojawia się przy niezsynchronizowanym dostępie do pamięci.

14.2 Uruchamianie wątków: `std::thread`

Nagłówek `<thread>` udostępnia typ `std::thread` (C++11). Utworzenie obiektu z funkcją uruchamia nowy wątek. Wywołanie `join()` czeka na jego zakończenie; `joinable()` informuje, czy obiekt nadal reprezentuje wątek wymagający obsłużenia.

```cpp
#include <iostream>
#include <thread>

void work(int id) {
    std::cout << "Worker " << id << "\n";
}

int main() {
    std::thread a(work, 1);
    std::thread b(work, 2);

    a.join();
    b.join();
}
```

Listing 14.1: Pierwszy program z dwoma wątkami (C++11).

Kolejność wypisywania nie jest określona, a fragmenty wypisywane przez różne wątki mogą się przeplatać. Jeżeli wymagana jest spójna linia wyjścia, można chronić wypisywanie mutexem albo użyć `std::osyncstream` (C++20).

14.2.1 Przekazywanie argumentów i czasu życia

Argumenty przekazywane do konstruktora `std::thread` są odpowiednio przechowywane na potrzeby nowego wątku; domyślnie nie oznacza to przekazania modyfikowalnego argumentu przez referencję. Referencję można przekazać za pomocą `std::ref` albo przechwycenia referencji w lambdzie.

```cpp
#include <functional>
#include <iostream>
#include <thread>

void increment(int& value) { ++value; }

int main() {
    int counter = 0;
    std::thread worker(increment, std::ref(counter));
    worker.join(); // counter nadal istnieje
    std::cout << counter << '\n'; // 1
}
```

Listing 14.2: Bezpieczne przekazanie referencji z zachowaniem jej czasu życia.

Destruktor obiektu `std::thread`, który nadal jest `joinable()`, wywołuje `std::terminate()`. Dlatego przy wielu ścieżkach wyjścia lepiej stosować RAII, np. `std::jthread`. Operacja `detach()` wymaga z kolei gwarancji, że wątek nie będzie używał już zniszczonych obiektów.

14.3 Wyścig danych, warunek wyścigu i relacja *happens-before*

Wyścig danych (*data race*) występuje, gdy co najmniej dwa potencjalnie współbieżne dostępy do tego samego miejsca pamięci są konfliktowe (co najmniej jeden modyfikuje), nie są odpowiednio uporządkowane przez synchronizację i nie są to dostępy atomowe w wymaganym sensie. W

standardowym C++ prowadzi to do **niezdefiniowanego zachowania**.

```
// Fragment celowo niepoprawny; NIE uruchamiac jako poprawnego przykladu.
int counter = 0;
// Watki rownoczesnie wykonuja ++counter bez synchronizacji.
```

Listing 14.3: Błędny przykład: niezdefiniowane zachowanie.

Warunek wyścigu (*race condition*) jest pojęciem szerszym: wynik programu zależy od względnej kolejności zdarzeń. Warunek wyścigu nie zawsze oznacza niezdefiniowane zachowanie; np. dwa poprawnie zsynchronizowane wątki mogą rywalizować o to, który pierwszy wykona zadanie.

Relacja *happens-before* formalizuje, jakie skutki wcześniejszych operacji muszą być widoczne dla późniejszych. Tworzy się m.in. przez porządek operacji w jednym wątku i przez operacje synchronizujące (np. odpowiednio sparowane `release`/`acquire`, odblokowanie/zablokowanie mutexa). To centralny element modelu pamięci C++.

14.4 Ochrona danych: `mutex` i blokady RAII

`std::mutex` pozwala maksymalnie jednemu wątkowi utrzymywać blokadę w danej chwili. Jeśli kilka operacji ma tworzyć niepodzielną *sekcję krytyczną*, umieszczamy je pod tą samą blokadą.

```
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>

int main() {
    int counter = 0;
    std::mutex mtx;
    std::vector<std::thread> workers;

    for (int i = 0; i < 4; ++i) {
        workers.emplace_back([&] {
            for (int j = 0; j < 10000; ++j) {
                std::lock_guard<std::mutex> lock(mtx);
                ++counter;
            }
        });
    }
    for (auto& t : workers) t.join();
    std::cout << counter << '\n'; // 40000
}
```

Listing 14.4: Bezpieczny licznik przy użyciu mutexa (C++11).

`lock_guard` automatycznie przejmuje mutex w konstruktorze i zwalnia w destruktorze. Dzięki temu zwolnienie blokady następuje także podczas rozwijania stosu po wyjątku. `unique_lock` jest bardziej elastyczny: pozwala jawnie odblokować i ponownie zablokować mutex oraz współpracuje z `condition_variable`.

14.4.1 Kilka mutexów i zakleszczenie

Jeżeli dwa wątki blokują mutexy w przeciwnej kolejności, może wystąpić *deadlock*: pierwszy czeka na drugi mutex, drugi czeka na pierwszy. W C++17 można bezpiecznie przejmować kilka mutexów przez `scoped_lock`.

```
#include <mutex>
```

```
2
3  int main() {
4      std::mutex a, b;
5      {
6          std::scoped_lock lock(a, b);
7          // Operacje chronione jednoczesnie przez a i b.
8      }
9  }
```

Listing 14.5: Łączne przejęcie dwóch mutexów (C++17).

Uwaga. Mutexy nie chronią „samej zmiennej” automatycznie: poprawność wymaga, aby *wszystkie* konfliktowe dostępy do chronionego stanu stosowały uzgodniony mechanizm synchronizacji.

14.5 Oczekiwanie na stan: `condition_variable`

Mutex zapewnia wyłączność, ale nie sygnalizuje gotowości danych. `std::condition_variable` pozwala czekać bez aktywnego odpytywania. Do oczekiwania z predykatem używamy blokady

`std::unique_lock<std::mutex>`.

Predykat opisuje stan, na który czeka konsument.

```
1  #include <condition_variable>
2  #include <iostream>
3  #include <mutex>
4  #include <thread>
5
6  int main() {
7      std::mutex mtx;
8      std::condition_variable cv;
9      bool ready = false;
10     int data = 0;
11
12     std::thread consumer([&] {
13         std::unique_lock<std::mutex> lock(mtx);
14         cv.wait(lock, [&] { return ready; });
15         std::cout << data << '\n'; // 42
16     });
17
18     std::thread producer([&] {
19         {
20             std::lock_guard<std::mutex> lock(mtx);
21             data = 42;
22             ready = true;
23         }
24         cv.notify_one();
25     });
26
27     producer.join();
28     consumer.join();
29 }
```

Listing 14.6: Producent i konsument z warunkiem gotowości (C++11).

Wywołanie `wait(lock, pred)` ma następujący sens:

1. podczas trzymania mutexa sprawdzany jest predykat;
2. jeśli jest fałszywy, wątek atomowo zwalnia mutex i przechodzi w oczekiwanie;

3. po powiadomieniu (lub pozornym wybudzeniu) odzyskuje mutex i znów sprawdza predykat;
4. kończy czekanie dopiero wtedy, gdy predykat jest prawdziwy.

Dopuszczalne są *spurious wakeups*, czyli wybudzenia bez zmiany interesującego nas stanu. Dlatego samo `cv.wait(lock)` zwykle nie wystarcza. Powiadomienie `notify_one()` budzi co najwyżej jeden oczekujący wątek, a `notify_all()` umożliwia wybudzenie wszystkich. Powiadomienia nie są kolejkowane jak komunikaty; stan `ready`, a nie samo powiadomienie, stanowi źródło prawdy.

14.5.1 Kolejka producent–konsument

W ogólniejszym przypadku producent umieszcza wiele elementów w kolejce, a konsument czeka na warunek „kolejka niepusta lub zakończono produkcję”. Flaga zakończenia jest konieczna, aby konsument nie oczekiwał bez końca po przetworzeniu ostatniego elementu.

```cpp
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <queue>
#include <thread>

int main() {
    std::mutex m;
    std::condition_variable cv;
    std::queue<int> q;
    bool done = false;

    std::thread consumer([&] {
        for (;;) {
            int item;
            {
                std::unique_lock<std::mutex> lock(m);
                cv.wait(lock, [&] { return done || !q.empty(); });
                if (q.empty() && done) break;
                item = q.front();
                q.pop();
            }
            std::cout << "Consumed " << item << '\n';
        }
    });

    std::thread producer([&] {
        for (int x = 1; x <= 3; ++x) {
            {
                std::lock_guard<std::mutex> lock(m);
                q.push(x);
            }
            cv.notify_one();
        }
        {
            std::lock_guard<std::mutex> lock(m);
            done = true;
        }
        cv.notify_all();
    });

    producer.join();
    consumer.join();
}
```

Listing 14.7: Współdzielona kolejka i sygnał zakończenia (C++11).

14.6 Operacje atomowe: `std::atomic`

`std::atomic<T>` udostępnia atomowe operacje na obiekcie `T`. Nie oznacza to ogólnie „braku jakichkolwiek blokad” w implementacji: nie każda instancja atomika jest *lock-free*. Można to sprawdzić metodą `is_lock_free()`.

```cpp
#include <atomic>
#include <iostream>
#include <thread>
#include <vector>

int main() {
    std::atomic<int> counter{0};
    std::vector<std::thread> threads;
    for (int i = 0; i < 4; ++i) {
        threads.emplace_back([&] {
            for (int j = 0; j < 10000; ++j)
                counter.fetch_add(1, std::memory_order_relaxed);
        });
    }
    for (auto& t : threads) t.join();
    std::cout << counter.load() << '\n'; // 40000
}
```

Listing 14.8: Licznik atomowy z porządkiem relaxed (C++11).

W przypadku samego licznika potrzebujemy niepodzielności inkrementacji, a nie synchronizacji dodatkowych danych. Stąd wystarcza `memory_order_relaxed`. Domyślnie operacje atomowe stosują `memory_order_seq_cst`, który narzuca silniejsze gwarancje porządkowania.

14.6.1 Operacja porównaj-i-zamień

`compare_exchange_weak(expected, desired)` atomowo sprawdza, czy obecna wartość jest równa `expected`. Przy powodzeniu zapisuje `desired`; przy niepowodzeniu aktualizuje `expected` do wartości odczytanej. Wersja `weak` może zgłosić niepowodzenie także pozornie, dlatego często umieszcza się ją w pętli.

```cpp
#include <atomic>
#include <iostream>

int main() {
    std::atomic<int> maximum{3};
    int desired = 10;
    int expected = maximum.load(std::memory_order_relaxed);

    while (expected < desired &&
           !maximum.compare_exchange_weak(
               expected, desired,
               std::memory_order_relaxed)) {
        // expected zawiera najnowsza odczytana wartosc
    }
    std::cout << maximum.load() << '\n'; // 10
}
```

Listing 14.9: Atomowe podnoszenie wartości do zadanego minimum (C++11).

14.6.2 Porządek pamięci: `release` i `acquire`

Atomowość jednej zmiennej nie wystarcza, gdy jednym sygnałem chcemy bezpiecznie opublikować inne dane. Stosujemy wówczas odpowiednio sparowane operacje `release` i `acquire`.

```cpp
#include <atomic>
#include <iostream>
#include <thread>

int main() {
    int data = 0;
    std::atomic<bool> ready{false};

    std::thread producer([&] {
        data = 42;
        ready.store(true, std::memory_order_release);
    });
    std::thread consumer([&] {
        while (!ready.load(std::memory_order_acquire)) {
            std::this_thread::yield();
        }
        std::cout << data << '\n'; // 42
    });

    producer.join();
    consumer.join();
}
```

Listing 14.10: Bezpieczna publikacja zwykłej zmiennej (C++11).

Gdy odczyt `acquire` zaobserwuje właściwy zapis `release`, operacje *synchronizes-with* gwarantują widoczność wcześniejszego zapisu `data = 42`. Nie wolno tu zamieniać obu porządków na `relaxed`.

Porządek	Zasadnicza gwarancja
`relaxed`	Atomowość bez synchronizacji pozostałych danych.
`release`	Publikowanie wcześniejszych efektów do odpowiedniego odczytu.
`acquire`	Odczyt odbierający efekty opublikowane przez `release`.
`acq_rel`	Połączenie obu kierunków dla operacji odczyt–modyfikacja–zapis.
`seq_cst`	Dodatkowy wspólny porządek operacji sekwencyjnie spójnych.

Istnieje również `memory_order_consume`, ale jego historia i obsługa przez kompilatory są skomplikowane; nie wykorzystujemy go w przykładach. Programiście początkującemu zalecamy domyślny `seq_cst`, a słabsze porządki dopiero po uzasadnieniu poprawności.

14.6.3 Czekanie na atomik bez aktywnej pętli: C++20

Od C++20 obiekt atomowy udostępnia `wait(old)` oraz `notify_one()/notify_all()`. Oczekiwanie kończy się, kiedy zaobserwowana wartość różni się od `old`; wątek nie musi stale wykonywać aktywnego odpytywania.

```cpp
#include <atomic>
#include <iostream>
#include <thread>

int main() {
    std::atomic<bool> ready{false};
    int payload = 0;

```

```
    std::jthread worker([&] {
        payload = 42;
        ready.store(true, std::memory_order_release);
        ready.notify_one();
    });

    ready.wait(false, std::memory_order_acquire);
    std::cout << payload << '\n'; // 42
}
```

Listing 14.11: Atomowe czekanie i wybudzanie (C++20).

Uwaga. Atomowy znacznik nie jest zamiennikiem mutexa chroniącego wiele powiązanych pól struktury. Algorytmy *lock-free* bywają trudniejsze od rozwiązania opartego na blokadzie; dodatkowo należy rozumieć m.in. problem ABA i czas życia współdzielonych obiektów.

14.7 Model pamięci C++: formalne podstawy synchronizacji

Sekcja 14.3 wprowadziła pojęcie wyścigu danych. Teraz sprecyzujemy, *dlaczego* poprawny program wielowątkowy nie może polegać na fizycznym porządku wykonywania instrukcji. Model pamięci C++ określa dopuszczalne obserwacje programu również wtedy, gdy kompilator zmienia kolejność instrukcji, procesor używa buforów zapisów, a rdzenie mają prywatne pamięci podręczne. Nie jest to model jednej konkretnej architektury (np. x86 albo ARM).

14.7.1 Trzy relacje: *sequenced-before*, *synchronizes-with* i *happens-before*

Niech A i B oznaczają dwa zdarzenia (ewaluacje) programu.

Sequenced-before
: Relacja porządkująca ewaluacje wewnątrz jednego wątku, wynikająca z zasad sekwencjonowania w C++. Nie wolno jej utożsamiać z samą kolejnością występowania tokenów w źródle: nie każda para podwyrażeń jest sekwencjonowana.

Synchronizes-with
: Relacja między odpowiednimi zdarzeniami synchronizacyjnymi, np. skutecznym odblokowaniem mutexa i późniejszym jego zablokowaniem albo zapisem `release` i odczytem `acquire`, który obserwuje zapis (lub odpowiedni element jego sekwencji *release*).

Happens-before
: Relacja logicznego porządku zdarzeń. W przykładach bez osobliwego porządku *consume* można ją konstruować z dwóch poprzednich relacji i przechodniości. Jeżeli A *happens-before* B, skutki A ograniczają dopuszczalne obserwacje zdarzenia B.

Dla trzech zdarzeń reguła przechodniości ma postać

$$A \operatorname{hb} B \ \wedge \ B \operatorname{hb} C \quad \Longrightarrow \quad A \operatorname{hb} C.$$

Przykładowy łańcuch synchronizacji można zapisać jako

$$\begin{aligned} A:\ \text{zapis } x &\xrightarrow{\text{sequenced-before}} B:\ \text{store(release)},\\ B &\xrightarrow{\text{synchronizes-with}} C:\ \text{load(acquire)},\\ C &\xrightarrow{\text{sequenced-before}} D:\ \text{odczyt } x. \end{aligned}$$

Z przechodniości wynika A hb D. Istotne jest jednak, że operacja `load(acquire)` musi rzeczywiście odczytać wartość pochodzącą z odpowiedniego zapisu `release` (lub jego sekwencji); samo użycie obu etykiet w dwóch wątkach nie wystarcza.

14.7.2 Wyścig danych a wyścig logiczny: przykład

```
#include <atomic>
#include <thread>

int main() {
    std::atomic<int> winner{0};
    std::jthread a([&] {
        int expected = 0;
        winner.compare_exchange_strong(expected, 1);
    });
    std::jthread b([&] {
        int expected = 0;
        winner.compare_exchange_strong(expected, 2);
    });
    a.join();
    b.join();
    // winner == 1 albo winner == 2.
}
```

Listing 14.12: Dwa atomiki eliminują data race, ale nie ustalają zwycięzcy.

Wynik zależy od harmonogramu, ale program nie ma wyścigu danych. *Race condition* jest błędem dopiero wtedy, gdy program wymaga konkretnego zwycięzcy albo narusza swój kontrakt w części harmonogramów.

14.7.3 Porządek modyfikacji pojedynczego atomika

Każdy obiekt atomowy ma własny, spójny dla wszystkich wątków *modification order*: całkowity porządek jego atomowych modyfikacji. Nie oznacza to wspólnego globalnego porządku wszystkich atomików. Dla modyfikacji `x` oraz `y` mogą więc istnieć obserwacje, które intuicyjnie zaskakują, mimo że program nie zawiera *data race*.

W poniższym przykładzie obie wartości startują od zera:

```
std::atomic<int> x{0}, y{0};
int r1 = -1, r2 = -1;

// Watek A:
x.store(1, std::memory_order_relaxed);
r1 = y.load(std::memory_order_relaxed);

// Watek B:
y.store(1, std::memory_order_relaxed);
r2 = x.load(std::memory_order_relaxed);
```

Listing 14.13: Litmus test: dwa zapisy, dwa odczyty (fragment dwóch wątków).

Przy porządku `relaxed` *dopuszczalne* jest $r_1 = r_2 = 0$. Zastąpienie *wszystkich czterech* operacji przez `seq_cst` wyklucza ten wynik: jeden wspólny porządek musiałby wtedy spełniać cykliczne sprzeczne ograniczenia. Test uruchomiony tylko na jednym procesorze niczego tu nie dowodzi: brak obserwacji nie oznacza zakazu w standardzie.

14.7.4 Porządki atomowe w praktyce

Porządek	Praktyczny sens i typowe zastosowanie
`relaxed`	Niepodzielność i porządek modyfikacji danego atomika; liczniki statystyk, gdy nie synchronizują innego stanu.
`release`	Publikuje efekty poprzedzających operacji; stosowany do zapisów lub odpowiednich operacji RMW.
`acquire`	Odbiera publikację po odczytaniu właściwej wartości; stosowany do odczytów lub odpowiednich RMW.
`acq_rel`	Oba kierunki synchronizacji podczas operacji typu read–modify–write.
`seq_cst`	Domyślny, mocniejszy porządek; dodatkowo wspólne uporządkowanie wszystkich operacji `seq_cst`.

`memory_order_consume` jest historycznie problematyczny; nie stanowi dobrego punktu wyjścia do pisania przenośnego kodu. Wybór słabszego porządku należy poprzedzić argumentem poprawności, a nie samym wynikiem benchmarku. Nie wszystkie porządki są dozwolone dla każdej operacji, np. zwykły `store` nie może użyć `memory_order_acquire`.

14.7.5 Publikacja danych i sekwencja *release*

Rozpatrzmy trzy zdarzenia: producent wypełnia dane, drugi wątek zwiększa atomowy licznik, a trzeci odbiera wynik. W C++20 (i wcześniej) sekwencja *release* po zapisie `release` może być kontynuowana przez kolejne atomowe operacje *read–modify–write* tego samego atomika, także wykonywane przez inne wątki. Dlatego odczyt `acquire` wartości zapisanej przez taką operację RMW może synchronizować się z początkowym zapisem `release`.

```
int payload = 0;
std::atomic<int> state{0};

// Producent (A):
payload = 42;
state.store(1, std::memory_order_release);

// Posrednik (B), DOPIERO po wykonaniu powyzszego store:
state.fetch_add(1, std::memory_order_relaxed); // 1 -> 2

// Konsument (C): odczytuje 2 z RMW posrednika:
if (state.load(std::memory_order_acquire) == 2) {
    // Wolno odczytac payload == 42.
}
```

Listing 14.14: Sekwencja release: fragment przy ustalonej kolejności operacji.

Warunek o kolejności jest częścią przykładu: bez niego pośrednik mógłby zwiększyć `state` jeszcze przed publikacją. Zwykły zapis wykonany przez inny wątek przerywa tę sekwencję; nie należy jej mylić z dowolną serią zapisów do tego samego atomika.

14.7.6 Operacje CAS: sukces i porażka

`compare_exchange_weak` i `compare_exchange_strong` mają przeciążenia z osobnymi porządkami pamięci dla sukcesu i porażki. Operacja nieudana jest *odczytem*, a nie zapisem. Dlatego jej porządek nie może być `release` ani `acq_rel` oraz nie może być silniejszy od porządku sukcesu w sensie wymagań standardu.

```
#include <atomic>

```

```cpp
void add_one(std::atomic<int>& x) {
    int expected = x.load(std::memory_order_relaxed);
    while (!x.compare_exchange_weak(
        expected, expected + 1,
        std::memory_order_acq_rel,
        std::memory_order_acquire)) {
        // expected zostalo zaktualizowane po niepowodzeniu.
    }
}
```

Listing 14.15: Pętla CAS: zwiększanie licznika, przykład praktyczny.

W tym przypadku `fetch_add(1)` jest prostsze i zwykle lepsze. CAS ma sens zwłaszcza przy warunkowej zmianie stanu lub złożonych strukturach atomowych. `weak` może nie powieść się pozornie, więc jest wygodny w pętli; `strong` lepiej nadaje się do pojedynczych prób. Użycie `acq_rel` w powyższym liczniku służy zilustrowaniu API; gdy licznik niczego nie publikuje, wystarczy `memory_order_relaxed`.

14.7.7 Bariery pamięci: `atomic_thread_fence`

`std::atomic_thread_fence(order)` nakłada ograniczenia porządkowania pamięci, ale *sama* bariera nie synchronizuje magicznie dwóch wątków. Potrzebna jest odpowiednia operacja atomowa, która połączy strony synchronizacji. Przykład celowo przenosi `release`/`acquire` z atomika na bariery:

```cpp
#include <atomic>
#include <thread>
#include <cassert>

int main() {
    int payload = 0;
    std::atomic<bool> ready{false};

    std::thread producer([&] {
        payload = 42;
        std::atomic_thread_fence(std::memory_order_release);
        ready.store(true, std::memory_order_relaxed);
    });
    std::thread consumer([&] {
        while (!ready.load(std::memory_order_relaxed)) {
            std::this_thread::yield();
        }
        std::atomic_thread_fence(std::memory_order_acquire);
        assert(payload == 42);
    });
    producer.join();
    consumer.join();
}
```

Listing 14.16: Publikacja przez dwie bariery i atomowy znacznik (C++11).

Dla tego układu bariery łączą się poprzez odczyt `ready`, który obserwuje zapis producenta. Zwykłe `ready.store(release)` oraz `ready.load(acquire)` byłoby prostsze. `atomic_signal_fence` ma inny cel: ogranicza określone przestawienia przez kompilator, a nie pełni roli zwykłej synchronizacji między wątkami.

14.7.8 Atomowość, blokady i gwarancje postępu

Blocking
Wątek może oczekiwać na zwolnienie blokady przez innego uczestnika, który np. został wywłaszczony przez system.

Lock-free
W standardowym ujęciu algorytmowym system jako całość może robić postęp mimo tego, że pewne wątki wielokrotnie ponawiają operację. Pojedynczy wątek może ulegać zagłodzeniu.

Wait-free
Każdy uczestnik kończy operację w ograniczonej liczbie własnych kroków, niezależnie od harmonogramu innych uczestników.

`atomic<T>::is_lock_free()` bada własność implementacji operacji na tym atomiku; nie jest dowodem, że dowolny algorytm zbudowany z atomików jest *lock-free*. Nie wolno też wnioskować, że *lock-free* zawsze oznacza szybszy kod. Rywalizacja o jedną linię cache może dominować koszt obliczeń.

14.7.9 Problem ABA i odzyskiwanie pamięci

Niech wierzchołek współbieżnego stosu zawiera wskaźnik `A`. Wątek 1 odczytuje `A` i zapamiętuje wskaźnik do następnego węzła, po czym zostaje wstrzymany. Wątek 2 usuwa `A`, usuwa kolejny element `B` i ponownie umieszcza `A` na szczycie. Wątek 1 widzi znowu `A`: porównanie wskaźnika przez CAS może się powieść, choć pomiędzy odczytem i zapisem struktura zmieniła się istotnie. Co gorsza, wcześniej zapamiętany węzeł mógł zostać już zwolniony. To właśnie *problem ABA*.

$$\boxed{\mathrm{A} \to \mathrm{B} \to \mathrm{C}} \quad \longrightarrow \quad \boxed{\mathrm{B} \to \mathrm{C}} \quad \longrightarrow \quad \boxed{\mathrm{A} \to \mathrm{C}}.$$

Do typowych narzędzi należą: oznaczanie wskaźnika licznikiem wersji (*tagged pointer*), *hazard pointers*, *epoch-based reclamation* i *reference counting*. Zabezpieczenie samego porównania wskaźników nie załatwia problemu *czasu życia obiektu*. Dla zwykłej kolejki aplikacyjnej `std::mutex` bywa najprostszy i najbardziej wiarygodny.

14.7.10 Zagłodzenie, *livelock* i błędne podwójne sprawdzenie

Deadlock oznacza brak postępu wskutek cyklicznego oczekiwania. *Livelock* występuje, gdy wątki aktywnie reagują na siebie, ale nie kończą pracy. *Starvation* to brak postępu konkretnego wątku mimo że inne wątki pracują. Te trzy zjawiska należy odróżniać.

W starszym kodzie spotyka się tzw. *double-checked locking*. Bez poprawnych atomików albo mutexa nie można go naprawić przez samo `volatile`. W nowym kodzie znacznie lepiej używać bezpiecznej inicjalizacji statycznej zmiennej lokalnej (C++11) lub `std::call_once`. `volatile` nie zapewnia atomowości ani synchronizacji wątków.

14.8 Wynik przyszłego obliczenia: `future` i `promise`

`std::future<T>` reprezentuje możliwość odebrania *jednego* wyniku typu `T`, który może jeszcze nie być dostępny. `std::promise<T>` umożliwia producentowi ten wynik ustawić. Te obiekty odnoszą się do wspólnego *stanu asynchronicznego* (ang. *shared state*). Sam `promise` nie uruchamia wątku.

`promise.set_value(x)` ⟶ stan współdzielony ⟶ `future.get()`

```cpp
#include <future>
#include <iostream>
#include <thread>
#include <utility>

void compute(std::promise<int> producer) {
    producer.set_value(6 * 7);
}

int main() {
    std::promise<int> promise;
    std::future<int> result = promise.get_future();

    std::thread worker(compute, std::move(promise));
    std::cout << result.get() << '\n'; // 42
    worker.join();
}
```

Listing 14.17: Ręczne przekazanie wyniku z wątku (C++11).

`future.get()` czeka, jeśli wynik nie jest jeszcze gotowy, po czym odbiera go (lub ponownie zgłasza wyjątek). Zwykłe `future` jest jednokrotnego odbioru: po `get()` obiekt przestaje być ważnym uchwytem do stanu i ponowne `get()` nie jest dozwolone. `wait()` tylko czeka, natomiast `wait_for()` i `wait_until()` pozwalają ograniczyć czas oczekiwania.

14.8.1 Przekazywanie wyjątku i *broken promise*

Producent może przekazać wyjątek za pomocą `set_exception()`. Jeśli `promise` zostanie zniszczony bez przekazania wyniku, stan otrzymuje błąd `broken_promise`, który zostanie zgłoszony podczas odbioru przez `future.get()`.

```cpp
#include <exception>
#include <future>
#include <iostream>
#include <stdexcept>
#include <thread>
#include <utility>

int main() {
    std::promise<int> producer;
    auto f = producer.get_future();

    std::thread worker([p = std::move(producer)]() mutable {
        try {
            throw std::runtime_error("Calculation failed");
        } catch (...) {
            p.set_exception(std::current_exception());
        }
    });

    try {
        std::cout << f.get() << '\n';
    } catch (const std::exception& e) {
        std::cout << e.what() << '\n';
    }
    worker.join();
}
```

Listing 14.18: Przekazanie wyjątku przez promise (C++14).

14.8.2 Wiele odbiorców: `shared_future`

`std::shared_future<T>` można kopiować i wielokrotnie odczytywać. Każdy odbiorca uzyskuje dostęp do tego samego wyniku. Uzyskujemy go np. przez `future.share()`.

```
#include <future>
#include <iostream>
#include <thread>

int main() {
    std::promise<int> p;
    std::shared_future<int> f = p.get_future().share();
    int a = 0, b = 0;

    std::thread first([&] { a = f.get(); });
    std::thread second([&] { b = f.get(); });

    p.set_value(42);
    first.join();
    second.join();
    std::cout << a << ' ' << b << '\n'; // 42 42
}
```

Listing 14.19: Dwa wątki odczytują ten sam wynik (C++11).

14.9 Uruchamianie zadań: `std::async`

`std::async` wiąże wywołanie funkcji ze stanem asynchronicznym i zwraca `std::future<R>` dla typu wyniku `R`. W odróżnieniu od `promise` nie trzeba samodzielnie obsługiwać przekazywania wartości ani wyjątków. Wariant `std::launch::async` uruchamia funkcję w innym wątku.

```
#include <future>
#include <iostream>

int square(int x) { return x * x; }

int main() {
    auto f1 = std::async(std::launch::async, square, 6);
    auto f2 = std::async(std::launch::async, square, 7);

    // Oba zadania wystartowaly przed pobraniem wynikow.
    std::cout << f1.get() + f2.get() << '\n'; // 85
}
```

Listing 14.20: Dwa niezależne obliczenia przez async (C++11).

14.9.1 Polityki uruchamiania

Polityka	Znaczenie
`std::launch::async`	Funkcja zostaje uruchomiona w osobnym wątku; wywołujący może kontynuować pracę.
`std::launch::deferred`	Funkcja zostaje wykonana dopiero przy pierwszym nieczasowym oczekiwaniu (np. `get()` lub `wait()`), w wątku, który tę operację wykonuje.
Brak jawnej polityki	Implementacja może wybrać `async`, `deferred` (lub dopuszczoną politykę implementacyjną).

Nie wolno utożsamiać samego wywołania `std::async(f)` z gwarancją równoległości. Jeśli potrzebujesz niezależnego wątku, wybierz jawnie `std::launch::async`.

```
#include <future>
#include <iostream>

int main() {
    auto f = std::async(std::launch::deferred, [] {
        std::cout << "Executing the task\n";
        return 42;
    });
    std::cout << "Before get\n";
    std::cout << f.get() << '\n';
}
```

Listing 14.21: Opóźnione wykonanie: deferred (C++11).

W tym przypadku komunikat `Before get` zawsze poprzedza wykonanie funkcji odroczonej, ponieważ została ona uruchomiona dopiero przez `get()`.

14.9.2 Destruktor tymczasowego `future` może blokować

Istotna pułapka polega na tym, że zniszczenie *ostatniego* uchwytu do niezakończonego stanu utworzonego przez `std::async` z polityką `async` może czekać na ukończenie zadania. Zapis poniżej nie uruchamia efektywnie obu prac obok siebie:

```
// Kazdy tymczasowy future jest niszczony na koncu instrukcji.
std::async(std::launch::async, expensive_a);
std::async(std::launch::async, expensive_b);
```

Listing 14.22: Pułapka: tymczasowe future; fragment ilustracyjny.

Poprawnie przechowujemy wyniki obu wywołań, zanim zaczniemy czekać:

```
auto a = std::async(std::launch::async, expensive_a);
auto b = std::async(std::launch::async, expensive_b);
a.wait();
b.wait();
```

Listing 14.23: Zadania mogą pracować równolegle; fragment.

14.9.3 Oczekiwanie z limitem czasu i polityka deferred

`wait_for()` zwraca jeden z trzech statusów: `ready`, `timeout` lub `deferred`. Przy zadaniu odroczonym samo `wait_for()` nie uruchamia pracy. Niepoprawna pętla sprawdzająca wyłącznie `ready` może więc kręcić się bez końca.

```
#include <chrono>
#include <future>
#include <iostream>

int main() {
    auto f = std::async(std::launch::deferred, [] { return 42; });
    auto status = f.wait_for(std::chrono::milliseconds(1));

    if (status == std::future_status::deferred) {
        std::cout << "Deferred: get starts the computation\n";
    } else if (status == std::future_status::timeout) {
        std::cout << "Not ready yet\n";
    } else {
```

```
        std::cout << "Already ready\n";
    }
    std::cout << f.get() << '\n';
}
```

Listing 14.24: Rozróżnianie statusów future (C++11).

14.9.4 Wyjątki z zadań i brak automatycznego anulowania

Jeśli funkcja uruchomiona przez `std::async` zgłosi wyjątek, wyjątek jest przechowywany w stanie asynchronicznym, a `get()` zgłasza go w wątku odbierającym wynik. `wait_for()` *nie anuluje* pracy przy przekroczeniu czasu: tylko informuje, że wynik jeszcze nie jest gotowy.

```
#include <future>
#include <iostream>
#include <stdexcept>

int main() {
    auto f = std::async(std::launch::async, []() -> int {
        throw std::runtime_error("Task failed");
    });
    try {
        std::cout << f.get() << '\n';
    } catch (const std::runtime_error& e) {
        std::cout << e.what() << '\n';
    }
}
```

Listing 14.25: Wyjątek z async zostaje przekazany przez get() (C++11).

14.10 `std::packaged_task`: funkcja z wynikiem future

`std::packaged_task<R(Args...)>` opakowuje funkcję w zadanie, którego wynik lub wyjątek trafia do powiązanego `future`. W odróżnieniu od `std::async` samo utworzenie `packaged_task` nie uruchamia żadnego wątku.

```
#include <future>
#include <iostream>
#include <thread>
#include <utility>

int main() {
    std::packaged_task<int(int)> task([](int x) {
        return x * x;
    });
    auto f = task.get_future();
    std::thread worker(std::move(task), 7);

    std::cout << f.get() << '\n'; // 49
    worker.join();
}
```

Listing 14.26: Packaged task i ręcznie uruchomiony wątek (C++11).

To jeden z elementów potrzebnych do zbudowania puli wątków: *task* trafia do kolejki, wątek roboczy go pobiera, a klient otrzymuje `future`. Pula wątków jest szczególnie przydatna dla bardzo wielu małych zadań, ponieważ tworzenie odrębnego wątku dla każdego zadania może być kosztowne.

14.11 `std::jthread` i kooperacyjne zatrzymywanie (C++20)

`std::jthread` jest obiektem zarządzającym wątkiem zgodnie z RAII. Jeżeli jest dołączalny, destruktor żąda zatrzymania przez `request_stop()`, a następnie wywołuje `join()`. Żądanie zatrzymania nie wymusza przerwania funkcji: wątek musi sam sprawdzać odpowiedni `stop_token`.

```
#include <chrono>
#include <iostream>
#include <thread>

int main() {
    std::jthread worker([](std::stop_token stop) {
        while (!stop.stop_requested()) {
            // Przykladowa porcja pracy.
            std::this_thread::sleep_for(std::chrono::milliseconds(5));
        }
        std::cout << "Stopped\n";
    });

    std::this_thread::sleep_for(std::chrono::milliseconds(25));
    worker.request_stop();
    // Destruktor worker dolaczy watek.
}
```

Listing 14.27: Kooperacyjne zatrzymywanie wątku (C++20).

Można również przekazywać `stop_token` do zadań, które reagują na anulowanie w większej liczbie punktów. Dla złożonych kolejek C++20 oferuje `condition_variable_any` z przeciążeniem obsługującym żądanie zatrzymania; zwykłe `condition_variable` nie ma analogicznego przeciążenia ze `stop_token`.

14.12 Inne przydatne mechanizmy biblioteki standardowej

14.12.1 `shared_mutex`: wielu czytelników, jeden pisarz

`std::shared_mutex` (C++17) pozwala wielu czytelnikom jednocześnie uzyskać blokadę współdzieloną, a pisarzowi — blokadę wyłączną. Czytanie chronimy przez `std::shared_lock`, zapisy przez `std::unique_lock`. Nie gwarantuje to braku zagłodzenia wątków; polityka przydziału blokad może zależeć od implementacji.

```
#include <iostream>
#include <mutex>
#include <shared_mutex>

int main() {
    std::shared_mutex m;
    int value = 10;
    {
        std::shared_lock<std::shared_mutex> read_lock(m);
        std::cout << value << '\n';
    }
    {
        std::unique_lock<std::shared_mutex> write_lock(m);
        value = 20;
    }
}
```

Listing 14.28: Blokada współdzielona i wyłączna (C++17).

14.12.2 `call_once` i inicjalizacja jednokrotna

`std::call_once` oraz `std::once_flag` pozwalają wykonać funkcję inicjalizującą co najwyżej raz pomyślnie. Jeżeli inicjalizacja rzuci wyjątek, późniejsze wywołanie może ponowić próbę. Lokalna statyczna zmienna ma w C++11 bezpieczną współbieżnie inicjalizację, co często jest prostszą alternatywą.

```
#include <iostream>
#include <mutex>
#include <thread>

std::once_flag init_flag;
int shared_value = 0;

void initialize() { shared_value = 42; }
void work() {
    std::call_once(init_flag, initialize);
    // Kazdy watek po call_once widzi zakonczona inicjalizacje.
}

int main() {
    std::thread a(work), b(work);
    a.join();
    b.join();
    std::cout << shared_value << '\n';
}
```

Listing 14.29: Inicjalizacja wykonywana raz (C++11).

14.12.3 `thread_local`: osobny egzemplarz dla wątku

Zmienne zadeklarowane `thread_local` mają osobny egzemplarz w każdym wątku. Nie rozwiązują automatycznie problemu komunikacji między wątkami, ale przydają się np. do lokalnych buforów roboczych.

```
#include <iostream>
#include <thread>

thread_local int local_counter = 0;
void count() { ++local_counter; }

int main() {
    std::thread a(count), b(count);
    a.join();
    b.join();
    std::cout << local_counter << '\n'; // 0 w watku main
}
```

Listing 14.30: Pamięć lokalna dla każdego wątku (C++11).

14.12.4 Semafory, zapadki i bariery w C++20

Ogranicza liczbę równoczesnych wejść. Operacja `acquire()` pobiera zezwolenie, a `release()` je zwraca.

Jednorazowy punkt synchronizacji: po odpowiedniej liczbie zgłoszeń `count_down()` czekające wątki mogą kontynuować.

Punkt synchronizacji wielokrotnego użytku dla kolejnych faz pracy.

```cpp
#include <chrono>
#include <iostream>
#include <semaphore>
#include <thread>
#include <vector>

int main() {
    std::counting_semaphore<2> slots(2);
    std::vector<std::jthread> workers;
    for (int i = 0; i < 4; ++i) {
        workers.emplace_back([&, i] {
            slots.acquire();
            // Jednoczesnie w sekcji moze byc najwyzej 2 watki.
            std::this_thread::sleep_for(std::chrono::milliseconds(10));
            slots.release();
        });
    }
    // Destruktory jthread dolaczaja watki przed destruktorem slots.
}
```

Listing 14.31: Semafor dopuszcza maksymalnie dwa zadania naraz (C++20).

Uwaga. W przykładzie z semaforem zmienne zostały zadeklarowane w kolejności, dzięki której wektor `workers` zostaje zniszczony przed `slots`. Ma to znaczenie dla czasu życia obiektów współdzielonych przez wątki.

14.13 Zadania, pule wątków i planowanie pracy

Wątek jest zasobem systemowym, a zadanie jest jednostką pracy. Wiele krótkich zadań nie powinno wymagać utworzenia osobnego wątku na każde wywołanie. *Thread pool* utrzymuje ograniczoną liczbę pracowników, którzy pobierają zadania ze współdzielonej kolejki.

14.13.1 Minimalna pula wątków ze zwracaniem `future`

Poniższa implementacja jest przykładem dydaktycznym w C++20. Łączy kilka poznanych mechanizmów:

- `std::counting_semaphore` / `std::jthread` do obsługi pracowników;
- `std::condition_variable` do obsługi pracowników;
- `std::packaged_task` i `std::future` do przekazywania wyników;
- `std::function` do przechowywania zadań w jednej kolejce.

Metoda `submit` zwraca `future` z wynikiem. Wyjątek zgłoszony przez zadanie zostaje ponowiony podczas `get()`.

```cpp
#include <condition_variable>
#include <cstddef>
#include <functional>
#include <future>
#include <memory>
#include <mutex>
#include <queue>
#include <stdexcept>
#include <thread>
#include <type_traits>
#include <utility>
#include <vector>

```

```cpp
class ThreadPool {
    std::mutex m_;
    std::condition_variable cv_;
    std::queue<std::function<void()>> tasks_;
    bool closing_ = false;
    // workers_ jest ostatni: jego destruktory wywolaja join
    // PRZED zniszczeniem m_, cv_ i kolejki.
    std::vector<std::jthread> workers_;

public:
    explicit ThreadPool(std::size_t count) {
        if (count == 0) throw std::invalid_argument("count=0");
        workers_.reserve(count);
        for (std::size_t i = 0; i < count; ++i) {
            workers_.emplace_back([this] {
                for (;;) {
                    std::function<void()> work;
                    {
                        std::unique_lock lock(m_);
                        cv_.wait(lock, [&] {
                            return closing_ || !tasks_.empty();
                        });
                        if (closing_ && tasks_.empty()) return;
                        work = std::move(tasks_.front());
                        tasks_.pop();
                    }
                    work(); // poza sekcja krytyczna
                }
            });
        }
    }

    ThreadPool(const ThreadPool&) = delete;
    ThreadPool& operator=(const ThreadPool&) = delete;

    template<class F>
    auto submit(F&& f)
        -> std::future<std::invoke_result_t<std::decay_t<F>&>> {
        using Result = std::invoke_result_t<std::decay_t<F>&>;
        auto task = std::make_shared<std::packaged_task<Result()>>(
            std::forward<F>(f));
        auto result = task->get_future();
        {
            std::lock_guard lock(m_);
            if (closing_)
                throw std::runtime_error("Pool is closing");
            tasks_.emplace([task] { (*task)(); });
        }
        cv_.notify_one();
        return result;
    }

    ~ThreadPool() {
        {
            std::lock_guard lock(m_);
            closing_ = true;
        }
        cv_.notify_all();
        // Destruktory jthread czekaja na pracownikow.
    }
};
```

Listing 14.32: Prosta pula: kompletna implementacja C++20.

```
#include <iostream>

int main() {
    ThreadPool pool(3);
    auto a = pool.submit([] { return 40 + 2; });
    auto b = pool.submit([] { return 6 * 7; });
    std::cout << a.get() << " " << b.get() << '\n';
}
```

Listing 14.33: Użycie puli i odbiór wyników; dołącz do programu z klasą ThreadPool.

Przykład nie implementuje wszystkich funkcji puli produkcyjnej: nie ma limitu długości kolejki, priorytetów, *work stealing*, możliwości natychmiastowego porzucenia zadania ani kooperacyjnego anulowania pojedynczego zadania. Destruktor wykonuje model *graceful shutdown*: przestaje przyjmować zadania, pozwala dokończyć prace oczekujące w kolejce i czeka na zakończenie pracowników. Wywołujący musi zadbać, żeby inne wątki nie używały obiektu puli podczas jego niszczenia. Nie należy też niszczyć puli z jej własnego pracownika.

Uwaga. Gdy pula ma jednego pracownika, zadanie oczekujące synchronicznie na `future` innego zadania umieszczonego w *tej samej* puli może wywołać zakleszczenie. Równie ważna jest kolejność niszczenia składowych wykorzystywanych przez pracowników.

14.13.2 Anulowanie zadania: prośba, nie wymuszone przerwanie

`std::stop_source` dostarcza możliwość wysłania żądania, a `std::stop_token` pozwala je obserwować. Mogą istnieć liczni odbiorcy jednego tokenu, a sygnał pozostaje ustawiony po wywołaniu `request_stop()`. Mechanizm sam nie przerywa instrukcji, blokującego wywołania systemowego ani obliczenia, które ignoruje token.

```
#include <chrono>
#include <iostream>
#include <stop_token>
#include <thread>

void work(std::stop_token token) {
    for (int i = 0; i < 100; ++i) {
        if (token.stop_requested()) {
            std::cout << "Cancelled after " << i << " steps\n";
            return;
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }
}

int main() {
    std::stop_source source;
    std::jthread worker(work, source.get_token());
    std::this_thread::sleep_for(std::chrono::milliseconds(5));
    source.request_stop();
}
```

Listing 14.34: Jedna prośba o zatrzymanie i kilka punktów kontroli.

Możliwa jest także reakcja w callbacku rejestrowanym przez `std::stop_callback`. Trzeba uważać, by callback nie blokował mutexa, który jest już trzymany przez wątek wywołujący `request_stop()`, ponieważ callback może zostać wykonany *synchronicznie* w tym wywołaniu.

14.13.3 Czekanie z możliwością zatrzymania

`std::condition_variable_any` ma w C++20 przeciążenie `wait(lock, token, pred)`. Dzięki niemu kolejka zadań może wybudzać się również po prośbie o zatrzymanie.

```
#include <condition_variable>
#include <mutex>
#include <stop_token>

std::mutex m;
std::condition_variable_any cv;
bool ready = false;

void wait_for_work(std::stop_token token) {
    std::unique_lock lock(m);
    bool completed = cv.wait(lock, token, [&] { return ready; });
    if (!completed) {
        // Stop requested zanim predykat stal sie prawdziwy.
        return;
    }
    // W tym miejscu trzymamy m, a ready jest prawdziwe.
}
```

Listing 14.35: Oczekiwanie na stan lub anulowanie w C++20.

Sam token nie zastępuje sygnalizacji `cv.notify_one()` po zmianie predykatu. Wykorzystanie tokenów nie zwalnia także z wymagania, by stan zadania i jego zasoby pozostawały ważne aż do zakończenia pracy.

14.13.4 Wiele zadań i wyjątki: kiedy użyć async, a kiedy puli?

`std::async(std::launch::async, f)` jest dobry, gdy liczba zadań jest mała, a każde zadanie względnie kosztowne. Pula sprawdza się, gdy zadań jest wiele i nie chcemy tworzyć osobnego wątku dla każdego. Z kolei `std::jthread` jest wygodny, gdy potrzebujemy długowiecznego pracownika wykonującego pętlę. W praktyce można też użyć gotowego *executora* lub biblioteki wyspecjalizowanej w planowaniu zadań.

Gdy funkcja w wątku rzuca wyjątek i nie przechwytuje go lokalnie, program może zakończyć się przez `std::terminate`. Wymianę wyjątków między wątkami organizujemy przez `future`, `promise::set_exception` lub `std::exception_ptr`.

14.14 Skalowanie i koszt współbieżności

Przyspieszenie równoległego algorytmu jest ograniczone nie tylko przez liczbę rdzeni, ale także przez odsetek części sekwencyjnej i koszty synchronizacji. Jeśli część $s \in [0, 1]$ pracy musi pozostać sekwencyjna, to idealizowane prawo Amdahla daje

$$S(p) \leq \frac{1}{s + (1 - s)/p},$$

gdzie p to liczba procesorów/rdzeni roboczych. Jest to uproszczony model pomijający np. pamięć, uruchamianie wątków i nierównomierność pracy.

W praktyce należy rozważyć: narzut utworzenia i obudzenia wątku, przeciążenie wspólnego mutexa, współdzielenie linii cache (*false sharing*), ograniczoną przepustowość pamięci i liczbę rzeczywiście fizycznych rdzeni. `std::thread::hardware_concurrency()` może być użyteczną wskazówką, ale nie gwarantuje liczby dostępnych rdzeni i może zwrócić zero.

Równoległość nie jest synonimem wydajności: drobne zadania, trudne do przewidzenia skoki oraz intensywna konkurencja o pamięć mogą działać wolniej od wariantu jednowątkowego. Pomiary należy wykonywać na reprezentatywnych danych w konfiguracji Release, a poprawność sprawdzać niezależnie od benchmarków.

14.15 Przykład numeryczny: równoległe sumowanie

Załóżmy, że mamy duży wektor wartości i chcemy policzyć sumę. Możemy podzielić zakres na dwie rozłączne części, jedną zsumować w zadaniu `std::async`, a drugą w bieżącym wątku.

```cpp
#include <future>
#include <iostream>
#include <numeric>
#include <vector>

int main() {
    std::vector<double> values(100000, 1.0);
    auto middle = values.begin() + values.size() / 2;

    auto right = std::async(std::launch::async, [&] {
        return std::accumulate(middle, values.end(), 0.0);
    });

    double left = std::accumulate(values.begin(), middle, 0.0);
    double total = left + right.get();
    std::cout << total << '\n'; // 100000
}
```

Listing 14.36: Dwa niezależne fragmenty obliczenia sumy (C++11).

Odczytywane fragmenty `values` pozostają niezmienne aż do zakończenia obliczeń, więc nie ma wyścigu danych. Trzeba jednak pamiętać, że *koszt uruchomienia zadania* może przewyższać korzyść, jeżeli obliczenia są zbyt krótkie. Przy sumach zmiennoprzecinkowych inna kolejność dodawania może ponadto zmienić wynik z powodu zaokrągleń.

14.16 OpenMP: programowanie równoległe przez dyrektywy

OpenMP (*Open Multi-Processing*) to standard API do programowania równoległego na pamięci współdzielonej, stosowany w C, C++ i Fortranie. Nie jest pojedynczą klasą ani tradycyjną biblioteką szablonową C++: łączy *dyrektywy kompilatora*, *bibliotekę funkcji wykonawczych* (`omp.h`) i *zmienne środowiskowe*. Posiada również konstrukcje do pracy na akceleratorach, lecz tutaj skupimy się na wielordzeniowym CPU.

Zaletą OpenMP jest możliwość stopniowego równoleglenia istniejących pętli. Kod źródłowy pozostaje zazwyczaj podobny do wersji sekwencyjnej. Ceną jest konieczność rozumienia prywatności zmiennych, zależności pomiędzy iteracjami i synchronizacji.

Uwaga. Specyfikacja OpenMP ma własne wersje (m.in. 5.2 i 6.0), niezależne od C++20. W przykładach poniżej używamy głównie klasycznych dyrektyw dostępnych w wielu kompilatorach. Samo istnienie mechanizmu w specyfikacji nie gwarantuje, że konkretna wersja GCC, Clang, MSVC lub Intel oneAPI obsługuje go w pełni.

14.16.1 Kompilacja i konfiguracja OpenMP

Dla GCC i typowych konfiguracji Clang można użyć flagi `-fopenmp`. W Clangu konieczne może być dodatkowo zainstalowanie odpowiedniego środowiska wykonawczego OpenMP (np. `libomp`). W MSVC wsparcie i zestaw flag zależą od wersji oraz wybranego trybu OpenMP.

```
g++ -std=c++20 -O2 -Wall -Wextra -fopenmp example.cpp -o example
./example

# Przykladowe uruchomienie z 4 watkami:
OMP_NUM_THREADS=4 ./example
```

Listing 14.37: Kompilacja programu OpenMP w GCC.

W CMake można uniknąć ręcznego wpisywania flag:

```
cmake_minimum_required(VERSION 3.16)
project(OpenMPDemo LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)
find_package(OpenMP REQUIRED COMPONENTS CXX)
add_executable(example example.cpp)
target_link_libraries(example PRIVATE OpenMP::OpenMP_CXX)
```

Listing 14.38: Minimalny CMakeLists.txt (tekst pliku).

Biblioteka wykonawcza udostępnia funkcje do odczytu stanu środowiska:

- `omp_get_thread_num()` — identyfikator wątku;
- `omp_get_num_threads()` — liczba uczestników aktualnego zespołu;
- `omp_get_max_threads()` — górna granica następnego zespołu;
- `omp_set_num_threads()` — ustawianie żądanej liczby wątków;
- `omp_get_wtime()` — czas ścienny w sekundach.

Poza regionem równoległym `omp_get_num_threads()` zwraca 1. Wartość `omp_get_max_threads()` nie jest liczbą fizycznych rdzeni.

14.16.2 Pierwszy region: `parallel`

`#pragma omp parallel` tworzy zespół wątków, którego uczestnicy wykonują blok. Wątek napotykający dyrektywę staje się wątkiem głównym tego zespołu (numer 0). Zakończenie bloku obejmuje domyślną synchronizację uczestników.

```
#include <iostream>
#include <omp.h>

int main() {
    #pragma omp parallel num_threads(4)
    {
        int id = omp_get_thread_num();
        int count = omp_get_num_threads();
        #pragma omp critical
        {
            std::cout << "Thread " << id
                      << " of " << count << '\n';
        }
    }
    std::cout << "Region finished\n";
}
```

Listing 14.39: Region równoległy OpenMP.

`num_threads(4)` jest żądaniem liczby wątków, a faktyczna liczba może zależeć od ograniczeń środowiska wykonawczego. `critical` chroni tu przed przeplataniem fragmentów wypisywanego tekstu, nie narzuca kolejności identyfikatorów wątków.

14.16.3 Równoległa pętla `parallel for`

Najczęstsze zastosowanie OpenMP polega na podziale *niezależnych* iteracji jednej pętli między uczestników zespołu.

```
#include <cstddef>
#include <vector>

int main() {
    constexpr long long n = 100000;
    std::vector<double> squares(static_cast<std::size_t>(n));

    #pragma omp parallel for default(none) shared(squares, n) \
        schedule(static)
    for (long long i = 0; i < n; ++i) {
        squares[static_cast<std::size_t>(i)] =
            static_cast<double>(i) * i;
    }
}
```

Listing 14.40: Wypełnianie wektora: niezależne iteracje.

`default(none)` wymaga jawnego określenia udziału zmiennych, które nie mają z góry ustalonych atrybutów współdzielenia. Jest wartościowym narzędziem kontroli poprawności. Zmienna sterująca pętlą `i` jest prywatna na mocy reguł dyrektywy `for`. Każdy wątek zapisuje inny element wektora; nie ma konfliktu, zakładając że nikt nie zmienia równolegle rozmiaru ani struktury wektora.

14.16.4 `shared`, `private`, `firstprivate`, `lastprivate`

Wątki odnoszą się do tego samego obiektu. Dostępy konfliktowe wymagają ochrony lub podziału danych.

Każdy uczestnik ma własny egzemplarz (z zachowaniem reguł inicjalizacji odpowiednich typów); nie należy zakładać skopiowania początkowej wartości zmiennej pierwotnej.

Nowy egzemplarz zostaje zainicjalizowany wartością odpowiadającej mu zmiennej sprzed regionu.

Po regionie wynik z logicznie ostatniej iteracji (niekoniecznie wykonanej na końcu w czasie rzeczywistym) zostaje przypisany do zmiennej pierwotnej.

```
#include <iostream>

int main() {
    int seed = 10;
    int last = -1;

    #pragma omp parallel for firstprivate(seed) lastprivate(last)
    for (int i = 0; i < 8; ++i) {
        int local = seed + i;
        last = local;
```

```
    }
    std::cout << last << '\n'; // 17
}
```

Listing 14.41: Przykład firstprivate i lastprivate w OpenMP.

Zmienna `seed` jest kopiowana do prywatnych egzemplarzy. Wartość `last` po zakończeniu odpowiada iteracji $i = 7$, nie zaś temu wątkowi, który jako ostatni fizycznie zakończył pracę.

14.16.5 Wyścig w redukcji i klauzula `reduction`

Poniższa konstrukcja jest **błędna**:

```
// Fragment celowo bledny:
// double sum = 0;
// #pragma omp parallel for
// for (int i = 0; i < n; ++i) sum += values[i];
```

Listing 14.42: NIEPOPRAWNIE: konfliktowe modyfikacje sumy.

Każdy wątek modyfikuje wspólny `sum`. Możemy chronić dodawanie blokadą, lecz w klasycznym sumowaniu wygodniejsze są prywatne sumy, łączone pod koniec obliczeń:

```
#include <iostream>
#include <vector>

int main() {
    std::vector<double> values(1000000, 1.0);
    const long long n = static_cast<long long>(values.size());
    double sum = 0.0;

    #pragma omp parallel for reduction(+:sum) schedule(static)
    for (long long i = 0; i < n; ++i) {
        sum += values[static_cast<std::size_t>(i)];
    }
    std::cout << sum << '\n'; // 1000000
}
```

Listing 14.43: Redukcja równoległa w OpenMP.

OpenMP tworzy prywatne wartości redukcji i łączy je operatorem `+`. Dostępne są również inne redukcje, np. `*`, `min` i `max` (zależnie od typu oraz konstrukcji). Ważne: dla liczb zmiennoprzecinkowych kolejność działań może być inna przy różnych zespołach i harmonogramach. Wyniki mogą zatem różnić się na ostatnich cyfrach, bo

$$(a + b) + c \neq a + (b + c)$$

może zachodzić w arytmetyce IEEE 754. Dla algorytmów numerycznych mających wymagania reprodukowalności nie wystarczy zakładać identycznych bitowo wyników każdej redukcji.

14.16.6 Harmonogramowanie iteracji: `schedule`

`shared(x)` `private(x)` `firstprivate(x)` `lastprivate(x)`

Tryb	Znaczenie
`static`	Przydział iteracji ustalany na początku; mały narzut, dobry dla równych kosztów.
`static,k`	Przydział kolejnych porcji po k iteracji w sposób cykliczny.
`dynamic,k`	Porcje po k iteracji przydzielane pracownikom w trakcie pracy; dobre dla nierównych obciążeń, większy narzut.
`guided,k`	Rozmiary przydzielanych porcji zasadniczo maleją, do narzuconego minimum; kompromis między narzutem a równoważeniem.
`runtime`	Ustawienia pobierane z środowiska, m.in. `OMP_SCHEDULE`.

```
#pragma omp parallel for schedule(dynamic, 8)
for (int i = 0; i < n; ++i) {
    perform_variable_cost_work(i);
}
```

Listing 14.44: Równoważenie kosztów obliczeń w pętli.

Przy równomiernych obliczeniach `static` bywa tańszy od `dynamic`. Dla zadań nieregularnych dynamiczny harmonogram zmniejsza bezczynność pracowników, ale może pogorszyć lokalność pamięci. Doboru dokonujemy na podstawie pomiarów, nie wyłącznie intuicji.

14.16.7 Synchronizacja: `critical`, `atomic`, `barrier`, `nowait`

Chroni region wykonywany jednocześnie najwyżej przez jeden wątek w danej grupie nazwanych regionów krytycznych.

Zapewnia atomową obsługę wspieranego wyrażenia (np. inkrementacji); nie obejmuje dowolnego bloku instrukcji.

Wymaga, żeby uczestnicy zespołu dotarli do punktu synchronizacji, zanim którykolwiek przejdzie dalej.

Usuwa domyślną barierę tam, gdzie klauzula jest dopuszczalna; wymaga osobnego rozumowania o zależnościach danych.

```
int counter = 0;

#pragma omp parallel for
for (int i = 0; i < 10000; ++i) {
    #pragma omp atomic update
    ++counter;
}
// counter == 10000
```

Listing 14.45: Dwie alternatywy zwiększania licznika.

Dla tej operacji `reduction(+:counter)` zwykle ogranicza rywalizację lepiej niż atomowa inkrementacja przy każdej iteracji. Z kolei `critical` jest przydatny, gdy atomowo musi wykonać się cały fragment kodu, a nie tylko jedno proste wyrażenie.

```
#pragma omp parallel
{
    prepare_local_data();
    #pragma omp barrier
    consume_results_after_everyone_prepared();
}
```

Listing 14.46: Bariera wewnątrz istniejącego zespołu.

Nie można wstawiać bariery w taki sposób, aby część wątków zespołu ją pomijała, a pozostałe na niej czekały. Podobnie `nowait` nie może być dodawane mechanicznie, jeśli następna pętla potrzebuje wyników pierwszej.

14.16.8 `sections` i `single`

Gdy mamy kilka niezależnych, różnych funkcji, wygodne bywa `sections`; gdy chcemy pojedynczego producenta zadań, wybieramy `single`:

```
#pragma omp parallel sections
{
    #pragma omp section
    load_input();

    #pragma omp section
    preprocess();
}

#pragma omp parallel
{
    #pragma omp single
    initialize_once();
    // Wszystkie watki mijaja koniec single po barierze.
}
```

Listing 14.47: Kilka różnych zadań i jedna sekcja inicjalizacyjna.

Powyższy przykład `sections` ma sens tylko wtedy, gdy `load_input()` i `preprocess()` są rzeczywiście niezależne. W przeciwnym razie potrzebna jest synchronizacja albo zadania z opisanymi zależnościami.

14.16.9 Zadania OpenMP: `task`, `taskwait`, `taskgroup`

Podział pętli `for` pasuje do regularnych obliczeń. Dla pracy nieregularnej, np. rekursywnego przetwarzania drzewa, używa się `task`. Dyrektywa tworzy jednostkę pracy, którą środowisko wykonawcze może przydzielić pracownikowi zespołu. Nie jest to równoznaczne z utworzeniem nowego wątku systemowego.

```
#include <iostream>

int main() {
    int results[4]{};
    #pragma omp parallel
    {
        #pragma omp single
        {
            for (int i = 0; i < 4; ++i) {
                #pragma omp task firstprivate(i) shared(results)
                {
                    results[i] = i * i;
                }
            }
            #pragma omp taskwait
        }
    }
    for (int x : results) std::cout << x << ' ';
}
```

Listing 14.48: Tworzenie i oczekiwanie na zadania OpenMP.

`taskwait` oczekuje na odpowiednie bezpośrednie zadania potomne bieżącego zadania. Gdy chcemy objąć również ich potomków, praktyczną konstrukcją jest `taskgroup`. Koniec regionu `parallel` również zapewnia zakończenie prac należących do tej struktury zadaniowej.

14.16.10 Zależności pomiędzy zadaniami: `depend`

```
#include <cassert>

int main() {
    int a = 0, b = 0;
    #pragma omp parallel
    {
        #pragma omp single
        {
            #pragma omp task shared(a) depend(out:a)
            {
                a = 21;
            }
            #pragma omp task shared(a,b) depend(in:a)
            {
                b = a * 2;
            }
            #pragma omp taskwait
        }
    }
    assert(b == 42);
}
```

Listing 14.49: Zadanie drugie korzysta z wyniku pierwszego.

Klauzule `depend(in:...)`, `depend(out:...)` i `depend(inout:...)` opisują zależności danych m.in. pomiędzy zadaniami rodzeństwa. Nie zastępują synchronizacji przy dowolnych dostępach do innych, nieopisanych zasobów.

14.16.11 `collapse` i `simd`: pętle wielowymiarowe i wektoryzacja

`collapse(2)` łączy przestrzeń iteracji dwóch odpowiednio zagnieżdżonych pętli, zwiększając liczbę możliwych porcji pracy. `simd` natomiast przekazuje kompilatorowi informację, że iteracje dają się wykonać w modelu SIMD. Wektoryzacja i równoległość wielowątkowa są różnymi poziomami przyspieszania.

```
#include <vector>

int main() {
    const int rows = 400, cols = 400;
    std::vector<double> a(rows * cols);
    #pragma omp parallel for collapse(2) schedule(static)
    for (int i = 0; i < rows; ++i)
        for (int j = 0; j < cols; ++j)
            a[i * cols + j] = i + 0.5 * j;
}
```

Listing 14.50: Tablica 2D: parallel for z collapse(2).

```
void scale(double* dest, const double* src, int n, double alpha) {
    #pragma omp simd
    for (int i = 0; i < n; ++i)
        dest[i] = alpha * src[i];
}
```

Listing 14.51: Wektoryzacja bez obowiązkowego tworzenia wielu wątków.

Prawidłowość `simd` wymaga faktycznej niezależności iteracji, w tym braku niedozwolonego nakładania się zapisów i odczytów. Nie należy wprowadzać dyrektywy w kodzie o nieustalonym aliasingu bez wcześniejszej analizy zależności.

14.16.12 Zmienne środowiskowe OpenMP i diagnostyka

`critical` `atomic` `barrier` `nowait`

Zmienna	Rola
`OMP_NUM_THREADS`	Żądana liczba wątków w zespołach.
`OMP_DYNAMIC`	Pozwala środowisku dynamicznie dostosowywać liczbę wątków.
`OMP_SCHEDULE`	Harmonogram stosowany przy `schedule(runtime)`.
`OMP_PROC_BIND`	Strategia wiązania wątków z zasobami obliczeniowymi.
`OMP_PLACES`	Definiuje miejsca wykonania dla mechanizmu wiązania.
`OMP_MAX_ACTIVE_LEVELS`	Ogranicza liczbę aktywnych poziomów zagnieżdżonych regionów.

Przykładowa sesja uruchomieniowa:

```
OMP_NUM_THREADS=8 OMP_SCHEDULE=dynamic,32 ./example
OMP_NUM_THREADS=4 OMP_PROC_BIND=close ./example
```

Listing 14.52: Sterowanie środowiskiem bez zmiany źródeł.

Rzeczywista dostępność i interpretacja opcji dotyczących wiązania wątków zależą od środowiska wykonawczego i maszyny. Zagnieżdżone regiony OpenMP oraz łączenie OpenMP z `std::thread` może spowodować *oversubscription*: więcej aktywnych wątków roboczych niż zasobów obliczeniowych.

14.16.13 OpenMP a model pamięci C++

OpenMP określa własny model pamięci, operację *flush* oraz semantykę synchronizacji dyrektyw, takich jak `barrier`, `critical` i `atomic`. *Flush* nie jest funkcją `std::atomic_thread_fence`, a samo słowo „flush” nie znaczy „wyczyść wszystkie cache procesora”.

W praktyce warto trzymać się jednej, konsekwentnej reguły:

- dane dzielone między iteracje OpenMP synchronizujemy przede wszystkim przez odpowiednie konstrukcje OpenMP;
- dane wymieniane pomiędzy własnymi wątkami C++ synchronizujemy przez `std::mutex`, `std::atomic` lub inne narzędzia C++;
- przy *mieszaniu* obu modeli wymagane jest szczególnie staranne rozumowanie o zgodności mechanizmów, zamiast zakładania, że przypadkowe `flush` naprawi wyścig danych.

Wyjątków C++ nie należy wypuszczać poza regiony OpenMP w sposób naruszający zasady specyfikacji. Jeżeli zadanie może zgłosić wyjątek, trzeba obsłużyć go wewnątrz odpowiedniego regionu i przekazać informację o błędzie w kontrolowany sposób po zakończeniu pracy.

14.16.14 OpenMP czy `std::thread`, `async`, a może Ranges?

Mechanizm	Typowy przypadek użycia
`std::thread/jthread`	Długowieczny wątek, własny protokół komunikacji, kontrola czasu życia.
`std::async`	Niewiele niezależnych obliczeń z wynikiem w `future`.
Pula wątków	Wiele krótkich lub średnich zadań, ograniczona liczba pracowników.
OpenMP `parallel for`	Obliczenia naukowe i numeryczne z niezależnymi iteracjami.
OpenMP `task`	Dynamiczne drzewo prac i zależności.
`std::execution::par`	Algorytmy standardowe C++17+ z odpowiednią polityką wykonania i dostępnym backendem.

Polityka `std::execution::par` nie stanowi gwarancji faktycznego wielowątkowego wykonania na każdym kompilatorze i bibliotece standardowej. *Ranges views* z C++20 są zazwyczaj przede wszystkim leniwymi adaptatorami zakresów, nie automatycznym mechanizmem równoległego planowania.

14.16.15 Studium numeryczne: porównanie trzech sposobów sumowania

Rozważmy trzy warianty: zwykły `std::accumulate`, równoległe sumowanie przez `std::async` oraz OpenMP `reduction`. W przykładzie[1] obliczamy sumę kwadratów wejściowych liczb przez OpenMP, a następnie porównujemy z referencyjną pętlą:

```cpp
#include <cmath>
#include <iostream>
#include <omp.h>
#include <vector>

int main() {
    constexpr int n = 1000000;
    std::vector<double> a(n);
    for (int i = 0; i < n; ++i) a[i] = i * 1e-5;

    double sequential = 0.0;
    double t0 = omp_get_wtime();
    for (double x : a) sequential += x * x;
    double t1 = omp_get_wtime();

    double parallel = 0.0;
    #pragma omp parallel for reduction(+:parallel) schedule(static)
    for (int i = 0; i < n; ++i)
        parallel += a[i] * a[i];
    double t2 = omp_get_wtime();

    std::cout << "sequential: " << sequential
              << ", time: " << t1 - t0 << " s\n";
    std::cout << "OpenMP: " << parallel
              << ", time: " << t2 - t1 << " s\n";
    std::cout << "relative error: "
              << std::abs(sequential - parallel) /
                 (1.0 + std::abs(sequential)) << '\n';
}
```

Listing 14.53: Praktyczny przykład: suma kwadratów OpenMP z pomiarem.

Uwaga. Ten program jest demonstracją interfejsu, a nie metodologicznie pełnym benchmarkiem. Należy wykonać rozgrzewkę, wiele powtórzeń, uwzględnić rozrzut wyników i sprawdzić, czy kompi-

[1] Przy dzieleniu dużego wektora na wiele części należy uwzględnić balans obciążenia, rozmiary porcji i koszt agregacji wyników.

lator nie usunął obliczeń. Koszt dostępu do pamięci może ograniczać przyspieszenie; przy małym `n` równoległa wersja może być wolniejsza.

14.17 Porównanie i zasady projektowania

Po zapoznaniu się z modelem pamięci i OpenMP potrafimy rozróżnić trzy poziomy pracy: *własność wątku* (`thread`/`jthread`), *własność zadania i jego wyniku* (`future`, pule) oraz *deklaratywny podział iteracji/zadań* (OpenMP). Wybór nie powinien zależeć jedynie od liczby linijek kodu.

Narzędzie	Kiedy używać
`thread`	Potrzebna jest bezpośrednia kontrola nad wątkiem i jego czasem życia.
`jthread`	W C++20, gdy przydaje się RAII i kooperacyjny sygnał zatrzymania.
`mutex`	Kilka operacji musi spójnie chronić współdzielony stan.
`condition_variable`	Wątek powinien zasnąć do spełnienia warunku na stanie.
`atomic`	Niezależne, atomowe operacje na zmiennej lub starannie zaprojektowana synchronizacja.
`promise`	Producent sam decyduje, kiedy dostarczyć wynik albo wyjątek.
`future`	Odbiór jednego wyniku i ewentualne oczekiwanie.
`shared_future`	Kilku odbiorców potrzebuje tej samej wartości.
`async`	Chcemy uruchomić funkcję i odebrać jej wynik bez zarządzania wątkiem.
`packaged_task`	Trzeba oddzielić zadanie od mechanizmu jego uruchamiania.
Pula wątków	Dużo zadań i ograniczona liczba pracowników; kontrolowane zamykanie.
OpenMP	Duże obliczenia z niezależnymi iteracjami albo grafem zadań.

14.17.1 Najczęstsze błędy

1. Użycie zwykłej zmiennej współdzielonej bez synchronizacji (*data race*, niezdefiniowane zachowanie).
2. Założenie, że `++x` na zwykłym `int` jest atomowe.
3. Zniszczenie aktywnego `std::thread` bez `join()` lub `detach()`.
4. Przekazanie referencji do obiektu, którego czas życia kończy się przed zakończeniem pracy wątku.
5. Oczekiwanie przez `condition_variable` bez predykatu.
6. Mylenie powiadomienia z trwałym zapisem informacji o stanie.
7. Założenie, że domyślne `std::async(f)` zawsze uruchamia nowy wątek.
8. Natychmiastowe niszczenie tymczasowego `future` z `std::async`.
9. Traktowanie `wait_for()` jako anulowania zadania.
10. Próba wielokrotnego `get()` na zwykłym `future`.
11. Nadmierne używanie `memory_order_relaxed` bez dowodu poprawności.
12. Zakładanie, że dla każdego drobnego zadania opłaca się uruchomić osobny wątek.
13. Przenoszenie `memory_order_relaxed` do algorytmu publikującego dane bez analizy zależności między wątkami.
14. Stosowanie `#pragma omp parallel for` do pętli z zależnościami między iteracjami albo wspólnym niechronionym akumulatorem.
15. Mieszanie własnej puli wątków z OpenMP bez kontroli liczby aktywnych pracowników; pojawia się ryzyko *oversubscription*.

14.17.2 Standard języka a dostępność API

Standard	Wybrane mechanizmy
C++11	`thread`, `mutex`, `condition_variable`, `atomic`, `promise`, `future`, `async`, `packaged_task`, `call_once`, `thread_local`.
C++14	M.in. ułatwienia związane z lambdami, np. inicjalizujące przechwycenie użyte w przykładzie `promise`.
C++17	`scoped_lock`, `shared_mutex`, `shared_lock` (ta ostatnia od C++14).
C++20	`jthread`, `stop_token`, `atomic::wait`, `semaphore`, `latch`, `barrier`, `osyncstream`.

Przykład z inicjalizującym przechwyceniem:

```
[p = std::move(producer)]
```

wymaga C++14, mimo że sam `std::promise` istnieje od C++11.

14.18 Kompilacja, sprawdzanie i diagnostyka

Każdy kompletny przykład można zapisać w osobnym pliku z rozszerzeniem `.cpp`. Dla kodu C++20 typowe wywołanie kompilatora GCC lub Clang to:

```
g++ -std=c++20 -O2 -Wall -Wextra -pthread example.cpp -o example
./example
```

Listing 14.54: Kompilacja przykładu i uruchomienie programu.

Podczas nauki szczególnie pomocny jest *ThreadSanitizer*, który może wykrywać wiele wyścigów danych (dostępność zależy od platformy oraz kompilatora):

```
g++ -std=c++20 -O1 -g -pthread -fsanitize=thread example.cpp -o example_tsan
./example_tsan
```

Listing 14.55: Przykładowe uruchomienie z ThreadSanitizer.

Nawet jeśli test przejdzie tysiąc razy, nie dowodzi to braku błędów współbieżności. Poprawność wymaga analizy synchronizacji i czasu życia; narzędzia diagnostyczne stanowią jej uzupełnienie.

Podsumowanie rozdziału

Z `std::thread` i `std::jthread` sterujemy *wątkami*, a z `std::async` i `std::packaged_task` opisujemy raczej *zadania*. `future`/`promise` tworzą interfejs dostarczania i odbioru jednego wyniku. `mutex` i `condition_variable` pozwalają chronić współdzielony stan i czekać na jego zmianę. `atomic` udostępnia operacje atomowe oraz mechanizmy porządkowania pamięci. Algorytmy *lock-free* wymagają ponadto zarządzania czasem życia węzłów i analizy problemu ABA. Pula wątków separuje zadania od fizycznych pracowników, a OpenMP udostępnia deklaratywny podział pracy na iteracje, redukcje i zadania. Żadne z tych narzędzi nie zastępuje pozostałych we wszystkich zastosowaniach.

Dobra architektura zaczyna się od pytania, *które dane są współdzielone, kto jest właścicielem zadania, kiedy wolno zakończyć jego pracę oraz jak przekazać wynik i wyjątek*. Dopiero potem należy wybrać konkretne narzędzia biblioteki standardowej.

Literatura i dokumentacja

- https://cppreference.com/cpp/thread/async — `std::async` i polityki uruchamiania.
- https://en.cppreference.com/w/cpp/thread/future — `std::future`.
- https://cppreference.com/cpp/thread/condition_variable/wait — oczekiwanie z predykatem.
- https://en.cppreference.com/w/cpp/atomic/atomic/wait — `atomic::wait` w C++20.
- https://en.cppreference.com/w/cpp/atomic/memory_order — porządek pamięci.
- https://en.cppreference.com/w/cpp/atomic/atomic/compare_exchange — CAS.
- https://www.openmp.org/specifications/ — oficjalne wersje i dokumentacja OpenMP (w tym 5.2 i 6.0).
- https://www.openmp.org/spec-html/5.2/openmp.html — treść specyfikacji OpenMP 5.2.
- https://www.openmp.org/spec-html/5.2/openmpse4.html — model pamięci OpenMP.
- Anthony Williams, *C++ Concurrency in Action*, wyd. 2, Manning, 2019.

Rozdział 15

Korutyny, zadania asynchroniczne i `std::async`

15.1 Po co C++ potrzebuje korutyn?

Zwykła funkcja ma zasadniczo jeden punkt wejścia. Może wywoływać inne funkcje, ale po powrocie oddaje sterowanie wywołującemu i nie posiada mechanizmu późniejszego wznowienia dokładnie w miejscu, w którym zakończyła wykonywanie. *Korutyna* (ang. *coroutine*) jest funkcją, której wykonanie można *zawiesić*, a następnie *wznowić*, zachowując wybrane zmienne lokalne i miejsce kontynuacji.

Korutyny C++20 wspierają między innymi:

- *generatory* — wytwarzanie kolejnych wartości na żądanie, bez budowania całej kolekcji;
- *zadania asynchroniczne* — opis operacji, których kolejne etapy uruchamiane są po zakończeniu wcześniejszych;
- *kooperacyjne planowanie* — oddawanie sterowania harmonogramowi zadań;
- *maszyny stanów* — zachowywanie stanu pomiędzy wznowieniami bez ręcznego kodowania każdej możliwej kontynuacji.

Najważniejsza zasada brzmi:

Korutyna nie jest wątkiem, a `co_await` *nie oznacza automatycznie „uruchom w nowym wątku”.*

Korutyna określa *jak zatrzymać i kontynuować obliczenie*; `std::async`, `std::thread` i pula wątków dotyczą również *gdzie i kiedy* wykonywać pracę. W szczególności kod korutyny może działać od początku do końca w jednym wątku, nawet jeśli wielokrotnie się zawiesza. Odwrotnie, korutyna może być wznawiana przez różne wątki, jeżeli tak zdecyduje dostarczony mechanizm planowania.

15.1.1 Trzy słowa kluczowe

C++20 wprowadził następujące konstrukcje składniowe:

`co_await expr` Zastosuj protokół oczekiwania do wyrażenia; zależnie od niego możesz zawiesić korutynę i później ją wznowić.

`co_yield expr` Udostępnij kolejną wartość generatora; składniowo jest to szczególna postać oczekiwania, wykorzystująca metodę `yield_value` obiektu `promise_type`.

`co_return expr` Zakończ korutynę i przekaż wynik przez odpowiednie `return_value` albo `return_void`.

Funkcja zawierająca jeden z tych operatorów (w odpowiednim kontekście) rozpoznawana jest jako korutyna. Typ jej zwracanej wartości musi udostępniać właściwy *typ obietnicy* `promise_type` (lub odpowiednią specjalizację `std::coroutine_traits`). To wymóg mechanizmu korutyn, a nie znanego już `std::promise`.

```
#include <future>

// Standardowe std::future<int> nie zapewnia promise_type:
// std::future<int> calculate() { co_return 42; } // BLAD!
```

Listing 15.1: Niepoprawna intuicja: std::future nie jest automatycznie typem korutyny (C++20); fragment celowo błędny.

W C++20 biblioteka standardowa dostarcza *mechanizm* korutyn (nagłówek `<coroutine>`), ale nie dostarcza gotowego, ogólnego `Task<T>` ani standardowego executora. Takie typy definiuje biblioteka lub programista. `std::generator` jest rozwiązaniem późniejszym, wprowadzonym dopiero w C++23, a jego wsparcie zależy od użytej biblioteki standardowej.

15.2 Różnica między zawieszeniem a zablokowaniem wątku

Rozważmy dwa sposoby wstrzymania działania programu:

- `std::this_thread::sleep_for(100ms)` *blokuje bieżący wątek* na pewien czas; sam wątek pozostaje zajętym zasobem systemowym, choć może nie zużywać wówczas czasu CPU;
- dobrze zaprojektowane `co_await` *zawiesza korutynę*, oddając wątek do wykonywania innych zadań; wznowienie może nastąpić po nadejściu wyniku operacji I/O, komunikatu lub upływie czasu.

Nie każde `co_await` daje zawieszenie: awaiter może zgłosić, że wynik jest już gotowy. Również samo użycie `co_await` nie gwarantuje nieblokującego I/O — zależy to od jego implementacji.

Właściwość	**Blokada wątku**	**Zawieszenie korutyny**
Zachowanie stanu	Na stosie wywołań wątku	Zwykle w ramce korutyny
Wątek potrzebny podczas oczekiwania	Tak, istnieje i czeka	Nie jest konieczny *dla samego oczekiwania*
Wznowienie	Planista systemu/odblokowanie	Mechanizm awaitera i właściciel kontynuacji
Koszt	M.in. wątek i jego stos	M.in. ramka i obsługa zawieszenia

15.3 Ramka korutyny i jej cykl życia

Kompilator przekształca korutynę w obiekt reprezentujący jej stan. Ten stan nazywamy *ramką korutyny* (ang. *coroutine frame*). Może ona obejmować obiekt `promise_type`, parametry i stan potrzebny do kontynuowania obliczenia po zawieszeniu. To, które zmienne lokalne trzeba zachować, wynika z przebiegu programu.

W ogólnym modelu wywołanie korutyny przebiega w następujących etapach:

1. Alokowana jest ramka, chyba że implementacja legalnie wyeliminuje tę alokację (np. przez zagnieżdżenie ramki w innym obiekcie).
2. Kopiowane lub przenoszone są parametry odpowiednio do ich rodzajów; *parametry referencyjne pozostają referencjami.*

3. Konstruowany jest obiekt obietnicy `promise_type`.
4. Wywoływana jest metoda `get_return_object()`.
5. Wykonywany jest `co_await promise.initial_suspend()`.
6. Uruchamiane jest ciało korutyny (od razu lub po wznowieniu).
7. `co_return` uruchamia `return_value` lub `return_void`; nieobsłużony wyjątek trafia do `unhandled_exception()`.
8. Niszczone są obiekty lokalne wymagające zniszczenia; wykonywany jest `co_await promise.final_suspend(`
9. Ramka zostaje ostatecznie zniszczona we właściwym momencie.

Kolejność i czas *zwolnienia ramki* zależą od przyjętej polityki własności. Dla `final_suspend = suspend_always` ramkę zwykle niszczy właściciel uchwytu, nawet po zakończeniu ciała korutyny. Przy `final_suspend = suspend_never` mechanizm korutyny może zakończyć się bez pozostawiania ramki zawieszonej do późniejszego zniszczenia przez właściciela. Tych strategii nie wolno mieszać.

15.3.1 Początek i koniec: `initial_suspend` i `final_suspend`

Dwa najczęściej używane typy z nagłówka `<coroutine>` to:

`std::suspend_never` Oczekiwanie jest natychmiast gotowe i nie powoduje zawieszenia.

`std::suspend_always` Oczekiwanie zawsze prowadzi do zawieszenia, dopóki ktoś nie wznowi korutyny.

Jeżeli `initial_suspend()` zwraca `suspend_always`, korutyna jest *lazy*: nie rozpoczyna wykonywania ciała przy samym wywołaniu funkcji. Natomiast `suspend_never` oznacza *eager start*: ciało może rozpocząć się natychmiast w wątku wywołującym i pracować aż do pierwszego rzeczywistego zawieszenia.

Ważne: nie wolno wywoływać `resume()` na korutynie, która jest zawieszona w swoim `final_suspend`. Jest to niezdefiniowane zachowanie. Uchwyt do takiej ramki może nadal służyć do `destroy()`, jeżeli ramka nie została już zniszczona.

15.4 Uchwyt `std::coroutine_handle` i własność ramki

`std::coroutine_handle<Promise>` jest lekkim uchwytem umożliwiającym m.in. wznowienie korutyny i dostęp do jej obietnicy. Sam `coroutine_handle` *nie zarządza automatycznie czasem życia ramki*. Skopiowanie uchwytu nie jest kopią korutyny, a zniszczenie uchwytu nie wywołuje automatycznie `destroy()`. Dlatego wygodna klasa będąca właścicielem ramki powinna stosować RAII, uniemożliwiać kopiowanie oraz poprawnie obsługiwać przenoszenie.

```
#include <coroutine>

using Handle = std::coroutine_handle<>;

// Zakladamy, ze h wskazuje na istniejaca, zawieszona korutyne.
// if (h && !h.done()) h.resume();
// h.destroy(); // dokladnie raz, przez wlasciciela ramki
// Po destroy() wszystkie kopie h sa wiszacymi uchwytami.
```

Listing 15.2: Operacje na uchwycie; fragment ilustracyjny.

Uchwyt może zostać przekazany do innego wątku lub executora, ale wówczas trzeba zagwarantować, że ramka nie zostanie zniszczona, zanim planista faktycznie wznowi zawieszoną korutynę. Jest to jedna z głównych trudności implementacyjnych asynchronicznego API C++20.

15.5 Własny generator: `co_yield` od podstaw

Zbudujemy teraz typ `Generator`, który produkuje liczby całkowite kolejno, *na żądanie*. Poniższy przykład jest samowystarczalny. Prywatny uchwyt do ramki stanowi jedynego właściciela zasobu; `next()` wznawia korutynę tylko wtedy, gdy nie zakończyła wykonywania. Własną obsługą wyjątku uzupełniamy przykład o poprawną propagację błędów.

```
#include <cassert>
#include <coroutine>
#include <exception>
#include <stdexcept>
#include <utility>

class Generator {
public:
    struct promise_type {
        int current = 0;
        std::exception_ptr error;

        Generator get_return_object() {
            return Generator{Handle::from_promise(*this)};
        }
        std::suspend_always initial_suspend() noexcept { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        std::suspend_always yield_value(int x) noexcept {
            current = x;
            return {};
        }
        void return_void() noexcept {}
        void unhandled_exception() noexcept {
            error = std::current_exception();
        }
    };

    using Handle = std::coroutine_handle<promise_type>;

    explicit Generator(Handle h) noexcept : handle_(h) {}
    Generator(const Generator&) = delete;
    Generator& operator=(const Generator&) = delete;

    Generator(Generator&& other) noexcept
        : handle_(std::exchange(other.handle_, {})),
          ready_(std::exchange(other.ready_, false)) {}

    Generator& operator=(Generator&& other) noexcept {
        if (this != &other) {
            if (handle_) handle_.destroy();
            handle_ = std::exchange(other.handle_, {});
            ready_ = std::exchange(other.ready_, false);
        }
        return *this;
    }

    ~Generator() {
        if (handle_) handle_.destroy();
    }

```

```
    bool next() {
        ready_ = false;
        if (!handle_ || handle_.done()) return false;
        handle_.resume();
        if (handle_.promise().error)
            std::rethrow_exception(handle_.promise().error);
        ready_ = !handle_.done();
        return ready_;
    }

    int value() const {
        if (!ready_)
            throw std::logic_error("No current value");
        return handle_.promise().current;
    }

private:
    Handle handle_{};
    bool ready_ = false;
};

Generator squares(int n) {
    for (int i = 1; i <= n; ++i)
        co_yield i * i;
}

int main() {
    auto gen = squares(4); // Cialo jeszcze sie nie wykonalo.
    int sum = 0;
    while (gen.next()) sum += gen.value();
    assert(sum == 1 + 4 + 9 + 16);
    assert(!gen.next());
}
```

Listing 15.3: Generator liczb całkowitych: co_yield i RAII (C++20); kompletny program.

15.5.1 Co dzieje się podczas `co_yield`?

Instrukcja `co_yield i * i` powoduje wywołanie `promise.yield_value(i * i)` i zastosowanie mechanizmu `co_await` do otrzymanego awaitera. W naszym przypadku `yield_value` zapisuje wartość w obietnicy i zwraca `suspend_always`. Funkcja `next()` otrzymuje wtedy sterowanie z powrotem i udostępnia zapisany element. Następne `next()` kontynuuje pętlę od poprzedniego miejsca zawieszenia.

Generator posiada tylko jeden bieżący wynik. Nie przechowuje całego ciągu jak `std::vector<int>`. To użyteczne przy dużych lub potencjalnie nieskończonych sekwencjach. *Nie oznacza to* jednak automatycznie mniejszego kosztu obliczeń — dochodzą koszty ramki i kolejnych wznowień.

Uwaga. Przedstawiony `Generator` jest celowo minimalny: nie udostępnia interfejsu iteratorów `begin()/end()`, nie jest bezpieczny do jednoczesnego wznawiania z kilku wątków i nie powinien być stosowany jako gotowa biblioteka produkcyjna.

15.6 Protokół `co_await`: `await_ready`, `await_suspend`, `await_resume`

Operator `co_await` nie jest synonimem oczekiwania na `future`. Kompilator uzyskuje obiekt *awaiter*, którego interfejs ma trzy fundamentalne operacje:

`await_ready()` Zwraca wartość logiczną: jeśli `true`, zawieszenie jest pomijane.

`await_suspend(handle)` Jeżeli `await_ready()` było fałszywe, metoda podejmuje decyzję o kontynuacji. Dopuszczalne postacie zwracają `void`, `bool` albo inny `std::coroutine_handle`.

`await_resume()` Wyznacza wynik całego wyrażenia `co_await`; może też zgłosić wyjątek.

Gdy `await_suspend` zwraca `bool`, wartość `false` oznacza kontynuację bez pozostawania w zawieszeniu, a `true` — pozostawienie korutyny w stanie zawieszonym. Przy zwrocie `void` korutyna pozostaje zawieszona do późniejszego wznowienia. Zwrócenie innego uchwytu powoduje *symmetric transfer*: sterowanie przechodzi do wskazanej korutyny.

Nie każdy typ bezpośrednio dostarcza trzy wymienione metody: język obsługuje też przeciążenie `operator co_await` oraz `await_transform` zdefiniowane w `promise_type`. Sposób uzyskania awaitera zależy od tych reguł.

15.6.1 Czysto lokalne zawieszenie bez nowego wątku

Poniższy typ liczy wznowienia. Wykorzystamy go z prostą korutyną, której właściciel ręcznie wywołuje `resume()`.

```
#include <cassert>
#include <coroutine>
#include <exception>
#include <utility>

struct Stepper {
    struct promise_type {
        Stepper get_return_object() {
            return Stepper{handle::from_promise(*this)};
        }
        std::suspend_always initial_suspend() noexcept { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        void return_void() noexcept {}
        void unhandled_exception() { std::terminate(); }
    };

    using handle = std::coroutine_handle<promise_type>;
    handle h;
    explicit Stepper(handle x) : h(x) {}
    Stepper(const Stepper&) = delete;
    Stepper(Stepper&& other) noexcept
        : h(std::exchange(other.h, {})) {}
    ~Stepper() { if (h) h.destroy(); }
    bool resume() {
        if (!h || h.done()) return false;
        h.resume();
        return !h.done();
    }
};

struct Pause {
    int& count;
    bool await_ready() const noexcept { return false; }
    void await_suspend(std::coroutine_handle<>) const noexcept {
        ++count; // Zawiesz, az wlasciciel wywola resume().
    }
    void await_resume() const noexcept {}
};

Stepper example(int& count) {
    ++count;                 // wykona sie przy pierwszym resume
    co_await Pause{count};  // pierwsze zawieszenie
    ++count;                 // wykona sie przy drugim resume
```

```
}

int main() {
    int count = 0;
    auto step = example(count);
    assert(count == 0);
    assert(step.resume() && count == 2);
    assert(!step.resume() && count == 3);
}
```

Listing 15.4: co_await z własnym awaiterem: żadnego nowego wątku (C++20); kompletny program.

Kod niczego nie oblicza równolegle. Wszystkie instrukcje wykonał wątek `main`. Korutyna jedynie oddała sterowanie, zachowując miejsce wznowienia.

15.7 Własne `Task<T>`: leniwe obliczenie a asynchroniczność

Zaprojektujemy najmniejszy rozsądny typ `Task<T>` zwracający wynik i obsługujący wyjątek. Celowo nie implementujemy jeszcze harmonogramu: pozwala to zrozumieć, że *sam typ korutyny nie jest schedulerem*. `get()` w tym przykładzie działa wyłącznie wtedy, gdy ciało może zakończyć się po pojedynczym wznowieniu.

```
#include <cassert>
#include <coroutine>
#include <exception>
#include <optional>
#include <stdexcept>
#include <utility>

template<class T>
class Task {
public:
    struct promise_type {
        std::optional<T> result;
        std::exception_ptr error;
        Task get_return_object() {
            return Task{Handle::from_promise(*this)};
        }
        std::suspend_always initial_suspend() noexcept { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        void return_value(T x) { result.emplace(std::move(x)); }
        void unhandled_exception() noexcept {
            error = std::current_exception();
        }
    };
    using Handle = std::coroutine_handle<promise_type>;

    explicit Task(Handle h) noexcept : h_(h) {}
    Task(const Task&) = delete;
    Task& operator=(const Task&) = delete;
    Task(Task&& x) noexcept : h_(std::exchange(x.h_, {})) {}
    ~Task() { if (h_) h_.destroy(); }

    T get() {
        if (!h_) throw std::logic_error("Empty task");
        if (!h_.done()) h_.resume();
        if (!h_.done())
            throw std::logic_error("Task needs an external scheduler");
        if (h_.promise().error)
            std::rethrow_exception(h_.promise().error);
        return std::move(*h_.promise().result);
```

```
    }

private:
    Handle h_{};
};

Task<int> calculate(int x) {
    co_return x * x;
}

int main() {
    auto task = calculate(7); // lazy, obliczenie jeszcze nie ruszylo
    assert(task.get() == 49);
}
```

Listing 15.5: Lazy Task<T> bez schedulera (C++20); kompletny program.

Ten `Task<T>` jest pedagogiczny: nie ma przeciążenia `operator co_await`, nie wspiera wielokrotnego pobierania wyniku i nie potrafi samodzielnie czekać na zdarzenia zewnętrzne. W szczególności `get()` nie może po prostu wielokrotnie wywoływać `resume()` na rzeczywistym zadaniu I/O: wznowienie przed ukończeniem operacji mogłoby naruszyć protokół awaitera. Produkcyjne `Task<T>` wymaga m.in. protokołu kontynuacji, jasnej semantyki własności i poprawnej synchronizacji z executorem.

15.8 Korutyna a `std::async`: fundamentalne porównanie

`std::async` (C++11, nagłówek `<future>`) jest gotową funkcją uruchamiającą zadanie i zwracającą `std::future<T>`. Nie wymaga od użytkownika implementowania `promise_type`. Przy polityce `std::launch::async` funkcja pracuje w odrębnym wątku; polityka `std::launch::deferred` odracza jej uruchomienie do pierwszego nieczasowego oczekiwania na wynik.

Z kolei korutyna C++20 definiuje sposób zapisywania *kontynuacji*. Nawet jeśli wynik ma charakter asynchroniczny, *faktyczne uruchamianie pracy* zależy od awaitera, event loop, schedulera lub innego mechanizmu zewnętrznego. Nie istnieje standardowa ogólna reguła „jedna korutyna = jeden wątek”.

Czy można napisać `co_await future`? Nie wprost: w C++20 standardowy `std::future<T>` nie udostępnia protokołu awaitera wymaganego przez `co_await`. `future.get()` jest operacją potencjalnie blokującą bieżący wątek. Istnieją biblioteki dostarczające awaitery dla przyszłych wyników, ale sam adapter nie zamienia blokującej operacji w nieblokującą. Naiwne uruchamianie osobnego wątku tylko po to, by wykonał `future.wait()` i wznowił korutynę, może zniweczyć zalety oszczędnego używania wątków. Dla rzeczywiście nieblokującego oczekiwania na I/O potrzeba integracji z właściwym mechanizmem asynchronicznym.

Cecha	`std::async`	**Korutyny C++20**
Co opisuje?	Zadanie ze stanem wyniku	Wykonanie z punktami zawieszenia
Tworzy nowy wątek?	`launch::async`: osobny wątek	Samo `co_await`: nie
Kiedy zaczyna pracę?	Zależnie od polityki	Zależnie od `initial_suspend` i schedulera
Odbieranie wyniku	`future::get()` (może blokować)	Zależy od typu `Task` i awaitera
Wyjątki	Przechowywane w stanie `future`	Obsługa przez `unhandled_exception` i typ wyniku
Anulowanie	Brak automatycznego anulowania przez `future`	Możliwe kooperacyjnie; wymaga protokołu
Pamięć/zasoby	Stan asynchroniczny, przy `async` również wątek	Ramka korutyny, ewentualnie executor
Gotowe narzędzie w C++20?	Tak	Składnia tak; ogólny `Task<T>` nie

15.8.1 Ten sam rachunek zapisany na dwa sposoby

Porównanie dwóch krótkich funkcji pokazuje różnicę semantyczną.

```
#include <cassert>
#include <future>

int square(int x) { return x * x; }

int main() {
    auto f = std::async(std::launch::async, square, 7);
    assert(f.get() == 49);
}
```

Listing 15.6: std::async uruchamia zadanie w odrębnym wątku (C++11); kompletny program.

```
// Przyjmujemy implementacje Task<T> z poprzedniej sekcji.
Task<int> square_coroutine(int x) {
    co_return x * x;
}

// Task<int> t = square_coroutine(7);
// int result = t.get(); // 49, w watku wywolujacym get()
```

Listing 15.7: Korutyna bez awaitera zewnętrznego nie tworzy wątku; fragment.

W obu przypadkach można otrzymać tę samą wartość, ale nie jest to to samo zachowanie wykonawcze. `std::async` z `launch::async` zarządza wykonaniem w innym wątku, a pokazany `Task` jedynie odkłada wywołanie do momentu ręcznego wznowienia.

15.8.2 Pułapka destruktora `future`

Ważna różnica dotyczy czasu życia. Gdy ostatni uchwyt do stanu asynchronicznego utworzonego przez `std::async` z polityką `launch::async` jest niszczony i praca wciąż trwa, destruktor może czekać na jej ukończenie (w typowym przypadku rzeczywiście czeka). Poniższe dwa wywołania mogą więc wykonać się praktycznie sekwencyjnie:

```
// Funkcje slow_a i slow_b wykonuja dlugie obliczenia.
// std::async(std::launch::async, slow_a);
// std::async(std::launch::async, slow_b);
```

Listing 15.8: Zignorowane future mogą wymuszać oczekiwanie; fragment.

Jeżeli chcemy umożliwić nakładanie się zadań, trzeba przechować ich uchwyty przed oczekiwaniem:

```
// auto fa = std::async(std::launch::async, slow_a);
// auto fb = std::async(std::launch::async, slow_b);
// auto a = fa.get();
// auto b = fb.get();
```

Listing 15.9: Dwa przyszłe wyniki powinny żyć jednocześnie; fragment.

Analogicznie w systemie korutyn należy znać politykę niszczenia `Task`: usunięcie właściciela zawieszonej korutyny bez koordynacji z executorem może zostawić wiszący uchwyt. Różne biblioteki oferują odmienne kontrakty anulowania i oczekiwania.

15.8.3 Asynchroniczne I/O a zadania CPU-bound

Przy *operacji I/O* większość czasu może upływać na oczekiwaniu na dysk, sieć lub timer. Duża liczba korutyn może opisywać wiele rozpoczętych operacji, podczas gdy mała pula wątków obsługuje zdarzenia gotowości i wznowienia. Korzyść nie bierze się z tego, że `co_await` „jest szybsze”, lecz z możliwości nieblokowania wątków podczas oczekiwania.

Przy *operacjach CPU-bound* (obliczenia numeryczne, kompresja, przetwarzanie obrazów) samo zawieszenie korutyny nie zwiększa liczby rdzeni wykonujących kod. Tu potrzebne jest rzeczywiste rozłożenie pracy na wiele wątków (np. `std::async`, pula wątków lub OpenMP). Korutyna może być wygodnym *interfejsem* do takiego executora, ale nie zastępuje go.

15.9 Rzeczywista asynchroniczność: korutyna i własny executor

Żeby zestawienie z `std::async` nie pozostało czysto teoretyczne, utworzymy prosty executor z jednym wątkiem roboczym. `co_await executor.schedule()` odłoży kontynuację do kolejki. Wznowienie wykona *wątek executora*, nie wywołujący. Wartością zwracaną będzie nasza klasa `FutureTask<T>`, wykorzystująca wewnętrznie `std::promise<T>` i `std::future<T>`.

W przeciwieństwie do poprzedniego przykładu ramka ma strategię *samozniszczenia* przez `final_suspend = suspend_never`. Nie przechowujemy już zewnętrznego uchwytu właścicielskiego: po zakończeniu korutyna sama domyka swój cykl życia, a klient posiada wyłącznie `future`. W tej uproszczonej konstrukcji executor musi żyć aż do zakończenia zadań, a pojedyncza korutyna jest wznawiana w kolejce co najwyżej raz.

```
#include <cassert>
#include <condition_variable>
#include <coroutine>
#include <deque>
#include <exception>
#include <future>
#include <mutex>
#include <stdexcept>
#include <thread>
#include <utility>

class Executor {
    std::mutex m_;
    std::condition_variable cv_;
    std::deque<std::coroutine_handle<>> queue_;
    bool closed_ = false;
    std::thread worker_;

    void run() {
        for (;;) {
            std::coroutine_handle<> h;
            {
                std::unique_lock lock(m_);
                cv_.wait(lock, [&] { return closed_ || !queue_.empty(); });
                if (queue_.empty() && closed_) return;
                h = queue_.front();
                queue_.pop_front();
            }
            h.resume(); // H moze sie tu samozniszczyc.
            // Nie korzystamy juz z h po resume().
        }
    }

public:
    Executor() : worker_([this] { run(); }) {}
```

```
    Executor(const Executor&) = delete;
    Executor& operator=(const Executor&) = delete;

    ~Executor() {
        {
            std::lock_guard lock(m_);
            closed_ = true;
        }
        cv_.notify_one();
        worker_.join();
    }

    void post(std::coroutine_handle<> h) {
        {
            std::lock_guard lock(m_);
            if (closed_) throw std::runtime_error("Executor closed");
            queue_.push_back(h);
        }
        cv_.notify_one();
    }

    struct Schedule {
        Executor& ex;
        bool await_ready() const noexcept { return false; }
        void await_suspend(std::coroutine_handle<> h) const {
            ex.post(h); // Po przekazaniu h nie uzywamy juz awaitera.
        }
        void await_resume() const noexcept {}
    };

    Schedule schedule() noexcept { return Schedule{*this}; }
};

template<class T>
class FutureTask {
    std::future<T> future_;
public:
    explicit FutureTask(std::future<T> f) : future_(std::move(f)) {}
    FutureTask(FutureTask&&) = default;
    FutureTask(const FutureTask&) = delete;

    T get() { return future_.get(); }

    struct promise_type {
        std::promise<T> result;
        FutureTask get_return_object() {
            return FutureTask{result.get_future()};
        }
        std::suspend_never initial_suspend() noexcept { return {}; }
        std::suspend_never final_suspend() noexcept { return {}; }
        void return_value(T x) { result.set_value(std::move(x)); }
        void unhandled_exception() noexcept {
            try { result.set_exception(std::current_exception()); }
            catch (...) { std::terminate(); }
        }
    };
};

FutureTask<int> square_on(Executor& ex, int x) {
    co_await ex.schedule(); // Tu oddajemy sterowanie.
    co_return x * x;        // Wykona sie w watku roboczym.
}

int main() {
```

```
    Executor ex;
    auto a = square_on(ex, 6);
    auto b = square_on(ex, 7);
    assert(a.get() + b.get() == 85);
    // Przed zniszczeniem ex oba zadania sie zakonczyly.
}
```

Listing 15.10: Korutyna przechodząca do innego wątku poprzez executor (C++20); kompletny program.

Co to porównanie rzeczywiście dowodzi? Kod z `square_on` jest asynchroniczny względem wywołującego, ponieważ po `co_await` kontynuację przejmuje inny wątek. Nie oznacza to jeszcze *równoległego wykonywania* obu obliczeń: executor ma tylko jednego pracownika, więc zadania są przez niego wykonywane kolejno. Do równoległego przetwarzania dwóch obliczeń należałoby dodać drugiego pracownika albo wykorzystać istniejącą pulę wątków.

Pokazany executor jest *szkicem edukacyjnym*. Nie obsługuje ogólnego anulowania ani ponownego planowania tej samej korutyny, nie rozwiązuje wszystkich problemów czasu życia i nie stanowi gotowej biblioteki. W szczególności klient nie ma uchwytu pozwalającego anulować i bezpiecznie zniszczyć zawieszone zadanie. W systemie produkcyjnym własność korutyny, zamknięcie executora i czas życia powiązanych zasobów muszą stanowić jeden, spójny kontrakt.

15.10 Obsługa wyjątków, anulowanie i czas życia danych

15.10.1 Wyjątki a `unhandled_exception`

Nieobsłużony wyjątek z ciała korutyny nie może zostać pozostawiony bez decyzji. Kompilator wywołuje `promise.unhandled_exception()`. W generatorze zapisaliśmy `std::exception_ptr`; przy kolejnym `next()` błąd zostanie zgłoszony w kodzie klienta. W naszym `FutureTask<T>` wyjątek trafia do stanu `std::future` i ujawnia się podczas `get()`. Jest to analogiczne do propagacji błędów w `std::async`, ale sam mechanizm wymagał jawnej implementacji w `promise_type`.

Należy też uwzględnić wyjątki zgłaszane przez operacje awaitera oraz błędy alokacji ramki. Awaria podczas tworzenia ramki może wystąpić jeszcze przed wykonaniem ciała korutyny. Typ obietnicy może opcjonalnie udostępniać `get_return_object_on_allocation_failure()` razem z odpowiednią, niezgłaszającą wyjątku alokacją, ale nie jest to mechanizm automatycznie dostępny każdemu typowi korutyny.

15.10.2 Referencje przechowywane w ramce

Argumenty referencyjne nie wydłużają czasu życia obiektów. Jeżeli korutyna przechowuje referencję do zmiennej lokalnej, która zniknie podczas zawieszenia, jej późniejsze wznowienie może prowadzić do użycia nieistniejącego już obiektu.

```
// Zakladamy lazy Task<int> z poprzedniej sekcji.
Task<int> read_later(const int& ref) {
    co_return ref;
}

// NIEPOPRAWNE:
// Task<int> bad() {
//     Task<int> t = read_later(123);
//     return t; // referencja w ramce moze byc juz wiszaca
// }
```

Listing 15.11: Niebezpieczny czas życia referencji w korutynie; fragment koncepcyjny.

Z reguły bezpieczniej przechowywać wewnątrz ramki wartości przeznaczone do późniejszego użycia (np. obiekt przekazany przez wartość), a nie referencje do tymczasowych obiektów. Trzeba też zwracać uwagę na czas życia domknięć lambda: korutyna będąca lambdą i wykorzystująca przechwycenia może po zawieszeniu odwoływać się do już zniszczonego obiektu domknięcia. Sam fakt przeniesienia wynikowej korutyny nie musi temu zapobiegać.

15.10.3 Anulowanie jest kooperacyjne

`std::stop_token` (C++20) umożliwia *żądanie*, a nie wymuszenie zakończenia zadania. Korutyna może sprawdzać token przed rozpoczęciem kolejnych faz albo korzystać z awaitera, który potrafi odpowiednio reagować na anulowanie. Nie wynika z tego jednak, że każde `co_await` automatycznie obsługuje `stop_token`.

```
// Zakladamy Task<int> z poprzedniej sekcji.
#include <stop_token>

Task<int> calculate(std::stop_token token) {
    if (token.stop_requested()) co_return -1;
    // Pierwszy krok obliczen...
    if (token.stop_requested()) co_return -1;
    // Kolejny krok obliczen...
    co_return 42;
}
```

Listing 15.12: Kooperacyjne anulowanie korutyny: sprawdzenie żądania w punktach kontrolnych; fragment.

W powyższym przykładzie token jest przekazywany przez wartość. Sam `stop_token` nie steruje czasem życia ramki i nie wznawia korutyny, która czeka na zdarzenie. Do pełnego anulowania operacji I/O potrzeba jeszcze protokołu z usługą, która wykonuje rzeczywistą operację, oraz reguł określających *kto* zniszczy korutynę i *kiedy* wolno to zrobić.

15.11 Pułapki synchronizacji i planowania

15.11.1 Równoczesne wznowienie tej samej korutyny

Nie można traktować `coroutine_handle::resume()` jak operacji atomowej nadającej się do równoczesnego wywoływania przez wiele wątków. Trzeba zapewnić, że dana zawieszona korutyna ma jednego logicznego właściciela uprawnienia do wznowienia. Ponowne wznowienie korutyny już aktywnej lub zakończonej może prowadzić do niezdefiniowanego zachowania.

15.11.2 Utracone powiadomienie i wyścig publikacji uchwytu

Awaiter może przekazać uchwyt do innego wątku wewnątrz `await_suspend`. Wówczas druga strona może wznowić korutynę *zanim* pierwotne wywołanie `await_suspend` powróci. Dlatego po opublikowaniu uchwytu kod awaitera nie powinien korzystać z obiektów, których czas życia mógł się już zakończyć.

W bardziej złożonych awaiterach trzeba również uniknąć zgubienia zdarzenia kończącego operację między sprawdzeniem gotowości a zarejestrowaniem oczekiwania. Poprawne rozwiązania stosują np.

mutex, operacje atomowe albo protokół jednorazowego przejęcia kontynuacji. Samo `await_ready()` nie zastępuje synchronizacji.

15.11.3 Brak własności `coroutine_handle`

Gdy planista ma w kolejce jedynie `coroutine_handle<>`, nie posiada informacji, kto może zniszczyć ramkę. Usunięcie obiektu `Task` podczas oczekiwania może prowadzić do późniejszego wznowienia już zwolnionej pamięci. Najbezpieczniejszy projekt definiuje własność od początku: *Task posiada ramkę*, *executor posiada ramkę* albo *ramka posiada samą siebie*, z dokładnym protokołem przekazania własności i zakończenia.

15.12 Koszty, optymalizacje i pomiary

Wydajność należy rozpatrywać w całym systemie, a nie tylko przez porównanie czasu jednego `co_await` i jednego `std::async`.

- **Utworzenie wątku.** `std::async(launch::async, ...)` uruchamia pracę w odrębnym wątku; bardzo duża liczba krótkich zadań może generować istotny narzut zarządzania wątkami.
- **Ramka.** Korutyna może wymagać alokacji pamięci, zachowywania stanu i wznawiania. Kompilator może czasem wyeliminować alokację ramki, ale nie jest to gwarancja.
- **Executor.** Kolejka, mutexy, przełączanie między wątkami oraz współdzielenie pamięci również kosztują.
- **Lokalność pamięci.** Duża liczba korutyn z rozproszonymi ramkami może prowadzić do dodatkowych nietrafień cache.
- **Skalowalność.** Dla I/O ważniejsza może być liczba oczekujących operacji niż przepustowość pojedynczego zadania. Dla CPU-bound kluczowa jest liczba rdzeni, wielkość porcji pracy i ograniczenie dostępu do współdzielonych zasobów.

Rzetelny benchmark powinien mierzyć co najmniej: całkowitą przepustowość, opóźnienia (również wartości odstające), liczbę wątków, liczbę alokacji oraz maksymalne zużycie pamięci. Porównywanie `std::async` z *nieuruchamiającą nowych wątków* korutyną wykonującą natychmiast prostą arytmetykę nie jest miarodajnym testem wydajności asynchronicznej.

15.12.1 Praktyczna decyzja: kiedy używać którego narzędzia?

Problem	Rozsądny pierwszy wybór
Jedno niezależne zadanie CPU z wynikiem	`std::async(std::launch::async, ...)`
Wiele krótkich zadań CPU	Pula wątków, executor lub OpenMP
Wiele połączeń I/O z długim oczekiwaniem	Korutyny + asynchroniczny runtime I/O
Leniwe generowanie sekwencji	Generator korutynowy
Wspólny licznik lub krótka sekcja krytyczna	`atomic` albo mutex, nie korutyna
Anulowalne zadanie działające długo	Przemyślany protokół `stop_token`
Interfejs biblioteki o złożonych zależnościach	`Task<T>` nad zdefiniowanym executorem

15.13 Związek z OpenMP, `thread` i `future`

Rozszerzony rozdział o współbieżności omawiał `std::thread`, `std::jthread`, atomiki, blokady, `std::async` oraz OpenMP. Korutyny nie unieważniają tych narzędzi; stanowią kolejną warstwę.

- `std::thread` opisuje konkretny wątek wykonania, a `std::jthread` dodatkowo pomaga zarządzać jego czasem życia i kooperacyjnym zatrzymywaniem.
- `std::async` tworzy stan przyszłego wyniku i uruchamia wywołanie zgodnie z przyjętą polityką.
- OpenMP rozdziela pętle i zadania między wątki obsługiwane przez środowisko wykonawcze OpenMP; szczególnie przydaje się w obliczeniach CPU-bound i kodzie numerycznym.
- Korutyny pozwalają wygodnie opisać wieloetapową logikę oczekiwania i kontynuacji. Ich wydajność i stopień równoległości zależą od dostarczonego środowiska wykonawczego.

Możliwe jest użycie *korutyn nad pulą wątków*: `co_await` przekazuje kontynuację do kolejki, a pracownik puli później ją wznawia. Nie należy jednak od razu łączyć wszystkich mechanizmów. Najpierw określamy, czy program czeka głównie na I/O, czy wykonuje obliczenia, a następnie wybieramy model wykonania i własności zasobów.

15.14 Najczęstsze błędy projektowe

1. Zakładanie, że `co_await` uruchamia nowy wątek.
2. Próba zwrócenia bezpośrednio `std::future<T>` z własnej korutyny bez odpowiedniego `promise_type`.
3. Dwukrotne wywołanie `destroy()` dla tej samej ramki.
4. Wywołanie `resume()` po osiągnięciu `final_suspend`.
5. Pozostawienie referencji lub przechwycenia lambdy do obiektu, który znika podczas zawieszenia.
6. Zniszczenie `Task` mimo tego, że executor posiada w kolejce uchwyt do jego ramki.
7. Mylenie automatycznej propagacji wyjątków w `std::async` z obowiązkami implementatora `promise_type`.
8. Traktowanie `std::stop_token` jako mechanizmu przymusowego przerwania dowolnego `co_await`.
9. Równoczesne wznawianie tej samej korutyny z kilku wątków.
10. Uznanie, że zawieszenie i wznowienie zawsze kosztuje mniej niż zwykłe wywołanie funkcji.
11. Uruchamianie `std::async` dla tysięcy mikro-zadań bez rozważenia puli wątków.
12. Zwracanie z `await_suspend` po opublikowaniu uchwytu i dalsze używanie potencjalnie zniszczonego awaitera.

Podsumowanie rozdziału

Korutyny C++20 są mechanizmem *zawieszenia i kontynuacji*, a nie samodzielnym środowiskiem asynchronicznym. Generator używa `co_yield`, aby leniwie udostępniać wartości. Typ zadania używa `co_return`, obiektu `promise_type` i awaiterów, aby opisać wykonanie z wynikiem i wyjątkami. `co_await` przekazuje kontrolę zgodnie z protokołem awaitera, a dopiero executor lub inne środowisko może wznowić kod w osobnym wątku.

`std::async` jest prostym narzędziem do uruchomienia funkcji i odebrania przyszłego wyniku. Korutyny są lepszym budulcem dla złożonych przepływów wieloetapowych, szczególnie I/O, ale wymagają większej odpowiedzialności za pamięć, synchronizację, anulowanie i czas życia ramki. W obu przypadkach należy rozdzielać trzy pytania: *kto wykonuje kod, kto posiada stan i kto decyduje o zakończeniu.*

Literatura i dokumentacja

- https://en.cppreference.com/w/cpp/language/coroutines — model korutyn C++20.
- https://en.cppreference.com/w/cpp/coroutine/coroutine_handle — uchwyty i zarządzanie ramką.
- https://en.cppreference.com/w/cpp/coroutine/suspend_always — podstawowe awaitery.
- https://en.cppreference.com/w/cpp/thread/async — `std::async` i polityki uruchamiania.
- https://en.cppreference.com/w/cpp/thread/future — `std::future`.
- https://en.cppreference.com/w/cpp/coroutine/generator — `std::generator` (C++23).
- Lewis Baker, *C++ Coroutines: Understanding the Compiler Transformations* — seria publikacji o modelu wykonania korutyn.
- David Mazières, *My Tutorial and Take on C++20 Coroutines*, 2021.

Rozdział 16

Zaawansowana współbieżność, model pamięci i algorytmy bez blokad

16.1 Cel rozdziału i powiązanie z podstawami

W rozdziale 1 poznaliśmy wątki, mutexy, zmienne warunkowe, operacje atomowe, `std::async`, pule wątków i OpenMP. Sama znajomość tych interfejsów nie wystarcza jednak do samodzielnego projektowania algorytmów współbieżnych. Trzeba odpowiedzieć na trudniejsze pytania:

- Kiedy zapis wykonany przez jeden wątek *musi* być widoczny dla drugiego?
- Które wyniki są dozwolone przez model pamięci, nawet gdy wydają się sprzeczne z intuicją opartą na sekwencyjnym wykonywaniu?
- Czym różni się atomowość pojedynczej operacji od niepodzielności całego algorytmu?
- Jak zaprojektować *postęp* algorytmu i bezpiecznie zwalniać pamięć, gdy inne wątki mogą jeszcze posiadać wskaźniki do obiektów?
- Kiedy rozwiązanie bez mutexa rzeczywiście poprawia wydajność?

Skupimy się na *uzasadnianiu poprawności*, a nie tylko na składni. W szczególności przedstawimy proste testy modelu pamięci (*litmus tests*), algorytm kolejki SPSC, publikację niezmiennych struktur danych, problem ABA, techniki odzyskiwania pamięci oraz praktyczne zasady pomiaru i diagnostyki. Nie zakładamy, że każdy `std::atomic<T>` jest realizowany bez blokad.

Uwaga. Wszystkie gwarancje sformułowane w rozdziale dotyczą abstrakcyjnego modelu języka C++. Rzeczywisty procesor może wykonywać instrukcje w innej kolejności i stosować bufory zapisów, ale program musi być poprawny na gruncie standardu, a nie tylko na jednym komputerze.

16.2 Formalny porządek zdarzeń w wielowątkowym programie

16.2.1 Zdarzenia i relacje *sequenced-before*, *synchronizes-with*

Rozpatrzmy pewne operacje na pamięci jako *zdarzenia*. Wśród relacji używanych do opisu ich uporządkowania najważniejsze są:

Sequenced-before (sb)
: Porządek określony regułami języka pomiędzy wybranymi operacjami w obrębie jednego wątku. Nie należy go mylić z dowolną kolejnością pojawiania się instrukcji w kodzie źródłowym: język zawiera także wyrażenia z nieokreśloną kolejnością ewaluacji.

Synchronizes-with (sw)
: Synchronizacja pomiędzy odpowiednimi operacjami różnych wątków: na przykład zapis `release` i odczyt `acquire`, który czyta wartość z odpowiedniego zapisu lub jego sekwencji *release*. Podobną synchronizację zapewniają właściwe operacje na mutexach.

Happens-before (hb)
: Relacja porządkująca zdarzenia, budowana w typowych przykładach przez sb, sw i ich przechodnie złożenia. Jeśli A hb B, skutki A są uporządkowane przed B w sensie formalnego modelu; nie jest to stwierdzenie o czasie zegarowym.

Dla przejrzystości będziemy używać następującego schematu:

$$\underbrace{A \xrightarrow{\text{sb}} R}_{\text{producent}} \xrightarrow{\text{sw}} \underbrace{Q \xrightarrow{\text{sb}} B}_{\text{konsument}} \implies A \xrightarrow{\text{hb}} B.$$

Operacja R jest publikacją, Q odpowiadającym jej odbiorem; A i B mogą dotyczyć zwykłej, nieatomowej zmiennej.

16.2.2 Atomowość nie zastępuje synchronizacji

W przeciwieństwie do zwykłego odczytu lub zapisu operacje na `std::atomic<T>` nie powodują wyścigu danych pomiędzy sobą. Nie oznacza to jednak, że każdy odczyt atomowy widzi ostatni zapis według zegara ściennego. Taki globalny *ostatni zapis* zwykle w ogóle nie istnieje jako zdefiniowane pojęcie modelu.

Z kolei jeśli dwa wątki odczytują i modyfikują to samo miejsce pamięci nieatomowo, a konfliktowe operacje nie są uporządkowane relacją *happens-before*, program ma *data race* i niezdefiniowane zachowanie. Samo oznaczenie obiektu przez `volatile` nie rozwiązuje tego problemu.

16.2.3 Porządek modyfikacji (*modification order*) i spójność

Dla *każdego pojedynczego* obiektu atomowego istnieje całkowity porządek jego modyfikacji (*modification order*). Nie wynika z tego jeden wspólny porządek modyfikacji *wszystkich* atomików. Ważne konsekwencje to między innymi:

- zapis do `x` uporządkowany przed innym zapisem do `x` musi być odpowiednio umieszczony przed nim w porządku modyfikacji;
- jeśli odczyt `x` jest uporządkowany przed kolejnym odczytem `x` w tym samym wątku, drugi odczyt nie może cofnąć się do wcześniejszej modyfikacji w porządku tego atomika;
- ograniczenia spójności dla jednego atomika nie zapewniają automatycznie spójnego obrazu *dwóch różnych* atomików.

16.3 Testy modelu pamięci: wyniki, które zaskakują

16.3.1 *Store buffering*: dwa odczyty zera

Załóżmy, że `x` i `y` początkowo są równe zero, a wątki wykonują następujące operacje:

Wątek 1	Wątek 2
$x \leftarrow 1$	$y \leftarrow 1$
$r_1 \leftarrow y$	$r_2 \leftarrow x$

```cpp
#include <atomic>
#include <iostream>
#include <thread>

int main() {
    std::atomic<int> x{0}, y{0};
    int r1 = -1, r2 = -1;

    std::jthread a([&] {
        x.store(1, std::memory_order_relaxed);
        r1 = y.load(std::memory_order_relaxed);
    });
    std::jthread b([&] {
        y.store(1, std::memory_order_relaxed);
        r2 = x.load(std::memory_order_relaxed);
    });
    a.join();
    b.join();
    std::cout << r1 << ' ' << r2 << '\n';
}
```

Listing 16.1: Store buffering: kompletna, wolna od wyścigu danych próba (C++20).

W tym programie $r_1 = r_2 = 0$ jest **dozwolonym** wynikiem modelu C++ dla `relaxed`; nie ma gwarancji, że zostanie zaobserwowany w praktycznym uruchomieniu. Jeśli *wszystkie cztery* operacje atomowe zamienimy na `memory_order_seq_cst`, wynik $(0, 0)$ stanie się niemożliwy: wspólny porządek operacji sekwencyjnie spójnych musiałby przeczyć kolejności w każdym z wątków.

Uwaga. Wynik pojedynczego eksperymentu, nawet powtórzony milion razy, nie dowodzi niedopuszczalności innego zachowania. Test tego rodzaju to materiał do rozumowania nad modelem, a nie pełny dowód poprawności.

16.3.2 *Message passing*: publikacja zwykłych danych

```cpp
#include <atomic>
#include <cassert>
#include <thread>

int main() {
    int payload = 0;
    std::atomic<bool> published{false};

    std::thread producer([&] {
        payload = 42;
        published.store(true, std::memory_order_release);
    });
    std::thread consumer([&] {
        while (!published.load(std::memory_order_acquire)) {
            std::this_thread::yield();
        }
        assert(payload == 42);
    });

    producer.join();
    consumer.join();
}
```

Listing 16.2: Publikacja danych przy użyciu release/acquire (C++11).

Odczyt flagi, który widzi `true`, synchronizuje się z zapisem `release`. Zapis `payload = 42` następuje

happens-before odczytu `payload` przez konsumenta. Flaga *publikuje* wcześniejszy stan; nie należy po publikacji modyfikować `payload` bez dalszej synchronizacji.

16.4 Porządki atomowe i ich dokładne zastosowania

Porządek	Typowe zastosowanie i ograniczenie
`relaxed`	Liczniki, statystyki, pola kontrolne, gdy sama atomowość wystarcza, a relacja synchronizacji wynika skądinąd.
`release`	Zapis lub operacja RMW publikująca wcześniejsze efekty wątku.
`acquire`	Odczyt lub operacja RMW odbierająca stan od publikującego wątku.
`acq_rel`	Operacja odczyt–modyfikacja–zapis wymagająca obu kierunków.
`seq_cst`	Operacje z dodatkową gwarancją pojedynczego wspólnego porządku dla operacji sekwencyjnie spójnych.

`memory_order_consume` ma długą i skomplikowaną historię standaryzacji. W praktycznym kodzie C++20 należy preferować `acquire` zamiast opierania przenośnej poprawności na `consume`. Uwaga: porządek `release` jest właściwy dla zapisu, `acquire` dla odczytu, a `acq_rel` dla operacji odczyt–modyfikacja–zapis; nie wszystkie porządki są poprawne dla każdego przeciążenia funkcji atomowej.

16.4.1 Operacje *read–modify–write*

Metody `fetch_add`, `fetch_sub`, `exchange` i udany `compare_exchange_*` wykonują operację RMW. Są one niepodzielne w odniesieniu do danego atomika, ale nie tworzą automatycznie transakcji obejmującej inne pola.

```
#include <atomic>
#include <cassert>
#include <thread>

int main() {
    std::atomic<int> next_id{0};
    int first = -1, second = -1;
    std::thread a([&] {
        first = next_id.fetch_add(1, std::memory_order_relaxed);
    });
    std::thread b([&] {
        second = next_id.fetch_add(1, std::memory_order_relaxed);
    });
    a.join();
    b.join();
    assert(first != second);
    assert(first + second == 1);
}
```

Listing 16.3: Przydzielanie kolejnych numerów: wystarcza relaxed (C++11).

Uzyskujemy unikatowe wartości, bo RMW mają jednoznaczny porządek modyfikacji `next_id`. Nie uzyskujemy przez to żadnej dodatkowej synchronizacji niezwiązanych danych.

16.4.2 *Release sequence*: łańcuch operacji RMW

W C++20 zapis z porządkiem `release` może rozpoczynać sekwencję *release* składającą się z następujących po nim operacji RMW na *tym samym* atomiku, wykonywanych także przez inne wątki. Odczyt `acquire` wartości pochodzącej z tej sekwencji może synchronizować się z początkowym zapisem `release`.

```cpp
#include <atomic>
#include <cassert>
#include <thread>

int main() {
    int payload = 7;
    std::atomic<int> state{0};

    std::thread producer([&] {
        payload = 42;
        state.store(1, std::memory_order_release);
    });
    std::thread intermediary([&] {
        int expected = 0;
        while ((expected = state.load(std::memory_order_acquire))
               == 0) {
            std::this_thread::yield();
        }
        state.fetch_add(1, std::memory_order_relaxed);
    });
    std::thread consumer([&] {
        while (state.load(std::memory_order_acquire) != 2) {
            std::this_thread::yield();
        }
        assert(payload == 42);
    });

    producer.join();
    intermediary.join();
    consumer.join();
}
```

Listing 16.4: Przekazanie publikacji przez relaxed RMW (C++20).

Wątek pośredni czeka, aż stan osiągnie 1, i wykonuje RMW, który zapisuje 2. Odczyt wartości 2 z `acquire` synchronizuje się z pierwotnym `release` przez sekwencję *release*. Nie można w ogólności zastąpić `fetch_add` zwykłym `store(2, relaxed)` i utrzymać tej samej argumentacji.

16.5 CAS: porównaj i zamień, pętle ponawiania i porządek niepowodzenia

`compare_exchange_weak(expected, desired, success, failure)` atomowo porównuje bieżącą wartość z `expected`. Gdy porównanie się powiedzie, zapisuje `desired`; w przeciwnym razie aktualizuje `expected` do wartości zastanej w atomiku. Wersja `weak` może ponadto nie powieść się pozornie (*spurious failure*). W pętli jest zwykle dobrym wyborem.

```cpp
#include <atomic>
#include <cassert>

void atomic_max(std::atomic<int>& target, int candidate) {
    int old = target.load(std::memory_order_relaxed);
    while (old < candidate &&
           !target.compare_exchange_weak(
               old, candidate,
               std::memory_order_relaxed,
               std::memory_order_relaxed)) {
        // Przy niepowodzeniu old jest aktualizowane.
    }
}

```

```
int main() {
    std::atomic<int> maximum{10};
    atomic_max(maximum, 8);
    atomic_max(maximum, 17);
    assert(maximum.load() == 17);
}
```

Listing 16.5: Atomowe maksimum z pętlą CAS (C++11).

Parametr `failure` opisuje porządek operacji *odczytu* w przypadku niepowodzenia. Nie może to być `release` ani `acq_rel`; musi też spełniać ograniczenia względem porządku sukcesu określone standardem. Typowy wzorzec publikacji nowej wartości to sukces z `acq_rel` i niepowodzenie z `acquire` (albo `relaxed`, jeśli tak wynika z dowodu).

16.5.1 `weak` kontra `strong`

`compare_exchange_strong` nie zgłasza pozornego niepowodzenia przy poprawnie porównanych wartościach. Może być użyteczne dla pojedynczej próby; `weak` bywa wygodniejsze w pętlach CAS. Nie ma uniwersalnej gwarancji, że jedna z tych wersji zawsze działa szybciej na danej platformie.

16.6 Bariery pamięci i `std::atomic_thread_fence`

Bariera `std::atomic_thread_fence(order)` jest operacją porządkowania pamięci, która sama nie odczytuje ani nie zapisuje konkretnego atomika. Synchronizacja z innym wątkiem wciąż wymaga odpowiednich operacji atomowych łączących obie strony. Samo wstawienie `fence` obok zwykłej zmiennej nie usuwa wyścigu danych.

```
#include <atomic>
#include <cassert>
#include <thread>

int main() {
    int payload = 0;
    std::atomic<bool> flag{false};

    std::thread producer([&] {
        payload = 42;
        std::atomic_thread_fence(std::memory_order_release);
        flag.store(true, std::memory_order_relaxed);
    });
    std::thread consumer([&] {
        while (!flag.load(std::memory_order_relaxed)) {
            std::this_thread::yield();
        }
        std::atomic_thread_fence(std::memory_order_acquire);
        assert(payload == 42);
    });

    producer.join();
    consumer.join();
}
```

Listing 16.6: Publikacja ze zwolnieniową i przejmującą barierą pamięci (C++11).

To przykład synchronizacji *fence–fence*: odczyt atomika widzi zapis następujący po barierze `release`, a bariera `acquire` jest wykonywana po tym odczycie. W nowym kodzie zwykle prostsze i czytelniejsze jest zastosowanie pary operacji `release`/`acquire` bez jawnych barier. Bariery nie są substytutem przemyślanego protokołu komunikacji.

16.7 Gwarancje postępu: *blocking*, *lock-free* i *wait-free*

Wielowątkowy algorytm można analizować pod względem *bezpieczeństwa* (*safety*: nic niedozwolonego się nie dzieje) i *żywotności* (*liveness*: coś pożądanego w końcu się wydarzy). Są to różne kwestie.

Blocking
: Wątek może oczekiwać na odblokowanie mutexa przez konkretny inny wątek. Jeśli właściciel przestanie robić postępy, pozostali również mogą utknąć.

Lock-free
: W standardowym ujęciu postępu algorytmowego, mimo rywalizacji wątków, system jako całość może kontynuować wykonywanie operacji: niepowodzenia jednej próby CAS zwykle oznaczają, że inny uczestnik wykonał postęp. Nie gwarantuje to postępu *każdego* wątku.

Wait-free
: Każda operacja danego wątku ma ograniczoną liczbę własnych kroków niezależnie od działań innych wątków (przy przyjętym modelu). To silniejszy warunek niż *lock-free*.

Obstruction-free
: Operacja postępuje, jeśli wątek może działać dostatecznie długo bez konfliktów ze strony innych wątków. Jest to słabsza gwarancja.

Lock-free nie znaczy „zawsze szybciej”. Nie oznacza również braku aktywnego oczekiwania ani braku zagłodzenia konkretnego wątku. Odróżniajmy też własność implementacji pojedynczego atomika (`is_lock_free()`) od postępu kompletnego algorytmu.

16.7.1 Prosta blokada wirująca (*spinlock*)

```cpp
#include <atomic>
#include <cassert>
#include <mutex>
#include <thread>

class SpinLock {
    std::atomic_flag flag = ATOMIC_FLAG_INIT;
public:
    void lock() noexcept {
        while (flag.test_and_set(std::memory_order_acquire)) {
            // Zamiast stale rywalizowac o cache line, czekamy.
            flag.wait(true, std::memory_order_relaxed);
        }
    }
    void unlock() noexcept {
        flag.clear(std::memory_order_release);
        flag.notify_one();
    }
};

int main() {
    SpinLock lock;
    int counter = 0;
    auto work = [&] {
        for (int i = 0; i < 10000; ++i) {
            std::lock_guard<SpinLock> guard(lock);
            ++counter;
        }
    };
    std::jthread a(work), b(work);
    a.join();
    b.join();
```

```
33      assert(counter == 20000);
34  }
```

Listing 16.7: Poprawna blokada wirująca z flagą atomową; ilustracja (C++20).

`test_and_set` ma semantykę RMW; odblokowanie jest operacją `release`. Metoda `wait(true)` może blokować, a zatem powyższy `SpinLock` *nie* jest algorytmem *lock-free*. Ponadto nie gwarantuje sprawiedliwości. W realnym oprogramowaniu `std::mutex` zwykle pozostaje lepszym wyborem, zwłaszcza gdy sekcja krytyczna może być długa lub obciążenie jest wysokie.

16.8 Problem ABA: poprawne CAS nie wystarcza

Rozważmy wierzchołek stosu reprezentowany wskaźnikiem atomowym. Wątek A czyta wskaźnik p i zamierza wykonać `compare_exchange` z wartości p na `p->next`. W tym czasie wątek B może:

1. zdjąć element p ze stosu;
2. zdjąć jeszcze jeden element;
3. ponownie umieścić p na wierzchołku stosu.

Wartość wskaźnika uległa zmianie $A \to B \to A$ (stąd ABA), ale CAS wątku A może zobaczyć ponownie tę samą wartość p. W międzyczasie struktura stosu mogła się całkowicie zmienić.

$$\boxed{\text{head} = p} \longrightarrow \boxed{\text{head} = q} \longrightarrow \boxed{\text{head} = p}$$

Dwa problemy trzeba odróżnić:

1. *ABA logiczne*: porównanie tej samej wartości nie wykrywa zmian stanu struktury.
2. *Bezpieczeństwo pamięci*: wątek A może próbować odczytać `p->next`, gdy inny wątek zniszczył już obiekt `*p`. Taki odczyt nie staje się poprawny tylko dlatego, że `head` jest atomowy.

16.8.1 Wskaźnik z licznikiem generacji

Jedna z technik polega na porównywaniu pary $(\text{wskaźnik}, \text{licznik wersji})$ zamiast samego wskaźnika. Wtedy ponowne pojawienie się tego samego adresu nie oznacza równości całego stanu. Licznik również ma ograniczony zakres i może się przepełnić; trzeba uzasadnić brak niebezpiecznego *wrap-around*.

Uwaga. Nie zakładaj, że `std::atomic<SomeStruct>` będzie bez blokad: implementacja może wykorzystać wewnętrzny mutex lub bibliotekę atomowych operacji. Możliwość zapisania stanu złożonego w `atomic` nie rozwiązuje też samodzielnie problemu czasu życia obiektów.

16.8.2 Hazard pointers, epoch-based reclamation i RCU

Hazard pointers pozwalają wątkom publikować informację: „zamierzam korzystać z tego obiektu”. Wątek usuwający obiekt odkłada go na listę *retired* i odracza zniszczenie do chwili, gdy żaden aktywny hazard pointer go nie wskazuje. Poprawna procedura zwykle musi ponownie zweryfikować odczyt wskaźnika po publikacji hazardu; samo opublikowanie dowolnego, potencjalnie już nieważnego adresu nie wystarcza.

Epoch-based reclamation grupuje operacje w epoki. Usunięte obiekty mogą zostać odzyskane dopiero wtedy, gdy wszystkie aktywne wątki zakończyły okresy korzystania z danych obejmujące ich usunięcie. Dłużej zatrzymany wątek może opóźniać zwolnienie pamięci.

Read-copy-update (RCU) korzysta z odczytów starych wersji danych i publikacji nowych wersji; usuwanie poprzednich odbywa się po odpowiednim okresie ochronnym (*grace period*). Szczegóły różnią się pomiędzy implementacjami.

Metoda	Kiedy rozważyć
Hazard pointers	Wiele odczytów wskaźników, częste usuwanie węzłów; możliwa kosztowna administracja hazardami.
Epoki	Duży ruch, kontrolowane sekcje dostępu; zatrzymany czytelnik może opóźnić odzyskanie pamięci.
RCU	Wzorce „wielu czytelników, niewielu pisarzy”, publikacja niezmiennych wersji danych.
`shared_ptr`	Gdy łatwość dowodu poprawności jest ważniejsza od kosztu liczenia referencji; nie obiecuje *lock-free*.

W C++20 standardowa biblioteka nie udostępnia jeszcze kompletnej, przenośnej abstrakcji hazard pointerów. Bezpieczna implementacja wymaga precyzyjnego protokołu pamięciowego, dlatego zamiast podawać uproszczony, ale błędny stos ze zwalnianiem węzłów, przedstawimy algorytm bez dynamicznego odzyskiwania pamięci: kolejkę SPSC.

16.9 Kolejka SPSC: praktyczny algorytm bez mutexa

Single Producer, Single Consumer (SPSC) oznacza dokładnie jednego producenta i jednego konsumenta. Bufor cykliczny może wtedy używać dwóch liczników:

- `head` — numer kolejnego miejsca zapisu, modyfikowany wyłącznie przez producenta;
- `tail` — numer kolejnego miejsca odczytu, modyfikowany wyłącznie przez konsumenta.

Producent zapisuje komórkę, a następnie publikuje nowy `head` operacją `release`. Konsument odczytuje `head` przez `acquire`, a więc może bezpiecznie zobaczyć zawartość wcześniej zapisanej komórki. Analogicznie konsument zwalnia komórkę, publikując nowy `tail`, a producent odbiera tę informację operacją `acquire`. Warunek pełności bufora o pojemności C to $\text{head} - \text{tail} = C$, a pustości: $\text{head} = \text{tail}$.

```
#include <array>
#include <atomic>
#include <cassert>
#include <cstddef>
#include <thread>

// Tylko jeden producent i dokladnie jeden konsument.
template<std::size_t Capacity>
class SpscQueue {
    static_assert(Capacity > 0);
    static_assert((Capacity & (Capacity - 1)) == 0);
    std::array<int, Capacity> buffer_{};
    std::atomic<std::size_t> head_{0};
    std::atomic<std::size_t> tail_{0};
public:
    bool try_push(int value) {
        const auto head = head_.load(std::memory_order_relaxed);
        const auto tail = tail_.load(std::memory_order_acquire);
        if (head - tail == Capacity) return false;
        buffer_[head & (Capacity - 1)] = value;
        head_.store(head + 1, std::memory_order_release);
        return true;
    }

    bool try_pop(int& result) {
```

```
        const auto tail = tail_.load(std::memory_order_relaxed);
        const auto head = head_.load(std::memory_order_acquire);
        if (tail == head) return false;
        result = buffer_[tail & (Capacity - 1)];
        tail_.store(tail + 1, std::memory_order_release);
        return true;
    }
};

int main() {
    constexpr int Count = 100000;
    SpscQueue<1024> queue;
    long long result = 0;

    std::jthread producer([&] {
        for (int x = 1; x <= Count; ++x) {
            while (!queue.try_push(x)) std::this_thread::yield();
        }
    });
    std::jthread consumer([&] {
        int item = 0;
        for (int n = 0; n < Count; ++n) {
            while (!queue.try_pop(item)) std::this_thread::yield();
            result += item;
        }
    });
    producer.join();
    consumer.join();
    assert(result == 1LL * Count * (Count + 1) / 2);
}
```

Listing 16.8: Kompletna kolejka SPSC dla int, ze statyczną pamięcią (C++20).

16.9.1 Dlaczego kod jest poprawny?

Dla danych w `buffer_` nie stosujemy operacji atomowych. Nie ma jednak konfliktowego dostępu bez synchronizacji, ponieważ:

1. przed odczytem komórki konsument otrzymuje informację o zapisie przez `head_.load(acquire)`;
2. przed ponownym nadpisaniem tej komórki producent otrzymuje informację o jej zwolnieniu przez `tail_.load(acquire)`;
3. każdy z liczników ma jednego piszącego, więc nie potrzeba CAS do aktualizowania `head_` ani `tail_`.

Przykład zakłada, że operacji `try_push` nie wywołuje kilku producentów, a `try_pop` kilku konsumentów, nawet jeśli same metody są dostępne publicznie. Złamane założenie SPSC powoduje utratę poprawności. Ograniczenia bufora, przepełnienie liczników i kwestie typów niebanalnych wymagają osobnej analizy. Użyliśmy potęgi dwójki, żeby indeksowanie przez maskę pozostało poprawne także przy modularnym zawijaniu licznika bez znaku.

Uwaga. Ten przykład jest edukacyjną, *nieblokującą* konstrukcją w sensie, że nie przejmuje mutexa, ale aktywne pętle producenta i konsumenta mogą czekać dowolnie długo. Metody `try_push` i `try_pop` są krótkie i nie wykonują pętli CAS. Bez uzasadnienia własności użytych atomików na danej platformie nie deklarujemy silniejszej gwarancji wydajnościowej.

16.10 Atomowa publikacja niezmiennych wersji danych

Wiele rzeczywistych aplikacji (konfiguracja serwera, słowniki, parametry modeli) posiada licznych czytelników i rzadkich pisarzy. Dobrą architekturą bywa utworzenie nowej, niezmiennej wersji struktury i jednorazowe opublikowanie wskaźnika do niej. W C++20 można do tego wykorzystać specjalizację `std::atomic<std::shared_ptr<T>>`.

```
#include <atomic>
#include <cassert>
#include <memory>
#include <thread>

struct Config {
    int threshold;
    double scale;
};

int main() {
    using Ptr = std::shared_ptr<const Config>;
    std::atomic<Ptr> current{
        std::make_shared<const Config>(Config{10, 1.0})
    };

    std::jthread writer([&] {
        Ptr next = std::make_shared<const Config>(Config{20, 2.0});
        current.store(std::move(next), std::memory_order_release);
    });
    std::jthread reader([&] {
        Ptr snap = current.load(std::memory_order_acquire);
        assert((snap->threshold == 10 && snap->scale == 1.0) ||
               (snap->threshold == 20 && snap->scale == 2.0));
    });
    writer.join();
    reader.join();
}
```

Listing 16.9: Bezpieczny snapshot konfiguracji (C++20).

Czytelnik otrzymuje *spójny* snapshot obu pól, a stara wersja jest automatycznie utrzymywana przy życiu, dopóki istnieją do niej uchwyty `shared_ptr`. Nie oznacza to, że przechowywanie `shared_ptr` w atomiku jest *lock-free*. Referencyjne liczenie i alokacje bywają istotnym kosztem przy bardzo częstych odczytach. Dla obciążenia z dużą przewagą czytelników należy porównać to podejście z `shared_mutex`, kopiowaniem małych struktur i technikami RCU.

16.10.1 `std::atomic_ref`: synchronizacja istniejącego obiektu

`std::atomic_ref<T>` (C++20) pozwala wykonywać operacje atomowe na istniejącym, odpowiednio wyrównanym obiekcie typu `T`. Przydatne bywa przy interoperacyjności ze strukturami danych, które nie przechowują `std::atomic<T>` jako pola.

```
#include <atomic>
#include <cassert>
#include <thread>

int main() {
    alignas(std::atomic_ref<int>::required_alignment) int counter = 0;
    {
        std::atomic_ref<int> shared_counter(counter);
        auto add = [&] {
            for (int i = 0; i < 10000; ++i)
```

```
                shared_counter.fetch_add(1, std::memory_order_relaxed);
        };
        std::jthread a(add), b(add);
        a.join();
        b.join();
    } // koniec zycia atomic_ref
    assert(counter == 20000);
}
```

Listing 16.10: Operacje atomowe na istniejącej zmiennej (C++20).

Dopóki istnieje `atomic_ref` odnoszący się do obiektu, dostęp do tego obiektu należy realizować przez instancje `atomic_ref` — nie tylko w chwilach równoczesnego dostępu. Dlatego zwykłe odczytanie `counter` wykonujemy *po* zakończeniu czasu życia referencji atomowej. Sam obiekt musi żyć dostatecznie długo, a jego wyrównanie musi spełniać wymagania `required_alignment`.

16.11 Planowanie zadań: pule wątków, *work stealing* i *backpressure*

Poza poprawną synchronizacją potrzebujemy rozsądnej strategii przypisywania prac do wątków. Wątek systemowy ma koszt utworzenia, przełączenia kontekstu i utrzymania stosu. Z tego powodu tysiące krótkich zadań nie powinny automatycznie oznaczać tysiąca wątków.

Thread pool
: Ustalona liczba wątków pobiera zadania z kolejki. Ogranicza koszt częstego tworzenia wątków i może ograniczać równoległe obciążenie.

Work stealing
: Każdy wątek ma własną kolejkę, a bezczynny pracownik próbuje przejąć część pracy od innych. Dobrze sprawdza się przy nierównych czasach trwania podzadań. Wymaga ostrożnego protokołu synchronizacji kolejek, szczególnie gdy korzystają z nich różni producenci.

Backpressure
: Mechanizm spowalniania producenta, gdy konsumenci nie nadążają. Bez niego kolejka bez ograniczenia pojemności może zużyć całą pamięć.

Structured concurrency
: Sposób organizacji zadań, w którym ich czas życia, anulowanie i błędy pozostają związane z zakresem nadrzędnej operacji. Sam `std::async` nie implementuje kompletnego frameworka takiego stylu.

16.11.1 Pula wątków: bezpieczne zamknięcie i propagacja wyjątków

Następny przykład rozwija pulę z rozdziału 1. Używa mutexa oraz zmiennej warunkowej. Łatwy dowód poprawności i przewidywalne zachowanie przy zamykaniu są często ważniejsze niż niepotwierdzony *lock-free*.

```
#include <condition_variable>
#include <functional>
#include <future>
#include <iostream>
#include <memory>
#include <mutex>
#include <queue>
#include <stdexcept>
#include <thread>
```

```cpp
#include <type_traits>
#include <utility>
#include <vector>

class ThreadPool {
    std::mutex mutex_;
    std::condition_variable cv_;
    std::queue<std::function<void()>> jobs_;
    bool closed_ = false;
    std::vector<std::jthread> workers_;

    void worker_loop() {
        for (;;) {
            std::function<void()> job;
            {
                std::unique_lock lock(mutex_);
                cv_.wait(lock, [&] {
                    return closed_ || !jobs_.empty();
                });
                if (closed_ && jobs_.empty()) return;
                job = std::move(jobs_.front());
                jobs_.pop();
            }
            job();  // packaged_task przechwyci wyjatek
        }
    }

public:
    explicit ThreadPool(unsigned count) {
        if (count == 0) throw std::invalid_argument("count=0");
        try {
            for (unsigned i = 0; i < count; ++i)
                workers_.emplace_back([this] { worker_loop(); });
        } catch (...) {
            close();  // zwolnij pracownikow przed rozwijaniem stosu
            throw;
        }
    }
    ThreadPool(const ThreadPool&) = delete;
    ThreadPool& operator=(const ThreadPool&) = delete;
    ~ThreadPool() { close(); }

    void close() {
        {
            std::lock_guard lock(mutex_);
            closed_ = true;
        }
        cv_.notify_all();
    }

    template<class F>
    auto submit(F&& callable)
        -> std::future<std::invoke_result_t<std::decay_t<F>&>> {
        using R = std::invoke_result_t<std::decay_t<F>&>;
        auto task = std::make_shared<std::packaged_task<R()>>(
            std::forward<F>(callable)
        );
        auto result = task->get_future();
        {
            std::lock_guard lock(mutex_);
            if (closed_) throw std::runtime_error("pool closed");
            jobs_.emplace([task] { (*task)(); });
        }
        cv_.notify_one();
```

```
        return result;
    }
};

int main() {
    ThreadPool pool(2);
    auto a = pool.submit([] { return 40; });
    auto b = pool.submit([] { return 2; });
    std::cout << a.get() + b.get() << '\n'; // 42
    pool.close(); // od tej chwili bez nowych zadan
}
```

Listing 16.11: Prosta pula wątków odprowadzająca kolejkę przed zamknięciem (C++20).

Ważne własności pokazanej implementacji:

- `close()` jest bezpieczne wobec równoległego `submit()`: oba działania są serializowane przez ten sam mutex;
- po `close()` pracownicy *kończą kolejkę*, a dopiero następnie opuszczają pętlę; destruktory `jthread` dołączają pracowników, zanim zniszczone zostaną mutex i kolejka;
- `packaged_task` zapisuje wartość lub wyjątek w stanie asynchronicznym, więc zwykłe wyjątki z zadań nie wychodzą bezpośrednio przez `worker_loop`;
- ta minimalna pula nie ma ograniczenia długości kolejki, priorytetów, terminów ani anulowania uruchomionych prac.

Uwaga. Nie wolno niszczyć takiej puli z zadania wykonywanego w tej samej puli: dołączenie własnego wątku prowadzi do błędu. Podobnie zadanie czekające na inne zadanie z tej samej puli może zakleszczyć program, jeśli wszyscy pracownicy pozostają zajęci oczekiwaniem. Produkcyjny executor wymaga rozwiązań dla takich sytuacji.

16.11.2 Kiedy potrzebny jest *work stealing*?

Załóżmy, że przetwarzamy drzewo lub dzielimy dużą macierz na bloki. Część bloków kończy się po mikrosekundach, inne zajmują milisekundy. Statyczny przydział zadań może zostawić jeden wątek przeciążony, gdy inne nie mają już pracy. Przechwytywanie pracy pomiędzy lokalnymi kolejkami poprawia *równoważenie obciążenia*.

Typowy projekt stosuje:

1. właścicielski dostęp pracownika do jednego końca jego kolejki;
2. kradzież elementów przez inne wątki z drugiego końca;
3. atomowe indeksy i protokół obsługujący konflikt właściciela ze złodziejem o ostatni element;
4. kontrolę czasu życia tablicy zadań oraz jej ewentualnej realokacji.

Takie struktury są znacznie bardziej subtelne niż `std::deque` z mutexem. W praktyce warto użyć sprawdzonego executora/biblioteki, a własną implementację traktować jako osobny, formalnie weryfikowany projekt.

16.12 Anulowanie, sygnały stopu i punkty oczekiwania

Kooperacyjne zatrzymywanie w C++20 jest oparte na `std::stop_source`, `std::stop_token` i `std::stop_callback`. Żądanie zatrzymania nie zabija wątku; umożliwia mu dobrowolne zakończenie po zweryfikowaniu stanu i odpowiednim uporządkowaniu zasobów.

```cpp
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <thread>

int main() {
    std::mutex mutex;
    std::condition_variable_any cv;
    bool ready = false;

    std::jthread worker([&](std::stop_token stop) {
        std::unique_lock lock(mutex);
        bool completed = cv.wait(lock, stop, [&] { return ready; });
        std::cout << (completed ? "ready\n" : "cancelled\n");
    });

    worker.request_stop();
    worker.join();
}
```

Listing 16.12: Wybudzanie i anulowanie czekania (C++20).

W tym przeciążeniu `wait` zwraca wartość predykatu po zakończeniu czekania. Jeśli żądanie zatrzymania nastąpi przed spełnieniem predykatu, przykład wypisze `cancelled`. Jeśli stan zmieniłby się w międzyczasie, wynik mógłby być inny. Zwykła `std::condition_variable` nie posiada w C++20 odpowiednika tego przeciążenia z `stop_token`.

16.12.1 Anulowanie i bezpieczeństwo wyjątkowe

W dobrym API warto określić, co dzieje się z wynikiem po anulowaniu:

- czy zwracany jest częściowy wynik;
- czy do `future` trafia własny wyjątek oznaczający anulowanie;
- czy potrzebne jest odwrócenie skutków wcześniej wykonanych operacji;
- jak zapewnić, że wszystkie zależne zadania zostaną zakończone przed zniszczeniem współdzielonych danych.

Żadne z tych pytań nie ma jednej odpowiedzi dla wszystkich bibliotek. Sama metoda `future.wait_for()` nie anuluje zadań, a korutyna z rozdziału 15 wymaga mechanizmu propagacji żądania anulowania przez awaitery i executor.

16.13 Fałszywe współdzielenie (*false sharing*) i cache coherence

Dwa niezależne atomiki mogą być zapisane blisko siebie w pamięci, tak że mieszczą się w tej samej jednostce przenoszenia danych pomiędzy pamięciami podręcznymi. Wtedy zapis przez różne rdzenie może wymagać kosztownych transferów linii cache, choć z punktu widzenia C++ nie ma wyścigu danych.

```cpp
#include <atomic>
#include <cstdint>
#include <iostream>
#include <new>
#include <thread>

struct alignas(std::hardware_destructive_interference_size) Counter {
```

```
    std::atomic<std::uint64_t> value{0};
};

int main() {
    Counter a, b;
    auto increment = [](Counter& c) {
        for (int i = 0; i < 100000; ++i)
            c.value.fetch_add(1, std::memory_order_relaxed);
    };
    std::jthread first([&] { increment(a); });
    std::jthread second([&] { increment(b); });
    first.join();
    second.join();
    std::cout << a.value << ' ' << b.value << '\n';
}
```

Listing 16.13: Rozdzielenie intensywnie zapisywanych liczników (C++17).

`std::hardware_destructive_interference_size` jest wskazówką implementacji dotyczącą rozdzielania obiektów często modyfikowanych przez różne wątki; w praktyce jej obsługa zależy od kompilatora i biblioteki standardowej. `alignas` może zwiększać zużycie pamięci, zatem nie należy bez pomiarów wyrównywać w ten sposób każdego pola. Rozdział 10 omawia szerzej cache locality i organizację danych AoS/SoA.

16.13.1 Co mierzyć przy ocenie skalowalności?

Pojedynczy czas wykonania jest mało informacyjny. Przy porównywaniu wersji jednowątkowej i wielowątkowej warto mierzyć:

- przepustowość (operacje/sekundę), a nie tylko opóźnienie pojedynczego zadania;
- *speedup* $S(p) = T(1)/T(p)$, dla p wątków;
- rozkład opóźnień, np. medianę i wysokie percentyle;
- liczbę alokacji, przełączeń kontekstu, braków w cache i konfliktów CAS;
- oddzielnie okres rozgrzewki oraz właściwy okres pomiarowy.

Według prawa Amdahla, jeśli część s programu musi pozostać sekwencyjna, to w idealizacji

$$S(p) \le \frac{1}{s + \frac{1-s}{p}}.$$

Wzór ten nie obejmuje kosztów synchronizacji, a więc rzeczywiste przyspieszenie może być jeszcze niższe. Jednocześnie program z dominującym kosztem dostępu do pamięci może nie skalować się nawet przy bardzo małej części sekwencyjnej.

16.14 Linearizowalność i niezmienniki współdzielonych struktur

Dla współbieżnych kolekcji często potrzebujemy mocniejszej własności niż brak wyścigów danych: *linearizowalności*. Operacja jest linearizowalna, jeśli można przypisać jej jeden umowny moment pomiędzy wywołaniem i zakończeniem, w którym wywołanie ma taki sam skutek jak operacja wykonana sekwencyjnie.

Dla prostej atomowej inkrementacji punktem linearyzacji jest sam RMW. Dla kolejki algorytm jest bardziej subtelny:

- producent najpierw zapisuje komórkę, a następnie publikuje nowy `head`;

- konsument najpierw odczytuje i przetwarza komórkę, a następnie publikuje nowy `tail`;
- sama komórka nie musi być atomowa, o ile zachowana jest właściwa synchronizacja i protokół własności miejsca w buforze.

W algorytmach złożonych, np. z dwoma atomikami `size` i `head`, każde z pól może być wolne od wyścigu danych, ale ich jednoczesny odczyt nie musi stanowić spójnego snapshotu. Osobny *niezmiennik* struktury wymaga własnego uzasadnienia.

16.14.1 Dlaczego prosty *seqlock* nie wystarczy dla zwykłych danych?

Popularny pomysł polega na tym, że pisarz zwiększa licznik wersji przed i po aktualizacji obiektu, a czytelnik ponawia odczyt, jeżeli wersja zmieniła się podczas odczytu. Jeżeli pola obiektu są zwykłymi zmiennymi C++, czytelnik nadal może czytać je jednocześnie z zapisem, co oznacza *data race*. Ponowienie odczytu *po fakcie* nie naprawia niezdefiniowanego zachowania, które już nastąpiło.

Poprawny wariant dla przenośnego C++ musi zastosować odpowiednią synchronizację lub atomowe pola danych, a nie tylko atomowy licznik wersji. Alternatywą jest pokazana wcześniej publikacja niezmiennych snapshotów.

16.15 Współbieżność a parallel STL, Ranges i OpenMP

Nie każda równoległa praca wymaga własnej kolejki czy atomowych liczników. Gdy zadanie polega na niezależnym przetwarzaniu elementów, lepszą abstrakcją może być algorytm równoległy lub dyrektywa OpenMP.

16.15.1 Polityki wykonania standardowych algorytmów

Nagłówek `<execution>` udostępnia od C++17 między innymi:

> Wykonanie sekwencyjne.
>
> Implementacja może wykonywać pracę równolegle; nie daje gwarancji, że rzeczywiście użyje wielu wątków.
>
> Zezwala dodatkowo na niesekwencyjne wykonywanie, w tym wektoryzację; nakłada ważne ograniczenia na wykonywane funkcje i synchronizację.

```
#include <algorithm>
#include <execution>
#include <numeric>
#include <vector>

int main() {
    std::vector<double> a(10000), b(a.size());
    std::iota(a.begin(), a.end(), 0.0);
    std::transform(std::execution::par,
                   a.begin(), a.end(), b.begin(),
                   [](double x) { return x * x; });
}
```

Listing 16.14: Równoległa transformacja bez współdzielonych zapisów (C++17).

Kod nie zakłada kolejności przetwarzania elementów. Wyjściowy zakres `b` nie nakłada się z `a`, a każde wywołanie funkcji zapisuje inny element. Rzeczywiste zrównoleglenie zależy od implementacji biblioteki standardowej i konfiguracji środowiska; niektóre kompilacje wymagają dodatkowej biblioteki wykonawczej.

16.15.2 C++ i OpenMP: dwa poziomy abstrakcji

OpenMP, omawiany szczegółowo w rozdziale 1, umożliwia między innymi równoległe pętle, redukcje i zależności pomiędzy zadaniami. W obliczeniach numerycznych typowe rozwiązanie wygląda tak:

```
#include <iostream>
#include <vector>

int main() {
    const std::vector<double> x(100000, 1.0);
    double sum = 0.0;

    #pragma omp parallel for reduction(+:sum) schedule(static)
    for (int i = 0; i < static_cast<int>(x.size()); ++i) {
        sum += x[static_cast<std::size_t>(i)];
    }
    std::cout << sum << '\n';
}
```

Listing 16.15: Wektorowa redukcja z OpenMP; wymagane -fopenmp.

Kodowanie redukcji przez `reduction` pozwala uniknąć współdzielenia jednego licznika zapisywanego przez wszystkie wątki. Kolejność sumowania liczb zmiennoprzecinkowych może być odmienna niż przy obliczeniu sekwencyjnym, a zatem wynik może różnić się przez zaokrąglenia. Liczba wątków i harmonogram OpenMP mogą być kontrolowane m.in. przez `OMP_NUM_THREADS`, `OMP_SCHEDULE` i funkcje biblioteczne OpenMP.

Uwaga. Mieszanie pul wątków, OpenMP, równoległych algorytmów STL oraz zadań asynchronicznych może prowadzić do *oversubscription*: liczba aktywnych wątków znacząco przewyższa dostępne zasoby CPU. Przy nakładaniu kilku warstw równoległości trzeba świadomie kontrolować budżet wątków i mierzyć efekt końcowy.

16.16 Diagnostyka: kompilatory, sanitizatory, testy przeciążeniowe

16.16.1 Kompilacja i uruchamianie przykładów

Dla przykładów wykorzystujących `std::jthread`, `atomic::wait` i `atomic<shared_ptr<T>>` wybieramy tryb C++20:

```
g++ -std=c++20 -O2 -Wall -Wextra -Wpedantic -pthread file.cpp -o app
./app
```

Listing 16.16: Kompilacja wielowątkowego programu C++20.

Dla przykładów OpenMP w GCC lub odpowiednio skonfigurowanym Clangu:

```
g++ -std=c++20 -O2 -Wall -Wextra -fopenmp file.cpp -o app
./app
```

Listing 16.17: Włączenie OpenMP.

16.16.2 ThreadSanitizer i inne narzędzia

ThreadSanitizer (TSan) potrafi wykrywać wiele wyścigów danych i wybrane błędy synchronizacji w uruchamianym programie:

```
g++ -std=c++20 -O1 -g -pthread -fsanitize=thread file.cpp -o app_tsan
```

```
./app_tsan
```

Listing 16.18: Budowanie i uruchamianie z ThreadSanitizer.

Nie wszystkie środowiska poprawnie wspierają TSan; szczególnie przy nietypowych bibliotekach wykonawczych, OpenMP i własnych mechanizmach synchronizacji należy weryfikować zakres instrumentacji. AddressSanitizer i UndefinedBehaviorSanitizer mogą dodatkowo wykrywać część błędów czasu życia oraz naruszeń zasad języka, lecz nie są zamiennikami TSan ani formalnego dowodu synchronizacji.

16.16.3 Jak testować algorytm współbieżny?

Dla poważniejszej struktury danych zaleca się następującą procedurę:

`std::execution::seq` `std::execution::par_unseq`

1. **Zdefiniuj kontrakt**: ilu producentów i konsumentów, dozwolone operacje oraz zasady zamknięcia i niszczenia struktury.
2. **Zidentyfikuj własność danych**: kto może zapisywać dane w każdej chwili; kiedy przechodzi własność konkretnej komórki.
3. **Narysuj relacje**: wskaż atomik, który publikuje zapis i odczyt, który tę publikację odbiera.
4. **Wskaż punkt linearyzacji**: jeśli API ma być linearizowalne, określ, kiedy operacja staje się widoczna logicznie.
5. **Przeanalizuj zakończenie**: co dzieje się przy anulowaniu, wyjątku, opóźnieniu konsumenta i niszczeniu obiektu.
6. **Testuj wielokrotnie**: zmieniaj liczbę wątków, rozmiary danych i częstotliwość przełączeń; uruchamiaj TSan tam, gdzie jest dostępny.
7. **Benchmarkuj osobno**: włącz optymalizacje i mierz przy realistycznym obciążeniu, ale nie myl szybkiego testu z dowodem poprawności.

16.17 Typowe błędy i zalecenia projektowe

Błąd	Dlaczego jest groźny / co wybrać
`volatile` jako synchronizacja	Nie eliminuje *data race*; użyj mutexa lub atomików z właściwym porządkiem.
`relaxed` do publikacji zwykłych danych	Nie wytwarza relacji synchronizacji; zazwyczaj potrzebne `release/acquire`.
Bezrefleksyjne `seq_cst`	Zwykle poprawniejsze niż źle dobrane słabsze porządki, ale nie zastąpi logicznego kontraktu obiektu.
Kasowanie węzła po udanym CAS	Inny wątek może nadal używać wskaźnika; zaprojektuj odzyskiwanie pamięci.
Dwa atomiki jako „jedna transakcja”	Brak automatycznego atomowego snapshotu pary; użyj blokady lub niezmiennej wersji.
Spinlock dla długiej pracy	Prowadzi do rywalizacji o CPU; zwykle użyj `std::mutex`.
Nieograniczona kolejka zadań	Może zużyć pamięć; dodaj limit i mechanizm *backpressure*.
Nadmierna liczba wątków	Konkurencja o rdzenie i cache; kontroluj równoległość i profiluj.
Mieszanie OpenMP i własnej puli bez planu	Może prowadzić do *oversubscription* i spadku wydajności.

16.17.1 Praktyczne reguły wyboru

- **Mutex** wybieraj jako punkt wyjścia, jeśli współdzielisz kilka powiązanych pól i potrzebujesz łatwo sprawdzalnych niezmienników.
- **Atomik** `relaxed` jest właściwy dla prostego licznika niezależnego od publikacji innych danych.
- `release/acquire` stosuj, gdy istnieje wyraźny punkt przekazania danych lub własności pomiędzy wątkami.
- **Lock-free** rozważaj dopiero wtedy, gdy pomiary pokażą realny koszt blokad, a projekt ma przejrzysty dowód poprawności oraz plan odzyskiwania pamięci.
- **OpenMP / parallel STL** wybieraj do operacji na niezależnych elementach, redukcji i równoległości danych.
- **Thread pool** wybieraj do dużej liczby krótkich zadań, gdy potrzebujesz ograniczać liczbę wątków, priorytety lub przepustowość; osobno zdefiniuj zasady zamykania.

Podsumowanie rozdziału

Zaawansowana współbieżność nie polega na mechanicznym zastępowaniu mutexów przez `std::atomic`. Najpierw trzeba określić *własność danych i kontrakt algorytmu*, następnie uzasadnić relacje *happens-before* i wyznaczyć punkty widoczności poszczególnych operacji. Dopiero wtedy warto wybierać porządki pamięciowe, typy kolejek, planistę i sposób odzyskiwania pamięci.

W szczególności brak wyścigu danych nie oznacza jeszcze poprawności logicznej, a nieblokujący dostęp do atomika nie gwarantuje nieblokującego ani szybkiego zachowania całego programu. Bezpieczna współbieżność to połączenie *modelu formalnego*, *kontroli czasu życia* i *rzetelnego pomiaru*.

Literatura i dokumentacja

- Anthony Williams, *C++ Concurrency in Action*, wyd. 2, Manning, 2019.
- ISO/IEC 14882:2020, rozdziały o modelu pamięci, wątkach, atomikach i synchronizacji.
- https://en.cppreference.com/w/cpp/atomic/memory_order — porządki pamięci i relacje synchronizacji.
- https://en.cppreference.com/w/cpp/atomic/atomic — typy atomowe i operacje RMW.
- https://en.cppreference.com/w/cpp/atomic/atomic_ref — atomowe odwołania do istniejących obiektów.
- https://en.cppreference.com/w/cpp/thread/jthread — `jthread`, czas życia i kooperacyjne zatrzymywanie.
- OpenMP Architecture Review Board, *OpenMP Application Programming Interface*; https://www.openmp.org/specifications/.

Rozdział 17

Kompilacja, linkowanie, ABI i moduły C++20

17.1 Dlaczego trzeba rozumieć proces budowania?

W poprzednich rozdziałach projektowaliśmy klasy, szablony, mechanizmy zarządzania pamięcią i współbieżność. Dopóki cały program mieści się w jednym pliku, wystarcza zwykle wywołanie `g++ main.cpp -o app`. W realnym projekcie kod dzieli się jednak na dziesiątki lub tysiące *jednostek translacji*, biblioteki zewnętrzne i moduły. To wtedy pojawiają się komunikaty `undefined reference`, `multiple definition`, konflikty wersji bibliotek, długi czas budowania oraz błędy ujawniające się dopiero po wymianie biblioteki współdzielonej.

Musimy rozdzielić trzy różne zagadnienia:

Poprawność programu w języku C++
: Czy deklaracje, definicje, typy i wywołania spełniają reguły C++, w szczególności *One Definition Rule* (ODR)?

Model budowania i linkowania
: Jakie pliki są kompilowane osobno, gdzie kompilator zapisuje symbole, w jakiej kolejności linker przeszukuje biblioteki oraz jak ładowane są biblioteki dynamiczne?

Zgodność binarna (ABI)
: Czy dwa osobno zbudowane komponenty zgadzają się co do reprezentacji typów, sposobu wywoływania funkcji, nazw symboli i pozostałych reguł interfejsu binarnego?

Uwaga. Norma ISO C++ opisuje semantykę programu, ale nie narzuca jednego formatu pliku obiektowego, konwencji *name mangling*, organizacji bibliotek `.so`/`.dll` ani poleceń linkera. Poniżej odróżniamy reguły języka od typowych rozwiązań narzędziowych.

17.2 Od pliku źródłowego do programu

17.2.1 Preprocesor, kompilator, asembler, linker i loader

W uproszczeniu klasyczny projekt z nagłówkami przechodzi przez etapy:

$$\text{źródła i nagłówki} \longrightarrow \text{preprocessing} \longrightarrow \text{kompilacja} \longrightarrow \text{plik obiektowy} \longrightarrow \text{linkowanie} \longrightarrow \text{program}.$$

Na niektórych platformach kompilator wytwarza tekst asemblerowy (`.s`) jako etap pośredni; w innych

przejście do kodu maszynowego jest zintegrowane. Linkowanie może odbywać się przy budowaniu programu, a dodatkowe rozwiązywanie symboli może następować w czasie ładowania lub później.

Prześledźmy etapy na samodzielnym programie.

```
#include <iostream>

int twice(int x) { return 2 * x; }

int main() {
    std::cout << twice(21) << '\n';
}
```

Listing 17.1: Plik hello.cpp: prosty program do obserwowania kompilacji.

```
g++ -std=c++20 -E hello.cpp -o hello.ii  # Preprocesor
# Produkt poprzedniego etapu moze byc duzy.
g++ -std=c++20 -S hello.cpp -o hello.s   # Asembler

g++ -std=c++20 -c hello.cpp -o hello.o   # Kod obiektowy
g++ hello.o -o hello                      # Linkowanie
./hello                                   # 42
```

Listing 17.2: Wybrane etapy budowania przy uzyciu GCC/Clang.

Opcja `-c` nie tworzy wykonywalnego programu; produkuje plik obiektowy, który może zawierać nierozwiązane odwołania do symboli pochodzących z innych jednostek translacji. Polecenie `-S` kończy budowanie na asemblerze, a `-E` na wstępnym przetwarzaniu. Warto odróżnić program sterujący kompilacją (np. `g++`) od samego frontendu C++, asemblera i linkera.

17.2.2 Pliki obiektowe, symbole i relokacje

Plik obiektowy zawiera zwykle kod maszynowy, dane, tablice symboli oraz *relokacje*, czyli informacje, które adresy muszą zostać skorygowane po ustaleniu ostatecznego układu programu. W środowisku Linux powszechny jest format ELF; Windows używa PE/COFF, a macOS Mach-O. Szczegóły tych formatów nie są częścią języka C++.

```
nm -C hello.o                # Symbole; -C demangluje nazwy C++
readelf -Ws hello.o          # Tablica symboli ELF
readelf -r hello.o           # Relokacje
objdump -drC hello.o         # Asembler i relokacje
```

Listing 17.3: Ogladanie symboli i relokacji (Linux/ELF).

Przykładowo litera `T` w typowym wyjściu `nm` informuje o symbolu definiowanym w sekcji kodu, a `U` o symbolu niezdefiniowanym w danym obiekcie, który trzeba rozwiązać. Nie należy traktować oznaczeń `nm` jako przenośnej własności C++.

17.2.3 Jednostka translacji i kompilacja oddzielna

W tradycyjnym modelu *jednostka translacji* (*translation unit*, TU) powstaje z pliku źródłowego po uwzględnieniu dyrektyw preprocesora, w szczególności zawartości dołączanych nagłówków. Każda TU jest analizowana niezależnie, a linker łączy otrzymane obiekty.

Rozważmy projekt:

```
// Plik: math.hpp
#ifndef CPP17_MATH_HPP
#define CPP17_MATH_HPP
```

```
int square(int x);
#endif

// Plik: math.cpp
#include "math.hpp"
int square(int x) { return x * x; }

// Plik: main.cpp
#include "math.hpp"
#include <iostream>
int main() { std::cout << square(7) << '\n'; }
```

Listing 17.4: Trzy pliki klasycznego projektu C++: API, implementacja i klient.

```
g++ -std=c++20 -Wall -Wextra -c math.cpp -o math.o
g++ -std=c++20 -Wall -Wextra -c main.cpp -o main.o
g++ main.o math.o -o app
./app                          # 49
```

Listing 17.5: Niezalezne budowanie TU i koncowe linkowanie.

`math.hpp` dostarcza deklarację klientowi i implementacji, ale nie jest zwykle oddzielnym obiektem linkowanym. Zmiana samego `math.cpp` nie musi wymuszać ponownej kompilacji `main.cpp`, o ile nie zmienił się kontrakt w nagłówku lub pozostałe zależności.

17.2.4 Przykładowe komendy w MSVC

W *Developer Command Prompt for Visual Studio* można zbudować analogiczny przykład za pomocą `cl.exe` i `link.exe`:

```
cl /std:c++20 /EHsc /c math.cpp
cl /std:c++20 /EHsc /c main.cpp
link main.obj math.obj /OUT:app.exe
app.exe
```

Listing 17.6: Kompilacja dwóch plików w środowisku MSVC/Windows.

Przy budowaniu programów C++ w Visual Studio warto również kontrolować zgodność wersji MSVC, ustawienia biblioteki uruchomieniowej (np. `/MD` wobec `/MT`) i konfigurację Debug/Release. Takie flagi nie mają bezpośrednich odpowiedników narzuconych przez standard języka; należą do konkretnego środowiska.

17.3 Nagłówki, makra i zasada jednej definicji

17.3.1 Include guards i `#pragma once`

Zawartość `#include` jest w uproszczeniu wstawiana do źródła przez preprocesor. Wielokrotne dołączenie tego samego nagłówka w jednej TU mogłoby więc prowadzić do powtórzenia definicji klasy. Chronimy się przez *include guard*:

```
#ifndef LIB_WIDGET_HPP
#define LIB_WIDGET_HPP

struct Widget {
    int value{};
};

#endif // LIB_WIDGET_HPP
```

Listing 17.7: Klasyczna os?ona nag?owka.

W praktyce popularna jest też dyrektywa `#pragma once`, obsługiwana przez najważniejsze kompilatory, lecz niestandaryzowana przez ISO C++. Oba rozwiązania chronią *jedną TU* przed powtórnym dołączeniem pliku. Nie rozwiązują automatycznie problemu *wielu definicji tego samego nie-inline symbolu w różnych TU*.

17.3.2 ODR: deklaracja kontra definicja

Deklaracja wprowadza nazwę i informację o typie, natomiast definicja zwykle dostarcza byt danej funkcji lub obiektu. Deklaracji pewnej funkcji może być wiele, ale *nie-inline* funkcja z zewnętrzną łącznością, używana w programie w sposób wymagający definicji, powinna mieć jedną definicję w programie. Reguły ODR zawierają wyjątki pozwalające na identyczne definicje w różnych TU, np. dla odpowiednich funkcji `inline`, klas i szablonów.

```
// BAD: wrong.hpp, dolaczony do dwoch plikow .cpp
int twice(int x) { return 2 * x; }
// Dwie zewnetrzne definicje -> naruszenie ODR.
```

Listing 17.8: Typowa pu?apka: definicja zwyk?ej funkcji w nag?owku.

```
// Plik: twice.hpp
#pragma once
inline int twice(int x) { return 2 * x; }
```

Listing 17.9: Definicja inline przeznaczona do nag?owka.

`inline` nie nakazuje kompilatorowi wstawienia instrukcji funkcji bezpośrednio w miejscu wywołania. Jego znaczenie dla organizacji programu polega przede wszystkim na dopuszczeniu definicji w wielu TU zgodnie z ODR (z odpowiednimi warunkami). Wszystkie takie definicje muszą być zgodne i spełniać reguły ODR; samo dopisanie `inline` nie naprawia różniących się implementacji w dwóch plikach.

17.3.3 Zmienne `extern`, `inline`, `const` i `static`

```
// Plik: settings.hpp
#pragma once
extern int request_limit;       // Tylko deklaracja

// Plik: settings.cpp
#include "settings.hpp"
int request_limit = 100;        // Definicja

// Plik: main.cpp
#include "settings.hpp"
int main() { return request_limit == 100 ? 0 : 1; }
```

Listing 17.10: Jedna definicja zmiennej globalnej, wiele deklaracji.

Od C++17 w nagłówkach można również definiować *zmienne inline*:

```
#pragma once
inline constexpr int default_timeout_ms = 250;
inline int global_counter = 0;
```

Listing 17.11: Zmienna inline w nag?owku (C++17).

W tym przypadku odpowiednie definicje we wszystkich TU mogą wskazywać na ten sam obiekt o zewnętrznej łączności. W kontraście zmienna `const` zdefiniowana w przestrzeni nazw bez `extern` ma w typowym przypadku łączność wewnętrzną; oddzielne TU mogą mieć wówczas odrębne egzemplarze. `static` na poziomie przestrzeni nazw także zapewnia łączność wewnętrzną, lecz na współczesnym C++ zwykle preferujemy anonimową przestrzeń nazw.

```
// Plik: helper.cpp
namespace {
    int internal_counter = 0;
    int twice_local(int x) { return x * 2; }
}

int perform(int x) {
    ++internal_counter;
    return twice_local(x);
}
```

Listing 17.12: Nazwy pomocnicze ograniczone do jednej TU.

17.3.4 Szablony: dlaczego definicje zwykle znajdują się w nagłówkach?

Kompilator instancjuje szablon dla określonych argumentów typowych w miejscu, w którym wymagana jest jego definicja. Samo udostępnienie deklaracji z pliku `.hpp` i ukrycie ogólnej definicji szablonu w oddzielnym `.cpp` może zatem prowadzić do błędu linkera.

```
// Plik: multiply.hpp
#pragma once

template<class T>
T multiply(T a, T b) {
    return a * b;
}
```

Listing 17.13: Najprostszy, tradycyjny sposob udostepniania szablonu.

Można także jawnie instancjować *konkretne* typy w pliku implementacji. Oddzielne jednostki translacji nie muszą wtedy instancjować tego samego kodu niezależnie:

```
// Plik: operation.hpp
#pragma once
template<class T>
T add(T a, T b);
extern template int add<int>(int, int);

// Plik: operation.cpp
#include "operation.hpp"
template<class T>
T add(T a, T b) { return a + b; }
template int add<int>(int, int);

// Plik: main.cpp
#include "operation.hpp"
int main() { return add(10, 32) == 42 ? 0 : 1; }
```

Listing 17.14: Jawna instancjacja okreslonego wariantu szablonu.

W tym przykładzie klient może korzystać z `add<int>`, ale instancjacja innych typów nie zostaje mu udostępniona automatycznie. Definicje szablonów umieszczane w interfejsach modułów podlegają nieco innemu modelowi widoczności, który omówimy dalej.

17.3.5 Naruszenie ODR bez łatwego komunikatu

Nie wszystkie naruszenia ODR skutkują natychmiastowym błędem linkera. Szczególnie groźne jest warunkowe definiowanie tej samej klasy w dwóch TU w inny sposób, np. przy użyciu sprzecznych makr kompilacji:

```
// Plik: config.hpp
struct Config {
#ifdef EXTRA_FIELD
    int extra;
#endif
    int value;
};

// Jedna TU z -DEXTRA_FIELD, inna bez tej flagi:
// rozne definicje klasy -> naruszenie ODR.
```

Listing 17.15: Przyk?ad niezgodnych definicji typu (celowo b?edny).

Taki program może się nawet zlinkować, lecz pozostaje niepoprawny. Dlatego opcje zmieniające publiczne definicje klas i struktur muszą być spójne we wszystkich TU danego programu.

17.4 Symbole, łączność i typowe błędy linkera

17.4.1 Łączność a widoczność symbolu

Linkage w języku C++ mówi, czy deklaracje w różnych zakresach i TU mogą nazywać tę samą encję. W praktyce spotykamy między innymi łączność zewnętrzną, wewnętrzną, a w jednostkach modułów także łączność modułową. Nie należy jednak utożsamiać tej własności języka z *visibility* na poziomie ELF ani z eksportem symbolu z biblioteki dynamicznej.

Mechanizm	Co określa?
`static` w przestrzeni nazw	Łączność wewnętrzna nazwy w TU.
`extern`	M.in. deklarowanie obiektu definiowanego w innym miejscu.
Anonimowa przestrzeń nazw	Wewnętrzna łączność elementów pomocniczych.
`inline`	Warunki wielokrotnych definicji zgodnych z ODR.
`-fvisibility=hidden`	Domyślna widoczność symboli w typowych bibliotekach ELF.
`__declspec(dllexport)`	Udostępnianie symboli biblioteki DLL w MSVC.

17.4.2 Name mangling i `extern "C"`

C++ dopuszcza przeciążanie funkcji, przestrzenie nazw oraz rozbudowane typy szablonowe. Aby móc odróżniać symbole po linkowaniu, implementacje zwykle kodują część tych informacji w nazwach binarnych (*name mangling*). Dokładny sposób kodowania nie jest ustalony przez ISO C++.

```
namespace calc {
    int f(int x) { return x + 1; }
    double f(double x) { return x + 1.0; }
}

extern "C" int api_add(int a, int b) {
    return a + b;
}
```

Listing 17.16: Przeciazanie i punkt wejscia C ABI.

Na typowej platformie ELF/Itanium ABI funkcje przeciążone otrzymują różne symbole zakodowane przez kompilator, podczas gdy `api_add` ma nazwę zgodną z łącznością języka C. Nie jest to gwarancja pełnej zgodności ABI z dowolną biblioteką C: znaczenie mają także deklaracje typów i konwencja wywołania ustalona dla danej platformy.

```
g++ -std=c++20 -c overload.cpp -o overload.o
nm overload.o
nm -C overload.o
c++filt _Z3fooi      # Ilustracja popularnego schematu manglingu
```

Listing 17.17: Demanglowanie i inspekcja symboli (przyk?ady narzedziowe).

17.4.3 `undefined reference`, `multiple definition`

Błąd `undefined reference` oznacza zwykle, że linker znalazł odwołanie do symbolu, lecz nie znalazł odpowiedniej definicji. Przykładowe przyczyny to pominięcie `math.o`, pominięcie biblioteki, niezgodny podpis funkcji albo brak jawnej instancjacji szablonu. Z kolei `multiple definition` sugeruje m.in. zwykłą funkcję lub zmienną zdefiniowaną w nagłówku dołączanym do wielu TU.

Przydatna lista kontrolna:

1. Sprawdź, czy wszystkie wymagane `.cpp` są kompilowane i linkowane.
2. Porównaj podpis funkcji: przestrzeń nazw, typy, kwalifikatory i ABI.
3. Upewnij się, że deklaracja i definicja pochodzą z tego samego interfejsu.
4. Sprawdź kolejność bibliotek statycznych i ich zależności.
5. Dla klas polimorficznych sprawdź brakujące definicje funkcji wirtualnych, w tym destruktora wirtualnego, który może wymagać definicji nawet jako funkcja `= 0`.
6. Dla szablonów sprawdź dostępność definicji w punkcie instancjacji lub odpowiednią instancjację jawną.

17.5 Biblioteki statyczne

Biblioteka statyczna jest typowo archiwum plików obiektowych. W Linux ma zwykle rozszerzenie `.a`, w MSVC `.lib`. Biblioteka sama nie jest osobnym procesem ani automatycznie tworzonym wątkiem. Linker wybiera z niej obiekty potrzebne do rozwiązania zależności i tworzy wynikowy program lub inną bibliotekę.

```
g++ -std=c++20 -O2 -c math.cpp -o math.o
ar rcs libmathstuff.a math.o

g++ -std=c++20 -O2 -c main.cpp -o main.o
g++ main.o -L. -lmathstuff -o app
./app
```

Listing 17.18: Budowanie i uzycie biblioteki statycznej (Linux/GCC).

W typowym poleceniu GNU linkera biblioteki archiwalne warto umieszczać *po* obiektach, które odwołują się do ich symboli. Przykładowo `g++ main.o -lmathstuff` jest zazwyczaj właściwszym porządkiem niż `g++ -lmathstuff main.o`. Linkowanie zależności cyklicznych bibliotek statycznych może wymagać ponownego wskazania biblioteki lub użycia grupy linkera (np. `--start-group`/`--end-group` w GNU ld); lepiej jednak unikać takich zależności architektonicznie.

Uwaga. Statyczna biblioteka nie oznacza automatycznie w pełni statycznego programu. Aplikacja może dalej korzystać z bibliotek współdzielonych systemu i standardowej biblioteki C++.

17.6 Biblioteki współdzielone i dynamiczne ładowanie

17.6.1 Budowanie biblioteki współdzielonej

Na systemach ELF biblioteki współdzielone są zazwyczaj plikami `.so`. Obiekty wykorzystywane w takich bibliotekach zwykle buduje się jako kod niezależny od położenia w pamięci (*PIC*).

```
g++ -std=c++20 -O2 -fPIC -c math.cpp -o math_pic.o
g++ -shared math_pic.o -o libmathstuff.so

g++ -std=c++20 -O2 main.cpp -L. -lmathstuff \
    '-Wl,-rpath,$ORIGIN' -o app
./app
```

Listing 17.19: Biblioteka .so oraz program klienta (Linux/GCC).

Wyrażenie `$ORIGIN` odnosi się w typowym loaderze ELF do katalogu, w którym znajduje się ładowany obiekt. Apostrofy w poleceniu powłoki zapobiegają wcześniejszej ekspansji `$ORIGIN` przez powłokę. W projekcie produkcyjnym należy przemyśleć ścieżki instalacyjne, pakowanie bibliotek i bezpieczeństwo ładowania.

Biblioteki współdzielone pozwalają wielu procesom mapować ich kod z tego samego pliku oraz wymieniać implementację bez konieczności pełnego linkowania programu w czasie jego budowania. Nie gwarantuje to jednak, że *dowolna* nowa wersja zachowa zgodność ABI.

17.6.2 Wyszukiwanie bibliotek: RPATH/RUNPATH, SONAME

Na Linux/ELF linker i loader korzystają z mechanizmów takich jak *SONAME*, *RPATH* i *RUNPATH*. Reguły wyszukiwania zależą od systemu i konfiguracji; środowisko wykonania może również uwzględniać ścieżki systemowe lub zmienne środowiskowe.

```
readelf -d ./app | grep -E 'NEEDED|RPATH|RUNPATH'
ldd ./app                  # Tylko dla zaufanych plikow
readelf -Ws libmathstuff.so
```

Listing 17.20: Diagnostyka zaleznosci dynamicznych (Linux).

Programów nieznanego pochodzenia nie należy lekkomyślnie przekazywać do narzędzi, które mogą uruchamiać kod lub loader; do statycznej inspekcji lepiej nadaje się `readelf`.

17.6.3 Eksportowanie symboli i kontrola API

Domyślne eksportowanie zbyt wielu symboli może powodować kolizje nazw i powiększać powierzchnię ABI. W ELF można budować bibliotekę z `-fvisibility=hidden`, a publiczne symbole jawnie oznaczać:

```
// Plik: api.hpp
#pragma once

#if defined(_WIN32)
  #if defined(MYLIB_BUILD_SHARED)
    #define MYLIB_API __declspec(dllexport)
  #elif defined(MYLIB_USE_SHARED)
```

```
    #define MYLIB_API __declspec(dllimport)
  #else
    #define MYLIB_API
  #endif
#elif defined(__GNUC__) || defined(__clang__)
  #define MYLIB_API __attribute__((visibility("default")))
#else
  #define MYLIB_API
#endif

extern "C" MYLIB_API int mylib_add(int, int);
```

Listing 17.21: Przenosne w granicach GCC/Clang/MSVC makro eksportu.

W MSVC zwykle stosuje się projekt DLL oraz odpowiednią bibliotekę importową; na macOS biblioteki mają najczęściej rozszerzenie `.dylib`. Flagi `-fPIC`, `-shared` i `-Wl,...` w powyższej postaci dotyczą konkretnego toolchainu, nie ISO C++.

17.6.4 `dlopen`, `dlsym` i granica interfejsu wtyczki

Bibliotekę można także załadować podczas działania procesu, bez linkowania jej wprost z aplikacją. Na POSIX często używa się `dlopen`/`dlsym`, natomiast na Windows odpowiedników `LoadLibrary`/`GetProcAddress`.

```
// Plik: plugin.cpp
extern "C" int plugin_square(int x) {
    return x * x;
}

// Plik: loader.cpp
#include <dlfcn.h>
#include <iostream>
#include <stdexcept>

int main() {
    void* lib = dlopen("./libplugin.so", RTLD_NOW);
    if (!lib) throw std::runtime_error(dlerror());

    using Fn = int (*)(int);
    void* address = dlsym(lib, "plugin_square");
    if (!address) {
        dlclose(lib);
        throw std::runtime_error("Missing plugin_square");
    }

    // POSIX dopuszcza konwersje dlsym -> wskaznik do funkcji.
    Fn square = reinterpret_cast<Fn>(address);
    std::cout << square(7) << '\n'; // 49
    dlclose(lib);
}
```

Listing 17.22: Prosta wtyczka C ABI: biblioteka i klient POSIX (dwa pliki).

```
g++ -std=c++20 -fPIC -shared plugin.cpp -o libplugin.so
g++ -std=c++20 loader.cpp -ldl -o loader
./loader
```

Listing 17.23: Budowanie przyk?adu wtyczki (Linux/ELF).

Uwaga. To przykład dla POSIX, a nie w pełni przenośna konstrukcja ISO C++. Również rzutowanie wyniku `dlsym` na wskaźnik funkcji opiera się na wymaganiach POSIX, a nie na ogólnej gwarancji C++. Dodatkowo nie wolno wyładowywać biblioteki, dopóki jakakolwiek funkcja lub obiekt z tej

biblioteki nadal może być użyty. Przy bardziej złożonych wtyczkach potrzebny jest kontrakt wersji ABI i zarządzania czasem życia.

17.7 ABI — Application Binary Interface

17.7.1 API a ABI: dwa różne kontrakty

API opisuje przede wszystkim dostępne nazwy, typy, funkcje i semantykę na poziomie źródła. *ABI* określa, jak te elementy reprezentowane są w kodzie maszynowym. ABI obejmuje między innymi:

- konwencje wywołania: przekazywanie parametrów, wartość zwracana, rejestry i ułożenie stosu;
- reprezentację i wyrównanie obiektów, w tym pól klas;
- nazwy symboli C++, RTTI, mechanizmy wyjątków i wirtualnych wywołań;
- reguły udostępniania i ładowania symboli bibliotek;
- ABI standardowej biblioteki oraz ustawienia kompilatora.

Dwie biblioteki mogą być zgodne źródłowo i jednocześnie binarnie niezgodne. W szczególności identyczny nagłówek nie oznacza, że plik `.so` zbudowany innym toolchainem lub z innym zestawem opcji na pewno współpracuje z aplikacją.

17.7.2 Zmiana układu obiektu łamie zgodność binarną

Rozważmy klasę publiczną:

```
// Wersja biblioteki 1
class Record {
    int id_{};
public:
    int id() const;
};

// Wersja biblioteki 2
class Record {
    int id_{};
    double score_{}; // Nawet prywatne pole zmienia uklad.
public:
    int id() const;
};
```

Listing 17.24: Zmiana prywatnych pol nadal moze zmienic ABI.

Klient kompiluje np. `sizeof(Record)` według nagłówka, który widział podczas kompilacji. Gdy zamienimy jedynie bibliotekę na wersję z innym układem `Record`, może dojść do naruszenia kontraktu pamięci, nawet jeśli `id()` nadal ma taki sam podpis. Podobne ryzyko dotyczy dodawania wirtualnych funkcji i zmian hierarchii dziedziczenia.

17.7.3 Wzorzec PImpl

Aby zmniejszyć zależności kompilacyjne i ukryć układ implementacji, można przechowywać wskaźnik do niekompletnego typu. Dzięki temu zmiana *pól implementacyjnych* często nie zmienia rozmiaru publicznego obiektu.

```
#pragma once
#include <memory>
```

```

class Widget {
public:
    Widget();
    ~Widget();

    Widget(Widget&&) noexcept;
    Widget& operator=(Widget&&) noexcept;

    Widget(const Widget&) = delete;
    Widget& operator=(const Widget&) = delete;

    int value() const;
    void set_value(int value);

private:
    struct Impl;
    std::unique_ptr<Impl> impl_;
};
```

Listing 17.25: PImpl: plik widget.hpp.

```
#include "widget.hpp"
#include <utility>

struct Widget::Impl {
    int value = 0;
    // Tu moga pojawiac sie dalsze pola.
};

Widget::Widget() : impl_(std::make_unique<Impl>()) {}
Widget::~Widget() = default;
Widget::Widget(Widget&&) noexcept = default;
Widget& Widget::operator=(Widget&&) noexcept = default;

int Widget::value() const { return impl_->value; }
void Widget::set_value(int value) { impl_->value = value; }
```

Listing 17.26: PImpl: plik widget.cpp.

Destruktor jest zdefiniowany w `widget.cpp`, gdzie `Impl` jest kompletny; to istotne przy `std::unique_ptr<Impl>`. PImpl nie zapewnia automatycznie pełnej stabilności ABI: nadal liczy się zgodność kompilatora, standardowej biblioteki, eksportu symboli, konwencji wywołania i pozostałych publicznych deklaracji. Dodatkowy wskaźnik i alokacja na stercie mogą zwiększać koszt dostępu.

17.7.4 C ABI jako ostrożna granica interoperacyjności

Dla wtyczek i bibliotek wykorzystywanych z różnych języków często projektuje się wąski interfejs oparty na funkcjach `extern "C"` oraz nieprzezroczystych uchwytach (*opaque handles*). Zamiast przekazywać `std::string` lub klasy z funkcjami wirtualnymi, przekazuje się proste typy i wskaźniki.

```
#ifdef __cplusplus
extern "C" {
#endif

struct engine_handle;

engine_handle* engine_create(void);
void engine_destroy(engine_handle* p);
int engine_eval(engine_handle* p, double x, double* result);

```

```
#ifdef __cplusplus
}
#endif
```

Listing 17.27: Przyk?adowy szkic stabilniejszego interfejsu binarnego.

Podpis funkcji nadal wymaga uzgodnienia ABI danej platformy, konwencji wywołania i kontraktu błędów. Przekraczanie takiej granicy wyjątkami C++, pamięcią zwalnianą przez niezgodny alokator lub obiektami standardowej biblioteki jest niebezpieczne. Bezpieczniej zwracać kody błędu, a alokację i zwalnianie łączyć w pary po tej samej stronie interfejsu.

17.7.5 Wersje kompilatora i biblioteki standardowej

Istnieją różne rodziny ABI C++ (np. rodzina Itanium ABI stosowana na wielu platformach uniksowych oraz ABI MSVC w Windows). Nawet w ramach jednej rodziny znaczenie mają flagi kompilatora, ustawienia standardowej biblioteki, opcje wyjątków, RTTI i ABI konkretnych typów. Przykładowo GCC/libstdc++ używa mechanizmu tzw. *dual ABI* dla niektórych typów standardowych (m.in. `std::string`); mieszanie niezgodnych ustawień może skutkować błędami linkera lub działania. Nie należy zakładać, że biblioteki zbudowane dowolnymi wersjami GCC, Clanga i MSVC są wymienne binarnie.

17.8 System budowania: CMake i projekt wieloplikowy

17.8.1 Dlaczego targety są ważniejsze niż globalne flagi?

Ręczne wywoływanie kompilatora dobrze tłumaczy mechanikę, lecz dla większych projektów potrzebujemy grafu zależności, wariantów Debug/Release, testów i kontroli opcji kompilatora. Współczesny CMake modeluje budowanie przez *targety*: bibliotekę, program, interfejsową bibliotekę nagłówkową i ich właściwości.

Przykładowa struktura:

```
project/
  CMakeLists.txt
  include/
    mathstuff/math.hpp
  src/
    math.cpp
    main.cpp
```

Listing 17.28: Uk?ad plikow klasycznej biblioteki CMake.

W tej konkretnej strukturze pliki źródłowe mogą zaczynać się od `#include "mathstuff/math.hpp"`, a `main.cpp` również korzysta z tej samej ścieżki nagłówka. To dzięki `target_include_directories` kompilator znajduje katalog `include` bez wpisywania ręcznych flag `-I`.

```
cmake_minimum_required(VERSION 3.20)
project(MathStuff LANGUAGES CXX)

add_library(mathstuff STATIC src/math.cpp)
target_compile_features(mathstuff PUBLIC cxx_std_20)
target_include_directories(mathstuff PUBLIC
    $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
    $<INSTALL_INTERFACE:include>
)

add_executable(app src/main.cpp)
```

```
12 target_link_libraries(app PRIVATE mathstuff)
```

Listing 17.29: CMakeLists.txt dla klasycznej biblioteki.

```
1 cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
2 cmake --build build -j
3 ./build/app
```

Listing 17.30: Konfigurowanie i budowanie projektu z CMake.

`PRIVATE`, `PUBLIC` i `INTERFACE` określają *propagację wymagań użycia* (usage requirements) między targetami:

Słowo	Znaczenie
`PRIVATE`	Wymaganie tylko na potrzeby budowania danego targetu.
`INTERFACE`	Wymaganie wyłącznie dla jego konsumentów.
`PUBLIC`	Wymaganie dla targetu i jego konsumentów.

Gdy publiczny nagłówek biblioteki wymaga pewnego katalogu nagłówków, ten katalog powinien zwykle propagować się do klientów. Gdy jednak biblioteka korzysta z zależności jedynie wewnątrz `.cpp`, często wystarcza `PRIVATE`. Właściwy zakres zależy też od tego, czy target jest biblioteką statyczną lub współdzieloną i jakie zależności są konieczne przy ostatecznym linkowaniu.

17.8.2 Biblioteka nagłówkowa i `INTERFACE`

```
1 add_library(my_headers INTERFACE)
2 target_include_directories(my_headers INTERFACE
3     ${CMAKE_CURRENT_SOURCE_DIR}/include
4 )
5 target_compile_features(my_headers INTERFACE cxx_std_20)
6
7 add_executable(tool src/tool.cpp)
8 target_link_libraries(tool PRIVATE my_headers)
```

Listing 17.31: Header-only target i wymagania dla klientow.

Taki target nie tworzy archiwum `.a`. Reprezentuje zestaw wymagań i publiczne nagłówki używane przez konsumentów.

17.8.3 Diagnostyka, sanitizatory, LTO i build types

```
1 g++ -std=c++20 -O0 -g -Wall -Wextra -Wpedantic \
2     -fsanitize=address,undefined main.cpp -o app_debug
3
4 g++ -std=c++20 -O2 -flto main.cpp math.cpp -o app_lto
```

Listing 17.32: Przyk?adowe flagi diagnostyczne GCC/Clang.

Link-Time Optimization (LTO) może umożliwiać optymalizacje pomiędzy TU, np. dodatkowe wstawianie funkcji do miejsc wywołania. Jest to jednak kosztowna funkcja toolchainu; jej efekty zależą od programu i opcji. Profilowanie i powtarzalne pomiary są ważniejsze niż sam fakt włączenia `-O3` lub `-flto`.

17.9 Moduły C++20: nowy mechanizm organizacji kodu

17.9.1 Motywacja: problem tekstowego dołączania nagłówków

W klasycznym modelu ten sam popularny nagłówek może być parsowany oddzielnie w wielu TU. Makra i kolejność `#include` wpływają na wynik preprocessingu. Moduły C++20 umożliwiają jawne eksportowanie deklaracji i importowanie ich bez tekstowego wklejania całych interfejsów do każdego klienta.

Zaletami modułów mogą być izolacja makr, bardziej jednoznaczne zależności i krótsze czasy kompilacji. Nie ma jednak gwarancji, że *każdy* program po migracji przyspieszy, a wdrożenie wymaga wsparcia kompilatora i systemu budowania.

17.9.2 Pierwszy moduł: `export module` i `import`

Rozważmy dwa pliki:

```
export module geometry;

export int square(int x) {
    return x * x;
}

int hidden_helper(int x) {
    return x + 1;
}
```

Listing 17.33: Interfejs modu?u geometry: plik geometry.cppm.

```
#include <iostream>
import geometry;

int main() {
    std::cout << square(7) << '\n'; // 49
    // hidden_helper(7); // Nie jest eksportowana.
}
```

Listing 17.34: Klient modu?u: plik main.cpp.

`export module geometry;` deklaruje *główną jednostkę interfejsu modułu* (*primary module interface unit*). `export int square(...)` udostępnia deklarację klientowi. Samo `import geometry;` nie oznacza linkowania kodu funkcji z powietrza: jeśli moduł generuje kod obiektowy, trzeba go również uwzględnić podczas linkowania.

17.9.3 BMI/CMI: moduł nie jest biblioteką binarną

Kompilator wytwarza dla interfejsu modułu własną reprezentację, często nazywaną *BMI* (*Binary Module Interface*) lub *CMI* (*Compiled Module Interface*). W Clangu spotyka się pliki `.pcm`, w GCC `.gcm`, a w MSVC pliki `.ifc`. Nie są to uniwersalne, przenośne między kompilatorami artefakty podobne do biblioteki `.a`. Importer musi znać interfejs przetworzony przez zgodną konfigurację kompilatora.

Zależności muszą być budowane w poprawnym porządku:

źródło interfejsu modułu $\longrightarrow$ BMI/CMI $\longrightarrow$ kompilacja importera $\longrightarrow$ linkowanie obiektów.

17.9.4 Rzeczywiste komendy: Clang i GCC

Dla prostego modułu `geometry.cppm` w Clangu obsługującym standardowe moduły można zastosować:

```
clang++ -std=c++20 geometry.cppm --precompile -o geometry.pcm
clang++ -std=c++20 -c geometry.pcm -o geometry.o
clang++ -std=c++20 -fmodule-file=geometry=geometry.pcm \
    -c main.cpp -o main.o
clang++ geometry.o main.o -o app
./app
```

Listing 17.35: Budowanie przyk?adowego modu?u w Clangu (tryb BMI).

Dla GCC 14 (wsparcie modułów C++20 było w tej wersji nadal rozwijane) trzeba dodatkowo włączyć eksperymentalny mechanizm `-fmodules-ts`. W przykładach z GCC 14 przydatne bywa jawne wskazanie języka przy rozszerzeniu `.cppm`:

```
g++ -std=c++20 -fmodules-ts -x c++ -c geometry.cppm \
    -o geometry.o
# GCC tworzy potrzebne dane interfejsu w gcm.cache/.
g++ -std=c++20 -fmodules-ts -c main.cpp -o main.o
g++ geometry.o main.o -o app
./app
```

Listing 17.36: Budowanie tego samego modu?u z GCC 14.

Dokładne opcje i formaty modułów mogą zmieniać się między wersjami narzędzi. W dużym projekcie lepiej powierzyć zależności systemowi budowania obsługującemu skanowanie importów, zamiast ręcznie zarządzać plikami BMI.

17.9.5 Jednostka interfejsu i jednostka implementacji

Moduł może rozdzielać interfejs od implementacji bez tradycyjnego nagłówka. Jednostka implementacji deklaruje `module name;`, a nie `export module name;`:

```
// Plik: engine.cppm
export module engine;
export int run(int value);

// Plik: engine.cpp
module engine;
int run(int value) { return 2 * value; }
```

Listing 17.37: Dwa pliki jednego modu?u: interface.cppm i implementation.cpp.

Importer widzi tylko deklaracje eksportowane przez interfejs. Jednostkę implementacji również trzeba skompilować i zlinkować. Konkretny system budowania musi uwzględnić fakt, że jednostka implementacji importuje interfejs swojego modułu niejawnie.

17.9.6 Global module fragment i nagłówki klasyczne

Tradycyjny nagłówek może wymagać makr konfigurujących jego treść. W takim przypadku można skorzystać z *global module fragment*:

```
module;
#include <string>
#include <utility>

```

```
export module greeting;

export std::string greet(std::string name) {
    return "Hello " + std::move(name);
}
```

Listing 17.38: Global module fragment: tradycyjny include przed modu?em.

Dyrektywa `module;` rozpoczyna fragment globalny, a następna deklaracja `export module greeting;` rozpoczyna właściwą jednostkę modułu. Importowanie nagłówków jednostkowych przez `import <string>;` jest innym mechanizmem, tzw. *header units*, których dostępność zależy od toolchainu. Nie utożsamiajmy go z nazwanym modułem standardowej biblioteki `import std;`.

17.9.7 Partycje i ponowny eksport importu

Duży moduł można podzielić na partycje. Nazwa po dwukropku oznacza partycję wewnątrz jednego modułu, a nie odrębny nazwany moduł.

```
// Plik: math_detail.cppm
export module mathlib:detail;
export int triple(int x) { return 3 * x; }

// Plik: mathlib.cppm
export module mathlib;
export import :detail;
export int times_six(int x) { return 2 * triple(x); }

// Plik: main.cpp
import mathlib;
int main() { return times_six(7) == 42 ? 0 : 1; }
```

Listing 17.39: Dwie jednostki interfejsu modu?u z partycja.

Klient importuje `mathlib`, a nie `mathlib:detail`. W tym przykładzie `export import :detail;` re-eksportuje eksporty partycji. Wymaga to prawidłowej kolejności budowania *najpierw partycji, potem głównego interfejsu, a na końcu klienta.*

17.9.8 Łączność modułowa i ODR

C++20 wprowadza reguły własności deklaracji (*module ownership*). Deklaracje przypisane do nazwanego modułu należą do niego, a pewne nieeksportowane nazwy uzyskują *module linkage*. Różni się to od mechanicznego zastąpienia `#include` przez `import`.

Przykładowo dodanie `export` nie sprawia automatycznie, że interfejs C++ będzie zgodny ABI między kompilatorami. Ponadto źródłowy kod szablonów nadal musi być semantycznie dostępny odpowiedniemu mechanizmowi instancjacji, choć nie musi być już tekstowo wklejany przez `#include`.

17.9.9 CMake i `FILE_SET CXX_MODULES`

Od CMake 3.28 dostępne są mechanizmy skanowania modułów i specjalny typ zbioru plików `CXX_MODULES`. Przykład projektu:

```
cmake_minimum_required(VERSION 3.28)
project(GeometryModules LANGUAGES CXX)

add_library(geometry)
target_compile_features(geometry PUBLIC cxx_std_20)
target_sources(geometry PUBLIC
```

```
    FILE_SET CXX_MODULES FILES geometry.cppm
)

add_executable(app main.cpp)
target_link_libraries(app PRIVATE geometry)
```

Listing 17.40: CMakeLists.txt projektu korzystajacego z modu?ow C++20.

```
cmake -S . -B build -G Ninja
cmake --build build
./build/app
```

Listing 17.41: Konfiguracja modu?ow przy uzyciu generatora Ninja.

Ten przykład pokazuje *składnię* CMake, ale jego zbudowanie wymaga zgodnego zestawu wersji CMake, kompilatora, generatora i skanera zależności. W szczególności kompilator akceptujący składnię modułów niekoniecznie współpracuje z automatycznym skanowaniem w danej konfiguracji CMake. Na współczesnych systemach używa się m.in. Ninja (co najmniej 1.11 w typowych konfiguracjach) i odpowiednich wersji Clanga lub MSVC. Obsługa modułów standardowej biblioteki oraz *header units* ma odrębne ograniczenia.

17.9.10 `import std;` nie jest tym samym co `#include`

Sam standard C++20 wprowadza językowy mechanizm modułów, ale *nie zobowiązuje* każdej biblioteki standardowej do dostarczenia nazwanego modułu `std` w tym standardzie. `import std;` jest standaryzowane w C++23; obsługa zależy od kompilatora, dostawcy biblioteki standardowej i wersji systemu budowania.

```
import std;

int main() {
    std::vector<int> xs{1, 2, 3};
    std::println("size = {}", xs.size());
}
```

Listing 17.42: Nazwany modu? standardowej biblioteki (C++23, zalezny od wsparcia).

Powyższy przykład wykorzystuje także `std::println` z C++23. Nie należy automatycznie kompilować go z `-std=c++20` ani zakładać, że `import std;` działa w każdym środowisku C++23.

17.10 Optymalizacja procesu budowania

17.10.1 Incremental build i zależności nagłówkowe

Zmiana publicznego nagłówka może wymusić ponowną kompilację wielu TU, podczas gdy zmiana prywatnego `.cpp` zwykle dotyka mniejszej części projektu. Dlatego ograniczanie niepotrzebnych dołączeń nagłówków, stosowanie deklaracji wyprzedzających tam, gdzie są prawidłowe, i PImpl mogą poprawiać czas budowania.

Moduły zmieniają strukturę zależności, ale nie usuwają konieczności ich śledzenia; importer musi otrzymać aktualną reprezentację interfejsu modułu.

17.10.2 Precompiled headers, unity builds, ccache

Popularne techniki przyspieszania kompilacji to:

Precompiled headers (PCH)
Przechowywanie wyniku parsowania stabilnych, obszernych nagłówków. Pomagają w projektach z dużą liczbą TU, ale wymagają spójności opcji kompilacji i zarządzania unieważnianiem pamięci podręcznej.

Unity builds
Łączenie kilku plików źródłowych we wspólną TU. Może ograniczać narzut kompilacji, ale też ujawniać kolizje anonimowych przestrzeni nazw, makr i definicji o łączności wewnętrznej.

Compilation cache
Narzędzia takie jak `ccache` unikają ponownego wykonywania kompilacji, jeżeli odpowiednio zdefiniowane wejścia pozostają takie same.

Parallel build
Kompilowanie niezależnych TU równolegle (np. `cmake --build build -j`) wykorzystuje wiele rdzeni, ale zwiększa chwilowe użycie pamięci.

17.10.3 Powtarzalność buildów i zależności transytywne

Dla bibliotek publicznych warto utrzymywać przejrzystą strukturę zależności. Przykładowo użytkownik nie powinien polegać na tym, że `#include <vector>` „przypadkiem" pojawiło się wskutek dołączenia innego nagłówka. Każdy nagłówek powinien zawierać potrzebne deklaracje w sposób niezależny od kolejności dołączeń.

Warto również zapisywać wersje kompilatora i bibliotek, flagi wpływające na ABI, wariant budowania oraz sposób instalowania artefaktów. *Reproducible builds* wymagają dodatkowo kontroli źródeł niedeterministyczności, np. znaczników czasu i ścieżek zapisywanych w plikach wynikowych.

17.11 Praktyczny przewodnik diagnostyczny

Objaw	Najbardziej prawdopodobne źródło problemu
`undefined reference`	Brakujący plik obiektowy, biblioteka, definicja albo instancjacja szablonu.
`multiple definition`	Niepoprawna definicja zwykłej funkcji lub zmiennej w nagłówku albo podwójnie linkowany obiekt.
`undefined reference to vtable`	Niezdefiniowana wymagana funkcja wirtualna (często destruktor) lub brak pliku implementacji.
`cannot find -lfoo`	Brak biblioteki lub nieprawidłowe katalogi przeszukiwania linkera.
`cannot open shared object file`	Problem ścieżki lub zależności ładowania, niekoniecznie problem kompilacji.
Kompilacja klienta działa, program kończy się błędem	Możliwa niezgodność ABI, naruszenie ODR, zły czas życia obiektu lub błąd programu.
Importer nie znajduje modułu	Nie zbudowano BMI/CMI, nie wskazano ścieżki albo konfiguracja skanera nie obsługuje importu.
Import działa, ale linker zgłasza brak funkcji	Brak pliku obiektowego modułu/implementacji lub potrzebnej biblioteki.
Zmiana nagłówka przebudowuje prawie wszystko	Silne zależności nagłówkowe; rozważ redukcję dołączeń, PImpl, moduły lub PCH.

17.11.1 Kiedy wybrać nagłówki, bibliotekę czy moduł?

- **Header-only** sprawdza się dla małych narzędzi, szablonów i funkcji `inline`; ceną mogą być zależności kompilacyjne.

- **Biblioteka statyczna** upraszcza dystrybucję kodu obiektowego wewnątrz aplikacji, ale nie usuwa wymagań ABI w czasie linkowania.
- **Biblioteka współdzielona** pozwala współdzielić kod i wymieniać implementację, lecz wymaga jasnej polityki ABI, widoczności symboli i wersjonowania.
- **Moduł C++20** służy do semantycznego podziału interfejsu źródłowego i kontroli importów; nie jest alternatywą wobec procesu linkowania ani automatycznym formatem dystrybucji ABI.
- **PImpl i C API** rozważaj na granicach bibliotek o długim cyklu życia, szczególnie gdy stabilność ABI ma duże znaczenie.

Podsumowanie rozdziału

Kompletny program C++ jest wynikiem wspólnej pracy preprocessora, kompilatora, linkera i loadera. Organizując projekt, trzeba osobno rozumieć *deklaracje i definicje*, *ODR i łączność*, *symbole i biblioteki* oraz *ABI*. Nagłówki i moduły pomagają udostępniać interfejsy źródłowe; biblioteki statyczne i współdzielone dostarczają kod do połączenia lub załadowania.

Dobra organizacja projektu polega na jasnym rozdziale API i implementacji, ograniczaniu zależności, korzystaniu z targetów CMake oraz świadomym wersjonowaniu interfejsu binarnego. Nowoczesne moduły upraszczają część problemów nagłówków, ale wprowadzają wymagania dotyczące narzędzi, BMI/CMI i kolejności kompilacji.

Literatura i dokumentacja

- ISO/IEC 14882:2020, zagadnienia *translation units*, *linkage*, *One Definition Rule* i *modules*.
- https://en.cppreference.com/w/cpp/language/definition — ODR, deklaracje i definicje.
- https://en.cppreference.com/w/cpp/language/modules — składnia i semantyka modułów.
- https://gcc.gnu.org/onlinedocs/gcc/C_002b_002b-Modules.html — implementacja modułów w GCC.
- https://clang.llvm.org/docs/StandardCPlusPlusModules.html — moduły w Clangu, BMI i komendy budowania.
- https://cmake.org/cmake/help/latest/manual/cmake-cxxmodules.7.html — CMake i zależności modułów.
- https://cmake.org/cmake/help/latest/command/target_sources.html — `FILE_SET CXX_MODULES`.
- https://gcc.gnu.org/onlinedocs/gcc/Code-Gen-Options.html — widoczność symboli w ELF.
- John Lakos, *Large-Scale C++ Software Design*, Addison-Wesley, 1996.